

Java từ Cơ bản đến Nâng cao — Tập 1

Sách lập trình Java cho người mới bắt đầu — dạy OOP từ số 0 (Tập 2: Java
J2EE/Jakarta EE, xuất bản sau)

https://github.com/phamtuanchip/java_book

Mục lục

Phần 0 — Chuẩn bị môi trường

1. Giới thiệu Java: JDK/JRE/JVM là gì
2. Cài đặt JDK và IDE
3. Chương trình Java đầu tiên
4. Làm quen IDE và giới thiệu Maven/Gradle

Phần 1 — Nền tảng lập trình

5. Biến, kiểu dữ liệu nguyên thủy, ép kiểu
6. Toán tử và biểu thức
7. Cấu trúc điều khiển: if/else, switch
8. Vòng lặp: for, while, do-while
9. Mảng một chiều và nhiều chiều
10. Phương thức (method) tĩnh
11. Chuỗi (String) và StringBuilder

Phần 2 — Nhập môn OOP

12. Vì sao cần OOP
13. Class & Object
14. Encapsulation (đóng gói)
15. Package & import

Phần 3 — OOP nâng cao

16. Kế thừa (Inheritance)
17. Đa hình (Polymorphism)
18. Abstract class & Interface
19. Lớp lồng, anonymous class, enum nâng cao
20. Lớp Object: equals/hashCode/toString

Phần 4 — Xử lý lỗi & Generics

21. Exception handling
22. Generics

Phần 5 — Collection Framework & Cấu trúc dữ liệu

23. Tổng quan Collection Framework
24. ArrayList & LinkedList
25. HashSet, TreeSet, LinkedHashSet
26. HashMap, TreeMap, LinkedHashMap
27. Duyệt & sắp xếp: Iterator, Comparator
28. Thuật toán tìm kiếm: Linear, Binary, Jump, Interpolation Search

29. Duyệt đồ thị/cây: BFS & DFS

Phần 6 — Lập trình hàm & Java hiện đại

30. Lambda expression & functional interface

31. Stream API

32. Optional và xử lý null an toàn

33. Java hiện đại: var, record, sealed, pattern matching

Phần 7 — Nhập/Xuất & Xử lý dữ liệu

34. File I/O: java.io và java.nio.file

35. Xử lý JSON/CSV

36. Serialization cơ bản

Phần 8 — Đa luồng & bất đồng bộ

37. Thread cơ bản

38. Đồng bộ hoá: synchronized, Lock

39. ExecutorService & CompletableFuture

40. Giới thiệu Virtual Thread

Phần 9 — Công cụ, kiểm thử & quản lý dự án

41. Unit test với JUnit 5

42. Build tool: Maven và Gradle

43. Debug & logging

Phần 10 — Kiến trúc & thực hành nâng cao

44. Design pattern cơ bản

45. Nguyên lý SOLID

46. Kết nối cơ sở dữ liệu: JDBC cơ bản

47. Dự án tổng hợp

Phụ lục

48. Phụ lục A — Lỗi thường gặp & cách xử lý

49. Phụ lục B — Tài liệu tham khảo

Chương 1 — Giới thiệu Java: JDK/JRE/JVM là gì

Mục tiêu học

Sau chương này, bạn sẽ:

- Biết Java là gì, dùng để làm gì, và vì sao nó vẫn là một trong những ngôn ngữ phổ biến nhất sau gần 30 năm.
- Phân biệt rõ ràng ba khái niệm hay bị nhầm lẫn: **JVM, JDK, JRE**.
- Hiểu cơ chế "viết một lần, chạy mọi nơi" (write once, run anywhere) hoạt động thế nào.
- Biết cách chọn phiên bản Java nào để học (bản LTS là gì, vì sao chương này chọn Java 21).

Chương này **không có code** — đây là chương nền tảng khái niệm. Từ Chương 3 bạn sẽ viết và chạy chương trình Java đầu tiên.

Java là gì?

Java là một ngôn ngữ lập trình **hướng đối tượng** (object-oriented), ra đời năm 1995 bởi Sun Microsystems (nay thuộc Oracle). Java được dùng để viết:

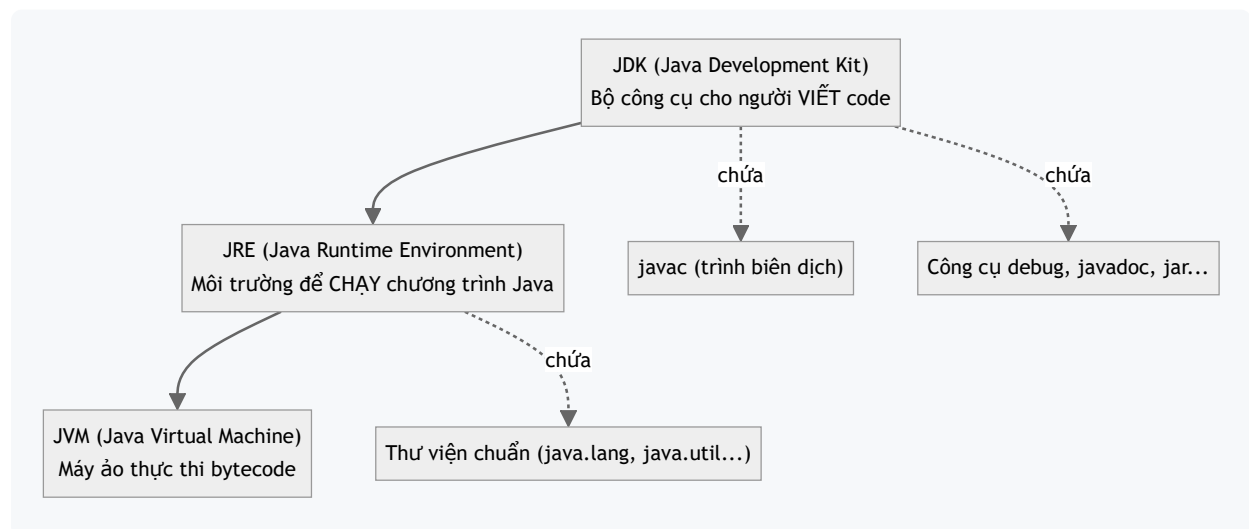
- Ứng dụng doanh nghiệp (backend server, hệ thống ngân hàng, bảo hiểm...) — phần lớn hệ thống lớn ở các công ty vẫn chạy Java.
- Ứng dụng Android (Android SDK dùng Java/Kotlin làm ngôn ngữ chính).
- Công cụ, hệ thống lớn (Hadoop, Kafka, Elasticsearch đều viết bằng Java).

Điều khiến Java tồn tại lâu và được dùng rộng rãi không phải vì cú pháp đẹp, mà vì **tính ổn định, khả năng chạy đa nền tảng, và hệ sinh thái thư viện khổng lồ**. Đây cũng là lý do sách này chọn Java làm ngôn ngữ đầu tiên để dạy tư duy hướng đối tượng (OOP): Java buộc bạn viết OOP đúng cách, không có đường tắt như một số ngôn ngữ khác.

Lưu ý quan trọng cho người mới: Java và JavaScript là **hai ngôn ngữ hoàn toàn khác nhau**, không liên quan gì ngoài cái tên. JavaScript chạy trong trình duyệt, dùng cho web frontend; Java là ngôn ngữ tổng quát, biên dịch và chạy trên máy ảo riêng của nó (JVM — sẽ giải thích ngay sau đây). Đừng nhầm lẫn hai cái này khi tìm tài liệu.

JVM, JDK, JRE — ba khái niệm nền tảng

Đây là câu hỏi gần như 100% người mới học Java sẽ bị rối ở tuần đầu tiên. Hãy hình dung theo lớp từ trong ra ngoài:



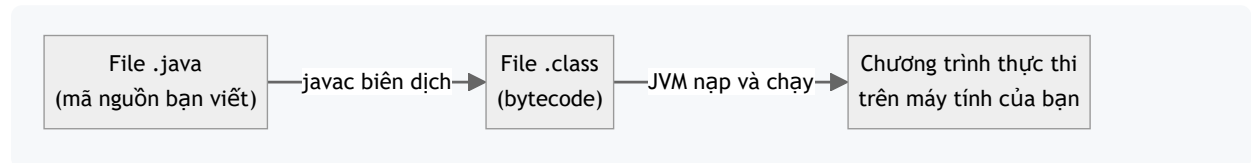
- **JVM (Java Virtual Machine)** — máy ảo thực thi. Đây là chương trình đọc và chạy **bytecode** (file `.class`) — không chạy trực tiếp mã nguồn `.java`. Mỗi hệ điều hành (Windows, macOS, Linux) có một bản JVM riêng, nhưng bytecode thì giống

hết nhau trên mọi nền tảng. Đây chính là cơ chế "viết một lần, chạy mọi nơi": bạn biên dịch code Java **một lần**, ra file `.class` chứa bytecode, rồi file đó chạy được trên bất kỳ máy nào có JVM — không cần biên dịch lại.

- **JRE (Java Runtime Environment)** — JVM cộng thêm các thư viện chuẩn cần thiết để **chạy** một chương trình Java đã biên dịch sẵn. Nếu máy bạn chỉ cần *chạy* ứng dụng Java (không viết code), về lý thuyết chỉ cần JRE là đủ.
- **JDK (Java Development Kit)** — JRE cộng thêm các công cụ để **viết và biên dịch** code, quan trọng nhất là `javac` (trình biên dịch, dịch file `.java` thành `.class`). JDK là thứ **bạn cần cài đặt** để học lập trình Java, vì nó bao gồm cả JRE lẫn JVM bên trong.

Tóm gọn một câu: **JDK** $\supset$ **JRE** $\supset$ **JVM** (JDK chứa JRE, JRE chứa JVM). Từ Java 11 trở đi, Oracle không còn phát hành JRE độc lập nữa — bạn chỉ cần tải và cài **JDK** là có đủ mọi thứ để vừa viết vừa chạy code. Chương 2 sẽ hướng dẫn cài đặt cụ thể.

Quy trình một chương trình Java chạy như thế nào



Khác với ngôn ngữ biên dịch trực tiếp ra mã máy (như C/C++, biên dịch ra file `.exe` chỉ chạy được trên đúng hệ điều hành đã biên dịch), Java biên dịch ra **bytecode** — một dạng mã trung gian độc lập hệ điều hành. Chính JVM (khác nhau theo từng OS) mới là thứ chuyển bytecode thành lệnh máy thực sự tại thời điểm chạy. Đánh đổi: Java thường chạy chậm hơn một chút so với code biên dịch thẳng ra máy (dù JVM hiện đại có JIT compiler tối ưu hoá rất mạnh), nhưng đổi lại bạn không phải biên dịch lại code cho từng hệ điều hành.

Hệ sinh thái Java

- **Java SE (Standard Edition)** — bộ nền tảng cơ bản, dùng cho ứng dụng desktop/console. Đây là thứ sách này dạy từ đầu đến cuối.
- **Java EE / Jakarta EE (Enterprise Edition)** — mở rộng cho ứng dụng server/web quy mô lớn (không nằm trong phạm vi sách này, nhưng kiến thức OOP + collection + exception trong sách là nền tảng bắt buộc trước khi học Jakarta EE hay Spring Framework).
- **Android** — Android SDK dùng Java (và Kotlin) làm ngôn ngữ phát triển ứng dụng chính.
- Các framework phổ biến trong công việc thực tế: **Spring/Spring Boot** (backend web), **Maven/ Gradle** (quản lý build & dependency — sẽ học ở Phần 9), **JUnit** (kiểm thử — Phần 9).

Phiên bản Java và bản LTS

Java phát hành phiên bản mới mỗi 6 tháng, nhưng không phải bản nào cũng đáng dùng lâu dài. Oracle đánh dấu một số bản là **LTS (Long-Term Support)** — được hỗ trợ và vá lỗi trong nhiều năm, phù hợp để học và dùng trong dự án thật. Các bản LTS gần đây: Java 8, 11, 17, 21.

Sách này dùng **Java 21 (LTS)** làm chuẩn cho toàn bộ code mẫu, vì đây là bản LTS mới nhất tại thời điểm biên soạn, có đầy đủ các tính năng hiện đại sẽ học ở Phần 6 (record, pattern matching, virtual thread...). Những chương dùng tính năng chỉ có ở Java 21 sẽ ghi chú rõ để bạn biết nếu đang dùng bản cũ hơn (ví dụ Java 8 hoặc Java 17) thì phần nào không chạy được.

Bài tập

1. Tự giải thích lại bằng lời của bạn (không nhìn sách): sự khác nhau giữa JDK, JRE, JVM là gì? Nếu máy bạn chỉ cài JRE (không có JDK), bạn có viết được code Java không? Vì sao?
2. Tìm hiểu: "bytecode" khác "mã máy" (machine code) ở điểm nào? Vì sao JVM cần tồn tại thay vì Java biên dịch thẳng ra mã máy như C?
3. Liệt kê 3 phần mềm hoặc hệ thống bạn biết (hoặc tra cứu) được viết bằng Java.

Hiểu lầm thường gặp

- **"Cứ cài JRE là học lập trình Java được"** — Sai. JRE chỉ để *chạy* chương trình đã biên dịch sẵn, không có `javac` nên không biên dịch được code bạn viết. Phải cài **JDK**.
- **"Java chậm hơn hẳn C/C++ nên không dùng được cho hệ thống lớn"** — Không chính xác trong thực tế: JVM hiện đại có JIT (Just-In-Time) compiler tối ưu hoá bytecode thành mã máy ngay khi chạy, hiệu năng đủ tốt cho tuyệt đại đa số ứng dụng doanh nghiệp — bằng chứng là các hệ thống lớn như LinkedIn, Netflix backend, hay toàn bộ Android đều dựa trên Java/JVM.
- **"Java và JavaScript liên quan đến nhau"** — Không. Tên giống nhau chỉ vì lý do marketing lịch sử (JavaScript ra đời sau, mượn tên Java lúc đó đang nổi để dễ tiếp cận), về mặt kỹ thuật hai ngôn ngữ không chia sẻ gì cả.

Chương 2 — Cài đặt JDK và IDE

Mục tiêu học

Sau chương này, bạn sẽ:

- Cài đặt được JDK trên máy (Windows, macOS, hoặc Linux).
- Kiểm tra được JDK đã cài đúng bằng dòng lệnh (`java -version` , `javac -version`).
- Chọn và cài một IDE phù hợp để viết code Java.
- Hiểu `PATH` và `JAVA_HOME` là gì, vì sao đôi khi cần cấu hình tay.

Chọn bản JDK nào

Có nhiều nhà cung cấp bản build JDK (đều tuân theo cùng chuẩn OpenJDK): Oracle JDK, Eclipse Temurin (Adoptium), Amazon Corretto, Azul Zulu... Với người mới học, khác biệt giữa các bản build gần như không ảnh hưởng gì. Sách này khuyến nghị **Eclipse Temurin 21 (LTS)** vì miễn phí hoàn toàn cho mọi mục đích sử dụng (kể cả thương mại) và không yêu cầu tài khoản để tải.

- Trang tải: adoptium.net → chọn **Version 21 (LTS)**, đúng hệ điều hành và kiến trúc CPU (x64 hoặc ARM64) của máy bạn.

Cài đặt theo hệ điều hành

Windows

1. Tải file `.msi` từ Adoptium cho Windows x64.
2. Chạy trình cài đặt, ở bước tùy chọn (Custom Setup) đảm bảo bật **"Set JAVA_HOME variable"** và **"Add to PATH"** — hai tùy chọn này giúp bạn không phải tự cấu hình biến môi trường tay.
3. Mở lại **PowerShell hoặc Command Prompt mới** (bắt buộc mở cửa sổ mới để nạp biến môi trường vừa cài) và kiểm tra:

```
java -version
javac -version
```

Nếu thấy hiện ra số phiên bản (ví dụ `openjdk version "21.0.x"`), cài đặt đã thành công.

Nếu vẫn báo 'java' is not recognized : trình cài đặt chưa thêm JDK vào `PATH` . Cách sửa thủ công: mở **Settings** → **System** → **About** → **Advanced system settings** → **Environment Variables**, thêm biến `JAVA_HOME` trỏ tới thư mục cài JDK (ví dụ `C:\Program Files\Eclipse Adoptium\jdk-21.0.x-hotspot`), sau đó thêm `%JAVA_HOME%\bin` vào biến `Path` . Mở cửa sổ terminal mới rồi kiểm tra lại.

macOS

Cách đơn giản nhất là dùng [Homebrew](https://brew.sh/):

```
brew install --cask temurin@21
java -version
```

Nếu chưa có Homebrew, tải file `.pkg` từ Adoptium và chạy trình cài đặt như ứng dụng macOS thông thường.

Linux (Debian/Ubuntu)

```
sudo apt update
sudo apt install temurin-21-jdk
java -version
```

Nếu package `temurin-21-jdk` không có sẵn trong repository mặc định, làm theo hướng dẫn thêm repository của Adoptium tại trang tải, hoặc tải file `.tar.gz` và giải nén thủ công, thêm đường dẫn vào `PATH`.

PATH và JAVA_HOME là gì?

- PATH** là danh sách thư mục mà hệ điều hành tìm chương trình thực thi khi bạn gõ lệnh trong terminal. Khi bạn gõ `java`, hệ điều hành tìm file thực thi tên `java` trong từng thư mục liệt kê ở `PATH`. Nếu thư mục `bin` của JDK không nằm trong `PATH`, hệ thống báo "not recognized"/"command not found" dù JDK vẫn cài đúng trên máy.
- JAVA_HOME** là một biến môi trường quy ước trỏ tới thư mục gốc cài JDK. Bản thân hệ điều hành không bắt buộc phải có biến này để chạy `java`, nhưng rất nhiều công cụ (Maven, Gradle, IDE) dựa vào `JAVA_HOME` để biết JDK nằm ở đâu — nên cần cấu hình đúng ngay từ đầu.

Chọn IDE

IDE (Integrated Development Environment) là công cụ giúp viết code có gợi ý, kiểm tra lỗi ngay khi gõ, chạy/debug bằng một nút bấm — thay vì phải gõ `javac` / `java` tay liên tục. Hai lựa chọn phổ biến nhất:

	IntelliJ IDEA Community	VS Code + Extension Pack for Java
Giá	Miễn phí	Miễn phí
Thế mạnh	Gợi ý code, refactor, debug mạnh nhất cho Java — chuẩn công nghiệp	Nhẹ, khởi động nhanh, dùng chung được cho nhiều ngôn ngữ khác
Phù hợp	Người tập trung học Java nghiêm túc, dự án lớn dần về sau	Người đã quen VS Code, muốn một công cụ dùng cho nhiều việc

Sách này minh họa bằng **IntelliJ IDEA Community Edition** (miễn phí, tải tại jetbrains.com/idea) vì đây là công cụ phổ biến nhất trong môi trường làm việc Java thực tế — nhưng mọi ví dụ trong sách đều chạy được bằng dòng lệnh thuần (`javac` / `java`) nếu bạn chọn dùng VS Code hoặc trình soạn thảo khác.

Cài đặt IntelliJ IDEA Community: tải trình cài đặt cho hệ điều hành của bạn, chạy theo mặc định. Không cần cấu hình gì thêm — IntelliJ sẽ tự phát hiện JDK đã cài ở bước trên khi bạn tạo project mới (Chương 4 sẽ hướng dẫn tạo project đầu tiên trong IDE).

Bài tập

- Cài JDK 21 theo hướng dẫn ở trên, chạy `java -version` và `javac -version`, chụp lại kết quả.
- Tìm trên máy bạn thư mục JDK vừa cài nằm ở đâu (đường dẫn cụ thể), xác nhận đó chính là giá trị nên gán cho `JAVA_HOME`.
- Cài IntelliJ IDEA Community (hoặc VS Code + Extension Pack for Java nếu bạn thích công cụ nhẹ hơn).

Lỗi thường gặp

- 'java' is not recognized as an internal or external command (Windows)** — JDK chưa được thêm vào `PATH`. Xem lại phần "Nếu vẫn báo..." ở trên.
- Cài nhiều bản JDK cùng lúc, java -version báo bản cũ** — `PATH` đang trỏ tới bản JDK cũ trước bản mới. Kiểm tra thứ tự các đường dẫn trong biến `PATH`, đường dẫn nào đứng trước sẽ được ưu tiên tìm thấy trước.
- macOS báo "java" không mở được vì không rõ nhà phát triển** — vào **System Settings** → **Privacy & Security**, cuộn xuống phần cảnh báo và chọn "Allow Anyway", sau đó thử lại.
- Cài JDK xong nhưng terminal cũ vẫn báo lỗi** — terminal đang mở **trước khi** cài đặt cập nhật biến môi trường, nó không tự nạp lại. Đóng và mở terminal mới.

Chương 3 — Chương trình Java đầu tiên

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết, biên dịch (`javac`) và chạy (`java`) được một chương trình Java bằng dòng lệnh, không cần IDE.
- Đọc hiểu từng dòng của một file `.java` tối thiểu: `class` , `public static void main` , vì sao cấu trúc đó bắt buộc phải có.
- Hiểu quy tắc đặt tên file phải khớp tên class public, và lý do đằng sau quy tắc đó.
- Biết cách nhận đối số dòng lệnh qua `String[] args` .

Code mẫu đầy đủ: <code/ch03-hello-world/> .

Viết chương trình đầu tiên

Tạo một file tên `HelloWorld.java` , nội dung:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Xin chào, Java!");  
    }  
}
```

Biên dịch và chạy bằng terminal (mở tại đúng thư mục chứa file):

```
javac HelloWorld.java  
java HelloWorld
```

Kết quả:

```
Xin chào, Java!
```

Chuyện gì vừa xảy ra? `javac HelloWorld.java` gọi trình biên dịch, đọc file mã nguồn, sinh ra file `HelloWorld.class` (bytecode) trong cùng thư mục. `java HelloWorld` gọi JVM nạp file `.class` đó và chạy nó — chú ý lệnh thứ hai **không có đuôi** `.java` , vì bạn đang chạy bytecode đã biên dịch, không phải mã nguồn.

Giải phẫu từng dòng

```
public class HelloWorld {
```

Mọi code Java (trừ vài trường hợp đặc biệt sẽ học sau) phải nằm trong một **class**. `HelloWorld` là tên class. Từ khoá `public` nghĩa là class này có thể được truy cập từ bất kỳ đâu (sẽ học kỹ ở Chương 14 — Encapsulation).

Quy tắc bắt buộc: nếu class được khai báo `public` , **tên file phải trùng chính xác tên class** (phân biệt hoa/thường), cộng đuôi `.java` . Class `HelloWorld` phải nằm trong file `HelloWorld.java` — không phải `helloworld.java` hay `Hello.java` . Trình biên dịch dùng tên file để biết class public nào cần tìm; sai tên là lỗi biên dịch ngay lập tức.

```
    public static void main(String[] args) {
```

Đây là **phương thức** `main` — điểm bắt đầu chạy chương trình. Khi bạn gõ `java HelloWorld` , JVM tìm trong class `HelloWorld` một phương thức có **chính xác** chữ ký này để bắt đầu chạy. Từng từ khoá có lý do tồn tại (sẽ hiểu sâu hơn ở Chương 10 và Chương 13, ở đây cứ ghi nhớ như một khuôn mẫu cố định):

- `public` — JVM (từ bên ngoài class) phải gọi được phương thức này.
- `static` — JVM gọi `main` **mà chưa có object nào được tạo** (object là gì sẽ học ở Chương 13). `static` nghĩa là phương thức thuộc về class, không cần tạo object mới gọi được.
- `void` — `main` không trả về giá trị nào.
- `String[] args` — mảng chuỗi chứa các đối số dòng lệnh truyền vào khi chạy chương trình.

Đổi bất kỳ phần nào trong chữ ký này (ví dụ bỏ `static`, hay đổi tên tham số thành kiểu khác) đều khiến JVM không tìm thấy điểm bắt đầu và báo lỗi `Error: Main method not found`.

```
System.out.println("Xin chào, Java!");
```

`System.out` là "cổng ra" chuẩn để in ra màn hình console. `println` in ra một dòng, tự động xuống dòng ở cuối. Dấu `;` bắt buộc kết thúc mỗi câu lệnh trong Java — quên dấu `;` là lỗi cú pháp phổ biến nhất với người mới.

Nhận đối số dòng lệnh

Xem file `Greeter.java` trong code mẫu:

```
public class Greeter {
    public static void main(String[] args) {
        String name = "học viên";
        if (args.length > 0) {
            name = args[0];
        }
        System.out.println("Xin chào, " + name + "!");
    }
}
```

Chạy `java Greeter Tuan` → `args` là mảng chứa `["Tuan"]`, `args[0]` là `"Tuan"`. Chạy `java Greeter` (không truyền gì) → `args.length` bằng 0, dùng giá trị mặc định. Đây là cách chương trình dòng lệnh nhận input mà chưa cần học đọc bàn phím (`Scanner` sẽ học ở Chương 5).

Chạy nhanh không cần biên dịch riêng (Java 11+)

Với một file duy nhất, bạn có thể bỏ qua bước `javac` để thử nhanh:

```
java HelloWorld.java
```

Lệnh này biên dịch trong bộ nhớ rồi chạy ngay, không sinh ra file `.class` trên đĩa. Tiện cho việc thử nghiệm nhanh, nhưng khi chương trình có nhiều file/nhiều class phụ thuộc nhau, bạn vẫn cần `javac` (sẽ gặp ở các chương sau khi chương trình lớn dần).

Bài tập

1. Sửa `HelloWorld.java` để in thêm dòng thứ hai là tên của bạn.
2. Thử xoá dấu `;` ở cuối dòng `System.out.println(...)`, biên dịch lại và đọc kỹ thông báo lỗi trình biên dịch đưa ra — tập làm quen đọc lỗi biên dịch, đây là kỹ năng dùng suốt quá trình học.
3. Sửa `Greeter.java` để nếu có từ 2 đối số trở lên, in ra lời chào cho **từng** đối số (gợi ý: dùng vòng lặp — nếu chưa học vòng lặp ở Chương 8, có thể tạm dùng `args[0]`, `args[1]` tay và quay lại bài tập này sau khi học xong Chương 8).

Lỗi thường gặp

- `error: class HelloWorld is public, should be declared in a file named HelloWorld.java` — tên file không khớp tên class public. Đổi tên file hoặc tên class cho khớp nhau.

- **Error: Main method not found in class HelloWorld** — chữ ký `main` sai (thiếu `static`, sai kiểu tham số, viết hoa/thường sai tên `main` ...). Đối chiếu lại đúng khuôn mẫu `public static void main(String[] args)`.
- **';' expected** — thiếu dấu chấm phẩy cuối câu lệnh. Trình biên dịch Java báo dòng và cột khá chính xác, đọc kỹ thông báo trước khi đoán mò.
- **java: command not found dù đã cài JDK** — xem lại Chương 2, phần cấu hình `PATH`.
- **Chạy `java HelloWorld.java` mà báo lỗi khác `java HelloWorld`** — hai lệnh không tương đương hoàn toàn khi chương trình có nhiều file; chạy bằng file nguồn đơn chỉ hỗ trợ đúng nghĩa "một file duy nhất".

Chương 4 — Làm quen IDE và giới thiệu Maven/Gradle

Mục tiêu học

Sau chương này, bạn sẽ:

- Tạo được một project mới trong IntelliJ IDEA, chạy chương trình bằng nút Run thay vì gõ `javac / java` tay.
- Biết cách đặt breakpoint và debug từng bước cơ bản.
- Hiểu vì sao project nhiều file cần công cụ quản lý build, và Maven/Gradle giải quyết vấn đề gì (chưa cần cài đặt hay dùng thật — sẽ học sâu ở Chương 42).

Code mẫu: [code/ch04-multi-file-project/](https://code.ch04-multi-file-project/).

Tạo project mới trong IntelliJ IDEA

1. Mở IntelliJ IDEA → **New Project**.
2. Chọn **Java** ở danh sách bên trái, đặt tên project (ví dụ `hello-intellij`).
3. Ở mục **JDK**, chọn bản JDK 21 đã cài ở Chương 2 (nếu IntelliJ không tự thấy, bấm **Add JDK...** và trở tới thư mục cài JDK).
4. Bỏ chọn (hoặc để mặc định, không ảnh hưởng ở giai đoạn này) tùy chọn tạo bằng Maven/Gradle — ở chương này ta tạo project Java thuần trước, chưa cần build tool.
5. IntelliJ tạo sẵn thư mục `src` — đây là nơi chứa toàn bộ file `.java` của bạn.

Chuột phải vào `src` → **New** → **Java Class**, đặt tên `Main`, IntelliJ tự sinh khung:

```
public class Main {  
    public static void main(String[] args) {  
  
    }  
}
```

Gõ `System.out.println("Hello");` vào bên trong `main`, bấm nút **(Run)** ở góc trái dòng khai báo `main` (hoặc bấm chuột phải trong file → Run 'Main.main()'). IntelliJ tự động biên dịch mọi file `.java` liên quan (tương đương chạy `javac` rồi `java` bạn đã làm tay ở Chương 3) và hiện kết quả ở panel **Run** phía dưới.

Debug cơ bản

Debug là kỹ năng quan trọng nhất khi chương trình chạy sai mà bạn không hiểu vì sao. Ba thao tác cần biết ngay từ đầu:

1. **Đặt breakpoint:** click vào lề trái cạnh số dòng — hiện một chấm đỏ. Chương trình sẽ **dừng lại** đúng tại dòng đó khi chạy tới, thay vì chạy xuyên suốt.
2. **Chạy ở chế độ Debug:** bấm nút  (Debug) thay vì **(Run)**. Chương trình chạy tới breakpoint thì dừng, một panel hiện ra cho thấy giá trị **hiện tại** của mọi biến.
3. **Step Over (F8) / Step Into (F7):** chạy tiếp từng dòng một, quan sát giá trị biến thay đổi ra sao sau mỗi dòng. Step Into đi vào bên trong một phương thức được gọi; Step Over chạy qua phương thức đó luôn mà không đi vào chi tiết bên trong.

Thử ngay với code mẫu: mở `Main.java` trong `code/ch04-multi-file-project/`, đặt breakpoint ở dòng gọi `MathUtils.square(4)`, chạy Debug, dùng Step Into để nhảy vào bên trong `MathUtils.square` và quan sát giá trị tham số `n`.

Vì sao cần Maven/Gradle?

Với chương trình 1-2 file như hiện tại, gõ `javac *.java` là đủ. Nhưng một dự án thực tế thường:

- Có hàng trăm file `.java`, tổ chức theo nhiều thư mục con.
- Phụ thuộc vào các **thư viện bên ngoài** (ví dụ thư viện đọc JSON sẽ dùng ở Chương 35) — bạn không tự viết những thư viện đó, mà cần tải về đúng phiên bản và cho trình biên dịch biết chỗ tìm chúng.

- Cần các bước lặp lại: biên dịch, chạy test, đóng gói thành file phân phối (`.jar`).

Maven và **Gradle** là hai công cụ phổ biến nhất giải quyết đúng ba vấn đề đó: khai báo thư viện cần dùng trong một file cấu hình (`pom.xml` với Maven, `build.gradle` với Gradle), công cụ tự tải về và biên dịch đúng thứ tự phụ thuộc. IntelliJ IDEA có hỗ trợ tạo project Maven/Gradle sẵn trong màn hình New Project ở bước trên — nhưng sách này **cố tình hoãn** việc dùng chúng tới Chương 42, để bạn tập trung học ngôn ngữ Java trước, tránh bị công cụ làm rối ở giai đoạn nhập môn. Toàn bộ code mẫu từ Chương 3 đến hết Phần 8 chạy được chỉ bằng `javac / java` thuần hoặc nút Run của IDE, không cần Maven/Gradle.

Bài tập

1. Tạo project IntelliJ mới, copy 2 file trong `code/ch04-multi-file-project/` vào, chạy bằng nút Run và xác nhận kết quả khớp với phần "Kết quả mong đợi" trong README của code mẫu.
2. Đặt breakpoint bên trong `MathUtils.isEven`, chạy Debug, dùng Step Over để xem giá trị `n % 2` được tính ra sao trước khi hàm trả về.
3. Thử xoá một dấu ngoặc `{` trong `Main.java` rồi bấm Run — quan sát cách IntelliJ báo lỗi *ngay khi gõ* (gạch đỏ dưới code) khác với việc phải đợi `javac` báo lỗi sau khi biên dịch.

Lỗi thường gặp

- **IntelliJ không thấy JDK nào để chọn khi tạo project** — JDK chưa cài đúng cách ở Chương 2, hoặc IntelliJ chưa quét lại. Dùng nút **Add JDK...** và trỏ thủ công tới thư mục cài JDK.
- **Bấm Run nhưng không thấy nút ▶ cạnh `main`** — thường do chữ ký `main` sai (xem lại Chương 3), IntelliJ không nhận diện được đây là điểm chạy chương trình.
- **Đặt breakpoint nhưng chương trình chạy xuyên qua không dừng** — bạn đang bấm **Run (▶)** chứ không phải **Debug (🐞)**. Chỉ chế độ Debug mới dừng lại ở breakpoint.
- **Copy code mẫu vào IntelliJ nhưng báo "class MathUtils is public, should be declared in a file named MathUtils.java"** — nhắc lại quy tắc ở Chương 3: tên file phải khớp tên class public, kể cả khi dùng IDE.

Chương 5 — Biến, kiểu dữ liệu nguyên thủy, ép kiểu

Mục tiêu học

Sau chương này, bạn sẽ:

- Khai báo và dùng được biến với 8 kiểu dữ liệu nguyên thủy (primitive type) của Java.
- Hiểu sự khác nhau giữa kiểu nguyên thủy và kiểu tham chiếu (sẽ đào sâu ở Chương 13, ở đây chỉ cần biết chúng khác nhau).
- Khai báo hằng số bằng `final`, biết vì sao nên dùng hằng số thay vì "số ma thuật" rải trong code.
- Hiểu ép kiểu tự động (widening) và ép kiểu thủ công (narrowing), và rủi ro mất dữ liệu khi ép kiểu.
- Đọc dữ liệu người dùng nhập từ bàn phím bằng `Scanner`.

Code mẫu đầy đủ: [code/ch05-bien-kieu-du-lieu/](#).

Biến là gì?

Biến (variable) là một **vùng nhớ có tên**, dùng để lưu một giá trị và có thể thay đổi giá trị đó theo thời gian chạy chương trình. Trong Java, khai báo biến luôn cần nói rõ **kiểu dữ liệu** (Java là ngôn ngữ **kiểu tĩnh** — statically typed: kiểu của biến được xác định lúc biên dịch, và không đổi kiểu trong suốt vòng đời biến đó).

```
int tuoi = 25;
```

Cú pháp: <kiểu dữ liệu> <tên biến> = <giá trị>;. `int` là kiểu, `tui` là tên biến, `25` là giá trị khởi tạo.

Quy ước đặt tên biến trong Java: dùng `camelCase` (chữ đầu thường, các từ ghép sau viết hoa chữ đầu) — ví dụ `soHocSinh`, `diemTrungBinh`. Đây là quy ước cộng đồng Java tuân theo nghiêm ngặt, không tuân theo sẽ khiến code bạn "trông lạ" với người khác đọc.

8 kiểu dữ liệu nguyên thủy

Java có đúng 8 kiểu nguyên thủy (primitive type) — đây là các kiểu **không phải object**, lưu trực tiếp giá trị (khác với kiểu tham chiếu như `String`, sẽ học kỹ lý do khác biệt ở Chương 13):

Kiểu	Kích thước	Phạm vi giá trị	Ví dụ
byte	8 bit	-128 đến 127	byte b = 100;
short	16 bit	-32,768 đến 32,767	short s = 1200;
int	32 bit	khoảng ±2.1 tỷ	int i = 98_000_000;
long	64 bit	khoảng ±9.2 tỷ tỷ	long l = 7_800_000_000L;
float	32 bit	số thực, độ chính xác đơn	float f = 8.5f;
double	64 bit	số thực, độ chính xác kép	double d = 3.14159;
char	16 bit	một ký tự Unicode	char c = 'J';
boolean	1 bit (logic)	true hoặc false	boolean done = false;

Vài lưu ý quan trọng:

- **Mặc định dùng `int` cho số nguyên, `double` cho số thực** — trừ khi có lý do cụ thể (tiết kiệm bộ nhớ với mảng cực lớn, hay số vượt phạm vi `int`). Người mới không cần cân nhắc `byte` / `short` trong phần lớn trường hợp.
- Số nguyên literal vượt phạm vi `int` **bắt buộc** có hậu tố `L` (ví dụ `7_800_000_000L`) — thiếu `L` trình biên dịch báo lỗi vì mặc định hiểu literal đó là `int`.

- Số thực literal mặc định là `double` ; muốn gán cho `float` phải thêm hậu tố `f` (ví dụ `8.5f`) — vì gán trực tiếp `double` vào biến `float` là ép kiểu thu hẹp (narrowing), không hợp lệ nếu không ép kiểu rõ ràng.
- Dùng dấu gạch dưới `_` để phân tách số cho dễ đọc: `98_000_000` — trình biên dịch bỏ qua dấu `_`, hoàn toàn tương đương `98000000`, chỉ để mắt người đọc dễ hơn.
- `char` dùng nhảy đơn `'j'`, khác với chuỗi ký tự dùng nhảy kép `"j"` (chuỗi sẽ học ở Chương 11).

Hằng số với `final`

```
final double LAI_SUAT_CO_DINH = 0.05;
```

Từ khoá `final` khiến biến **chỉ gán được giá trị một lần** — gán lại lần thứ hai là lỗi biên dịch. Quy ước đặt tên hằng số: viết hoa toàn bộ, cách nhau bằng `_` (ví dụ `MAX_SO_LUONG`, `LAI_SUAT_CO_DINH`). Dùng hằng số thay vì viết thẳng con số ("magic number") rải rác trong code giúp code dễ đọc hơn (đọc `LAI_SUAT_CO_DINH` hiểu ngay ý nghĩa, đọc `0.05` một mình thì không) và dễ sửa hơn (đổi giá trị một chỗ duy nhất thay vì tìm sửa khắp file).

Ép kiểu (type casting)

Ép kiểu tự động (widening) — an toàn

Khi gán giá trị từ kiểu **nhỏ hơn** sang kiểu **lớn hơn**, Java tự động chuyển đổi, không cần cú pháp gì thêm, vì không có nguy cơ mất dữ liệu:

```
int soNguyen = 10;
double soThuc = soNguyen; // int -> double, tu dong, an toan
```

Thứ tự từ nhỏ đến lớn (mỗi kiểu widening được sang mọi kiểu bên phải nó): `byte` → `short` → `int` → `long` → `float` → `double`.

Ép kiểu thủ công (narrowing) — có thể mất dữ liệu

Chiều ngược lại (kiểu lớn sang kiểu nhỏ) **không** tự động — bạn phải viết rõ ràng bằng cú pháp (kiểu đích), để xác nhận bạn hiểu và chấp nhận rủi ro mất dữ liệu:

```
double gia = 19.99;
int giaLamTron = (int) gia; // ket qua la 19 - CAT phan thap phan, khong lam tron
```

Hiểu lầm rất phổ biến: `(int) gia` **không làm tròn** — nó **cắt bỏ hoàn toàn** phần thập phân. `(int) 19.99` cho ra `19`, không phải `20`. Muốn làm tròn đúng nghĩa, dùng `Math.round(gia)` (sẽ gặp lại khi học thư viện `Math` chi tiết hơn ở các chương sau).

Ép kiểu số quá lớn vào kiểu quá nhỏ còn gây ra **tràn số (overflow)** — không chỉ mất phần thập phân mà giá trị có thể sai lệch hoàn toàn:

```
int soLon = 130;
byte soBiTran = (byte) soLon; // KHONG ra 130 - byte chỉ chưa được tới 127
```

Chạy thử trong code mẫu để thấy `soBiTran` thực sự in ra giá trị âm — vì `byte` "vòng lại" khi vượt quá phạm vi biểu diễn của nó (cơ chế này gọi là *integer overflow*, liên quan cách máy tính biểu diễn số nguyên trong bộ nhớ, không nằm trong phạm vi chương này).

Đọc dữ liệu người dùng nhập vào với `Scanner`

Để chương trình tương tác được (thay vì chỉ in ra giá trị cố định), dùng lớp `Scanner`:

```
import java.util.Scanner;
```

```

public class TemperatureConverter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Nhập nhiệt độ theo độ C: ");
        double celsius = scanner.nextDouble();

        double fahrenheit = celsius * 9 / 5 + 32;
        System.out.println(celsius + " độ C = " + fahrenheit + " độ F");

        scanner.close();
    }
}

```

- `import java.util.Scanner;` — nạp lớp `Scanner` từ thư viện chuẩn để dùng được trong file này (khái niệm `import /package` sẽ học kỹ ở Chương 15).
- `new Scanner(System.in)` — tạo một `Scanner` đọc từ bàn phím (`System.in` là "cổng vào" chuẩn, đối lập với `System.out` đã dùng ở Chương 3).
- `scanner.nextDouble()` — đợi người dùng gõ một số thực rồi nhấn Enter, trả về giá trị đó dưới dạng `double`. Có các phương thức tương tự: `nextInt()`, `nextLine()` (đọc cả dòng, kể cả có dấu cách — sẽ dùng nhiều ở Chương 11 khi làm việc với `String`).
- `scanner.close()` — đóng scanner khi dùng xong, giải phóng tài nguyên (thói quen tốt, dù chương trình nhỏ ở đây không ảnh hưởng nhiều nếu quên).

Bài tập

1. Viết chương trình khai báo biến `double luongThang` và biến `final int SO_THANG_MOT_NAM = 12`, tính và in ra tổng thu nhập một năm.
2. Sửa `TemperatureConverter.java` thành chương trình đổi ngược: nhập độ F, in ra độ C (`celsius = (fahrenheit - 32) * 5 / 9`).
3. Giải thích bằng lời (không chạy thử trước): `(int) 7.9` cho ra kết quả gì? `(int) -7.9` cho ra kết quả gì? Sau đó chạy thử để kiểm tra dự đoán.
4. Thử gán `float f = 3.14;` (không có hậu tố `f` ở cuối số) — đọc lỗi biên dịch, giải thích vì sao lỗi xảy ra dựa trên phần "ép kiểu thủ công" ở trên.

Lỗi thường gặp

- **`incompatible types: possible lossy conversion from double to int`** — bạn đang gán trực tiếp giá trị `double / float` cho biến `int` mà không ép kiểu. Thêm `(int)` phía trước nếu chấp nhận mất phần thập phân, hoặc đổi kiểu biến sang `double`.
- **`error: incompatible types: possible lossy conversion from double to float`** khi viết `float f = 3.14;` ****** — literal số thực mặc định là `double`; thêm hậu tố `f`: `float f = 3.14f;`.
- **Ép kiểu `(int)` xong tưởng đã làm tròn** — nhắc lại: `(int)` cắt bỏ phần thập phân, không làm tròn. Dùng `Math.round(...)` nếu cần làm tròn đúng nghĩa.
- **Gán lại giá trị cho biến `final`** — báo lỗi `cannot assign a value to final variable`. Đây là hành vi đúng theo thiết kế, không phải bug: `final` tồn tại chính để ngăn việc này.
- **`InputMismatchException` khi chạy `Scanner`** — người dùng nhập chữ trong khi chương trình gọi `nextDouble()` / `nextInt()`. Đây là một dạng lỗi runtime (exception) — cách xử lý đúng cách sẽ học ở Chương 21.

Chương 6 — Toán tử và biểu thức

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng đúng các nhóm toán tử: số học, so sánh, logic, tăng/giảm.
- Phân biệt phép chia số nguyên và chia số thực — lỗi rất phổ biến với người mới.
- Hiểu khác biệt giữa `x++` và `++x`.
- Hiểu cơ chế **short-circuit evaluation** của `&&` và `||`, và vì sao nó quan trọng không chỉ về hiệu năng mà cả về tránh lỗi runtime.
- Đọc đúng độ ưu tiên toán tử (operator precedence) trong biểu thức phức tạp.

Code mẫu đầy đủ: <code/ch06-toan-tu-bieu-thuc/>.

Toán tử số học

`+` `-` `*` `/` `%` — cộng, trừ, nhân, chia, chia lấy dư. Cẩn thận với phép chia:

```
int a = 17, b = 5;
System.out.println(a / b);    // 3 - chia SỐ NGUYÊN, cat phần dư, KHÔNG làm tròn
System.out.println(a % b);    // 2 - phần dư của phép chia (17 = 5*3 + 2)
System.out.println(17.0 / 5); // 3.4 - chia SỐ THỰC vì có một toán hạng là double
```

Quy tắc quan trọng nhất phần này: nếu **cả hai toán hạng** của phép `/` đều là kiểu số nguyên (`int`, `long` ...), kết quả là số nguyên, phần dư bị cắt bỏ hoàn toàn — dù bạn có gán kết quả cho biến `double` thì phép chia vẫn đã bị cắt **trước khi** gán:

```
double ketQua = 17 / 5; // ket qua là 3.0, KHÔNG phải 3.4 - phép chia đã xảy ra trên số nguyên trước
double ketQuaDung = 17.0 / 5; // 3.4 - dung vì ép một toán hạng thành double trước khi chia
```

Muốn chia ra kết quả thực, phải đảm bảo **ít nhất một** toán hạng là `double` / `float` (bằng cách viết literal có dấu chấm thập phân, hoặc ép kiểu một biến sang `double`).

Toán tử tăng/giảm: `++` và `--`

```
int x = 5;
int y = x++; // y nhận giá trị CU của x (5), SAU ĐÓ x mới tăng lên 6
// x = 6, y = 5

int m = 5;
int n = ++m; // m tăng lên 6 TRƯỚC, n nhận giá trị MỚI (6)
// m = 6, n = 6
```

- `x++` (post-increment): dùng giá trị hiện tại trước, rồi mới tăng.
- `++x` (pre-increment): tăng trước, rồi mới dùng giá trị mới.

Khi đứng một mình trên một dòng riêng (`dem++`;), hai cách viết cho kết quả giống hệt nhau. Khác biệt chỉ lộ ra khi bạn dùng **kết quả** của biểu thức tăng/giảm đó ngay trong cùng dòng (như ví dụ trên) — tình huống này dễ gây nhầm lẫn, nên nhiều lập trình viên tránh viết `x++ / ++x` lồng trong biểu thức phức tạp, tách thành hai dòng riêng cho rõ ràng.

Toán tử so sánh

`>` `<` `>=` `<=` `==` `!=` — trả về `boolean` (`true` / `false`). Lưu ý `==` so sánh **giá trị** với kiểu nguyên thủy (`int`, `double` ...), nhưng với kiểu tham chiếu (như `String`) `==` lại so sánh khác — đây là một trong những lỗi kinh điển nhất của người mới học Java, sẽ

giải thích kỹ và cách so sánh đúng ở Chương 11 (String) và Chương 20 (equals).

Toán tử logic

`&&` (AND — cả hai đều đúng), `||` (OR — ít nhất một đúng), `!` (NOT — phủ định):

```
boolean coVe = true;
boolean duTuoi = false;
coVe && duTuoi    // false - can CA HAI dieu kien
coVe || duTuoi    // true  - can IT NHAT MOT dieu kien
!coVe             // false - phu dinh cua true
```

Short-circuit evaluation — cơ chế "tắt sớm"

`&&` và `||` trong Java **không luôn đánh giá cả hai vế**:

- `&&` : nếu vế trái đã `false` , Java **không đánh giá vế phải nữa** — vì kết quả chắc chắn `false` rồi, không cần kiểm tra thêm.
- `||` : nếu vế trái đã `true` , Java **không đánh giá vế phải** — kết quả chắc chắn `true` rồi.

Đây không chỉ là tối ưu hiệu năng — nó thường được dùng **cố ý** để tránh lỗi runtime:

```
int[] mang = {};
if (mang.length > 0 && mang[0] == 1) {
    // ...
}
```

Nếu `mang` rỗng, `mang.length > 0` là `false` , Java dừng lại ngay, **không bao giờ chạy tới** `mang[0]` — nếu chạy tới, truy cập phần tử thứ 0 của mảng rỗng sẽ ném lỗi (Chương 9 sẽ học về mảng, Chương 21 học cách xử lý những lỗi kiểu này). Đây là mẫu (pattern) rất phổ biến: "kiểm tra điều kiện an toàn trước, rồi mới kiểm tra điều kiện phụ thuộc vào nó" — luôn đặt điều kiện an toàn ở **vế trái** của `&&` .

Độ ưu tiên toán tử (operator precedence)

Giống toán học phổ thông, Java có thứ tự ưu tiên: `*` , `/` , `%` được tính **trước** `+` , `-` ; các toán tử so sánh được tính trước toán tử logic; ngoặc đơn `()` luôn được tính trước tiên và có thể dùng để ép thứ tự bạn muốn:

```
int ketQua = 2 + 3 * 4;           // 14, không phải 20 - * tính trước +
int ketQuaCoNgoac = (2 + 3) * 4; // 20 - ngoac don thay doi thu tu tinh
```

Lời khuyên thực tế: đừng cố nhớ hết bảng độ ưu tiên đầy đủ (có hàng chục toán tử với các mức ưu tiên khác nhau). Khi biểu thức có từ 2 toán tử khác nhóm trở lên, **cứ thêm ngoặc đơn** cho rõ ràng — code rõ ràng quan trọng hơn code ngắn gọn nhưng khó đọc.

Bài tập

1. Không chạy code, dự đoán kết quả của `10 % 3` , `-10 % 3` , `10 % -3` , sau đó chạy thử để kiểm tra dự đoán (dấu của phép `%` trong Java theo dấu của số bị chia — số đứng trước).
2. Viết biểu thức tính trung bình cộng của 3 số nguyên `a` , `b` , `c` , đảm bảo kết quả là số thực chính xác (không bị cắt do chia số nguyên).
3. Giải thích: đoạn code `if (danhSach != null && danhSach.size() > 0)` dựa vào cơ chế nào trong chương này để không bị lỗi khi `danhSach` là `null` ? (Sẽ hiểu đầy đủ khái niệm `null` ở Chương 13, ở đây chỉ cần suy luận dựa trên short-circuit evaluation vừa học.)

Lỗi thường gặp

- **Chia hai số nguyên ra kết quả sai (bị cắt) dù gán cho biến `double`** — phép chia số nguyên xảy ra **trước** khi gán; xem lại phần "Toán tử số học" ở trên, ép ít nhất một toán hạng sang `double` trước khi chia.
- **Nhầm `=` (gán) với `==` (so sánh)** trong điều kiện `if` — ví dụ viết `if (x = 5)` thay vì `if (x == 5)`. Với kiểu `boolean`, Java **không cho phép** biên dịch lỗi này (khác với C/C++), vì `x = 5` trả về `int` chứ không phải `boolean` — nhưng nếu biến là `boolean`, lỗi này biên dịch được và rất khó phát hiện, cần cẩn thận đọc lại code.
- **Dùng `==` để so sánh `String`** — sẽ giải thích kỹ ở Chương 11, ghi nhớ trước: dùng `.equals()`, không dùng `==`, cho `String`.
- **Viết biểu thức phức tạp không có ngoặc, đọc lại không nhớ thứ tự tính** — thêm ngoặc đơn, đừng ngại code dài hơn vài ký tự.

Chương 7 — Cấu trúc điều khiển: if/else, switch

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết được rẽ nhánh với `if / else if / else`, hiểu rõ thứ tự kiểm tra điều kiện.
- Dùng toán tử ba ngôi (ternary `? :`) thay `if/else` khi chỉ cần gán một giá trị.
- Viết được `switch` statement kiểu cũ (kèm `break`) và `switch` expression kiểu mới (Java 14+), hiểu vì sao kiểu mới an toàn hơn.
- Hiểu hiện tượng "fall-through" trong `switch` cũ — khi nào nó là lỗi, khi nào nó được dùng có chủ đích.

Code mẫu đầy đủ: [code/ch07-cau-truc-dieu-khien/](#).

if / else if / else

```
if (diem >= 9.0) {
    xepLoai = "Xuat sac";
} else if (diem >= 8.0) {
    xepLoai = "Gioi";
} else if (diem >= 6.5) {
    xepLoai = "Kha";
} else {
    xepLoai = "Trung binh hoac Yeu";
}
```

Java kiểm tra các điều kiện **theo thứ tự từ trên xuống**, chạy vào nhánh đầu tiên có điều kiện đúng rồi **bỏ qua mọi nhánh còn lại** — kể cả khi điều kiện của những nhánh sau đó cũng đúng. Đây là lý do thứ tự các điều kiện trong chuỗi `if/else if` quan trọng: nếu bạn viết điều kiện `diem >= 6.5` **trước** `diem >= 9.0`, một học sinh 9.5 điểm sẽ bị rơi vào nhánh "Kha" vì `9.5 >= 6.5` đúng và được kiểm tra trước — đây là lỗi logic rất phổ biến, luôn sắp điều kiện từ **chật nhất đến lỏng nhất** khi các điều kiện có thể cùng đúng.

Dấu ngoặc nhọn `{ }` không bắt buộc nếu thân nhánh chỉ có một câu lệnh, nhưng **luôn nên dùng** — thiếu ngoặc nhọn là nguồn lỗi kinh điển khi bạn thêm dòng thứ hai vào thân nhánh mà quên thêm ngoặc, dòng mới sẽ chạy **vô điều kiện**, không còn thuộc về nhánh `if` nữa.

Toán tử ba ngôi (ternary `? :`)

```
String ketQua = (diem >= 5.0) ? "Dat" : "Khong dat";
```

Đọc là: nếu điều kiện đúng, lấy giá trị trước dấu `:`; nếu sai, lấy giá trị sau dấu `:`. Đây là cách viết gọn của:

```
String ketQua;
if (diem >= 5.0) {
    ketQua = "Dat";
} else {
    ketQua = "Khong dat";
}
```

Chỉ nên dùng ternary khi biểu thức **đơn giản, một dòng**. Lồng nhiều ternary vào nhau (`a ? b : c ? d : e`) làm code khó đọc — trong trường hợp đó, dùng `if/else` hoặc `switch` tường minh hơn.

switch statement (kiểu truyền thống)

```
switch (thu) {
    case 1:
        System.out.println("Thu Hai");
        break;
    case 2:
        System.out.println("Thu Ba");
        break;
    default:
        System.out.println("Ngày không hợp lệ");
}
```

`switch` so khớp giá trị của biến (`thu`) với từng `case` . **`break` bắt buộc phải có** ở cuối mỗi nhánh — nếu thiếu, chương trình **tiếp tục chạy xuống nhánh `case` tiếp theo** dù điều kiện của nhánh đó không khớp, hiện tượng gọi là **fall-through**:

```
switch (thu) {
    case 1:
        System.out.println("Thu Hai");
        // QUÊN break!
    case 2:
        System.out.println("Thu Ba");
        break;
}
```

Với `thu = 1` , đoạn trên in ra **cả hai dòng** "Thu Hai" và "Thu Ba" — vì thiếu `break` , luồng chạy rơi tiếp xuống `case 2` . Đây là một trong những lỗi phổ biến và khó phát hiện nhất với người mới dùng `switch` kiểu cũ (chương trình vẫn biên dịch được bình thường, không có cảnh báo).

Fall-through đôi khi được dùng **có chủ đích**, để gộp nhiều `case` xử lý chung một khối lệnh (xem `DayOfWeekSwitch.java` , phần tính số ngày trong tháng — `case 1: case 3: case 5: ...` gộp các tháng có 31 ngày dùng chung một `break` ở cuối).

switch expression (Java 14+) — cách viết hiện đại, an toàn hơn

```
String tenThu = switch (thu) {
    case 1 -> "Thu Hai";
    case 2 -> "Thu Ba";
    case 3 -> "Thu Tu";
    default -> "Ngày không hợp lệ";
};
```

Khác biệt cốt lõi so với `switch` statement:

- Dùng mũi tên `->` thay vì dấu `:` , **không cần `break`** — mỗi nhánh chỉ chạy đúng một biểu thức rồi kết thúc, **không có fall-through**. Đây là lý do nên ưu tiên dùng `switch` expression khi có thể: loại bỏ hẳn một nhóm lỗi mà `switch` statement cũ có thể gặp phải.
- `switch` expression **trả về giá trị trực tiếp** (có thể gán thẳng cho biến như trên), khác với `switch` statement chỉ thực thi các câu lệnh, không trả giá trị.
- Trình biên dịch bắt buộc bạn xử lý **đầy đủ mọi trường hợp** (yêu cầu có `default` , trừ khi `switch` trên kiểu `enum` liệt kê đủ mọi giá trị có thể — sẽ gặp lại khi học `enum` ở Chương 19).

Lời khuyên: với code mới (Java 14 trở lên), ưu tiên `switch` expression. Sách vẫn dạy `switch` statement kiểu cũ vì bạn sẽ gặp rất nhiều code cũ ngoài thực tế dùng cú pháp này.

Bài tập

- Viết chương trình dùng `if/else if/else` phân loại BMI (chỉ số khối cơ thể) thành "Gầy", "Bình thường", "Thừa cân", "Béo phì" dựa trên các mốc chuẩn (tự tra cứu mốc BMI).

2. Cố tình xoá một dòng `break` trong `DayOfWeekSwitch.java` (không phải dòng trong nhóm gộp case có chủ đích), chạy lại và quan sát hiện tượng fall-through xảy ra thế nào.
3. Viết lại toàn bộ `switch` statement trong bài tập 2 dưới dạng `switch` expression, xác nhận không còn khả năng xảy ra lỗi fall-through nữa dù bạn cố tình bỏ sót.

Lỗi thường gặp

- **Quên `break` trong `switch` statement, chạy sai kết quả (fall-through ngoài ý muốn)** — lỗi không được trình biên dịch cảnh báo, chỉ phát hiện được khi chạy và thấy kết quả in ra nhiều hơn mong đợi. Cách phòng ngừa tốt nhất: ưu tiên `switch` expression cho code mới.
- **Sắp thứ tự điều kiện `if/else if` sai (điều kiện lỏng đứng trước điều kiện chặt)** — dẫn tới rơi vào nhánh sai dù về mặt cú pháp code hoàn toàn hợp lệ. Xem lại phần "if/else if/else" ở trên.
- **Thiếu `{ }` khi thân nhánh có từ 2 dòng trở lên** — chỉ dòng ngay sau `if` thuộc về nhánh đó, các dòng sau chạy vô điều kiện. Luôn dùng `{ }` kể cả khi thân nhánh chỉ có một dòng, để tránh lỗi này khi code được sửa thêm sau này.
- **`switch` trên kiểu không hỗ trợ** (ví dụ `double`) — `switch` (cả hai kiểu) chỉ hỗ trợ `byte` / `short` / `char` / `int` (và kiểu wrapper tương ứng), `String`, và `enum` — không hỗ trợ `double` / `float` / `boolean` / `long`.

Chương 8 — Vòng lặp: for, while, do-while

Mục tiêu học

Sau chương này, bạn sẽ:

- Chọn đúng loại vòng lặp (`for` , `while` , `do-while`) tùy tình huống.
- Dùng `break` và `continue` đúng cách, hiểu khác biệt giữa hai lệnh này.
- Viết được vòng lặp lồng nhau, và dùng nhãn (label) khi cần thoát nhiều tầng vòng lặp cùng lúc.
- Tránh được lỗi vòng lặp vô hạn ngoài ý muốn.

Code mẫu đầy đủ: <code/ch08-vong-lap/>.

for — khi biết trước (hoặc tính được) số lần lặp

```
for (int i = 1; i <= 5; i++) {  
    System.out.println("i = " + i);  
}
```

Ba phần trong ngoặc `for` , phân cách bởi `;` :

1. **Khởi tạo** (`int i = 1`) — chạy **đúng một lần** trước khi vòng lặp bắt đầu.
2. **Điều kiện** (`i <= 5`) — kiểm tra **trước mỗi lần lặp**; còn đúng thì tiếp tục, sai thì dừng.
3. **Cập nhật** (`i++`) — chạy **sau mỗi lần lặp**, trước khi kiểm tra điều kiện lần tiếp theo.

Biến `i` khai báo trong phần khởi tạo chỉ tồn tại **bên trong** vòng lặp (phạm vi biến — scope — đã nhắc sơ ở Chương 6, sẽ nói kỹ hơn ở Chương 10).

while — khi chưa biết trước số lần lặp

```
double giaTri = 100;  
while (giaTri >= 1) {  
    System.out.println("giaTri = " + giaTri);  
    giaTri = giaTri / 2;  
}
```

`while` kiểm tra điều kiện **trước** mỗi lần chạy thân vòng lặp — nếu điều kiện sai ngay từ đầu, thân vòng lặp **không chạy lần nào cả**. Dùng `while` khi số lần lặp phụ thuộc vào một điều kiện thay đổi dần trong quá trình chạy (ở ví dụ trên: chia đôi giá trị tới khi nhỏ hơn 1 — không biết trước chính xác sẽ lặp bao nhiêu lần nếu giá trị ban đầu thay đổi).

do-while — luôn chạy ít nhất một lần

```
int soLanThu = 0;  
do {  
    soLanThu++;  
    System.out.println("Lan thu " + soLanThu);  
} while (soLanThu < 3);
```

Khác biệt duy nhất nhưng quan trọng so với `while` : `do-while` kiểm tra điều kiện **sau** khi chạy thân vòng lặp, nên thân vòng lặp **luôn chạy ít nhất một lần**, dù điều kiện sai ngay từ đầu. Dùng khi bài toán tự nhiên đòi hỏi "làm trước, rồi mới kiểm tra có nên làm tiếp không" — ví dụ điển hình: hiện menu và đọc lựa chọn người dùng ít nhất một lần trước khi hỏi có muốn tiếp tục không.

break — thoát khỏi vòng lặp ngay lập tức

```
for (int i = 1; i <= 100; i++) {
    if (i % 7 == 0) {
        System.out.println("Tim thay: " + i);
        break; // thoát vòng lặp NGAY, không chạy tiếp các giá trị i còn lại
    }
}
```

`break` dừng vòng lặp hoàn toàn, chương trình tiếp tục chạy từ dòng code **ngay sau** vòng lặp.

continue — bỏ qua phần còn lại của lần lặp hiện tại

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // bỏ qua phần còn lại CỦA LẦN LẶP NÀY, nhảy sang i tiếp theo
    }
    System.out.println("Số lẻ: " + i);
}
```

Khác với `break`, `continue` **không dừng cả vòng lặp** — nó chỉ bỏ qua phần code còn lại trong thân vòng lặp **ở lần lặp hiện tại**, rồi tiếp tục sang lần lặp kế tiếp như bình thường (với `for`, phần cập nhật `i++` vẫn chạy trước khi kiểm tra điều kiện tiếp theo).

Vòng lặp lồng nhau (nested loop)

```
for (int a = 1; a <= 3; a++) {
    for (int b = 1; b <= 3; b++) {
        System.out.println(a + " x " + b + " = " + (a * b));
    }
}
```

Vòng lặp trong (`b`) chạy **trọn vẹn từ đầu đến cuối** ứng với **mỗi** lần lặp của vòng lặp ngoài (`a`). Với ví dụ trên: `a = 1` → vòng `b` chạy đủ 1,2,3; rồi `a = 2` → vòng `b` lại chạy đủ 1,2,3; tương tự cho `a = 3`. Tổng số lần thân vòng lặp trong chạy là `3 × 3 = 9` lần.

Nhãn (label) — thoát nhiều tầng vòng lặp cùng lúc

`break` thông thường chỉ thoát vòng lặp **gần nhất** chứa nó. Muốn thoát cả vòng lặp ngoài từ bên trong vòng lặp trong, đặt **nhãn** trước vòng lặp ngoài:

```
outerLoop:
for (int a = 1; a <= 3; a++) {
    for (int b = 1; b <= 3; b++) {
        if (a * b == 6) {
            System.out.println("Dùng tại a=" + a + ", b=" + b);
            break outerLoop; // thoát CẢ HAI vòng lặp, không chỉ vòng trong
        }
    }
}
```

`continue` nhãn cũng hoạt động tương tự — bỏ qua lần lặp hiện tại của vòng lặp **có nhãn đó**, chứ không phải vòng lặp gần nhất. Nhãn không dùng thường xuyên trong code thực tế (thường có cách viết khác rõ ràng hơn, ví dụ tách thành phương thức riêng — sẽ học ở Chương 10), nhưng cần biết để đọc hiểu khi gặp trong code người khác.

Tránh vòng lặp vô hạn ngoài ý muốn

```
int n = 1;
while (true) {
    if (n > 3) {
        break;
    }
    System.out.println("n = " + n);
    n++;
}
```

`while (true)` tạo vòng lặp chạy mãi mãi — chỉ dừng được nhờ `break` bên trong. Đây là mẫu hợp lệ và hữu ích (ví dụ vòng lặp chính của một chương trình chờ nhập lệnh liên tục), nhưng **lỗi phổ biến nhất của người mới** là quên viết điều kiện dừng (`break` , hoặc quên cập nhật biến điều kiện trong `while` / `for`), khiến chương trình treo và phải tự tắt (Ctrl+C trong terminal, hoặc nút Stop trong IDE).

Bài tập

1. Chạy `FizzBuzz.java` , sau đó tự viết lại từ đầu **không nhìn code mẫu** — đây là bài tập kinh điển để luyện phản xạ với `for` + `if/else if/else` .
2. Viết chương trình dùng `while` tính tổng các số nguyên từ 1 đến khi tổng vượt quá 1000 lần đầu tiên, in ra tổng đó và số cuối cùng đã cộng vào.
3. Viết vòng lặp lồng nhau in ra hình tam giác dấu `*` :

```
*
**
***
****
*****
```

4. Cố tình viết một vòng lặp `while` quên cập nhật biến điều kiện (gây vòng lặp vô hạn), quan sát chương trình treo, rồi dừng bằng Ctrl+C (terminal) hoặc nút Stop (IDE) — làm quen cảm giác này để nhận ra ngay khi gặp lại trong thực tế.

Lỗi thường gặp

- **Vòng lặp vô hạn do quên cập nhật biến điều kiện** — ví dụ quên `i++` trong `while (i <= 10)` . Luôn kiểm tra: biến trong điều kiện có chắc chắn thay đổi dần tới khi điều kiện sai không?
- **Lỗi "off-by-one"**: dùng `<` thay vì `<=` (hoặc ngược lại) khiến vòng lặp chạy thiếu/thừa đúng một lần. Đây là lỗi cực kỳ phổ biến — luôn kiểm tra kỹ giá trị biên (giá trị đầu và cuối) có được xử lý đúng như mong đợi không.
- **Nhầm `break` với `continue`** — `break` thoát hẳn vòng lặp, `continue` chỉ bỏ qua lần lặp hiện tại. Nhầm hai lệnh này cho ra kết quả sai nhưng chương trình vẫn chạy bình thường (không có lỗi biên dịch/runtime), rất dễ bỏ sót khi kiểm tra.
- **Sửa giá trị biến điều khiển vòng lặp (`i`) bên trong thân `for`** — về mặt cú pháp hợp lệ, nhưng làm logic vòng lặp khó đoán (giá trị `i` bị thay đổi bởi cả phần cập nhật của `for` lẫn code trong thân), nên tránh trừ khi thực sự cần thiết và ghi chú rõ lý do.

Chương 9 — Mảng một chiều và nhiều chiều

Mục tiêu học

Sau chương này, bạn sẽ:

- Khai báo, khởi tạo, đọc/ghi phần tử của mảng một chiều.
- Duyệt mảng bằng `for` có chỉ số và `for-each`, biết khi nào dùng cách nào.
- Hiểu mảng là **kiểu tham chiếu**, không phải kiểu nguyên thủy — hệ quả khi gán/truyền mảng.
- Làm việc với mảng hai chiều (ma trận) và mảng không đều (jagged array).
- Nhận diện và hiểu lỗi truy cập ngoài phạm vi (`ArrayIndexOutOfBoundsException`).

Code mẫu đầy đủ: [code/ch09-mang/](#).

Khai báo và khởi tạo mảng

```
int[] diem = new int[5]; // mảng 5 phần tử kiểu int, giá trị mặc định là 0
diem[0] = 8;
diem[1] = 7;
```

`new int[5]` tạo một mảng có **kích thước cố định 5 phần tử**, đánh chỉ số từ `0` đến `4` (chỉ số **luôn bắt đầu từ 0**, không phải `1` — đây là quy ước hầu hết ngôn ngữ lập trình dùng, khác thói quen đếm `1, 2, 3...` ngoài đời). Kích thước mảng **không đổi** được sau khi tạo — muốn "thêm phần tử" thực chất phải tạo mảng mới lớn hơn và sao chép dữ liệu (Chương 24 sẽ giới thiệu `ArrayList`, cấu trúc dữ liệu giải quyết đúng hạn chế này bằng cách tự động quản lý việc đó cho bạn).

Cách khởi tạo trực tiếp với giá trị có sẵn (kích thước tự suy ra từ số phần tử liệt kê):

```
String[] ten = {"An", "Binh", "Chi", "Dung", "Em"};
```

Mọi kiểu — kể cả kiểu tham chiếu như `String` — đều tạo mảng được theo cùng cú pháp.

Duyệt mảng

`for` có chỉ số — khi cần biết vị trí phần tử

```
for (int i = 0; i < diem.length; i++) {
    System.out.println(ten[i] + ": " + diem[i] + " diem");
}
```

`diem.length` (không có ngoặc đơn — khác với `String.length()` sẽ học ở Chương 11, đây là điểm dễ nhầm) trả về số phần tử của mảng. Dùng `for` có chỉ số khi bạn cần chỉ số để làm việc khác (ví dụ ở đây: dùng cùng chỉ số `i` để lấy tương ứng cả `ten[i]` lẫn `diem[i]` từ hai mảng song song).

`for-each` — khi chỉ cần giá trị, không cần chỉ số

```
int tong = 0;
for (int d : diem) {
    tong += d;
}
```

Đọc là "với mỗi `d` trong `diem`". Ngắn gọn và ít lỗi hơn (không có nguy cơ viết sai điều kiện biên như `<=` thay vì `<`), nhưng **không cho biết chỉ số hiện tại** và **không sửa được phần tử gốc trong mảng** qua biến `d` (`d` chỉ là bản sao giá trị tại mỗi lượt lặp, với mảng kiểu nguyên thủy). Dùng `for-each` khi bạn chỉ cần đọc qua từng giá trị, không cần chỉ số.

Mảng là kiểu tham chiếu

Đây là khái niệm quan trọng, sẽ đào sâu đầy đủ ở Chương 13 khi học về object — nhưng cần biết ngay từ bây giờ vì rất dễ gây lỗi khó hiểu:

```
int[] diemBanSao = diem;
diemBanSao[0] = 100;
System.out.println(diem[0]); // in ra 100, KHÔNG phải giá trị cũ!
```

`diemBanSao = diem` **không sao chép nội dung mảng** — nó chỉ tạo ra một biến khác **cùng trỏ tới mảng gốc trong bộ nhớ**. Sửa qua `diemBanSao` chính là sửa mảng gốc, vì cả hai biến trỏ tới cùng một chỗ. Đây khác hẳn với biến kiểu nguyên thủy (`int a = b`; thực sự sao chép giá trị).

Muốn sao chép **thật sự** nội dung mảng, dùng `Arrays.copyOf` :

```
import java.util.Arrays;

int[] diemSaoChepThat = Arrays.copyOf(diem, diem.length);
diemSaoChepThat[0] = -1;
// diem[0] không đổi, vì đây là mảng gốc hoàn toàn độc lập
```

In mảng đúng cách

```
System.out.println(diem); // in ra thu gì đó như [I@1b6d3586 - VÔ ICH
System.out.println(Arrays.toString(diem)); // in ra [8, 7, 9, 6, 10] - DÙNG Ý MUỐN
```

Mảng không override cách hiển thị dạng chuỗi mặc định (lý do kỹ thuật sẽ hiểu rõ ở Chương 20 khi học `toString()`), nên `println` trực tiếp trên mảng cho ra một chuỗi mã định danh vô nghĩa với người đọc. Luôn dùng `Arrays.toString(...)` (cho mảng một chiều) hoặc `Arrays.deepToString(...)` (cho mảng nhiều chiều) khi cần in mảng ra để debug.

Mảng hai chiều

```
int[][] banCo = new int[3][4]; // 3 hàng, 4 cột
banCo[1][2] = 99; // hàng 1, cột 2 (chỉ số đều bắt đầu từ 0)
```

Về bản chất, mảng hai chiều trong Java là **mảng của các mảng** — `banCo` là một mảng gồm 3 phần tử, mỗi phần tử lại là một mảng `int[4]` . Duyệt bằng vòng lặp lồng nhau:

```
for (int hang = 0; hang < banCo.length; hang++) {
    for (int cot = 0; cot < banCo[hang].length; cot++) {
        // xử lý banCo[hang][cot]
    }
}
```

Mảng không đều (jagged array)

Vì mảng 2 chiều thực chất là mảng của mảng, mỗi "hàng" có thể có **độ dài khác nhau**:

```
int[][] mangKhongDeu = new int[3][];
mangKhongDeu[0] = new int[]{1};
mangKhongDeu[1] = new int[]{1, 2};
mangKhongDeu[2] = new int[]{1, 2, 3};
```

Khai báo `new int[3][]` tạo mảng ngoài có 3 phần tử, nhưng **chưa** tạo các mảng con bên trong (mỗi phần tử tạm thời là `null` — khái niệm `null` sẽ học kỹ ở Chương 13) — bạn tự gán từng mảng con với độ dài tùy ý. Đây là cấu trúc hữu ích khi dữ liệu tự nhiên không đều (ví dụ: số ngày trong từng tháng của một năm).

Bài tập

- Viết chương trình tạo mảng 10 số nguyên ngẫu nhiên (`int`) (`Math.random() * 100`), in ra mảng, tìm và in giá trị lớn nhất, nhỏ nhất bằng vòng lặp.
- Viết hàm (tạm thời viết trực tiếp trong `main` , học cách tách phương thức riêng ở Chương 10) đảo ngược thứ tự phần tử của một mảng `int[]` tại chỗ (không tạo mảng mới).
- Tạo mảng hai chiều 3x3 đại diện một bàn cờ caro, gán giá trị `1` cho các ô trên đường chéo chính, in ra ma trận kết quả.
- Bỏ comment dòng `diem[10]` trong `ArrayBasicsDemo.java` , chạy lại, đọc kỹ thông báo lỗi `ArrayIndexOutOfBoundsException` — nó cho biết chỉ số nào bị truy cập sai.

Lỗi thường gặp

- `ArrayIndexOutOfBoundsException` — truy cập chỉ số `< 0` hoặc `>= length` . Nhắc lại: chỉ số hợp lệ của mảng `n` phần tử là từ `0` đến `n - 1` , **không phải** đến `n` .
- Nhầm `.length` (thuộc tính, không ngoặc đơn) với `.length()` (phương thức của `String` , có ngoặc đơn)** — dùng sai gây lỗi biên dịch ngay lập tức, nhưng người mới hay mất thời gian tìm hiểu vì sao.
- Tưởng gán mảng (`b = a`) là sao chép, sửa `b` rồi ngạc nhiên vì `a` cũng đổi theo** — xem lại phần "Mảng là kiểu tham chiếu" ở trên, dùng `Arrays.copyOf` khi cần bản sao thật sự độc lập.
- In mảng trực tiếp bằng `println(mang)` rồi thắc mắc sao ra chuỗi kỳ lạ** — dùng `Arrays.toString(mang)` .
- Quên rằng kích thước mảng cố định, cố "thêm phần tử" vào mảng đã đầy** — mảng không hỗ trợ thêm phần tử động; cần cấu trúc dữ liệu khác (`ArrayList` , Chương 24) nếu số lượng phần tử thay đổi trong lúc chạy.

Chương 10 — Phương thức (method) tĩnh

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết được phương thức tĩnh (`static`) có/không tham số, có/không giá trị trả về.
- Dùng overloading (nạp chồng) đúng cách — nhiều phương thức cùng tên, khác tham số.
- Hiểu phạm vi biến (scope) — biến khai báo trong một phương thức không "nhìn thấy được" từ phương thức khác.
- Hiểu chính xác Java truyền tham số **theo giá trị** (pass-by-value) — kể cả với kiểu tham chiếu như mảng, và vì sao điều này hay bị hiểu nhầm.

Code mẫu đầy đủ: <code/ch10-phuong-thuc/> .

Vì sao cần phương thức?

Bạn đã viết `main` chứa toàn bộ code từ Chương 3. Khi chương trình lớn dần, gộp mọi thứ vào một `main` khiến code dài, lặp lại, khó đọc. **Phương thức** (method) là cách gói một đoạn logic có tên riêng, có thể **gọi lại nhiều lần** từ nhiều chỗ khác nhau mà không cần chép lại code.

```
static int add(int a, int b) {  
    return a + b;  
}
```

Cấu trúc: `<phạm vi> static <kiểu trả về> <tên>(<tham số>) { ... } .`

- `static` — phương thức thuộc về **class**, gọi được trực tiếp qua tên class (`MethodsDemo.add(3, 5)`) mà không cần tạo object (khái niệm object sẽ học ở Chương 13 — vì sao chương này chỉ dạy phương thức `static` : tránh nhắc tới object trước khi bạn hiểu nó là gì. Từ Chương 13 trở đi, phần lớn phương thức bạn viết **sẽ không còn** `static`).
- `int` (trước tên phương thức) — **kiểu dữ liệu trả về**. Nếu phương thức không trả về gì, dùng `void` (đã thấy ở `main` từ Chương 3).
- `(int a, int b)` — danh sách **tham số** (parameter), mỗi tham số cần khai báo kiểu dữ liệu.
- `return a + b;` — trả giá trị về nơi gọi. Phương thức có kiểu trả về khác `void` **bắt buộc** phải có `return` ở mọi nhánh có thể chạy tới cuối phương thức, nếu không trình biên dịch báo lỗi.

Gọi phương thức:

```
int tong = add(3, 5); // tong = 8
```

Tham số (parameter) và đối số (argument)

Phân biệt hai từ hay dùng lẫn: **tham số** (parameter) là biến khai báo trong định nghĩa phương thức (`a` , `b` ở trên); **đối số** (argument) là giá trị cụ thể truyền vào lúc gọi (`3` , `5`).

Overloading (nạp chồng phương thức)

```
static int add(int a, int b) { return a + b; }  
static int add(int a, int b, int c) { return a + b + c; }  
static double add(double a, double b) { return a + b; }
```

Overloading là việc định nghĩa nhiều phương thức **cùng tên** nhưng khác nhau về **số lượng tham số** hoặc **kiểu tham số** (chỉ khác kiểu trả về thôi thì **không đủ** — trình biên dịch không phân biệt được, sẽ báo lỗi trùng khai báo). Khi gọi `add(...)` , trình

biên dịch tự chọn đúng phiên bản dựa trên số lượng và kiểu đối số bạn truyền vào — việc này xảy ra **lúc biên dịch**, không phải lúc chạy.

Overloading giúp code gọi tự nhiên hơn (`add(3, 5)` và `add(3, 5, 7)` đều đọc hợp lý), thay vì phải đặt tên khác nhau như `addTwo`, `addThree`. Bạn sẽ thấy overloading dùng rất nhiều trong thư viện chuẩn Java, ví dụ `println(int)`, `println(String)`, `println(double)` ... đều là các bản overload của cùng một phương thức `println`.

Phạm vi biến (scope)

```
static void scopeDemo() {
    int x = 999;
    System.out.println(x);
}

public static void main(String[] args) {
    int x = 100;
    scopeDemo();
    System.out.println(x); // vẫn là 100, không bị ảnh hưởng bởi scopeDemo
}
```

Biến khai báo bên trong một phương thức **chỉ tồn tại và chỉ "nhìn thấy được"** trong phạm vi phương thức đó (cụ thể hơn: trong khối `{ }` nó được khai báo). Hai biến `x` ở trên là **hai biến hoàn toàn độc lập**, dù trùng tên — trùng tên biến giữa các phương thức khác nhau không gây xung đột, vì phạm vi của chúng tách biệt nhau. Khi phương thức kết thúc chạy, mọi biến cục bộ khai báo bên trong nó bị giải phóng khỏi bộ nhớ.

Truyền tham số: pass-by-value

Đây là khái niệm hay gây hiểu nhầm nhất chương này. **Java luôn truyền tham số theo giá trị (pass-by-value)** — không có ngoại lệ. Nhưng hệ quả khác nhau tùy kiểu dữ liệu:

Với kiểu nguyên thủy — sửa tham số không ảnh hưởng biến gốc

```
static void tangGap10(int n) {
    n = n * 10;
}

int soGoc = 10;
tangGap10(soGoc);
System.out.println(soGoc); // vẫn là 10, KHÔNG đổi
```

Khi gọi `tangGap10(soGoc)`, Java **sao chép giá trị** của `soGoc` vào tham số cục bộ `n` bên trong phương thức. `n` và `soGoc` là hai vùng nhớ độc lập; sửa `n` không ảnh hưởng gì tới `soGoc`.

Với mảng (kiểu tham chiếu) — sửa nội dung bên trong ảnh hưởng ra ngoài

```
static void nhanDoiPhanTu(int[] mang) {
    for (int i = 0; i < mang.length; i++) {
        mang[i] = mang[i] * 2;
    }
}

int[] arr = {1, 2, 3};
nhanDoiPhanTu(arr);
System.out.println(Arrays.toString(arr)); // [2, 4, 6] - CÓ đổi!
```

Điều này **không mâu thuẫn** với "Java luôn pass-by-value" — thứ được sao chép ở đây là **giá trị của tham chiếu** (địa chỉ trỏ tới mảng trong bộ nhớ), không phải nội dung mảng. Tham số `arr` bên trong phương thức và biến `arr` bên ngoài là hai biến độc lập, nhưng **cả hai cùng trỏ tới đúng một mảng** — sửa phần tử qua biến nào cũng thấy thay đổi qua biến kia, vì chỉ có một mảng duy nhất tồn tại trong bộ nhớ. Khái niệm này sẽ trở nên rõ ràng và tự nhiên hơn nhiều khi học object ở Chương 13 — mảng thực chất hoạt động giống hệt object về mặt tham chiếu.

Bài tập

- Viết phương thức `static boolean isPrime(int n)` kiểm tra số nguyên tố, gọi thử với vài giá trị trong `main`.
- Viết 3 phiên bản overload của phương thức `max`: `max(int, int)`, `max(int, int, int)`, và `max(double, double)`.
- Dự đoán (không chạy thử trước) kết quả của đoạn code sau, sau đó chạy để kiểm tra:

```
static void doiTen(String ten) {  
    ten = "Da doi";  
}  
  
String ten = "Ten goc";  
doiTen(ten);  
System.out.println(ten);
```

Gợi ý: `String` là kiểu tham chiếu nhưng bất biến (immutable) — kết quả sẽ giải thích trọn vẹn ở Chương 11, thử tự suy luận trước dựa trên phần "pass-by-value" vừa học.

Lỗi thường gặp

- missing return statement** — phương thức khai báo kiểu trả về khác `void` nhưng có nhánh logic (ví dụ trong `if`) không có `return`. Đảm bảo **mọi đường chạy có thể tới cuối phương thức** đều có `return`.
- Overload chỉ khác kiểu trả về, giữ nguyên tham số** — báo lỗi biên dịch trùng khai báo. Kiểu trả về không được tính khi trình biên dịch phân biệt các bản overload.
- Tướng sửa tham số nguyên thủy bên trong phương thức sẽ ảnh hưởng biến gốc bên ngoài** — xem lại phần pass-by-value ở trên, đây là hiểu lầm phổ biến nhất chương này.
- Trùng tên biến giữa các phương thức rồi lo lắng bị xung đột** — không xảy ra, phạm vi biến của mỗi phương thức độc lập hoàn toàn (trừ biến `static` cấp class — sẽ học ở Chương 13).

Chương 11 — Chuỗi (String) và StringBuilder

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng thành thạo các thao tác `String` cơ bản: độ dài, chuỗi con, tìm kiếm, thay thế, tách chuỗi.
- Hiểu `String` là **bất biến (immutable)** — vì sao, và hệ quả của điều đó.
- Biết chính xác vì sao **không được dùng** `==` **để so sánh nội dung** `String`, và `String pool` là gì.
- Dùng `StringBuilder` khi cần nối/sửa chuỗi nhiều lần, hiểu vì sao nó hiệu quả hơn `+`.

Code mẫu đầy đủ: [code/ch11-chuoi-string/](#).

Các thao tác String cơ bản

```
String ten = "Nguyen Van A";

ten.length()           // 12 - số ký tự
ten.toUpperCase()      // "NGUYEN VAN A"
ten.toLowerCase()      // "nguyen van a"
ten.charAt(0)          // 'N' - ký tự tại vị trí 0 (chỉ số bắt đầu từ 0, giống mảng)
ten.substring(0, 6)    // "Nguyen" - từ vị trí 0 đến TRƯỚC vị trí 6
ten.substring(7)       // "Van A" - từ vị trí 7 đến hết chuỗi
ten.indexOf("Van")     // 7 - vị trí bắt đầu xuất hiện, -1 nếu không tìm thấy
ten.contains("Nguyen") // true
ten.replace("A", "B")  // "Nguyen Van B" - thay THE TẤT CẢ ký tự/chuỗi con khớp
" xin chào ".trim()    // "xin chào" - xóa khoảng trắng thừa ở đầu/cuối
ten.split(" ")         // ["Nguyen", "Van", "A"] - tách thành mảng String[]
```

`substring(start, end)` lấy từ chỉ số `start` đến **trước** chỉ số `end` (không bao gồm `end`) — giống quy tắc `[start, end)` sẽ gặp lại nhiều lần trong lập trình. `substring(start)` (một tham số) lấy từ `start` đến hết chuỗi.

String là bất biến (immutable)

Đây là tính chất **quan trọng nhất** của `String` trong Java: **mọi thao tác "sửa" chuỗi thực chất tạo ra một chuỗi mới**, chuỗi gốc không bao giờ thay đổi.

```
String goc = "Java";
String daSua = goc.toUpperCase();
System.out.println(goc);    // "Java" - KHÔNG đổi
System.out.println(daSua);  // "JAVA" - chuỗi MỚI, khác object với goc
```

Nếu bạn gọi `goc.toUpperCase()`; mà **không gán** kết quả cho biến nào, chuỗi kết quả bị "bỏ rơi", `goc` vẫn y nguyên — đây là lỗi rất phổ biến với người mới (tưởng gọi phương thức là chuỗi tự đổi).

So sánh String: `==` vs `.equals()`

Đây là lỗi kinh điển bậc nhất của người học Java. Nhắc lại từ Chương 6: `==` với kiểu **tham chiếu** (như `String`) so sánh **hai biến có trỏ tới cùng một object trong bộ nhớ hay không**, chứ không so sánh nội dung:

```
String a = new String("hello");
String b = new String("hello");
System.out.println(a == b);    // false! - hai object KHÁC NHAU trong bộ nhớ
System.out.println(a.equals(b)); // true - nội dung GIỐNG NHAU
```

Luôn dùng `.equals()` để so sánh nội dung hai chuỗi, không bao giờ dùng `==` (trừ trường hợp cố ý kiểm tra hai biến có cùng trỏ một object hay không — hiếm gặp khi mới học).

Vậy sao đôi khi `==` lại "hoạt động đúng" với String?

```
String c = "hello";
String d = "hello";
System.out.println(c == d); // true !!
```

Java có một vùng nhớ đặc biệt gọi là **String pool**: khi bạn viết một chuỗi **literal** (viết thẳng trong code bằng dấu nháy kép, như "hello"), Java tái sử dụng **cùng một object** cho mọi literal có nội dung giống hệt nhau, để tiết kiệm bộ nhớ. Đó là lý do `c == d` cho `true` — cả hai đều trỏ tới cùng một chuỗi trong pool. Nhưng `new String("hello")` **ép buộc tạo object mới**, nằm ngoài pool, nên `==` giữa nó và bất kỳ chuỗi nào khác đều `false` dù nội dung giống hệt.

Kết luận thực dụng: đừng dựa vào String pool để dùng `==`. Hành vi String pool là chi tiết cài đặt bên trong, không phải quy tắc bạn nên lập trình dựa vào — code của bạn có thể nhận `String` từ nhiều nguồn khác nhau (nhập từ bàn phím, đọc từ file...) mà không phải lúc nào cũng đi qua String pool. **Quy tắc duy nhất cần nhớ: so sánh nội dung String luôn dùng `.equals()`.**

StringBuilder — khi cần nối/sửa chuỗi nhiều lần

Vì `String` bất biến, mỗi lần "nối chuỗi" bằng `+` thực chất **tạo ra một chuỗi hoàn toàn mới**, sao chép lại toàn bộ nội dung cũ cộng thêm phần mới:

```
String ketQua = "";
for (int i = 0; i < 50_000; i++) {
    ketQua = ketQua + i; // MOI LAN LAP tao mot chuoi moi, sao chep lai TOAN BO noi dung cu
}
```

Với vòng lặp ít lần, chuyện này không đáng lo. Nhưng lặp hàng chục nghìn lần trở lên, chi phí sao chép lặp lại tăng rất nhanh (chạy thử `StringBuilderDemo.java` để thấy chênh lệch thời gian thực tế). Giải pháp: dùng `StringBuilder` — một cấu trúc dữ liệu **có thể thay đổi (mutable)**, chuyên để xây dựng chuỗi dần dần mà không tạo object mới ở mỗi bước:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 50_000; i++) {
    sb.append(i); // sua NOI DUNG tai cho, khong tao chuoi moi
}
String ketQua = sb.toString(); // chuyen thanh String khi can dung nhu String binh thuong
```

Các phương thức `StringBuilder` hay dùng: `append(...)` (nối thêm vào cuối), `insert(vi_tri, ...)` (chèn vào giữa), `replace(start, end, ...)` (thay thế đoạn), `reverse()` (đảo ngược), `toString()` (chuyển thành `String` thông thường khi cần).

Quy tắc thực dụng: nối vài chuỗi cố định, viết bằng `+` cho dễ đọc ("Xin chào, " + ten) — hiệu năng không đáng lo trong trường hợp này, trình biên dịch Java thậm chí tự tối ưu một số trường hợp đơn giản. Chỉ chuyển sang `StringBuilder` khi nối chuỗi **trong vòng lặp** hoặc số lượng lớn.

Bài tập

- Viết chương trình nhận một câu (dùng `Scanner.nextLine()` — đọc cả dòng, kể cả có dấu cách), đếm số từ trong câu đó (gợi ý: dùng `split(" ")` rồi lấy độ dài mảng kết quả).
- Viết chương trình kiểm tra một chuỗi có phải là "palindrome" (đọc xuôi và đọc ngược giống nhau, ví dụ "madam") hay không, dùng `StringBuilder.reverse()`.
- Giải thích bằng lời: vì sao đoạn code sau **không** in ra "JAVA"?

```
String s = "java";
s.toUpperCase();
System.out.println(s);
```

Lỗi thường gặp

- Dùng `==` để so sánh nội dung `String`, thỉnh thoảng "vẫn đúng" (nhờ `String pool`) rồi ngỡ đó là cách làm đúng — đây là lỗi tiềm ẩn nguy hiểm nhất chương: code có thể chạy đúng trong lúc test (vì dùng toàn literal) nhưng sai khi chạy thật (chuỗi đến từ `new String(...)`, `Scanner.nextLine()`, ghép chuỗi lúc chạy...). Luôn dùng `.equals()`.
- Gọi phương thức "sửa" chuỗi mà không gán lại kết quả — ví dụ `ten.trim()`; một mình trên một dòng không làm gì cả, vì kết quả (chuỗi mới) bị bỏ đi ngay. Phải viết `ten = ten.trim()`.
- `StringIndexOutOfBoundsException` khi gọi `substring` / `charAt` với chỉ số vượt quá độ dài chuỗi — tương tự lỗi mảng ở Chương 9, luôn kiểm tra chỉ số hợp lệ trước khi truy cập.
- Nối chuỗi bằng `+` trong vòng lặp lớn rồi thắc mắc sao chương trình chạy chậm — chuyển sang `StringBuilder` khi số lần nối lớn (hàng nghìn lần trở lên).
- `NullPointerException` khi gọi phương thức trên biến `String` chưa được gán giá trị (`null`) — sẽ hiểu đầy đủ khái niệm `null` ở Chương 13; ghi nhớ trước: luôn đảm bảo biến `String` đã có giá trị thật trước khi gọi phương thức trên nó.

Chương 12 — Vì sao cần OOP

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu lối viết code **thủ tục** (procedural) mà bạn đã dùng từ Chương 3 đến 11 gặp vấn đề gì khi chương trình lớn dần.
- Hiểu ý tưởng cốt lõi của lập trình **hướng đối tượng** (OOP): gộp dữ liệu và hành vi thao tác trên dữ liệu đó vào cùng một đơn vị.
- Biết tên và ý nghĩa sơ lược của 4 trụ cột OOP — sẽ học chi tiết từng cái trong các chương tiếp theo (Chương 13-20), chương này chỉ xây dựng bức tranh tổng quan.

Chương này **không có code mẫu riêng** — đây là chương chuyển tiếp tư duy, chuẩn bị cho Chương 13 bắt đầu viết class đầu tiên.

Lối viết code bạn đã quen: thủ tục (procedural)

Từ Chương 3 đến giờ, mọi chương trình bạn viết đều có dạng: dữ liệu nằm trong các biến rời rạc, và các phương thức `static` thao tác lên những biến đó. Ví dụ hãy tưởng tượng bạn cần quản lý thông tin một sinh viên — theo lối viết đã học:

```
String tenSinhVien = "Nguyen Van A";
int[] diemCacMon = {8, 7, 9};

// Muon tinh diem trung binh, viet mot phuong thuc static riêng
static double tinhDiemTrungBinh(int[] diem) {
    int tong = 0;
    for (int d : diem) {
        tong += d;
    }
    return (double) tong / diem.length;
}
```

Cách này **hoạt động tốt** với chương trình nhỏ. Nhưng thử hình dung bạn cần quản lý **100 sinh viên**, mỗi người có tên, danh sách điểm, ngày sinh, lớp học... Vấn đề bắt đầu xuất hiện:

- Bạn phải tạo **rất nhiều mảng song song** (`String[] ten`, `int[][] diem`, `String[] lop`...) và tự đảm bảo chỉ số `i` ở mọi mảng luôn khớp đúng cùng một sinh viên — chỉ cần một chỗ lệch chỉ số là dữ liệu sai lệch hoàn toàn mà không có cách nào trình biên dịch phát hiện giúp bạn.
- Dữ liệu (`tenSinhVien`, `diemCacMon`) và hành vi thao tác lên nó (`tinhDiemTrungBinh`) nằm **tách rời nhau** — không có gì trong code nói rõ "hàm này thuộc về, và chỉ nên dùng với, dữ liệu sinh viên". Bất kỳ đoạn code nào ở bất kỳ đâu cũng có thể sửa trực tiếp `diemCacMon` mà không qua kiểm soát nào, kể cả gán giá trị vô lý (điểm âm, điểm > 10).

Ý tưởng cốt lõi của OOP: gộp dữ liệu + hành vi vào một đơn vị

Lập trình hướng đối tượng giải quyết đúng vấn đề trên bằng cách tạo ra một **khuôn mẫu** (gọi là **class**, học chi tiết ở Chương 13) mô tả một sinh viên gồm **cả dữ liệu lẫn các thao tác hợp lệ trên dữ liệu đó**, gộp chung một chỗ:

```
class SinhVien {
    String ten;
    int[] diem;

    double tinhDiemTrungBinh() {
        int tong = 0;
        for (int d : diem) {
            tong += d;
        }
    }
}
```

```
    }
    return (double) tong / diem.length;
}
}
```

Từ khuôn mẫu (class) đó, bạn tạo ra nhiều **đối tượng** (object) — mỗi object là một sinh viên cụ thể, tự mang theo dữ liệu riêng của nó:

```
SinhVien sv1 = new SinhVien(); // sinh vien thu nhat
SinhVien sv2 = new SinhVien(); // sinh vien thu hai, HOAN TOAN doc lap voi sv1
sv1.ten = "Nguyen Van A";
sv2.ten = "Tran Thi B";
System.out.println(sv1.tinhDiemTrungBinh()); // moi object tu goi phuong thuc voi DU LIEU CUA CHINH NO
```

Không còn mảng song song, không còn nguy cơ lệch chỉ số — mỗi `SinhVien` tự mang theo đúng dữ liệu của nó, và phương thức `tinhDiemTrungBinh()` gắn chặt với đúng dữ liệu đó. Đây chính xác là nội dung Chương 13 sẽ hướng dẫn viết chi tiết, cú pháp đầy đủ.

Bốn trụ cột của OOP (tổng quan)

Toàn bộ Phần 2 và Phần 3 của sách xoay quanh 4 ý tưởng nền tảng này. Ở đây chỉ giới thiệu tên và ý nghĩa sơ lược — mỗi cái sẽ có một hoặc nhiều chương riêng đi sâu:

1. **Encapsulation (đóng gói)** — Chương 14. Giấu chi tiết dữ liệu bên trong object, chỉ cho phép truy cập/thay đổi thông qua các "cổng" được kiểm soát (phương thức) — ví dụ ngăn không cho gán điểm số âm cho sinh viên, thay vì để bất kỳ đoạn code nào cũng gán trực tiếp giá trị bất kỳ.
2. **Inheritance (kế thừa)** — Chương 16. Cho phép một class **tái sử dụng** dữ liệu và hành vi của một class khác, rồi mở rộng hoặc điều chỉnh thêm — ví dụ `SinhVienDaiHoc` và `SinhVienCaoHoc` có thể cùng kế thừa từ một `SinhVien` chung, tránh viết lặp lại phần giống nhau.
3. **Polymorphism (đa hình)** — Chương 17. Cho phép xử lý nhiều loại object **khác nhau** thông qua **cùng một cách gọi**, mỗi loại tự "biết" cách phản ứng đúng theo bản chất riêng của nó — ví dụ một danh sách chứa cả `SinhVienDaiHoc` và `SinhVienCaoHoc`, gọi cùng một phương thức `tinhHocPhi()` trên từng phần tử, nhưng mỗi loại tự tính ra kết quả khác nhau theo đúng quy tắc của loại đó.
4. **Abstraction (trừu tượng hoá)** — Chương 18. Định nghĩa "**cái gì cần làm được**" mà không quy định chi tiết "**làm như thế nào**", cho phép nhiều class khác nhau cùng tuân theo một "hợp đồng" chung — ví dụ định nghĩa rằng "mọi hình dạng đều tính được diện tích", mà không quan tâm hình tròn hay hình vuông tính diện tích bằng công thức gì.

Bốn ý tưởng này **không tách rời nhau** — chúng phối hợp với nhau trong hầu hết code OOP thực tế. Đừng cố học thuộc lòng định nghĩa ở giai đoạn này; hãy đọc tiếp các chương sau, viết code thực tế, rồi quay lại đọc lại đoạn tổng quan này — lúc đó các khái niệm sẽ trở nên rõ ràng và cụ thể hơn nhiều so với đọc định nghĩa suông.

OOP không phải "cách làm đúng duy nhất"

Một lưu ý quan trọng để tránh hiểu sai: OOP là **một công cụ tư duy**, không phải chân lý tuyệt đối. Có những bài toán/ngôn ngữ phù hợp hơn với lối viết khác (lập trình hàm — functional programming, sẽ chạm tới ở Phần 6 khi học lambda/stream). Java **buộc** bạn viết theo OOP ở mức cấu trúc cơ bản (mọi thứ đều nằm trong class), nên việc hiểu OOP vững chắc là điều kiện bắt buộc để viết Java hiệu quả — đó là lý do phần lớn số chương còn lại của sách xoay quanh chủ đề này.

Bài tập

1. Viết lại (chỉ trên giấy hoặc mô tả bằng lời, chưa cần chạy được) ý tưởng một class `SanPham` có dữ liệu gì (tên, giá, số lượng tồn kho...) và những hành vi/phương thức nào hợp lý nên gắn liền với nó (tính tổng giá trị tồn kho, giảm giá...).
2. Tự giải thích lại bằng lời của bạn: vì sao gộp dữ liệu và hành vi vào cùng một đơn vị (class) lại giảm được nguy cơ lỗi so với dùng nhiều mảng song song?

3. Đọc lại đoạn "Bốn trụ cột của OOP" — ghi chú lại câu hỏi nào bạn còn mơ hồ, giữ lại để đối chiếu sau khi học xong Chương 13-18.

Chương 13 — Class & Object

Mục tiêu học

Sau chương này, bạn sẽ:

- Định nghĩa được một class với field và phương thức instance.
- Tạo object từ class bằng `new`, hiểu chuyện gì xảy ra trong bộ nhớ khi làm vậy.
- Viết constructor (kể cả nhiều constructor overload), hiểu khi nào Java tự tạo constructor mặc định cho bạn.
- Dùng đúng từ khoá `this`, hiểu vì sao đôi khi bắt buộc phải dùng nó.
- Hiểu `null` là gì — giá trị "chưa trỏ tới object nào" của kiểu tham chiếu.

Code mẫu đầy đủ: [code/ch13-class-va-object/](#).

Class là khuôn mẫu, Object là thực thể cụ thể

Tiếp nối Chương 12: **class** là bản thiết kế/khuôn mẫu mô tả một loại thứ gì đó có những dữ liệu gì (field) và làm được những gì (phương thức). **Object** là một thực thể **cụ thể** được tạo ra từ khuôn mẫu đó, tồn tại thật trong bộ nhớ lúc chương trình chạy.

Ẩn dụ quen thuộc: class `SinhVien` giống như **bản thiết kế nhà** — nó mô tả nhà có mấy phòng, mấy cửa sổ, nhưng bản thân bản thiết kế không phải là một căn nhà bạn ở được. Mỗi lần bạn `new SinhVien(...)`, bạn "xây" một **căn nhà cụ thể** theo đúng bản thiết kế đó — có địa chỉ riêng (vị trí trong bộ nhớ), độc lập hoàn toàn với những căn nhà khác cũng xây từ cùng bản thiết kế.

Định nghĩa class

```
public class SinhVien {
    String ten;
    int[] diem;

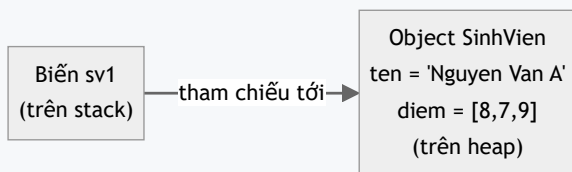
    public double tinhDiemTrungBinh() {
        int tong = 0;
        for (int d : diem) {
            tong += d;
        }
        return (double) tong / diem.length;
    }
}
```

- `String ten;` và `int[] diem;` là **field** (thuộc tính) — dữ liệu mà **mỗi object** của class này sẽ tự mang theo một bản riêng.
- `tinhDiemTrungBinh()` là **phương thức instance** — khác với phương thức `static` học ở Chương 10, phương thức instance **không có** từ khoá `static`, và luôn thao tác trên dữ liệu của **đúng object đang gọi nó** (bên trong thân phương thức, `diem` chính là field `diem` của object gọi phương thức này — không phải một biến trôi nổi đâu đó).

Tạo object với new

```
SinhVien sv1 = new SinhVien();
```

`new SinhVien()` làm hai việc: (1) **cấp phát vùng nhớ mới** trong một khu vực bộ nhớ gọi là **heap**, đủ chứa các field của `SinhVien`; (2) gọi **constructor** để khởi tạo giá trị ban đầu cho vùng nhớ đó. Biến `sv1` **không chứa trực tiếp object** — nó chứa một **tham chiếu** (giống địa chỉ) trỏ tới object vừa tạo trong heap. Đây chính là khái niệm "kiểu tham chiếu" đã nhắc tới khi học mảng ở Chương 9 — thực ra mảng trong Java **cũng là object**, hoạt động theo đúng cơ chế này.



Hệ quả quan trọng, đúng như đã thấy với mảng: `SinhVien sv2 = sv1;` **không tạo object mới**, chỉ tạo thêm một biến trỏ tới **cùng** object — sửa qua `sv2` sẽ ảnh hưởng tới những gì `sv1` "thấy", vì cả hai cùng trỏ tới một chỗ. Muốn hai object thật sự độc lập, phải gọi `new` riêng cho từng cái (như `sv1` và `sv2` trong code mẫu `Main.java`).

Constructor

Constructor là một khối code đặc biệt chạy **đúng một lần** khi object được tạo bằng `new`, dùng để thiết lập giá trị ban đầu cho các field.

```
public SinhVien(String ten, int[] diem) {
    this.ten = ten;
    this.diem = diem;
}
```

Đặc điểm nhận diện constructor: **tên trùng chính xác tên class**, và **không có kiểu trả về** (kể cả không có `void`). Gọi bằng `new SinhVien("Nguyen Van A", new int[]{8, 7, 9})`.

Constructor mặc định (default constructor)

Nếu bạn **không viết bất kỳ constructor nào**, Java tự động cung cấp một constructor không tham số, rỗng (không làm gì ngoài khởi tạo field về giá trị mặc định: số là `0`, `boolean` là `false`, kiểu tham chiếu là `null` — xem phần dưới). **Ngay khi bạn viết ít nhất một constructor của riêng mình**, Java **không còn tự cung cấp** constructor mặc định nữa — nếu vẫn muốn gọi được `new SinhVien()` không tham số, bạn phải **tự viết** constructor đó (như trong `SinhVien.java` ở code mẫu).

Nhiều constructor (constructor overloading)

Gống overloading phương thức đã học ở Chương 10, một class có thể có **nhiều constructor**, khác nhau về tham số:

```
public SinhVien() {
    this.ten = "Chua co ten";
    this.diem = new int[0];
}

public SinhVien(String ten, int[] diem) {
    this.ten = ten;
    this.diem = diem;
}
```

Java tự chọn đúng constructor dựa trên đối số bạn truyền vào lúc gọi `new`.

Từ khoá this

`this` là một tham chiếu **ngầm định** tồn tại bên trong mọi phương thức instance và constructor, trỏ tới **chính object đang gọi phương thức đó**. Dùng `this` phổ biến nhất khi:

Tham số trùng tên với field (rất thường gặp trong constructor, như ở trên) — `this.ten` chỉ rõ "field `ten` của object này", phân biệt với `ten` (tham số cục bộ) đứng một mình:

```
public SinhVien(String ten, int[] diem) {
    this.ten = ten; // this.ten (field) = ten (tham so) - BAT BUOC can 'this' o day
    this.diem = diem;
}
```

Nếu bỏ `this.` ở đây, dòng `ten = ten;` sẽ hiểu là **gán tham số cho chính nó**, field `ten` của object **không hề được gán giá trị** — lỗi logic khó phát hiện vì trình biên dịch không báo lỗi gì (cả hai `ten` đều là biến hợp lệ, chỉ là không phải cái bạn muốn).

Khi tên tham số **không trùng** tên field (ví dụ `tinhDiemTrungBinh()` dùng thẳng `diem` mà không cần viết `this.diem`), `this.` là **tuỳ chọn** — Java tự hiểu `diem` trong thân phương thức instance nghĩa là field của object hiện tại, trừ khi có một biến cục bộ/tham số cùng tên che khuất nó. Nhiều lập trình viên vẫn viết `this.` tưởng mình dù không bắt buộc, để code rõ ràng hơn khi đọc lại.

null — giá trị "chưa trở tới object nào"

```
String ten; // neu chua gan gia tri, mac dinh la null
```

Với **kiểu tham chiếu** (mọi class, mảng, `String` ...), giá trị mặc định khi chưa được gán là `null` — nghĩa là "biến này hiện không trở tới object nào cả". Khác với kiểu nguyên thủy (luôn có giá trị mặc định cụ thể như `0`, `false`), kiểu tham chiếu có thêm khả năng "không trở đi đâu cả".

Gọi bất kỳ phương thức nào trên một biến đang là `null` sẽ khiến chương trình ném ra lỗi `NullPointerException` — lỗi runtime **cực kỳ phổ biến** trong Java (đủ phổ biến để có biệt danh "tỷ đô la lỗi" trong giới lập trình). Chương 21 sẽ học cách xử lý các lỗi runtime kiểu này đúng cách; ở giai đoạn này, ghi nhớ nguyên tắc: **luôn đảm bảo biến tham chiếu đã được gán một object thật sự (bằng `new` hoặc gán từ nơi khác) trước khi gọi phương thức trên nó**.

Bài tập

- Tạo class `SanPham` với field `ten` (`String`), `gia` (`double`), `soLuong` (`int`); viết constructor nhận đủ 3 giá trị; viết phương thức `tinhTongGiaTri()` trả về `gia * soLuong`.
- Trong `Main`, tạo 3 object `SanPham` khác nhau, in tổng giá trị từng sản phẩm, và tính tổng giá trị của cả 3 sản phẩm cộng lại.
- Cố tình bỏ `this.` trong một constructor có tham số trùng tên field (như ví dụ lỗi ở phần "Từ khoá `this`"), chạy thử và quan sát field không được gán đúng giá trị mong đợi — dù chương trình biên dịch và chạy bình thường, không báo lỗi gì.
- Khai báo một biến `SinhVien sv`; (không `new`), thử gọi `sv.inThongTin()`; ngay sau đó — đọc kỹ thông báo lỗi `NullPointerException` xuất hiện.

Lỗi thường gặp

- Quên `this.` khi tham số trùng tên field trong constructor** — field không được gán giá trị đúng, lỗi logic âm thầm không có cảnh báo biên dịch. Xem lại phần "Từ khoá `this`".
- `NullPointerException`** — gọi phương thức hoặc truy cập field trên một biến tham chiếu đang là `null`. Kiểm tra lại: biến đó đã từng được `new` hoặc gán từ một nguồn hợp lệ chưa?
- Viết constructor riêng rồi ngạc nhiên vì `new SinhVien()` không tham số không còn gọi được** — nhắc lại: một khi bạn viết bất kỳ constructor nào, Java ngừng tự cung cấp constructor mặc định. Muốn giữ khả năng gọi không tham số, phải tự viết thêm constructor đó.
- Tưởng `sv2 = sv1` tạo ra object mới, sửa `sv2` rồi ngạc nhiên vì `sv1` cũng đổi** — đúng hành vi của kiểu tham chiếu, xem lại phần "Tạo object với `new`". Muốn bản sao độc lập, phải `new` riêng.
- Nhầm phương thức instance với phương thức `static` đã học ở Chương 10** — phương thức instance cần có một object cụ thể để gọi (`sv1.tinhDiemTrungBinh()`), không gọi được qua tên class như phương thức `static` (`SinhVien.tinhDiemTrungBinh()` sẽ báo lỗi biên dịch vì phương thức không phải `static`).

Chương 14 — Encapsulation (đóng gói)

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu 4 mức truy cập (access modifier) của Java: `public`, `private`, `protected`, `package-private`, và dùng đúng chúng cho field/phương thức.
- Viết được getter/setter, hiểu vì sao setter nên kiểm tra tính hợp lệ của dữ liệu.
- Thiết kế class theo nguyên tắc "chỉ mở đúng cổng cần thiết", thay vì để mọi field `public`.
- Biết khi nào nên **không** cung cấp setter, chỉ cung cấp các phương thức thao tác có kiểm soát.

Code mẫu đầy đủ: [code/ch14-encapsulation/](#).

Vấn đề với field `public`

Ở Chương 13, `SinhVien.ten` và `SinhVien.diem` không có access modifier riêng (mặc định là `package-private`, xem phần dưới) — trong ví dụ `Main.java` của chương đó, bạn từng gán trực tiếp `sv3.diem = new int[]{6, 7};`. Điều này **hoạt động được**, nhưng ẩn chứa rủi ro: **không có gì ngăn code ở đâu đó gán giá trị vô lý**, ví dụ:

```
sv3.diem = new int[]{-5, 15, 999}; // diem am, diem > 10 - HOAN TOAN hop le ve mat cu phap!
```

Không có lỗi biên dịch, không có cảnh báo — chương trình chạy "bình thường" với dữ liệu vô nghĩa, lỗi chỉ lộ ra khi bạn tính điểm trung bình ra một con số kỳ quặc, và lúc đó rất khó truy ngược lại xem chỗ nào đã gán sai. Đây chính là vấn đề **encapsulation** giải quyết.

Bốn mức truy cập của Java

Modifier	Trong cùng class	Trong cùng package	Ở class con (khác package)	Ở mọi nơi khác
<code>private</code>	✓	✗	✗	✗
<i>(không ghi gì — package-private)</i>	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

Khái niệm "package" sẽ học kỹ ở Chương 15, "class con"/kế thừa sẽ học ở Chương 16 — ở chương này, tập trung vào hai mức quan trọng nhất khi mới bắt đầu: **`private`** (giấu hoàn toàn, chỉ dùng được bên trong class) và **`public`** (mở hoàn toàn, dùng được từ mọi nơi).

Nguyên tắc thiết kế mặc định: field nên là `private`. Chỉ mở `public` cho phương thức nào bạn **chủ động muốn** cho phép gọi từ bên ngoài.

Getter và Setter

Khi field là `private`, code bên ngoài class không đọc/ghi trực tiếp được nữa. Muốn cho phép đọc, cung cấp **getter**; muốn cho phép ghi (có kiểm soát), cung cấp **setter**:

```
public class NgườiDùng {
    private String email;

    public String getEmail() {
        return email;
    }
}
```

```

    public void setEmail(String email) {
        if (email == null || !email.contains("@")) {
            System.out.println("Canh bao: email khong hop le, giu nguyen gia tri cu");
            return;
        }
        this.email = email;
    }
}

```

Điểm mấu chốt: **setter không chỉ đơn thuần gán giá trị** — nó là nơi đặt các quy tắc kiểm tra tính hợp lệ. Mọi chỗ trong chương trình muốn đổi `email` của một `NguoiDung` đều **buộc phải đi qua** `setEmail`, nghĩa là **buộc phải đi qua** kiểm tra hợp lệ đó — không có đường tắt nào khác để gán trực tiếp một giá trị bậy vào field, vì field đã là `private`.

Quy ước đặt tên: `get<TenField>()` cho getter, `set<TenField>(...)` cho setter (viết hoa chữ đầu tên field). Với `boolean`, quy ước thường dùng `is<TenField>()` thay vì `get` (ví dụ `isHopLe()`) — sẽ gặp lại quy ước này trong thư viện chuẩn Java.

Khi nào KHÔNG nên cung cấp setter

Không phải mọi field đều cần setter. Xem lại `TaiKhoanNganHang.java` trong code mẫu:

```

public class TaiKhoanNganHang {
    private double soDu;

    public double getSoDu() {
        return soDu;
    }

    public void napTien(double soTien) {
        if (soTien <= 0) { /* tu choi */ return; }
        soDu += soTien;
    }

    public void rutTien(double soTien) {
        if (soTien <= 0 || soTien > soDu) { /* tu choi */ return; }
        soDu -= soTien;
    }

    // CO CHU DICH: khong co setSoDu(double)
}

```

Nếu class này có `setSoDu(double)`, bất kỳ đoạn code nào cũng có thể gán thẳng một số dư bất kỳ, bỏ qua hoàn toàn logic nghiệp vụ (không được âm, thay đổi phải thông qua giao dịch nạp/rút cụ thể). Bằng cách **chỉ cung cấp** `napTien` / `rutTien` — hai phương thức mô tả đúng những **hành động hợp lệ trong thực tế** — class tự đảm bảo số dư luôn ở trạng thái hợp lý, không cần "hy vọng" người gọi code cẩn thận.

Nguyên tắc thiết kế: đặt câu hỏi "trong thực tế, dữ liệu này được phép thay đổi theo những cách nào?" — rồi chỉ cung cấp đúng những phương thức phản ánh các cách đó, thay vì mặc định thêm getter/ setter cho mọi field.

Bài tập

- Refactor lại class `SanPham` bạn viết ở bài tập Chương 13: chuyển field sang `private`, thêm getter cho `ten` / `gia` / `soLuong`, thêm setter cho `gia` với điều kiện không cho gán giá trị âm.
- Thêm phương thức `nhapKho(int soLuongThem)` và `xuatKho(int soLuongXuat)` vào `SanPham`, đảm bảo `xuatKho` từ chối nếu số lượng xuất vượt quá tồn kho hiện có — theo đúng tinh thần của `TaiKhoanNganHang.napTien` / `rutTien`.
- Giải thích bằng lời: vì sao dòng `tk.soDu = -999_999_999`; (đã comment sẵn trong `Main.java`) gây lỗi **biên dịch** chứ không phải lỗi lúc chạy? Điều này khác gì so với việc chỉ đơn giản "hy vọng" không ai viết dòng code đó?

Lỗi thường gặp

- **Thêm getter/setter cho mọi field theo phản xạ, kể cả khi không cần** — nếu một field không bao giờ cần đọc hoặc sửa từ bên ngoài, đừng thêm getter/setter cho nó. Encapsulation không chỉ là "field private + getter/setter", mà là **suy nghĩ có chủ đích** field nào thực sự cần lộ ra.
- **Setter chỉ gán giá trị, không kiểm tra gì** — làm mất hết ý nghĩa của việc dùng `private` + setter, tương đương với field `public` nhưng dài dòng hơn. Luôn cân nhắc điều kiện hợp lệ nào cần kiểm tra.
- **Getter trả về trực tiếp một mảng/object `private` mà không sao chép** — người gọi `getDiem()` nhận về **chính tham chiếu** tới mảng nội bộ, có thể sửa nó từ bên ngoài dù field là `private`, vô hiệu hoá tác dụng bảo vệ (bài toán này gọi là "rò rỉ tham chiếu" — sẽ quay lại kỹ hơn khi học về bất biến hoá object nâng cao). Cách khắc phục cơ bản: trả về bản sao (`Arrays.copyOf(...)`) thay vì trả thẳng tham chiếu nội bộ, khi field là kiểu tham chiếu có thể thay đổi được.
- **Nhầm lẫn `private` nghĩa là "không ai truy cập được kể cả để đọc"** — `private` chỉ giới hạn truy cập **trực tiếp từ bên ngoài class**; bên trong chính class đó (mọi phương thức, kể cả constructor), field `private` truy cập bình thường không hạn chế gì.

Chương 15 — Package & import

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu package là gì, vì sao Java cần nó khi project có nhiều class.
- Khai báo đúng `package` trong file `.java`, biết quy tắc thư mục phải khớp tên package.
- Dùng `import` để dùng class từ package khác.
- Hiểu lại rõ hơn mức truy cập **package-private** đã nhắc sơ ở Chương 14, qua ví dụ cụ thể.
- Biết quy ước đặt tên package theo tên miền đảo ngược.

Code mẫu đầy đủ: [code/ch15-package-import/](#).

Package là gì, vì sao cần?

Từ Chương 3 đến giờ, mọi class bạn viết đều nằm trong "**gói mặc định**" (default package) — không khai báo `package` gì cả. Cách này ổn với project nhỏ vài file, nhưng có vấn đề khi project lớn dần: **tên class phải là duy nhất trong toàn bộ project**. Hai lập trình viên trong cùng team, mỗi người viết một class tên `Utils`, sẽ xung đột ngay nếu cùng nằm trong gói mặc định.

Package là cách Java tổ chức class thành các "thư mục" logic, giải quyết đúng vấn đề đó — hai class trùng tên vẫn tồn tại được miễn là nằm trong package khác nhau (`com.truong.Utils` khác với `com.congty.Utils`, dù cùng tên lớp `Utils`).

Khai báo package

```
package com.example.school;

public class SinhVien {
    // ...
}
```

Dòng `package ...`; **phải là dòng code đầu tiên** trong file (chỉ có thể đứng sau comment, không được đứng sau bất kỳ dòng code nào khác).

Quy tắc bắt buộc: thư mục phải khớp tên package

Nếu class khai báo `package com.example.school`; , file chứa nó **phải** nằm trong đường dẫn thư mục `com/example/school/` (mỗi dấu `.` trong tên package tương ứng một cấp thư mục). Đây là quy tắc bắt buộc, tương tự quy tắc "tên file phải khớp tên class public" đã học ở Chương 3 — trình biên dịch dựa vào cấu trúc thư mục để tìm và tổ chức các class theo đúng package của chúng.

```
com/example/school/SinhVien.java -> package com.example.school;
```

import — dùng class từ package khác

```
package com.example.app;

import com.example.school.SinhVien;

public class Main {
    public static void main(String[] args) {
        SinhVien sv = new SinhVien("Nguyen Van A");
        // ...
    }
}
```

```
}  
}
```

`import` cho phép bạn viết tên class ngắn gọn (`SinhVien`) thay vì phải viết tên đầy đủ kèm package mỗi lần dùng (`com.example.school.SinhVien`). Về bản chất, `import` chỉ là một "khai báo tắt" — bạn vẫn có thể dùng tên đầy đủ mà không cần `import` gì cả:

```
com.example.school.SinhVien sv = new com.example.school.SinhVien("Nguyen Van A");
```

Nhưng cách viết tắt với `import` gần như luôn được ưu tiên vì ngắn gọn, dễ đọc hơn nhiều.

Lưu ý: các class trong package `java.lang` (như `String`, `System`, `Math` ...) được **tự động import**, không cần khai báo `import` gì — đây là lý do bạn dùng `String` thoải mái từ Chương 5 mà chưa từng viết `import java.lang.String;`. Mọi package khác trong thư viện chuẩn (`java.util.Scanner` đã dùng ở Chương 5, `java.util.Arrays` ở Chương 9) đều cần `import` tương minh.

Package-private — nhắc lại và làm rõ

Ở Chương 14, bảng 4 mức truy cập có nhắc tới **package-private** (không ghi access modifier gì) — giờ bạn đã hiểu package là gì, ý nghĩa của mức này trở nên rõ ràng: một class/field/phương thức không có modifier chỉ truy cập được từ **các class khác trong cùng package**, hoàn toàn "vô hình" với package khác — kể cả khi package khác có `import` cũng không giúp gì, vì bản thân class đó không được phép nhìn thấy từ bên ngoài package của nó.

Xem `MaSinhVienGenerator.java` trong code mẫu — class này **không có `public`**, chỉ `SinhVien.java` (cùng package `com.example.school`) gọi được nó; `Main.java` (package `com.example.app` khác) không thể, dù có cố `import` cũng không được (chính `import` cũng đòi hỏi class đích phải `public` mới nhập được từ package khác).

Ứng dụng thực tế của package-private: đây là cách gói gọn các class "chi tiết cài đặt nội bộ" mà bạn không muốn code bên ngoài package biết tới hay phụ thuộc vào — chỉ những gì thật sự cần dùng từ bên ngoài mới nên là `public`. Đây là **encapsulation ở cấp độ package**, mở rộng ý tưởng đã học ở Chương 14 (encapsulation ở cấp độ field) lên quy mô lớn hơn.

Quy ước đặt tên package

Cộng đồng Java quy ước đặt tên package theo **tên miền đảo ngược** của tổ chức sở hữu code, viết thường toàn bộ: công ty sở hữu tên miền `example.com` đặt package bắt đầu bằng `com.example.*`. Quy ước này giúp package của các tổ chức khác nhau không trùng nhau trên phạm vi toàn cầu (vì tên miền vốn đã là duy nhất). Với project cá nhân/học tập, quy ước phổ biến là dùng tên riêng của bạn, ví dụ `com.tenban.tenproject`.

Bài tập

- Tạo package `com.tenban.hinhhoc` chứa một class `HinhTron` (có field `banKinh`, phương thức `tinhDienTich()`), và package `com.tenban.app` chứa `Main` gọi tới `HinhTron` qua `import`.
- Thử xóa dòng `import` trong `Main.java` của bài tập 1, thay bằng gọi tên đầy đủ có package (`com.tenban.hinhhoc.HinhTron`), xác nhận vẫn biên dịch và chạy được — chứng minh `import` chỉ là cách viết tắt, không phải yêu cầu bắt buộc để dùng được class.
- Trong code mẫu, thử đổi `MaSinhVienGenerator` từ package-private sang `public`, biên dịch lại, rồi bỏ comment dòng gọi trực tiếp từ `Main.java` — xác nhận giờ đã gọi được, để thấy rõ tác dụng của từ khoá `public`.

Lỗi thường gặp

- `error: class SinhVien is public, should be declared in a file named SinhVien.java` hoặc lỗi tương tự về vị trí file — thư mục chứa file không khớp tên package khai báo. Kiểm tra lại đường dẫn thư mục đúng theo từng cấp của tên package.
- `package com.example.school does not exist hoặc cannot find symbol` khi biên dịch — thường do biên dịch thiếu file, hoặc chạy `javac` không đúng từ thư mục gốc chứa `com/`. Xem lại phần "Biên dịch và chạy" trong README code

mẫu.

- **Chạy** `java SinhVien` **thay vì** `java com.example.school.SinhVien` — khi class thuộc một package, phải chạy bằng **tên đầy đủ kèm package**, không dùng được tên ngắn.
- **Cố import một class package-private từ package khác** — báo lỗi biên dịch vì class đó "không tồn tại" theo nghĩa nhìn thấy được từ package hiện tại. Chỉ `public` class mới import được từ package khác.

Chương 16 — Kế thừa (Inheritance)

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng `extends` để một class kế thừa field và phương thức từ class khác.
- Gọi constructor lớp cha bằng `super(...)`, hiểu vì sao gần như luôn bắt buộc.
- Override (ghi đè) phương thức của lớp cha, dùng `@Override` đúng cách.
- Gọi lại phiên bản của lớp cha từ lớp con bằng `super.tenPhuongThuc(...)`.
- Phân biệt **overriding** (Chương này) với **overloading** (đã học ở Chương 10) — hai khái niệm rất dễ nhầm vì tên gần giống nhau.

Code mẫu đầy đủ: [code/ch16-ke-thua/](#).

Vấn đề: trùng lặp code giữa các class liên quan

Giả sử bạn cần mô hình hoá nhiều loại nhân viên: nhân viên thường, nhân viên bán hàng (có thêm hoa hồng), nhân viên quản lý (có thêm phụ cấp). Không có kế thừa, bạn phải viết lặp lại `ten`, `luongCoBan`, `getTen()` trong **cả ba** class riêng biệt — trùng lặp, và nếu sau này cần sửa cách lưu `ten`, phải sửa ở cả ba nơi.

`extends` — kế thừa từ một class khác

```
public class NhanVien {
    protected String ten;
    protected double luongCoBan;

    public NhanVien(String ten, double luongCoBan) {
        this.ten = ten;
        this.luongCoBan = luongCoBan;
    }

    public String getTen() {
        return ten;
    }

    public double tinhLuong() {
        return luongCoBan;
    }
}
```

```
public class NhanVienBanHang extends NhanVien {
    private double doanhSo;

    public NhanVienBanHang(String ten, double luongCoBan, double doanhSo) {
        super(ten, luongCoBan);
        this.doanhSo = doanhSo;
    }
}
```

`NhanVienBanHang extends NhanVien` nghĩa là `NhanVienBanHang` **tự động có sẵn** mọi field và phương thức `public` / `protected` của `NhanVien` (`ten`, `luongCoBan`, `getTen()`, `tinhLuong()`), mà không cần viết lại — gọi `NhanVien` là **lớp cha (superclass/parent class)**, `NhanVienBanHang` là **lớp con (subclass/child class)**.

Vì sao field cần protected thay vì private ở đây?

Ở Chương 14, nguyên tắc là field nên `private`. Nhưng `private` **giấu hoàn toàn**, kể cả với lớp con — lớp con sẽ **không truy cập trực tiếp được** field `private` của lớp cha. `protected` (nhắc lại bảng ở Chương 14) mở thêm một mức: **cho phép lớp con** (kể cả ở package khác) truy cập trực tiếp, trong khi vẫn giấu khỏi phần còn lại của chương trình. Đây là lý do `NhanVien.ten` và `NhanVien.luongCoBan` khai báo `protected` — để `NhanVienQuanLy.tinhLuong()` dùng được `luongCoBan` trực tiếp (xem phần override bên dưới).

`super(...)` — gọi constructor lớp cha

```
public NhanVienBanHang(String ten, double luongCoBan, double doanhSo) {
    super(ten, luongCoBan); // PHẢI là dòng đầu tiên trong constructor
    this.doanhSo = doanhSo;
}
```

`super(...)` gọi constructor của lớp cha, để lớp cha tự khởi tạo đúng phần dữ liệu của nó (`ten`, `luongCoBan`). Quy tắc bắt buộc: nếu bạn gọi `super(...)`, nó **phải là dòng đầu tiên** trong constructor lớp con. Nếu bạn **không** viết `super(...)` tường minh, Java **tự động chèn** một lời gọi `super()` không tham số ở đầu constructor — nếu lớp cha **không có** constructor không tham số (như `NhanVien` ở đây — nó chỉ có constructor 2 tham số), việc này gây **lỗi biên dịch**, buộc bạn phải tự viết `super(...)` với đúng tham số phù hợp.

Override (ghi đè) phương thức

```
public class NhanVienBanHang extends NhanVien {
    private static final double TY_LE_HOA_HONG = 0.05;

    @Override
    public double tinhLuong() {
        return luongCoBan + doanhSo * TY_LE_HOA_HONG;
    }
}
```

Override là việc lớp con định nghĩa lại một phương thức đã có ở lớp cha, **cùng tên, cùng tham số, cùng kiểu trả về** — thay thế hoàn toàn hành vi của lớp cha bằng hành vi mới khi phương thức đó được gọi trên object của lớp con.

`@Override` là một **annotation** (chú thích đặc biệt, không ảnh hưởng logic chạy) — báo cho trình biên dịch biết "tôi cố ý override phương thức này". Lợi ích: nếu bạn gõ sai tên phương thức, sai kiểu tham số (vô tình tạo ra một phương thức overload mới thay vì override như ý muốn), `@Override` khiến trình biên dịch **báo lỗi ngay** thay vì âm thầm để bạn tạo nhầm một phương thức khác. **Luôn dùng `@Override`** khi có ý định ghi đè — đây là thói quen tốt gần như bắt buộc trong code Java thực tế.

`super.tenPhuongThuc(...)` — gọi lại phiên bản của lớp cha

```
public class NhanVienQuanLy extends NhanVien {
    private double phuCapQuanLy;

    @Override
    public double tinhLuong() {
        return super.tinhLuong() + phuCapQuanLy;
    }
}
```

Đôi khi lớp con muốn **mở rộng** hành vi của lớp cha thay vì thay thế hoàn toàn — `super.tinhLuong()` gọi đúng phiên bản `tinhLuong()` của `NhanVien`, rồi cộng thêm phần riêng của `NhanVienQuanLy`. Cách này tránh việc phải chép lại logic `return`

`LuongCoBan`; một lần nữa trong `NhanVienQuanLy` — nếu sau này cách tính lương cơ bản ở `NhanVien` thay đổi, `NhanVienQuanLy` tự động thừa hưởng thay đổi đó mà không cần sửa gì.

Overriding khác Overloading như thế nào?

Đây là cặp khái niệm **rất dễ nhầm** vì tên gần giống nhau trong tiếng Anh lẫn cách phát âm:

	Overloading (Chương 10)	Overriding (chương này)
Xảy ra giữa	Nhiều phương thức cùng một class	Lớp con và lớp cha (hai class khác nhau)
Chữ ký (tên + tham số)	Phải khác nhau	Phải giống hệt nhau
Chọn phiên bản nào chạy	Lúc biên dịch, dựa trên đối số truyền vào	Lúc chạy, dựa trên object thực sự đang gọi (sẽ hiểu sâu ở Chương 17 — đa hình)

Bài tập

- Tạo class `DongVat` với field `ten`, phương thức `keu()` in ra `"..."` (âm thanh chung chung). Tạo `Cho` extends `DongVat` và `Meo` extends `DongVat`, mỗi lớp override `keu()` in ra âm thanh riêng (`"Gau gau!"`, `"Meo meo!"`).
- Thêm vào `NhanVien` (code mẫu) một phương thức `thuong(double soTien)` cộng thêm tiền thưởng một lần vào `luongCoBan`; xác nhận `NhanVienBanHang` và `NhanVienQuanLy` đều dùng được phương thức này mà không cần viết lại.
- Thử xoá dòng `super(ten, luongCoBan)`; trong `NhanVienBanHang`, biên dịch lại và đọc lỗi — giải thích vì sao lỗi xảy ra dựa trên phần `"super(...)"` ở trên.

Lỗi thường gặp

- Quên gọi `super(...)` khi lớp cha không có constructor không tham số** — lỗi biên dịch `constructor NhanVien in class NhanVien cannot be applied to given types` hoặc tương tự. Luôn gọi `super(...)` tường minh với đúng tham số khi lớp cha không có constructor rỗng.
- Sai chữ ký khi định tâm override, vô tình tạo ra overload** — ví dụ viết `public double tinhLuong(int x)` (thêm tham số) thay vì `tinhLuong()` — đây không còn là override, mà là một phương thức hoàn toàn khác. Dùng `@Override` để trình biên dịch tự phát hiện lỗi này.
- Field `private` ở lớp cha, lớp con tưởng dùng được trực tiếp** — `private` không kế thừa xuống được, kể cả lớp con. Cần `protected` (hoặc getter/setter `public`) nếu lớp con cần truy cập trực tiếp.
- Java không hỗ trợ đa kế thừa class** (một class chỉ extends được **đúng một** class cha, khác với một số ngôn ngữ khác cho phép kế thừa nhiều class cùng lúc) — muốn "kế thừa" nhiều nguồn hành vi khác nhau, dùng `interface` (Chương 18) thay vì cố tìm cách kế thừa nhiều class.

Chương 17 — Đa hình (Polymorphism)

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu đa hình là gì và tại sao nó là một trong những lý do OOP mạnh mẽ.
- Phân biệt **upcasting** (tự động, an toàn) và **downcasting** (thủ công, có rủi ro).
- Dùng `instanceof` để kiểm tra kiểu thực sự của object trước khi downcasting.
- Hiểu **liên kết động (dynamic binding)** — cơ chế Java quyết định gọi phiên bản phương thức nào lúc **chạy**, dựa trên kiểu **thực sự** của object, không phải kiểu khai báo của biến.

Code mẫu đầy đủ: [code/ch17-da-hinh/](#).

Đa hình là gì?

Tiếp tục ví dụ `NhanVien` / `NhanVienBanHang` / `NhanVienQuanLy` từ Chương 16. **Đa hình** (polymorphism — tiếng Hy Lạp nghĩa là "nhiều hình dạng") là khả năng xử lý nhiều loại object khác nhau **thông qua cùng một kiểu tham chiếu chung** (ở đây: `NhanVien`), gọi **cùng một phương thức** (`tinhLuong()`), nhưng mỗi object **tự chạy đúng logic riêng của loại thật sự của nó**:

```
NhanVien[] danhSach = {
    new NhanVien("A", 10_000_000),
    new NhanVienBanHang("B", 8_000_000, 100_000_000),
    new NhanVienQuanLy("C", 15_000_000, 3_000_000)
};

for (NhanVien nv : danhSach) {
    System.out.println(nv.getTen() + ": " + nv.tinhLuong());
}
```

Vòng lặp trên **không cần biết** (và không cần quan tâm) mỗi phần tử thực sự là loại nhân viên nào — nó chỉ gọi `tinhLuong()` một cách thống nhất, và mỗi object tự "biết" cách tính lương đúng theo bản chất riêng của nó. Đây chính là giá trị thực tiễn lớn nhất của đa hình: viết code xử lý chung **một lần**, hoạt động đúng cho **mọi** loại con hiện tại lẫn **mọi loại con sẽ được thêm sau này** (thêm `NhanVienThucTap` extends `NhanVien` mới, vòng lặp trên vẫn chạy đúng mà không cần sửa gì).

Upcasting — tự động, luôn an toàn

```
NhanVien nv = new NhanVienBanHang("B", 8_000_000, 100_000_000);
```

Gán một object lớp con cho biến khai báo kiểu lớp cha gọi là **upcasting** ("đi lên" theo cây kế thừa). Việc này **luôn hợp lệ và tự động** — logic: một `NhanVienBanHang` chắc chắn là một `NhanVien` (nó kế thừa mọi thứ của `NhanVien`), nên gán được cho biến kiểu `NhanVien` không có gì rủi ro.

Hệ quả quan trọng: biến `nv` (kiểu khai báo `NhanVien`) chỉ "nhìn thấy" được các phương thức **định nghĩa trong** `NhanVien` — dù bản chất thật sự của object là `NhanVienBanHang`, bạn **không gọi được trực tiếp** `nv.capNhatDoanhSo(...)` (phương thức chỉ có ở `NhanVienBanHang`) qua biến `nv` — trình biên dịch báo lỗi, vì nó chỉ xét **kiểu khai báo của biến**, không xét bản chất thật sự của object lúc chạy.

Downcasting — thủ công, cần kiểm tra trước

Muốn gọi được phương thức riêng của lớp con qua một biến đang khai báo kiểu lớp cha, phải **downcasting** ("đi xuống") — ép kiểu tưởng mình:

```
if (nv instanceof NhanVienBanHang) {
    NhanVienBanHang nvbh = (NhanVienBanHang) nv;
    nvbh.capNhatDoanhSo(150_000_000);
}
```

`instanceof` kiểm tra xem object mà `nv` đang trỏ tới **có thực sự là** (hoặc là lớp con của) `NhanVienBanHang` hay không, trả về `boolean`. **Luôn kiểm tra `instanceof` trước khi downcasting** — nếu bạn ép kiểu sai (object thật sự không phải loại đó), chương trình ném `ClassCastException` lúc **chạy**:

```
NhanVien nv1 = new NhanVien("A", 10_000_000);
NhanVienBanHang sai = (NhanVienBanHang) nv1; // ClassCastException! nv1 KHÔNG phải NhanVienBanHang
```

Pattern matching cho instanceof (Java 16+)

Cú pháp hiện đại gộp kiểm tra và ép kiểu thành một bước, tránh phải khai báo biến ép kiểu riêng:

```
if (nv instanceof NhanVienBanHang nvbh) {
    // bên trong khối if này, 'nvbh' đã sẵn sàng dùng, KIEU NhanVienBanHang
    nvbh.capNhatDoanhSo(150_000_000);
}
```

Sách sẽ dùng lại cú pháp này nhiều hơn ở Chương 33 (Java hiện đại) — ở đây chỉ cần biết nó tồn tại và làm đúng việc gì.

Liên kết động (dynamic binding) — vì sao đa hình hoạt động được

Khi bạn gọi `nv.tinhLuong()`, Java **không** quyết định ngay lúc biên dịch phiên bản `tinhLuong()` nào sẽ chạy dựa trên kiểu khai báo (`NhanVien`) của biến `nv`. Thay vào đó, **lúc chạy**, JVM nhìn vào **object thực sự** mà `nv` đang trỏ tới, và gọi đúng phiên bản `tinhLuong()` đã được **override** ở lớp thực sự của object đó (`NhanVienBanHang`, `NhanVienQuanLy`, hay giữ nguyên bản gốc của `NhanVien` nếu không override). Cơ chế "quyết định lúc chạy, dựa trên object thật" này gọi là **liên kết động (dynamic binding)** hay **virtual method invocation** — đây chính là cơ chế nền tảng khiến vòng lặp `for (NhanVien nv : danhSach)` ở phần đầu chương hoạt động đúng cho từng loại nhân viên khác nhau mà không cần bất kỳ `if/else` hay `instanceof` nào để phân biệt loại.

So sánh với overloading (Chương 10): overloading được quyết định **lúc biên dịch** (dựa trên kiểu tham số bạn viết trong code), còn override/đa hình được quyết định **lúc chạy** (dựa trên object thực tế) — đây là khác biệt cốt lõi khiến đa hình linh hoạt và mạnh hơn nhiều so với chỉ overloading.

Bài tập

- Tiếp nối bài tập Chương 16 (`DongVat`, `Cho`, `Meo`), tạo mảng `DongVat[]` chứa cả hai loại, viết vòng lặp gọi `keu()` trên từng phần tử — quan sát đa hình hoạt động.
- Thêm class `NhanVienThucTap` extends `NhanVien` (không có gì đặc biệt, dùng nguyên `tinhLuong()` của lớp cha), thêm vào mảng `danhSachNhanVien` trong `Main.java`, chạy lại và xác nhận vòng lặp vẫn hoạt động đúng **mà không cần sửa bất kỳ dòng nào khác**.
- Bỏ comment dòng `NhanVienBanHang epSai = (NhanVienBanHang) nv1;` trong `Main.java`, chạy và đọc kỹ thông báo `ClassCastException` — nó cho biết đang cố ép object thuộc lớp nào sang lớp nào.

Lỗi thường gặp

- `ClassCastException` — downcasting sai loại thực sự của object. Luôn `instanceof` trước khi ép kiểu, hoặc dùng pattern matching `instanceof` để gộp làm một bước an toàn.
- Tưởng gọi được phương thức riêng của lớp con qua biến khai báo kiểu lớp cha** — trình biên dịch chỉ xét kiểu khai báo của biến, không xét object thật sự lúc chạy. Cần downcasting trước.

- **Nhầm lẫn thời điểm quyết định: overloading là lúc biên dịch, override/đa hình là lúc chạy** — đọc lại phần "Liên kết động" nếu còn mơ hồ, đây là khái niệm nền tảng cho toàn bộ phần OOP nâng cao còn lại.
- **Lạm dụng instanceof + downcasting để giả lập đa hình bằng if/else dài dằng dặc** — nếu bạn thấy mình viết `if (x instanceof A) ... else if (x instanceof B) ...` lặp đi lặp lại ở nhiều nơi để xử lý khác nhau theo từng loại, đó thường là dấu hiệu nên **override phương thức** ở từng lớp con thay vì kiểm tra kiểu thủ công — để chính cơ chế đa hình tự làm việc đó cho bạn.

Chương 18 — Abstract class & Interface

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết được abstract class với phương thức `abstract` (bắt buộc override) trộn cùng phương thức thường (dùng chung, kế thừa nguyên).
- Viết được interface, hiểu `default method` và `static method` trong interface (Java 8+).
- Biết khi nào chọn abstract class, khi nào chọn interface.
- Hiểu vì sao một class `implements` được nhiều interface, trong khi chỉ `extends` được một class.

Code mẫu đầy đủ: [code/ch18-abstract-interface/](#).

Abstract class — lớp cha "chưa hoàn chỉnh" có chủ đích

Ở Chương 16-17, `NhanVien` là một class **hoàn chỉnh**, `new NhanVien(...)` tạo object bình thường được. Nhưng có những trường hợp lớp cha **chỉ nên tồn tại để kế thừa**, không nên tạo object trực tiếp — ví dụ khái niệm "hình học" chung chung: không có object nào thực sự là "một hình học" trần trụi, chỉ có hình tròn, hình chữ nhật... cụ thể. `abstract class` diễn đạt đúng ý định đó:

```
public abstract class HìnhHoc {
    protected String tenHinh;

    public HìnhHoc(String tenHinh) {
        this.tenHinh = tenHinh;
    }

    public abstract double tinhDienTich(); // KHÔNG có thân hàm
    public abstract double tinhChuVi();   // KHÔNG có thân hàm

    public void inThongTin() {             // CÓ thân hàm đầy đủ
        System.out.println(tenHinh + ": diện tích = " + tinhDienTich());
    }
}
```

Hai điểm quan trọng:

- `abstract class HìnhHoc { ... }` **không** `new` **trực tiếp được** — `new HìnhHoc("test")` là lỗi biên dịch. Chỉ tạo object được từ **lớp con cụ thể** (`HìnhTron`, `HìnhChuNhat`) đã override đầy đủ mọi phương thức `abstract`.
- Abstract class **trộn lẫn được** phương thức `abstract` (không thân hàm, bắt buộc lớp con override — như `tinhDienTich()`) với phương thức thường có sẵn thân hàm (như `inThongTin()`), lớp con kế thừa nguyên, dùng chung không cần viết lại. Đây là điểm khác biệt quan trọng so với interface kiểu cũ (xem phần dưới).

Lớp con kế thừa abstract class **bắt buộc phải override tất cả** phương thức `abstract` của nó (trừ khi lớp con cũng là abstract class), nếu không sẽ bị lỗi biên dịch — trình biên dịch **ép bạn không được quên** implement bất kỳ hành vi bắt buộc nào, khác với chỉ dùng class thường và "hy vọng" lớp con nhớ override.

Interface — "hợp đồng" thuần túy

```
public interface CoTheVe {
    void ve(); // interface tu hieu day la abstract, khong can ghi tu khoa 'abstract'

    default void veDamNet() {
        System.out.println("=== VE DAM NET ===");
    }
}
```

```

        ve();
        System.out.println("=== HET ===");
    }

    static void inHuongDan() {
        System.out.println("Goi ve() de ve hình...");
    }
}

```

Interface định nghĩa "**cái gì phải làm được**" mà không quan tâm "làm bằng cách nào" — mọi phương thức khai báo bình thường trong interface (`void ve();`) mặc định là `abstract` , **không có thân hàm**, và class `implements` interface đó **bắt buộc** phải cung cấp phần thân hàm cụ thể.

default method (Java 8+)

```

default void veDamNet() {
    System.out.println("=== VE DAM NET ===");
    ve();
    System.out.println("=== HET ===");
}

```

Khác với phương thức `abstract` thông thường của interface, `default method` **có sẵn thân hàm ngay trong interface**. Mọi class `implements` interface này **tự động kế thừa** hành vi mặc định đó, **không bắt buộc** phải override (nhưng vẫn override được nếu muốn hành vi khác). Tính năng này được thêm vào Java 8 chủ yếu để **mở rộng interface có sẵn** mà không phá vỡ code cũ đang implement interface đó (nếu thêm một phương thức `abstract` mới vào interface đã tồn tại, mọi class cũ `implements` nó sẽ lỗi biên dịch ngay; thêm `default method` thì không).

static method trong interface

```

static void inHuongDan() {
    System.out.println("...");
}

```

Gọi trực tiếp qua tên interface (`CoTheVe.inHuongDan();`), tương tự phương thức `static` của class đã học ở Chương 10 — không gắn với object cụ thể nào, thường dùng cho các hàm tiện ích liên quan tới interface đó.

Đa kế thừa interface — điều `extends` không cho phép

```

public class HìnhTron extends HìnhHoc implements CoTheVe, CoTheSoSanhDienTich {
    // ...
}

```

Nhắc lại từ Chương 16: một class chỉ `extends` được **đúng một** class cha. Nhưng một class `implements` **được nhiều interface cùng lúc**, phân cách bằng dấu phẩy. Đây là cách Java cho phép một object "vừa là cái này, vừa là cái kia" theo nhiều khía cạnh độc lập — `HìnhTron` vừa **là một** `HìnhHoc` (qua kế thừa), vừa **có khả năng** `CoTheVe` (vẽ được), vừa **có khả năng** `CoTheSoSanhDienTich` (so sánh diện tích được) — ba khía cạnh này độc lập với nhau, không có quan hệ cha-con giữa chúng.

Khi nào chọn `abstract class`, khi nào chọn `interface`?

Tiêu chí	Abstract class	Interface
Số lượng kế thừa/implements cùng lúc	Chỉ 1	Nhiều

Tiêu chí	Abstract class	Interface
Có field lưu trạng thái (như <code>tenHinh</code>)	Có	Không (interface không có field lưu trạng thái thực thụ, chỉ có hằng số <code>public static final</code> ngầm định)
Có phương thức đã có sẵn thân hàm, dùng chung	Có (phương thức thường)	Có (từ Java 8, qua <code>default</code>)
Ý nghĩa diễn đạt	"X LÀ MỘT loại Y" (quan hệ chặt, is-a)	"X CÓ KHẢ NĂNG làm được Z" (quan hệ lỏng hơn, capability)

Quy tắc thực dụng cho người mới: nếu các lớp con chia sẻ **dữ liệu chung** và có quan hệ "là một loại của nhau" rõ ràng (như mọi hình học đều có `tenHinh`), dùng `abstract class`. Nếu bạn chỉ muốn đảm bảo nhiều class **khác nhau, không liên quan họ hàng** đều có một khả năng chung (vẽ được, so sánh được, đọc/ghi được...), dùng `interface`.

Bài tập

- Tạo `abstract class` `PhuongTien` với field `bienSo`, phương thức `abstract dichChuyen()`; tạo `XeMay` và `OToTruong` `extends PhuongTien` (dùng tên khác `OTo` để tránh trùng nếu bạn từng đặt tên đó trước đó), mỗi lớp `override dichChuyen()` in ra cách di chuyển riêng.
- Tạo `interface` `CoTheSac` với phương thức `int soChoNgoi()`, cho cả `XeMay` và `OToTruong` cùng `implements` `interface` này (cùng với `extends PhuongTien`), xác nhận một class vừa `extends` vừa `implements` được cùng lúc.
- Thêm một `default method` vào `CoTheSac` in ra "Xe có X chỗ ngồi" dựa trên `soChoNgoi()`, xác nhận cả `XeMay` và `OToTruong` dùng chung được `default method` này mà không cần viết lại.

Lỗi thường gặp

- Cố `new` trực tiếp một `abstract class`** — lỗi biên dịch `HinhHoc is abstract; cannot be instantiated`. Chỉ tạo object được từ lớp con cụ thể đã `override` đầy đủ phương thức `abstract`.
- Quên `override` một phương thức `abstract` khi kế thừa/`implements`** — lỗi biên dịch báo rõ tên phương thức còn thiếu, class con phải `override` đủ **tất cả** phương thức `abstract` của lớp cha/ `interface` (hoặc chính nó cũng phải khai báo `abstract`).
- Cố `extends` hai class cùng lúc** (`class X extends A, B`) — Java không hỗ trợ, đây là lỗi cú pháp. Nếu cần kết hợp nhiều nguồn hành vi, dùng `implements` nhiều `interface` thay vì cố kế thừa nhiều class.
- Nhầm `interface` có thể lưu trạng thái như field bình thường** — mọi biến khai báo trong `interface` mặc định là `public static final` (hằng số dùng chung cho mọi class `implements`, không phải dữ liệu riêng từng object) — nếu cần dữ liệu riêng từng object, phải dùng (`abstract`) class, không phải `interface`.

Chương 19 — Lớp lồng, anonymous class, enum nâng cao

Mục tiêu học

Sau chương này, bạn sẽ:

- Viết được **inner class** (lớp lồng không static) và **static nested class**, hiểu khác biệt.
- Dùng **anonymous class** để tạo object implements interface "ngay tại chỗ", không cần đặt tên class riêng.
- Viết enum có field, constructor, phương thức riêng — vượt xa enum "danh sách tên suông" bạn có thể đã gặp ở nơi khác.
- Viết enum với phương thức khác nhau cho từng hằng số (constant-specific class body).

Code mẫu đầy đủ: [code/ch19-nested-enum/](#).

Lớp lồng (nested class) là gì?

Java cho phép khai báo một class **bên trong** một class khác, khi class đó **chỉ có ý nghĩa gắn liền** với class chứa nó — không cần tồn tại độc lập bên ngoài. Có hai loại chính:

Static nested class — độc lập, chỉ "đóng gói" cho gọn

```
public class DoiBongDa {
    public static class ThongKe {
        public int tongSoTran;
        public int tongBanThang;

        public ThongKe(int tongSoTran, int tongBanThang) {
            this.tongSoTran = tongSoTran;
            this.tongBanThang = tongBanThang;
        }
    }
}
```

`static class ThongKe` hoạt động gần như một class bình thường, độc lập hoàn toàn với bất kỳ object `DoiBongDa` nào — chỉ khác là tên đầy đủ của nó là `DoiBongDa.ThongKe`, và bạn khai báo nó bên trong `DoiBongDa` đơn thuần vì nó **có ý nghĩa liên quan chặt** tới `DoiBongDa` (thống kê của một đội bóng), giúp code dễ tìm và tổ chức hơn. Tạo object: `new DoiBongDa.ThongKe(10, 18)`.

Inner class (không static) — luôn gắn liền một object outer cụ thể

```
public class DoiBongDa {
    private String tenDoi;

    public class CauThu {
        private String ten;

        public void gioiThieu() {
            System.out.println(ten + " - doi " + tenDoi); // truy cap TRUC TIEP field outer
        }
    }
}
```

Khác với static nested class, **inner class luôn gắn liền với một object cụ thể** của class ngoài (gọi là "outer instance") — bên trong `CauThu`, bạn dùng được trực tiếp field `tenDoi` của `DoiBongDa`, dù không truyền nó vào constructor `CauThu` — Java tự "nhớ" `CauThu` này thuộc về object `DoiBongDa` nào. Muốn tạo inner class **từ bên ngoài** class chứa nó, cú pháp cần một object outer đứng trước:

```
DoiBongDa doi = new DoiBongDa("FC Java");
DoiBongDa.CauThu ct = doi.new CauThu("Le Van C", 9); // outerObject.new InnerClass(...)
```

Khi nào dùng loại nào? Nếu class lồng cần truy cập dữ liệu của outer instance và luôn "thuộc về" một outer cụ thể (như `CauThu` luôn thuộc về một `DoiBongDa`), dùng inner class. Nếu class lồng không cần gì từ outer instance, chỉ đơn thuần liên quan chủ đề, dùng static nested class — đây là lựa chọn phổ biến hơn trong thực tế vì đơn giản hơn.

Anonymous class — tạo object "ngay tại chỗ"

```
HanhDong chao = new HanhDong() {
    @Override
    public void thucHien() {
        System.out.println("Xin chao tu anonymous class!");
    }
};
chao.thucHien();
```

Anonymous class (lớp vô danh) tạo ra **một object duy nhất** implements một interface (hoặc kế thừa một class) **ngay tại nơi khai báo**, không cần đặt tên class riêng ở đâu khác. Cú pháp: `new <Interface hoặc Class>() { <phần thân override> }`.

Dùng phổ biến nhất khi bạn cần một hành vi **chỉ dùng đúng một lần**, không đáng để tạo hẳn một class riêng có tên:

```
static void thucHienHanhDong(HanhDong hd) {
    hd.thucHien();
}

// gọi trực tiếp với một anonymous class làm đối số:
thucHienHanhDong(new HanhDong() {
    @Override
    public void thucHien() {
        System.out.println("Hành động được truyền trực tiếp vào tham số!");
    }
});
```

Ghi chú hướng tới sau này: từ Chương 30, bạn sẽ học **lambda expression** — cú pháp Java hiện đại hơn nhiều để thay thế phần lớn trường hợp dùng anonymous class implements interface chỉ có **một phương thức** (như `HanhDong` ở đây). Chương này dạy anonymous class trước vì (1) nó vẫn xuất hiện rất nhiều trong code Java hiện có, và (2) hiểu anonymous class giúp lambda ở Chương 30 trở nên dễ hiểu hơn nhiều (lambda gần như chỉ là "anonymous class viết gọn hơn" cho đúng một trường hợp riêng).

Enum nâng cao: field, constructor, phương thức riêng

Bạn có thể đã thấy enum dạng đơn giản (`enum NgayTrongTuan { THU_HAI, THU_BA, ... }`). Enum trong Java thực chất **mạnh hơn nhiều** — về bản chất, mỗi hằng số enum **là một object**, và bản thân `enum` là một dạng class đặc biệt, có thể có field, constructor, phương thức như class bình thường:

```
public enum HanhTinh {
    TRAI_DAT(5.976e24, 6.37814e6),
    SAO_HOA(6.421e23, 3.3972e6),
    SAO_MOC(1.9e27, 7.1492e7);

    private final double khoiLuong;
    private final double banKinh;

    HanhTinh(double khoiLuong, double banKinh) {
        this.khoiLuong = khoiLuong;
    }
}
```

```

        this.bankKinh = bankKinh;
    }

    public double tinhTrongLuongBeMat() {
        return 6.673E-11 * khoiLuong / (bankKinh * bankKinh);
    }
}

```

Mỗi hằng số (`TRAI_DAT` , `SAO_HOA` ...) tự gọi **constructor tương ứng đúng một lần** khi enum được nạp. Constructor của enum **luôn ngầm định private** — không có cách nào tự `new HanhTinh(...)` từ bên ngoài; danh sách hằng số khai báo sẵn chính là **toàn bộ** các object của enum này tồn tại trong suốt chương trình.

Gọi phương thức: `HanhTinh.TRAI_DAT.tinhTrongLuongBeMat()` . Duyệt qua mọi hằng số bằng `HanhTinh.values()` (trả về mảng chứa tất cả hằng số, theo đúng thứ tự khai báo).

Enum với phương thức riêng cho từng hằng số (constant-specific class body)

```

public enum PhuongThucThanhToan {
    TIEN_MAT {
        @Override
        public void xuLy(double soTien) {
            System.out.println("Nhan " + soTien + " tien mat.");
        }
    },
    THE_TIN_DUNG {
        @Override
        public void xuLy(double soTien) {
            double phi = soTien * 0.02;
            System.out.println("Tru the " + (soTien + phi));
        }
    };

    public abstract void xuLy(double soTien);
}

```

Khi mỗi hằng số cần **hành vi hoàn toàn khác nhau** cho cùng một phương thức, thay vì viết một phương thức chung chứa `if/else` / `switch` dài dằng dặc kiểm tra "hằng số nào đang gọi", mỗi hằng số tự cung cấp **thân hàm riêng** ngay sau tên nó. Đây là một cách dùng enum khá nâng cao nhưng rất phổ biến trong code thực tế — bạn sẽ nhận ra nó là một dạng cụ thể của **đa hình** (Chương 17) áp dụng cho enum: gọi `PhuongThucThanhToan.TIEN_MAT.xuLy(...)` và `THE_TIN_DUNG.xuLy(...)` giống hệt cách gọi, nhưng mỗi hằng số tự chạy đúng logic riêng.

Bài tập

1. Thêm static nested class `DoiBongDa.HuanLuyenVien` với field `ten` , `soNamKinhNghiem` .
2. Viết enum `MucDoUuTien` gồm `THAP` , `TRUNG_BINH` , `CAO` , mỗi hằng số có field `heSoCanhBao` (int), phương thức `hienThiCanhBao()` in ra thông báo khác nhau tùy mức độ.
3. Viết một anonymous class implements `Comparator` -giống-tự-định-nghĩa (tạo interface riêng `SoSanh` { `int soSanh(int a, int b);` }), truyền vào một phương thức nhận tham số kiểu đó để sắp xếp một mảng `int[]` theo thứ tự giảm dần (gợi ý: viết thuật toán sắp xếp đơn giản như bubble sort, gọi `soSanh` để quyết định thứ tự thay vì so sánh `<` / `>` trực tiếp).

Lỗi thường gặp

- **Cố `new InnerClass(...)` trực tiếp mà không qua object outer** — inner class (không static) **luôn** cần một outer instance đứng trước: `outerObject.new InnerClass(...)` , hoặc tạo từ bên trong chính outer class như code mẫu.
- **Dùng inner class khi không thực sự cần truy cập dữ liệu outer** — gây phức tạp không cần thiết (bắt buộc phải có outer instance mới tạo được). Nếu class lồng không cần gì từ outer, dùng `static class` sẽ đơn giản hơn.

- **Cố gọi** `new PhươngThucThanhToan(...)` — enum không bao giờ `new` được từ bên ngoài, kể cả khi có constructor với tham số. Danh sách hằng số khai báo sẵn là cách duy nhất có object của enum đó.
- **Quên** `abstract` **cho phương thức chung khi dùng `constant-specific class body`** — nếu enum có phương thức mà **mọi** hằng số phải tự override riêng (như `xuLy` ở trên), phương thức đó phải khai báo `abstract` trong thân enum, nếu không trình biên dịch không bắt buộc từng hằng số phải cung cấp thân hàm riêng.

Chương 20 — Lớp Object: equals/hashCode/toString

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu mọi class trong Java đều ngầm kế thừa từ `Object`, và biết 3 phương thức quan trọng nhất nó cung cấp: `toString()`, `equals()`, `hashCode()`.
- Override `toString()` để in object có ý nghĩa.
- Override `equals()` và `hashCode()` đúng quy tắc để so sánh object theo **nội dung**.
- Phân biệt `Comparable` (thứ tự tự nhiên, định nghĩa trong chính class) và `Comparator` (thứ tự tùy chọn, định nghĩa riêng bên ngoài).

Code mẫu đầy đủ: <code/ch20-object-class/>.

Chương này dùng cú pháp generics (`Comparable<SanPham>`) trước khi Chương 22 dạy đầy đủ về generics — ở đây chỉ cần hiểu `<SanPham>` nghĩa là "phiên bản dành riêng cho kiểu `SanPham`", chưa cần hiểu cơ chế bên trong.

Mọi class đều kế thừa từ Object

Bạn chưa từng viết `class SanPham extends Object`, nhưng thực tế **mọi class trong Java**, nếu không `extends` class nào khác, **tự động kế thừa từ class `Object`** — lớp cha chung của toàn bộ hệ thống kiểu trong Java. `Object` cung cấp sẵn một số phương thức mà mọi object nào cũng có, quan trọng nhất: `toString()`, `equals(Object)`, `hashCode()`.

toString() — biểu diễn object dưới dạng chuỗi

Nhắc lại từ Chương 9: in một mảng trực tiếp cho ra chuỗi vô nghĩa như `[I@1b6d3586`. Chuyện tương tự xảy ra với object thường:

```
SanPham sp = new SanPham("But bi", 5000);
System.out.println(sp); // mac dinh: SanPham@1b6d3586 - vo nghia
```

Đó là hành vi **mặc định** của `toString()` kế thừa từ `Object` — nó in ra tên class cộng một mã định danh bộ nhớ, không nói lên gì về nội dung object. **Override lại** để có kết quả hữu ích:

```
@Override
public String toString() {
    return "SanPham{ten='" + ten + "', gia=" + gia + "}";
}
```

Sau khi override, `System.out.println(sp)` và cả nối chuỗi `"" + sp` **tự động gọi** `sp.toString()` — bạn không bao giờ cần gọi `.toString()` tường minh trong những trường hợp này, Java tự làm điều đó.

equals() — so sánh nội dung, đúng cách

Nhắc lại từ Chương 11 và 13: `==` với kiểu tham chiếu so sánh **có phải cùng một object trong bộ nhớ hay không**. Hành vi mặc định của `equals()` kế thừa từ `Object` cũng làm **đúng như vậy** (tương đương `==`) — nếu bạn không override, `equals()` không giúp gì hơn `==` cả. Muốn so sánh theo **nội dung**, phải tự override:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
```

```

    if (o == null || getClass() != o.getClass()) return false;
    SanPham sp = (SanPham) o;
    return Double.compare(sp.gia, gia) == 0 && Objects.equals(ten, sp.ten);
}

```

Khuôn mẫu chuẩn cần nhớ:

1. **Kiểm tra tham chiếu giống hệt** (`this == o`) — nếu đúng, chắc chắn bằng nhau, trả `true` ngay không cần so sánh gì thêm (tối ưu, cũng đúng logic).
2. **Kiểm tra null và kiểu** (`o == null || getClass() != o.getClass()`) — nếu `o` là `null` hoặc thuộc class khác hoàn toàn, chắc chắn không bằng nhau.
3. **Ép kiểu** (`(SanPham) o`) — an toàn vì bước 2 đã đảm bảo `o` đúng là `SanPham`.
4. **So sánh từng field có ý nghĩa** — dùng `Objects.equals(a, b)` cho field kiểu tham chiếu (tự xử lý an toàn cả trường hợp `null`, tương đương `a == null ? b == null : a.equals(b)` nhưng ngắn gọn hơn và không có nguy cơ `NullPointerException`), dùng `Double.compare / ==` trực tiếp cho field kiểu nguyên thủy.

hashCode() — luôn override cùng equals()

```

@Override
public int hashCode() {
    return Objects.hash(ten, gia);
}

```

Quy tắc bắt buộc, không có ngoại lệ: nếu `a.equals(b)` trả về `true`, thì **bắt buộc** `a.hashCode() == b.hashCode()` cũng phải đúng. Vì sao quy tắc này quan trọng: các cấu trúc dữ liệu như `HashMap`, `HashSet` (sẽ học ở Chương 25-26) dùng `hashCode()` để **tim nhanh** object trong tập hợp, rồi mới dùng `equals()` để xác nhận chính xác — nếu hai object "bằng nhau" theo `equals()` nhưng có `hashCode()` khác nhau, các cấu trúc dữ liệu đó sẽ **hoạt động sai** (không tìm thấy object lẽ ra phải tìm thấy), dù code biên dịch và chạy không báo lỗi gì. Đây là lỗi rất khó phát hiện nếu không biết trước quy tắc này.

Ghi nhớ: `equals()` và `hashCode()` **luôn đi thành cặp** — override một trong hai mà không override cái kia gần như luôn là lỗi.

Comparable — thứ tự tự nhiên, định nghĩa trong chính class

```

public class SanPham implements Comparable<SanPham> {
    @Override
    public int compareTo(SanPham kia) {
        return Double.compare(this.gia, kia.gia);
    }
}

```

`Comparable<T>` là interface chuẩn của Java, chỉ có một phương thức `compareTo(T kia)` — trả về số **âm** nếu object hiện tại "nhỏ hơn" `kia`, số **dương** nếu "lớn hơn", `0` nếu "bằng nhau" (theo tiêu chí bạn tự định nghĩa). Khi class implements `Comparable`, các phương thức thư viện chuẩn như `Arrays.sort(mang)` **tự động dùng** `compareTo()` này để sắp xếp mà không cần bạn chỉ định gì thêm — đây gọi là **thứ tự tự nhiên (natural ordering)** của kiểu đó (giống cách `Arrays.sort` trên `int[]` tự hiểu sắp xếp tăng dần theo giá trị số).

Comparator — thứ tự tùy chọn, định nghĩa riêng bên ngoài

Vấn đề: `SanPham` chỉ có **một** thứ tự tự nhiên (theo giá). Nếu đôi khi bạn cần sắp theo **tên** thay vì giá, không thể sửa `compareTo()` (chỉ được định nghĩa một lần duy nhất trong class). Giải pháp: `Comparator<T>` — một interface **riêng biệt**, định nghĩa thứ tự **bên ngoài** class, có thể tạo nhiều `Comparator` khác nhau cho cùng một kiểu:

```

Comparator<SanPham> theoTen = new Comparator<SanPham>() {
    @Override
    public int compare(SanPham sp1, SanPham sp2) {

```

```
        return sp1.getTen().compareTo(sp2.getTen());
    }
};
Arrays.sort(danhSach, theoTen); // truyen THEM Comparator, ghi de thu tu tu nhien
```

`String` cũng implements `Comparable<String>` sẵn (thứ tự tự nhiên: theo bảng chữ cái/mã Unicode), đó là lý do `sp1.getTen().compareTo(sp2.getTen())` gọi được trực tiếp.

Tóm tắt khác biệt: `Comparable` — class **tự định nghĩa** đúng **một** thứ tự tự nhiên của chính nó (implements trong class đó). `Comparator` — thứ tự **tuỳ chọn**, định nghĩa **bên ngoài**, có thể có **nhều Comparator khác nhau** cho cùng một kiểu, không cần sửa class gốc. Chương 27 sẽ học sâu hơn về `Comparator` (kể cả cách viết ngắn gọn hơn nhiều bằng lambda từ Chương 30, thay vì anonymous class dài dòng như ở đây).

Bài tập

1. Thêm `toString()`, `equals()`, `hashCode()` cho class `SanPham` / `SinhVien` bạn đã viết ở các chương trước (Chương 13/14), theo đúng khuôn mẫu trong chương này.
2. Cho class đó implements `Comparable`, định nghĩa một thứ tự tự nhiên hợp lý.
3. Viết thêm ít nhất một `Comparator` khác cho cùng class đó, sắp xếp theo tiêu chí khác thứ tự tự nhiên.
4. Giải thích bằng lời: vì sao nếu chỉ override `equals()` mà quên override `hashCode()`, một `HashSet` (sẽ học ở Chương 25) có thể chứa **hai object trùng lặp** dù `equals()` giữa chúng trả về `true`? (Chưa cần code, chỉ cần suy luận dựa trên phần "`hashCode()`" vừa học — sẽ kiểm chứng lại bằng code thật ở Chương 25.)

Lỗi thường gặp

- **Override `equals()` mà quên override `hashCode()`** (hoặc ngược lại) — vi phạm quy tắc bắt buộc, gây lỗi khó phát hiện khi dùng với `HashMap` / `HashSet`. IDE hiện đại (như IntelliJ) thường cảnh báo ngay khi bạn chỉ override một trong hai.
- **So sánh `o == null || getClass() != o.getClass()` bị bỏ sót** trong `equals()` — dẫn tới `NullPointerException` hoặc `ClassCastException` khi so sánh với `null` hoặc object khác kiểu.
- **Dùng `==` thay vì `Double.compare` / `.equals()` khi so sánh field kiểu `double` / `float` trong `equals()`** — số thực có các trường hợp đặc biệt (như `NaN`) khiến `==` cho kết quả không trực quan; `Double.compare` xử lý đúng các trường hợp đó.
- **Quên `@Override` khi override `equals(Object o)`, vô tình viết `equals(SanPham o)`** — đây không còn là override `Object.equals`, mà là một phương thức **overload** hoàn toàn khác (nhắc lại phân biệt overriding/overloading ở Chương 16) — các thư viện chuẩn gọi `equals(Object)` sẽ không bao giờ dùng tới phiên bản overload sai này. Luôn dùng `@Override` để trình biên dịch bắt lỗi này ngay.

Chương 21 — Exception handling

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng `try/catch/finally` để bắt và xử lý lỗi runtime, thay vì để chương trình crash đột ngột.
- Phân biệt **checked exception** và **unchecked exception**, hiểu vì sao Java tách biệt hai loại.
- Tạo được exception tự định nghĩa (custom exception) cho lỗi nghiệp vụ riêng của chương trình.
- Dùng `try-with-resources` để tự động giải phóng tài nguyên đúng cách.

Code mẫu đầy đủ: [code/ch21-exception-handling/](#).

Exception là gì, vì sao cần xử lý?

Từ các chương trước, bạn đã nhiều lần gặp chương trình "crash" giữa chừng: `ArrayIndexOutOfBoundsException` (Chương 9), `NullPointerException` (Chương 13), `ClassCastException` (Chương 17)... Khi những lỗi này xảy ra mà không được xử lý, JVM in ra "stack trace" (dấu vết lỗi) rồi **dừng chương trình ngay lập tức**. Với chương trình thực tế (ví dụ server chạy liên tục phục vụ nhiều người dùng), để một lỗi nhỏ làm sập toàn bộ chương trình là không chấp nhận được — **exception handling** là cơ chế cho phép bạn "bắt" những lỗi này và quyết định cách phản ứng phù hợp, thay vì để chương trình chết.

try / catch cơ bản

```
try {
    int ketQua = 10 / 0; // gay ra ArithmeticException
    System.out.println("Khong bao gio in ra dong nay");
} catch (ArithmeticException e) {
    System.out.println("Bat duoc loi: " + e.getMessage());
}
```

- Code có khả năng gây lỗi đặt trong khối `try`.
- Nếu lỗi xảy ra, Java **lập tức nhảy** ra khỏi khối `try` (mọi dòng code sau chỗ gây lỗi trong `try`, kể cả khi chưa lỗi, **không chạy nữa**), tìm khối `catch` phù hợp với **loại** lỗi đã xảy ra.
- `e` là object exception, chứa thông tin về lỗi — `e.getMessage()` lấy thông điệp mô tả lỗi.
- Nếu **không có** lỗi nào xảy ra trong `try`, mọi khối `catch` bị bỏ qua hoàn toàn, chương trình chạy tiếp bình thường sau `try/catch`.

finally — luôn chạy, dù có lỗi hay không

```
try {
    System.out.println("Dang thu...");
    int[] mang = new int[3];
    System.out.println(mang[5]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Bat duoc loi truy cap mang: " + e.getMessage());
} finally {
    System.out.println("Khoi finally LUON chay, bat ke co loi hay khong");
}
```

Khối `finally` (tùy chọn) **luôn được thực thi**, bất kể `try` có ném lỗi hay không, và bất kể lỗi đó có được `catch` bắt được hay không. Dùng `finally` cho các thao tác **bắt buộc phải làm** dù kết quả ra sao — ví dụ đóng kết nối, giải phóng tài nguyên (dù từ Java 7 trở đi, `try-with-resources` — xem phần dưới — thường là lựa chọn tốt hơn `finally` cho đúng mục đích này).

Bắt nhiều loại lỗi — multi-catch

```
try {
    int soNguyen = Integer.parseInt(s);
} catch (NumberFormatException | NullPointerException e) {
    System.out.println("Lỗi: " + e.getClass().getSimpleName());
}
```

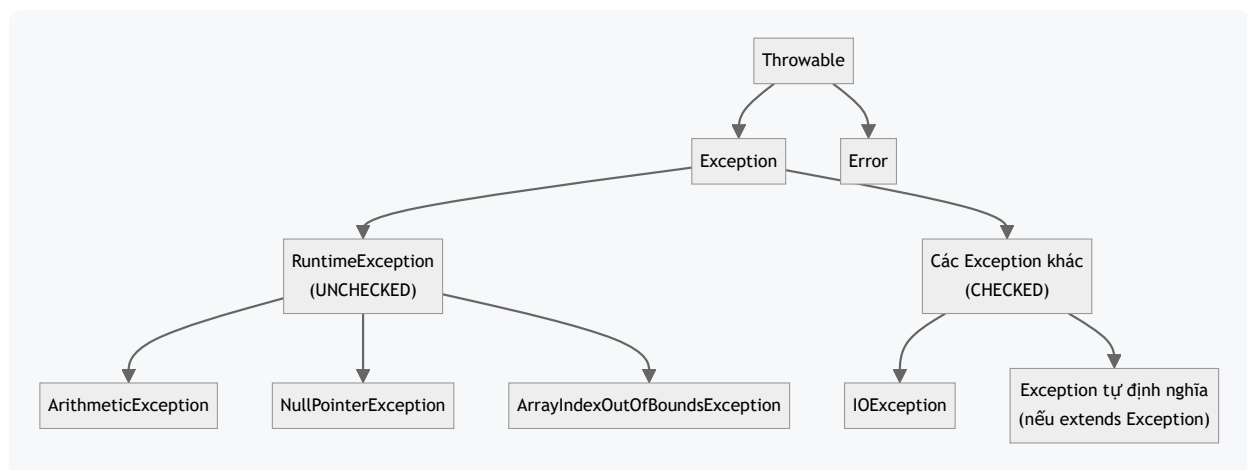
Dấu `|` cho phép **một khối catch xử lý nhiều loại exception** giống hệt nhau, tránh viết lặp lại nhiều khối `catch` có nội dung xử lý giống nhau. Chỉ dùng được khi các loại lỗi đó **không có quan hệ cha-con** với nhau (nếu có, chỉ cần bắt loại cha là đủ — xem phần thứ tự `catch` bên dưới).

Thứ tự các khối catch quan trọng

Nếu bắt nhiều loại lỗi có quan hệ kế thừa (Chương 16-17), khối `catch` cho lớp **con** (cụ thể hơn) phải đặt **trước** khối `catch` cho lớp **cha** (chung chung hơn) — nếu đặt ngược lại, khối `catch` lớp cha sẽ "bắt hết" trước khi khối con có cơ hội chạy, và trình biên dịch **báo lỗi** vì phát hiện khối `catch` con sẽ không bao giờ chạy tới được (`unreachable catch block`).

Checked exception vs Unchecked exception

Đây là khái niệm quan trọng và **riêng có của Java** so với nhiều ngôn ngữ khác:



- **Unchecked exception** — kế thừa từ `RuntimeException` (như `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException` bạn đã gặp). Trình biên dịch **không bắt buộc** bạn phải `try/catch` hay khai báo gì — chương trình vẫn biên dịch được dù không xử lý. Thường đại diện cho **lỗi lập trình** (bug) — về nguyên tắc nên sửa code để lỗi không xảy ra, hơn là "che" nó bằng `try/catch`.
- **Checked exception** — kế thừa trực tiếp từ `Exception` nhưng **không** qua `RuntimeException` (như `IOException` sẽ gặp ở Chương 34). Trình biên dịch **bắt buộc** bạn phải `try/catch`, hoặc khai báo `throws` để "đẩy trách nhiệm xử lý lên" nơi gọi nó — không làm gì cả sẽ là **lỗi biên dịch**. Thường đại diện cho **tình huống có thể lường trước** trong hoạt động bình thường của chương trình (file không tồn tại, kết nối mạng mất...), buộc lập trình viên phải chủ động quyết định xử lý ra sao.

Tự định nghĩa exception (custom exception)

```
public class SoDuKhongDuException extends Exception {
    private double soThieu;

    public SoDuKhongDuException(String thôngDiep, double soThieu) {
        super(thôngDiep);
        this.soThieu = soThieu;
    }
}
```

```

    public double getSoThieu() {
        return soThieu;
    }
}

```

`extends Exception` (không phải `RuntimeException`) tạo ra một **checked exception mới**, đại diện cho một lỗi **nghiệp vụ** riêng của chương trình bạn — ở đây: rút tiền vượt quá số dư. Dùng `throw` để **chủ động ném lỗi** khi phát hiện tình huống không hợp lệ:

```

public void rutTien(double soTien) throws SoDuKhongDuException {
    if (soTien > soDu) {
        throw new SoDuKhongDuException("Khong du so du", soTien - soDu);
    }
    soDu -= soTien;
}

```

`throws SoDuKhongDuException` trong chữ ký phương thức là **bắt buộc** vì đây là checked exception — mọi nơi gọi `rutTien(...)` buộc phải `try/catch` nó, hoặc tự khai báo `throws` tiếp lên trên.

So sánh với cách tiếp cận ở Chương 14 (`TaiKhoanNganHang.napTien / rutTien` chỉ in cảnh báo và `return` khi dữ liệu không hợp lệ): dùng exception (như chương này) buộc nơi gọi **phải chủ động quyết định** cách xử lý lỗi (không có cách nào "vô tình bỏ qua" lỗi mà không biết), trong khi cách in cảnh báo + `return` ở Chương 14 dễ bị bỏ sót nếu người gọi không kiểm tra kỹ. Với lỗi nghiệp vụ quan trọng, dùng exception thường là lựa chọn chắc chắn hơn.

try-with-resources — tự động giải phóng tài nguyên

```

try (NgonTaiNguyen tn = new NgonTaiNguyen("file.txt")) {
    tn.doc();
} // 'tn.close()' TU DONG duoc goi o day, KE CA khi co exception xay ra ben trong try

```

Bất kỳ class nào implements `AutoCloseable` (chỉ có một phương thức `close()`) đều dùng được trong cú pháp `try (...)` này. Java **tự động gọi `close()`** ngay khi ra khỏi khối `try` — dù thoát bình thường hay do exception — bạn không cần tự viết `finally { tn.close(); }` thủ công nữa. Đây là cách an toàn và ngắn gọn nhất để đảm bảo tài nguyên (file, kết nối mạng, kết nối database...) luôn được giải phóng đúng cách; sẽ dùng lại rất nhiều ở Chương 34 (File I/O) trở đi.

Bài tập

- Viết custom checked exception `TuoiKhongHopLeException`, dùng nó trong một phương thức `setTuoi(int tuoi)` (tương tự bài tập Chương 14) — thay vì chỉ in cảnh báo, `throw exception` khi `tuoi < 0`.
- Viết vòng lặp thử `rutTien` nhiều lần với các giá trị khác nhau (có hợp lệ, có không), dùng `try/catch` để chương trình **tiếp tục chạy** sau mỗi lần rút thất bại thay vì dừng hẳn.
- Tạo một class `AutoCloseable` khác (ví dụ mô phỏng kết nối database), dùng trong `try-with-resources` với **hai** resource cùng lúc (`try (A a = ...; B b = ...) { ... }`), quan sát thứ tự `close()` được gọi (gợi ý: thứ tự ngược lại với thứ tự mở).

Lỗi thường gặp

- Bắt exception quá chung chung** (`catch (Exception e)`) **rồi không làm gì cả** (khối `catch` rỗng) — lỗi bị "nuốt" hoàn toàn, chương trình chạy tiếp với trạng thái sai mà không có dấu vết gì để debug sau này. Ít nhất nên in ra thông tin lỗi (`e.printStackTrace()` hoặc `log` — sẽ học logging đúng cách ở Chương 43).
- Đặt khối `catch` cho lớp cha trước lớp con** — lỗi biên dịch `unreachable catch block`. Luôn đặt `catch` cụ thể nhất lên trước.
- Quên khai báo `throws` (hoặc quên `try/catch`) khi gọi phương thức ném checked exception** — lỗi biên dịch `unhandled exception`. Đây là hành vi **cố ý** của Java, không phải bug — nhắc bạn phải xử lý tình huống đó.

- **Dùng exception để điều khiển luồng chương trình bình thường** (ví dụ dùng exception thay cho `if` để kiểm tra điều kiện thông thường) — exception có chi phí hiệu năng cao hơn nhiều so với `if/else` thông thường, và làm code khó đọc. Chỉ dùng exception cho tình huống **thực sự bất thường**, không phải luồng logic chính.
- **Quên rằng `finally` chạy cả khi có `return` trong `try / catch`** — nếu `finally` cũng có `return`, giá trị trả về của `finally` sẽ **ghi đè** giá trị trả về dự định trong `try / catch` — đây là nguồn lỗi khó hiểu, tốt nhất tránh đặt `return` trong khối `finally`.

Chương 22 — Generics

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vấn đề generics giải quyết: dùng chung một class/phương thức cho nhiều kiểu dữ liệu mà vẫn an toàn về kiểu (type-safe).
- Viết được generic class và generic method.
- Dùng bounded type (`<T extends ...>`) khi cần giới hạn `T` phải là một loại cụ thể.
- Đọc hiểu wildcard (`? extends` , `? super`) khi gặp trong code/thư viện chuẩn — nền tảng quan trọng trước khi học Collection Framework ở Phần 5.

Code mẫu đầy đủ: [code/ch22-generics/](#).

Bạn đã dùng generics từ Chương 20 (`Comparable<SanPham>`) mà chưa hiểu cơ chế — chương này giải thích đầy đủ cú pháp `< >` đó nghĩa là gì.

Vấn đề generics giải quyết

Giả sử bạn cần một "cái hộp" chứa được một giá trị bất kỳ, không biết trước kiểu gì. Không có generics, một cách làm (tồi) là dùng `Object` :

```
public class HopChuaCu {
    private Object noiDung;

    public Object layNoiDung() {
        return noiDung;
    }
}
```

Vấn đề: `layNoiDung()` trả về `Object` , buộc người dùng phải **tự ép kiểu** mỗi lần lấy ra:

```
HopChuaCu hop = new HopChuaCu();
hop.datNoiDung("Nguyen Van A");
String ten = (String) hop.layNoiDung(); // phai TU EP KIEU, de sai
```

Nếu ai đó vô tình đặt sai kiểu vào hộp (`hop.datNoiDung(123)`), trình biên dịch **không phát hiện được gì cả** — lỗi chỉ lộ ra lúc **chạy**, dưới dạng `ClassCastException` (Chương 17), tại đúng chỗ ép kiểu, có thể cách rất xa chỗ đã đặt sai giá trị vào — rất khó truy vết.

Generic class — `<T>`

```
public class HopChua<T> {
    private T noiDung;

    public HopChua(T noiDung) {
        this.noiDung = noiDung;
    }

    public T layNoiDung() {
        return noiDung;
    }
}
```

```
}  
}
```

`<T>` ngay sau tên class khai báo **tham số kiểu** (type parameter) — `T` chỉ là một cái tên quy ước (viết tắt của "Type"; các tên quy ước khác hay gặp: `E` cho Element, `K / V` cho Key/Value — sẽ gặp lại ở Chương 26 khi học `Map`), sẽ được **thay thế bằng một kiểu cụ thể** khi bạn dùng class này:

```
HopChua<String> hopTen = new HopChua<>("Nguyen Van A");  
HopChua<Integer> hopTuoi = new HopChua<>(25);  
  
String ten = hopTen.layNoiDung(); // KHÔNG cần ép kiểu - trình biên dịch TU BIET là String
```

Cú pháp `new HopChua<>(...)` — cặp `<>` rỗng (gọi là "diamond operator", có từ Java 7) — cho phép trình biên dịch **tự suy luận** kiểu generic từ ngữ cảnh (ở đây: từ kiểu khai báo biến `HopChua<String>`), không cần viết lặp lại `new HopChua<String>(...)`.

Lợi ích cốt lõi: nếu bạn cố gán sai kiểu (`hopTuoi.datNoiDung("chuoii");` — trong khi `hopTuoi` là `HopChua<Integer>`), trình biên dịch **báo lỗi ngay lúc biên dịch**, không đợi tới lúc chạy mới phát hiện. Đây chính là giá trị lớn nhất của generics: chuyển lỗi từ "phát hiện lúc chạy" (tệ) sang "phát hiện lúc biên dịch" (tốt hơn nhiều — bạn thấy lỗi ngay khi viết code, không phải đợi người dùng gặp phải khi chương trình đã chạy).

Generic method

```
public static <T> void inMoiPhanTu(T[] mang) {  
    for (T phanTu : mang) {  
        System.out.println("- " + phanTu);  
    }  
}
```

Khai báo `<T>` **ngay trước kiểu trả về** — khác với generic class, tham số kiểu này chỉ có hiệu lực **trong phạm vi phương thức**, không cần class chứa nó phải là generic class. Gọi được với bất kỳ mảng kiểu nào:

```
Utils.inMoiPhanTu(new String[]{"An", "Binh"});  
Utils.inMoiPhanTu(new Integer[]{1, 2, 3});
```

Bounded type — giới hạn T phải là một loại cụ thể

```
public class HopSo<T extends Number> {  
    private T giaTri;  
  
    public double layGiaTriThuc() {  
        return giaTri.doubleValue(); // hop le vi MOI Number deu co doubleValue()  
    }  
}
```

`<T extends Number>` giới hạn: `T` **chỉ được phép** là `Number` hoặc lớp con của `Number` (`Integer`, `Double`, `Long`...) — cố dùng `HopSo<String>` sẽ là lỗi biên dịch, vì `String` không kế thừa `Number`. Đối lại, **bên trong** class, bạn được phép gọi các phương thức của `Number` (như `doubleValue()`) trên biến kiểu `T`, dù không biết trước `T` cụ thể sẽ là `Integer` hay `Double` — trình biên dịch biết chắc `T` **ít nhất** cũng là một `Number`, nên các phương thức của `Number` chắc chắn tồn tại.

(Lưu ý từ khoá dùng luôn là `extends`, kể cả khi giới hạn theo interface — không có từ khoá `implements` riêng trong ngữ cảnh bounded type.)

Wildcard — ? extends

```
public static void inGiaTriSo(HopChua<? extends Number> hop) {
    Number giaTri = hop.layNoiDung();
    System.out.println("Gia tri so: " + giaTri);
}
```

Khác với bounded type (`<T extends Number>` — dùng khi khai báo **cả class**), wildcard `?` dùng khi bạn chỉ cần khai báo kiểu cho **một tham số phương thức cụ thể**, chấp nhận "hộp chứa của **bất kỳ** kiểu con nào của `Number`" — gọi được với cả `HopChua<Integer>` lẫn `HopChua<Double>`, mà **không cần** viết riêng hai phiên bản overload cho từng kiểu.

`? extends Number` chỉ cho phép **đọc** (lấy giá trị ra, xem như `Number`), **không cho phép ghi** (gọi `hop.datNoiDung(...)` qua tham chiếu kiểu `HopChua<? extends Number>` là lỗi biên dịch) — lý do kỹ thuật: trình biên dịch không biết chính xác `T` là `Integer` hay `Double` hay kiểu con nào khác của `Number`, nên không thể đảm bảo giá trị bạn định ghi vào có đúng kiểu hay không, và **chặn hẳn** khả năng ghi để tránh sai kiểu tiềm ẩn.

Wildcard — `?` super (chỉ cần biết tên, dùng nhiều ở Chương 27-31)

Ngược lại với `? extends`, `? super T` nghĩa là "kiểu `T` hoặc **bất kỳ lớp cha** nào của `T`", thường dùng khi bạn cần **ghi** giá trị kiểu `T` vào, không cần đọc ra dùng ngay. Nguyên tắc ghi nhớ kinh điển gọi là **PECS** (Producer Extends, Consumer Super): nếu tham số chỉ **cung cấp** dữ liệu cho bạn đọc, dùng `extends`; nếu tham số chỉ **nhận** dữ liệu bạn đưa vào, dùng `super`. Sách sẽ gặp lại wildcard cụ thể hơn khi học Stream API (Chương 31).

Bài tập

- Viết generic class `Cap<A, B>` (Pair) chứa hai giá trị kiểu khác nhau (`A` và `B`), có getter cho cả hai, và `toString()` in ra dạng `(giaTriA, giaTriB)`.
- Viết generic method `<T> T layPhanTuCuoi(T[] mang)` trả về phần tử cuối cùng của mảng bất kỳ kiểu nào.
- Thử tạo `HopSo<String>` (vi phạm bounded type), đọc lỗi biên dịch, xác nhận đúng như phần "Bounded type" giải thích.

Lỗi thường gặp

- Dùng generics với kiểu nguyên thủy** (`HopChua<int>`) — không hợp lệ, generics **chỉ hoạt động với kiểu tham chiếu**. Phải dùng **kiểu bọc (wrapper type)**: `HopChua<Integer>` thay vì `HopChua<int>` (Java tự động chuyển đổi qua lại giữa `int` và `Integer` — cơ chế gọi là "autoboxing/unboxing" — khi cần, không cần bạn tự ép kiểu tường minh).
- Quên diamond operator, viết `new HopChua<String>()` dài dòng thay vì `new HopChua<>()`** — không sai, chỉ là thừa; `<>` để trình biên dịch tự suy luận là đủ trong hầu hết trường hợp.
- Cố gọi `datNoiDung(...)` qua tham chiếu `? extends Number`** — lỗi biên dịch cố ý (xem phần wildcard `? extends` ở trên), không phải bug — đây là cách generics đảm bảo an toàn kiểu.
- Nhầm generic method với generic class** — generic method khai báo `<T>` trước kiểu trả về, tồn tại độc lập, không đòi hỏi class chứa nó phải là generic class như `HopChua<T>`. Đọc kỹ chữ ký phương thức để phân biệt.

Chương 23 — Tổng quan Collection Framework

Mục tiêu học

Sau chương này, bạn sẽ:

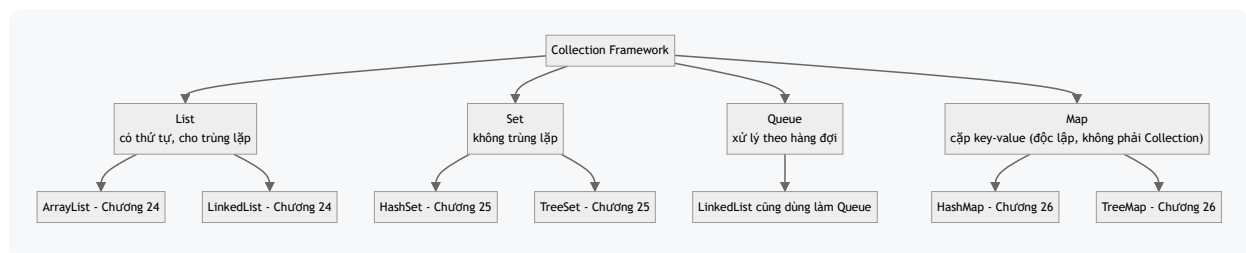
- Hiểu Collection Framework là gì, vì sao nó ra đời để thay thế mảng trong nhiều trường hợp.
- Biết 4 nhóm cấu trúc dữ liệu chính: `List`, `Set`, `Map`, `Queue` — khác nhau ở điểm nào.
- Có bức tranh tổng quan để chọn đúng cấu trúc cho từng bài toán, trước khi học sâu từng loại ở các chương tiếp theo.

Code mẫu đầy đủ: [code/ch23-tong-quan-collection/](https://code.ch23-tong-quan-collection/).

Vì sao cần Collection Framework, mảng chưa đủ sao?

Nhắc lại hạn chế của mảng đã học ở Chương 9: **kích thước cố định** (không thêm/bớt phần tử động được), không có sẵn thao tác tìm kiếm/loại trùng lặp/sắp xếp theo key... Bạn phải tự viết tay mọi logic đó nếu chỉ dùng mảng thuần. **Collection Framework** là tập hợp các cấu trúc dữ liệu chuẩn có sẵn trong `java.util`, đã được tối ưu và kiểm chứng kỹ lưỡng, giải quyết đúng những hạn chế đó.

Bốn nhóm chính



- **List** — dãy phần tử **có thứ tự** (index từ 0, giống mảng nhưng kích thước động), **cho phép phần tử trùng lặp**. Dùng khi thứ tự và số lần xuất hiện đều quan trọng — ví dụ danh sách công việc theo thứ tự nhập vào.
- **Set** — tập hợp **không cho phần tử trùng lặp** (thêm phần tử đã tồn tại sẽ bị bỏ qua âm thầm, không báo lỗi). Dùng khi bạn chỉ quan tâm "có mặt hay không", không quan tâm số lần/thứ tự — ví dụ tập hợp các mã số sinh viên đã điểm danh.
- **Map** — lưu trữ theo **cặp key-value**, mỗi key là duy nhất (gán giá trị mới cho key đã tồn tại sẽ **ghi đè**, không tạo thêm bản ghi). Dùng khi bạn cần tra cứu nhanh giá trị dựa trên một khoá — ví dụ tra điểm một môn học dựa trên tên môn.
- **Queue** — hàng đợi, mặc định xử lý theo nguyên tắc **vào trước ra trước** (FIFO — First In First Out), giống hàng người xếp hàng thật. Dùng khi cần xử lý các phần tử **theo đúng thứ tự chúng được thêm vào**, và loại bỏ dần sau khi xử lý — ví dụ hàng đợi in tài liệu, hàng đợi tác vụ cần xử lý.

Lưu ý về mặt kỹ thuật: `Map` **không** kế thừa từ interface `Collection` chung (khác với `List` / `Set` / `Queue`) vì bản chất nó lưu **cặp** dữ liệu, không phải **từng phần tử đơn lẻ** — nhưng về mặt khái niệm, `Map` vẫn thuộc "họ" Collection Framework và luôn được nhắc tới cùng nhóm.

Bảng chọn nhanh cấu trúc dữ liệu

Cần gì?	Chọn
Danh sách có thứ tự, cho phép trùng lặp, hay thêm/bớt ở giữa	<code>ArrayList</code> hoặc <code>LinkedList</code> (Chương 24)
Tập hợp không trùng lặp, không cần thứ tự cụ thể	<code>HashSet</code> (Chương 25)
Tập hợp không trùng lặp, cần tự động sắp xếp	<code>TreeSet</code> (Chương 25)
Tra cứu nhanh theo khoá	<code>HashMap</code> (Chương 26)
Tra cứu theo khoá, cần khoá luôn được sắp xếp	<code>TreeMap</code> (Chương 26)

Cần gì?	Chọn
Xử lý tuần tự theo hàng đợi (vào trước ra trước)	<code>LinkedList</code> (dùng qua interface <code>Queue</code>)

Bảng này sẽ trở nên cụ thể và dễ hiểu hơn nhiều sau khi bạn học chi tiết từng cấu trúc ở các chương tiếp theo — hãy quay lại đọc bảng này lần nữa sau khi học xong Chương 24-26.

Cùng một tinh thần generics đã học ở Chương 22

Mọi cấu trúc trong Collection Framework đều dùng generics: `List<String>`, `Map<String, Integer>` ... — tham số kiểu (`<String>`, `<String, Integer>`) khai báo rõ collection này chứa đúng kiểu gì, giúp trình biên dịch bắt lỗi kiểu ngay lúc biên dịch thay vì phải ép kiểu thủ công như thời trước Java 5 (khi Collection Framework còn dùng `Object` thuần).

Bài tập

- Không cần code, chỉ cần trả lời: bạn sẽ chọn cấu trúc dữ liệu nào để lưu (a) danh sách các bài hát trong một playlist (có thể trùng tên bài hát, thứ tự phát quan trọng), (b) tập hợp các email đã đăng ký (không được trùng), (c) tra cứu giá sản phẩm theo mã sản phẩm?
- Chạy `CollectionOverviewDemo.java`, thử thêm một phần tử trùng lặp vào `Set` và quan sát `tapMon.size()` không tăng lên.
- Thử đổi `diemMon.put("Toan", 10);` thành một key mới hoàn toàn (`"Ly"`), chạy lại và quan sát `Map` giờ có 3 cặp thay vì 2.

Lỗi thường gặp

- Dùng `List` khi thực ra chỉ cần kiểm tra "có tồn tại hay không"** — dùng `Set` sẽ hiệu quả hơn nhiều cho thao tác kiểm tra tồn tại (sẽ thấy rõ khác biệt hiệu năng ở Chương 25).
- Tưởng `Map` cũng là một dạng `Collection` giống `List` / `Set`** — về mặt kỹ thuật `Map` là interface riêng, không kế thừa `Collection`. Chi tiết này ít khi gây lỗi thực tế nhưng hay bị hỏi nhầm khi mới học.
- Kỳ vọng `Set` giữ đúng thứ tự đã thêm vào** — `HashSet` (loại `Set` phổ biến nhất) **không đảm bảo thứ tự** — sẽ giải thích rõ lý do kỹ thuật ở Chương 25, cùng với `LinkedHashSet` (biến thể có giữ thứ tự nếu cần).

Chương 24 — ArrayList & LinkedList

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng thành thạo `ArrayList` : thêm, đọc, sửa, xóa, duyệt.
- Hiểu vì sao nên khai báo biến bằng kiểu **interface** (`List<T>`) thay vì kiểu cài đặt cụ thể (`ArrayList<T>`).
- Hiểu cơ chế bên trong khiến `ArrayList` và `LinkedList` có đặc tính hiệu năng khác nhau, chọn đúng loại cho từng tình huống.

Code mẫu đầy đủ: <code/ch24-arraylist-linkedlist/> .

ArrayList — danh sách động dựa trên mảng

```
List<String> danhSach = new ArrayList<>();
danhSach.add("An");
danhSach.add("Binh");
danhSach.get(0);           // "An"
danhSach.set(0, "An 2");   // sua phan tu vi tri 0
danhSach.remove(0);        // xoa theo CHI SO
danhSach.remove("Binh");   // xoa theo GIA TRI (dung .equals())
danhSach.size();           // so phan tu HIEN TAI
danhSach.contains("An");   // kiem tra ton tai
```

`ArrayList` giải quyết đúng hạn chế lớn nhất của mảng thuần (Chương 9): **kích thước tự động thay đổi** khi bạn `add` / `remove` , không cần biết trước số lượng phần tử.

Vì sao khai báo `List<String> danhSach = new ArrayList<>()`; thay vì `ArrayList<String> danhSach = ...?`

Đây là thói quen rất phổ biến và nên theo trong code Java thực tế: khai báo **kiểu biến** bằng **interface** (`List`), chỉ dùng **kiểu cài đặt cụ thể** (`ArrayList`) ở phía bên phải khi tạo object. Lý do: phần lớn code của bạn chỉ cần các thao tác **chung** mà mọi `List` đều có (`add` , `get` , `size` ...), không quan tâm cụ thể là `ArrayList` hay `LinkedList` bên dưới. Nếu sau này bạn đổi ý muốn dùng `LinkedList` thay vì `ArrayList` (vì lý do hiệu năng — xem phần dưới), bạn chỉ cần sửa **đúng một chỗ** (`new ArrayList<>()` → `new LinkedList<>()`), mọi chỗ khác dùng biến `danhSach` không cần sửa gì — đây chính là ứng dụng thực tế của **đa hình** (Chương 17) và **abstraction** (nhắc ở Chương 12/18): lập trình theo interface, không lập trình theo cài đặt cụ thể.

Cơ chế bên trong ArrayList

`ArrayList` **thực chất được cài đặt bằng một mảng bên trong** — khi mảng đó đầy và bạn `add` thêm, `ArrayList` tự tạo một mảng **mới lớn hơn** (thường gấp khoảng 1.5 lần), sao chép toàn bộ phần tử cũ sang, rồi mới thêm phần tử mới. Quá trình này diễn ra **âm thầm**, bạn không cần biết chi tiết để dùng `ArrayList` , nhưng hiểu nó giải thích được đặc tính hiệu năng:

- Truy cập theo chỉ số (`get(i)`) — **rất nhanh**, vì mảng bên dưới cho phép nhảy thẳng tới đúng vị trí trong bộ nhớ.
- Thêm/xóa ở **cuối** danh sách — nhanh (thường không cần dịch chuyển gì).
- Thêm/xóa ở **đầu hoặc giữa** danh sách — **chậm dần khi danh sách lớn**, vì phải **dịch chuyển** mọi phần tử phía sau vị trí đó sang một ô để lấy chỗ trống (hoặc lấp chỗ trống khi xóa).

LinkedList — danh sách liên kết

```
LinkedList<String> danhSach = new LinkedList<>();
danhSach.addFirst("A");    // them vao DAU - RAT NHANH
```

```
danhSach.addLast("D");    // thêm vào CUOI - RAT NHANH
danhSach.getFirst();
danhSach.getLast();
danhSach.removeFirst();
```

`LinkedList` cài đặt theo cơ chế **khác hẳn**: mỗi phần tử là một "nút" (node) riêng lẻ trong bộ nhớ, **tự giữ liên kết** (con trỏ) tới nút trước và nút sau nó — không có mảng liên tục nào cả.



Hệ quả hiệu năng **ngược lại** với `ArrayList` :

- Thêm/xóa ở **đầu hoặc giữa** danh sách — **rất nhanh**, chỉ cần nối lại vài liên kết xung quanh vị trí đó, **không** cần dịch chuyển bất kỳ phần tử nào khác.
- Truy cập theo chỉ số (`get(i)`) — **chậm hơn** `ArrayList` đáng kể, vì phải **đi từng bước** qua các liên kết từ đầu (hoặc cuối, tùy vị trí gần đầu hay cuối hơn) cho tới đúng vị trí `i` , không nhảy thẳng được như mảng.

Chạy `LinkedListDemo.java` để thấy chênh lệch thời gian **thực tế** khi thêm 100.000 phần tử vào đầu danh sách — `LinkedList` nhanh hơn `ArrayList` rất nhiều trong tình huống này.

Chọn ArrayList hay LinkedList?

Tình huống	Nên chọn
Chủ yếu đọc/truy cập theo chỉ số, ít thêm/xóa ở giữa	<code>ArrayList</code>
Thêm/xóa thường xuyên ở đầu hoặc giữa danh sách	<code>LinkedList</code>
Không chắc, hoặc danh sách nhỏ	<code>ArrayList</code> (mặc định thực tế phổ biến nhất — đơn giản, hiệu năng tốt trong đa số trường hợp thông thường)

Lời khuyên thực dụng cho người mới: mặc định dùng `ArrayList` trừ khi bạn có lý do cụ thể (đã đo hoặc phân tích) cho thấy cần đặc tính của `LinkedList` . Phần lớn code Java thực tế dùng `ArrayList` nhiều hơn hẳn `LinkedList` .

Bài tập

1. Viết chương trình dùng `ArrayList<Integer>` lưu điểm số nhập từ bàn phím (dùng vòng lặp `while` kết hợp `Scanner` , dừng khi người dùng nhập `-1`), sau đó in ra điểm trung bình.
2. Viết phương thức `xoaTrungLap(List<String> danhSach)` loại bỏ các phần tử trùng lặp, giữ lại thứ tự xuất hiện đầu tiên (gợi ý: dùng `contains()` để kiểm tra trước khi thêm vào danh sách kết quả).
3. Chạy thử `LinkedListDemo.java` với `soLuong = 20_000` thay vì `100_000` thêm vào **cuối** danh sách (`add(danhSach.size(), i)` hoặc đơn giản là `add(i)`) thay vì đầu — so sánh kết quả, giải thích vì sao chênh lệch giữa `ArrayList` và `LinkedList` không còn rõ rệt như khi thêm vào đầu.

Lỗi thường gặp

- `IndexOutOfBoundsException` khi `get` / `set` / `remove` với chỉ số ngoài phạm vi `[0, size())` — giống hệt lỗi mảng ở Chương 9, nhưng tên exception khác (`ArrayIndexOutOfBoundsException` dành cho mảng thuần, `IndexOutOfBoundsException` dành cho `List`).
- Gọi `remove(int)` **khi thực ra muốn xóa theo giá trị Integer** — với `List<Integer>` , `remove(0)` xóa **phần tử ở chỉ số 0**, không phải xóa giá trị `0` — vì có overload `remove(int index)` và `remove(Object o)` cùng tồn tại (Chương 10). Dùng `remove(Integer.valueOf(0))` nếu thật sự muốn xóa theo giá trị `0` .

- **Cố add / remove trên danh sách tạo bằng `List.of(...)` — `List.of()` tạo danh sách **bất biến (immutable)**, mọi thao tác sửa đổi ném `UnsupportedOperationException` lúc chạy. Dùng `new ArrayList<>(List.of(...))` nếu cần một danh sách khởi tạo sẵn giá trị nhưng vẫn sửa được.**
- **Dùng `LinkedList` mặc định "vì nghe nói nhanh hơn" mà không xét thao tác thực tế cần làm** — như bảng so sánh ở trên, `LinkedList` chỉ nhanh hơn cho một số thao tác cụ thể (đầu/giữa danh sách); với `get(i)` ngẫu nhiên, `ArrayList` nhanh hơn nhiều. Chọn dựa trên thao tác bạn thực sự dùng nhiều nhất, không dựa trên cảm tính.

Chương 25 — HashSet, TreeSet, LinkedHashSet

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng `HashSet` để lưu tập hợp không trùng lặp, hiệu năng cao.
- Dùng `LinkedHashSet` khi cần giữ đúng thứ tự đã thêm vào.
- Dùng `TreeSet` khi cần tập hợp luôn được sắp xếp tự động.
- Thực hiện các phép toán tập hợp: hợp, giao, hiệu.
- Hiểu **sâu hơn** vì sao `equals()` / `hashCode()` (Chương 20) bắt buộc phải override đúng cách khi dùng object tự định nghĩa trong `HashSet`.

Code mẫu đầy đủ: <code/ch25-set/>.

HashSet — nhanh nhất, không đảm bảo thứ tự

```
Set<String> hashSet = new HashSet<>();
hashSet.add("Cam");
hashSet.add("Tao");
hashSet.add("Cam"); // trung lap - bi bo qua am tham, KHONG bao loi
System.out.println(hashSet.size()); // 2, không phải 3
```

`HashSet` dùng cơ chế **băm (hashing)** dựa trên `hashCode()` của phần tử để quyết định vị trí lưu trữ nội bộ — nhờ đó, kiểm tra "phần tử đã tồn tại chưa" (`contains`), và cả `add` tự kiểm tra trùng) **cực nhanh** (gần như không phụ thuộc số lượng phần tử đã có), nhanh hơn hẳn việc dò tuần tự trong `ArrayList`. Đánh đổi: **không đảm bảo bất kỳ thứ tự nào** khi duyệt — thứ tự phần tử khi in ra `HashSet` có thể khác thứ tự bạn `add` vào, và không nên dựa vào bất kỳ thứ tự cụ thể nào quan sát được (nó có thể khác nhau giữa các lần chạy hoặc phiên bản Java khác nhau).

LinkedHashSet — giữ đúng thứ tự thêm vào

```
Set<String> linkedHashSet = new LinkedHashSet<>();
linkedHashSet.add("Cam");
linkedHashSet.add("Tao");
linkedHashSet.add("Buoi");
System.out.println(linkedHashSet); // luôn là [Cam, Tao, Buoi] - đúng thu tu đã thêm
```

`LinkedHashSet` là biến thể của `HashSet`, giữ thêm một danh sách liên kết nội bộ để nhớ đúng thứ tự các phần tử được thêm vào, đồng thời **vẫn giữ được** tốc độ kiểm tra trùng lặp gần như `HashSet` (chỉ chậm hơn một chút do phải duy trì thêm thông tin thứ tự). Dùng khi bạn cần **cả hai**: không trùng lặp **và** giữ thứ tự.

TreeSet — tự động sắp xếp

```
Set<String> treeSet = new TreeSet<>();
treeSet.add("Cam");
treeSet.add("Tao");
treeSet.add("Buoi");
System.out.println(treeSet); // luôn là [Buoi, Cam, Tao] - theo thu tu BANG CHU CAI
```

`TreeSet` luôn giữ phần tử theo **thứ tự đã sắp xếp**, dựa trên `Comparable` (Chương 20 — thứ tự tự nhiên) của kiểu phần tử, hoặc một `Comparator` tùy chỉnh truyền vào lúc tạo (`new TreeSet<>(comparator)`). Vì phải duy trì thứ tự sắp xếp liên tục,

`TreeSet` **chậm hơn** `HashSet` / `LinkedHashSet` cho thao tác thêm/kiểm tra ($O(\log n)$) thay vì gần như $O(1)$ — sẽ hiểu rõ hơn ký hiệu độ phức tạp này ở Chương 28), nhưng đổi lại luôn duyệt được theo thứ tự mà không cần tự sắp xếp thêm bước nào.

Chọn loại Set nào?

Cần gì?	Chọn
Chỉ cần không trùng lặp, không quan tâm thứ tự, ưu tiên tốc độ	<code>HashSet</code>
Không trùng lặp và giữ đúng thứ tự thêm vào	<code>LinkedHashSet</code>
Không trùng lặp và luôn cần duyệt theo thứ tự sắp xếp	<code>TreeSet</code>

Các phép toán tập hợp

```
Set<Integer> tapA = new HashSet<>(Arrays.asList(1, 2, 3, 4));
Set<Integer> tapB = new HashSet<>(Arrays.asList(3, 4, 5, 6));

Set<Integer> hop = new HashSet<>(tapA);
hop.addAll(tapB);      // PHEP HOP: {1,2,3,4,5,6}

Set<Integer> giao = new HashSet<>(tapA);
giao.retainAll(tapB);  // PHEP GIAO: {3,4} - chỉ giữ phần tử CÓ Ở CẢ HAI

Set<Integer> hieu = new HashSet<>(tapA);
hieu.removeAll(tapB);  // PHEP HIEU: {1,2} - giữ phần tử CHỈ CÓ Ở tapA
```

Lưu ý quan trọng: `addAll` / `retainAll` / `removeAll` **sửa trực tiếp** tập hợp đang gọi — đó là lý do code trên luôn tạo `new HashSet<>(tapA)` (bản sao) trước khi thao tác, để không làm thay đổi `tapA` gốc ngoài ý muốn.

Vì sao `equals()`/`hashCode()` bắt buộc phải đúng khi dùng object tự định nghĩa trong Set

Đây là ứng dụng thực tế quan trọng nhất của quy tắc đã học ở Chương 20. `HashSet` xác định "phần tử này đã tồn tại chưa" bằng cách: (1) tính `hashCode()` để tìm nhanh tới đúng "khu vực" có thể chứa phần tử trùng, (2) trong khu vực đó, dùng `equals()` để so sánh chính xác từng phần tử. **Nếu bạn không override `equals()` / `hashCode()`**, hai object có nội dung giống hệt nhau vẫn được coi là **khác nhau** (theo hành vi mặc định của `Object` — tương đương `==`), và `HashSet` sẽ **không loại được trùng lặp theo nội dung** dù bạn mong đợi điều đó:

```
Set<DiemToaDo> tap = new HashSet<>();
tap.add(new DiemToaDo(1, 2));
tap.add(new DiemToaDo(1, 2)); // NEU DiemToaDo KHÔNG override equals/hashCode:
System.out.println(tap.size()); // ra 2, KHÔNG phải 1 như bạn có thể mong đợi!
```

Chạy `SetDemo.java` để thấy trực tiếp sự khác biệt này giữa hai class gần như giống hệt nhau, chỉ khác ở việc có override `equals` / `hashCode` hay không.

Bài tập

- Viết chương trình đọc một câu (dùng `Scanner.nextLine()`, tách bằng `split(" ")` như Chương 11), dùng `HashSet<String>` để đếm số từ **khác nhau** (không tính trùng lặp) trong câu đó.
- Dùng `TreeSet<Integer>` để tự động có được danh sách các số duy nhất, đã sắp xếp, từ một mảng `int[]` có nhiều giá trị trùng lặp (gợi ý: thêm từng phần tử mảng vào `TreeSet`, sau đó duyệt `TreeSet` để có kết quả).
- Viết một class `SanPham` (tương tự Chương 20) với `equals` / `hashCode` dựa trên **mã sản phẩm** (không dựa trên tên/giá), thêm nhiều object `SanPham` có cùng mã nhưng giá khác nhau vào một `HashSet` — quan sát chỉ giữ lại được một object

đầu tiên cho mỗi mã trùng.

Lỗi thường gặp

- Dùng object tự định nghĩa trong `HashSet` mà quên override `equals / hashCode` — lỗi phổ biến và khó nhận ra nhất chương này, vì chương trình **không báo lỗi gì**, chỉ đơn giản chứa nhiều phần tử trùng lặp hơn mong đợi. Luôn tự hỏi: object dùng trong `Set` đã override đúng cặp `equals / hashCode` chưa?
- Kỳ vọng `HashSet` giữ thứ tự đã thêm vào — dùng `LinkedHashSet` nếu cần điều đó.
- Sửa tapA gốc ngoài ý muốn khi tính phép giao/hiệu — quên tạo bản sao trước khi gọi `retainAll / removeAll`, làm mất dữ liệu gốc. Luôn `new HashSet<>(tapGoc)` trước khi thao tác nếu cần giữ nguyên tập gốc.
- Dùng `TreeSet` với kiểu phần tử không implements `Comparable`, và không truyền `Comparator` — ném `ClassCastException` lúc chạy ngay khi `add` phần tử thứ hai (`TreeSet` cần so sánh được các phần tử để biết thứ tự sắp xếp).

Chương 26 — HashMap, TreeMap, LinkedHashMap

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng thành thạo `HashMap`: `put`, `get`, `remove`, `containsKey`, `getOrDefault`.
- Duyệt `Map` bằng 3 cách: `keySet()`, `values()`, `entrySet()` — biết khi nào dùng cách nào.
- Phân biệt `HashMap` / `LinkedHashMap` / `TreeMap`, tương tự bộ ba `Set` đã học ở Chương 25.
- Hiểu lại rõ hơn vai trò của `hashCode()` / `equals()` (Chương 20) trong việc `HashMap` tìm đúng key.
- Giải bài toán "đếm số lần xuất hiện" — một trong những ứng dụng phổ biến nhất của `Map`.

Code mẫu đầy đủ: [code/ch26-map/](#).

Thao tác cơ bản với HashMap

```
Map<String, Integer> diem = new HashMap<>();
diem.put("Toan", 9);
diem.put("Toan", 10); // GHI DE - không tạo thêm cặp mới, "Toan" vẫn là MỘT key duy nhất

diem.get("Toan");           // 10
diem.get("Ly");              // null - key không tồn tại, KHÔNG ném lỗi
diem.getOrDefault("Ly", 0); // 0 - trả về giá trị mặc định nếu key không tồn tại
diem.containsKey("Van");    // true/false
diem.remove("Toan");        // xóa cặp có key "Toan"
diem.size();                 // số cặp key-value hiện có
```

Điểm khác biệt quan trọng nhất so với `List` / `Set` (chỉ số/phần tử): `Map` thao tác theo **khóa (key)**, mỗi khóa tương ứng **đúng một** giá trị (value) tại một thời điểm — `put` với khóa đã tồn tại sẽ **ghi đè** giá trị cũ, không tạo thêm cặp mới.

Lưu ý về `get()` khi key không tồn tại: trả về `null` (không ném lỗi) — nhắc lại nguy cơ `NullPointerException` từ Chương 13 nếu bạn dùng ngay giá trị trả về mà không kiểm tra. Dùng `getOrDefault(key, giáTriMặcĐịnh)` là cách an toàn hơn khi bạn muốn có sẵn một giá trị dự phòng.

Ba cách duyệt Map

```
// Cách 1: chỉ cần KEY
for (String mon : diem.keySet()) {
    System.out.println(mon);
}

// Cách 2: chỉ cần VALUE
for (int diemSo : diem.values()) {
    System.out.println(diemSo);
}

// Cách 3: cần CẢ HAI key và value cùng lúc
for (Map.Entry<String, Integer> entry : diem.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}
```

`entrySet()` là cách **hiệu quả nhất** khi bạn cần cả key lẫn value trong cùng vòng lặp — mỗi `Map.Entry` là một "cặp" đại diện đúng một dòng dữ liệu trong `Map`. Tránh cách viết kém hiệu quả (dù vẫn đúng): duyệt `keySet()` rồi gọi `diem.get(key)` bên trong vòng lặp để lấy value — cách này phải tra cứu lại `Map` một lần nữa cho mỗi key, trong khi `entrySet()` đã có sẵn cả hai.

LinkedHashMap và TreeMap

Hoàn toàn tương tự bộ ba `Set` đã học ở Chương 25, áp dụng cho `Map` :

```
Map<String, Integer> theoThuTu = new LinkedHashMap<>(); // giữ dung thu tu KEY đã thêm vào
Map<String, Integer> sapXep = new TreeMap<>();          // tu dong sap xep theo KEY
```

Cần gì?	Chọn
Chỉ cần tra cứu nhanh, không quan tâm thứ tự	HashMap
Tra cứu nhanh, giữ đúng thứ tự các key đã thêm vào	LinkedHashMap
Tra cứu, và key luôn cần duyệt theo thứ tự sắp xếp	TreeMap

Vai trò của hashCode()/equals() trong HashMap

`HashMap` dùng **key** (không phải value) để tính `hashCode()` xác định vị trí lưu trữ, y hệt cách `HashSet` dùng phần tử (Chương 25) — thực tế, `HashSet` **bên trong được cài đặt dựa trên** `HashMap` (mỗi phần tử của `Set` là một key trong `HashMap` ẩn, value chỉ là giá trị đánh dấu). Hệ quả: nếu bạn dùng một object tự định nghĩa **làm key** mà quên override `equals` / `hashCode` đúng cách, `get()` với một object khác (dù nội dung giống hệt key gốc) sẽ **không tìm thấy** giá trị đã lưu — vì `HashMap` coi hai object đó là hai key khác nhau hoàn toàn (giống hệt lỗi đã minh hoạ với `HashSet` ở Chương 25).

Bài tập kinh điển: đếm số lần xuất hiện

```
Map<String, Integer> demSoLan = new HashMap<>();
for (String tu : mangTu) {
    if (demSoLan.containsKey(tu)) {
        demSoLan.put(tu, demSoLan.get(tu) + 1);
    } else {
        demSoLan.put(tu, 1);
    }
}
```

Đây là một trong những khuôn mẫu (pattern) dùng `Map` phổ biến nhất trong lập trình thực tế: đếm tần suất xuất hiện của từng phần tử trong một tập dữ liệu. Cách viết gọn hơn dùng `merge()` :

```
demSoLan.merge(tu, 1, Integer::sum);
```

`merge(key, giaTriMoi, hamKetHop)` tự động xử lý cả hai trường hợp: nếu key **chưa có**, gán `giaTriMoi` (1); nếu key **đã có**, áp dụng `hamKetHop` lên (giá trị cũ, giá trị mới) để ra giá trị cuối cùng. `Integer::sum` là **method reference** (Chương 30) — hiểu đơn giản ở đây là "hàm cộng hai số lại" — trong trường hợp này nghĩa là "cộng dồn số đếm cũ với 1".

Bài tập

- Viết chương trình đếm số lần xuất hiện của từng ký tự trong một chuỗi (dùng `Map<Character, Integer>`, duyệt chuỗi bằng `charAt(i)` như Chương 11).
- Dùng `Map<String, List<String>>` để nhóm một danh sách tên theo chữ cái đầu tiên (ví dụ: "An", "Anh" thuộc nhóm "A") — cần khởi tạo `List` mới cho key lần đầu xuất hiện trước khi thêm phần tử vào.
- Viết phương thức tìm key có value **lớn nhất** trong một `Map<String, Integer>`, dùng vòng lặp `entrySet()`.

Lỗi thường gặp

- `NullPointerException` khi dùng luôn giá trị trả về của `get()` mà không kiểm tra `null` — key không tồn tại trả về `null`, không ném lỗi. Dùng `getOrDefault` hoặc kiểm tra `containsKey` trước khi thao tác tiếp với giá trị lấy ra.

- Dùng object tự định nghĩa làm key mà quên override `equals / hashCode` — `get()` với một object khác object gốc (dù nội dung giống hệt) sẽ không tìm thấy giá trị, dù bạn "biết chắc" nó phải có trong `Map`. Xem lại phần "Vai trò của `hashCode()/equals()`" ở trên.
- Nhầm lẫn `put()` ghi đè giá trị, tưởng nó "cộng thêm" — `put("Toan", 5)` sau `put("Toan", 3)` cho kết quả `key "Toan" = 5`, không phải `8`. Muốn cộng dồn, dùng `merge()` hoặc tự lấy giá trị cũ ra cộng thêm rồi `put()` lại.
- Sửa `Map` (thêm/xóa cặp) ngay trong vòng lặp `for-each` duyệt trên chính `Map` đó — ném `ConcurrentModificationException` lúc chạy (sẽ giải thích rõ nguyên nhân kỹ thuật ở Chương 27, khi học `Iterator`). Nếu cần sửa trong lúc duyệt, dùng `Iterator` trực tiếp thay vì `for-each`.

Chương 27 — Duyệt & sắp xếp: Iterator, Comparator nâng cao

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vì sao sửa collection trong lúc duyệt bằng for-each gây lỗi, và cách khắc phục bằng `Iterator`.
- Dùng `Comparator.comparing()`, `thenComparing()`, `reversed()` — cách viết hiện đại, ngắn gọn hơn nhiều so với anonymous class đã học ở Chương 20.
- Dùng các phương thức tiện ích của lớp `Collections`: sắp xếp, đảo ngược, tìm max/min, xáo trộn, tạo danh sách bất biến.

Code mẫu đầy đủ: [code/ch27-duyet-sap-xep/](#).

Chương này dùng cú pháp method reference (`NguoiDung::getTuoi`) trước khi Chương 30 dạy đầy đủ — ở đây chỉ cần hiểu nó nghĩa là "lấy giá trị trả về của phương thức đó để dùng làm tiêu chí".

ConcurrentModificationException — vì sao xảy ra

```
List<Integer> danhSach = new ArrayList<>(List.of(1, 2, 3, 4, 5));
for (int so : danhSach) {
    if (so % 2 == 0) {
        danhSach.remove(Integer.valueOf(so)); // SUA danh sach TRONG LUC duyet
    }
}
// ConcurrentModificationException!
```

Vòng lặp for-each (Chương 9) thực chất được Java "dịch ngầm" thành việc dùng một **Iterator** — một object di chuyển tuần tự qua từng phần tử, tự theo dõi "tôi đang ở vị trí nào". Khi bạn `remove()` trực tiếp trên `danhSach` (không qua Iterator) trong lúc Iterator đang duyệt dở, cấu trúc dữ liệu bên trong bị thay đổi bất ngờ so với những gì Iterator đang mong đợi — Java phát hiện sự "mất đồng bộ" này và chủ động ném `ConcurrentModificationException` để cảnh báo, thay vì để chương trình chạy tiếp với hành vi khó đoán (có thể bỏ sót phần tử, hoặc lỗi nghiêm trọng hơn).

Iterator — cách sửa collection an toàn trong lúc duyệt

```
Iterator<Integer> it = danhSach.iterator();
while (it.hasNext()) {
    int so = it.next();
    if (so % 2 == 0) {
        it.remove(); // xoa AN TOAN, Iterator TU CAP NHAT trang thai noi bo
    }
}
```

`Iterator<T>` có 3 phương thức chính: `hasNext()` (còn phần tử để duyệt tiếp không), `next()` (lấy phần tử tiếp theo, đồng thời di chuyển Iterator tới), `remove()` (xóa **đúng phần tử vừa lấy ra bởi next()** gần nhất, an toàn vì Iterator tự cập nhật trạng thái nội bộ đồng thời). Đây là cách **duy nhất an toàn** để xóa phần tử trong lúc đang duyệt collection — nếu cần xóa theo điều kiện trong lúc duyệt, luôn dùng `Iterator` trực tiếp thay vì for-each.

Comparator.comparing() — cách viết hiện đại

Nhắc lại Chương 20: viết `Comparator` bằng anonymous class khá dài dòng. Từ Java 8, có cách viết ngắn gọn hơn nhiều:

```
List<NguoiDung> danhSach = ...;
danhSach.sort(Comparator.comparing(NguoiDung::getTuoi));
```

`Comparator.comparing(NguoiDung::getTuoi)` tạo ra một `Comparator<NguoiDung>` so sánh dựa trên giá trị trả về của `getTuoi()` — ngắn gọn hơn hẳn so với việc tự viết anonymous class implements `compare(...)` như ở Chương 20.

`thenComparing()` — sắp xếp theo nhiều tiêu chí

```
danhSach.sort(
    Comparator.comparing(NguoiDung::getTen)
                .thenComparing(NguoiDung::getTuoi)
);
```

Sắp xếp trước theo `getTen()`; khi hai phần tử có **tên bằng nhau**, dùng tiếp `getTuoi()` để phân định thứ tự giữa chúng. Có thể nối tiếp nhiều `thenComparing()` nếu cần nhiều tiêu chí phân định dần.

`reversed()` — đảo ngược thứ tự

```
danhSach.sort(Comparator.comparing(NguoiDung::getTuoi).reversed());
```

Đảo ngược kết quả so sánh của `Comparator` đứng trước nó — sắp xếp giảm dần thay vì tăng dần.

Lớp tiện ích Collections

Khác với `Collection` (interface chung của `List` / `Set` / `Queue`, chú ý không có "s" ở cuối), `Collections` (có "s") là một **class tiện ích**, chứa toàn các phương thức `static` thao tác trên collection:

```
Collections.sort(danhSachSo);           // sap xep tang dan (thu tu tu nhien)
Collections.reverse(danhSachSo);        // dao nguoc thu tu hien tai
Collections.max(danhSachSo);            // gia tri lon nhat
Collections.min(danhSachSo);            // gia tri nho nhat
Collections.shuffle(danhSachSo);        // xao tron ngau nhien
Collections.unmodifiableList(danhSach); // tao mot "khung nhin" BAT BIEN cua danh sach
```

`Collections.sort(list)` là cách viết **trước Java 8** (khi chưa có `list.sort(...)`) — cả hai cách đều còn dùng phổ biến, bạn sẽ gặp cả hai trong code thực tế; `list.sort(comparator)` (gọi trực tiếp trên object `List`) là cách viết được khuyến khích hơn cho code mới vì gọn hơn.

`Collections.unmodifiableList(...)` tạo ra một **view bất biến** — không phải bản sao, mà là một "lớp bọc" chặn mọi thao tác sửa đổi (`add`, `remove` ...) qua tham chiếu đó, ném `UnsupportedOperationException` nếu cố sửa. Lưu ý: danh sách **gốc** đứng sau view này vẫn sửa được bình thường qua tham chiếu gốc — view chỉ chặn thao tác qua **chính tham chiếu bất biến** đó.

Bài tập

- Viết vòng lặp dùng `Iterator` để xoá mọi phần tử `String` có độ dài nhỏ hơn 3 ký tự khỏi một `List<String>`.
- Với danh sách `NguoiDung` trong code mẫu, viết một `Comparator` sắp xếp theo tuổi **giảm dần**, nếu tuổi bằng nhau thì theo tên **tăng dần** (kết hợp `reversed()` và `thenComparing()`).
- Dùng `Collections.max()` với một `Comparator` tùy chỉnh (có overload `max(collection, comparator)`) để tìm người dùng **lớn tuổi nhất** trong danh sách mà không cần tự viết vòng lặp so sánh thủ công.

Lỗi thường gặp

- `ConcurrentModificationException`** — sửa collection trực tiếp trong lúc duyệt for-each. Dùng `Iterator.remove()` thay thế.

- **Gọi `it.remove()` mà chưa gọi `it.next()` trước đó (hoặc gọi hai lần `remove()` liên tiếp)** — ném `IllegalStateException`. `remove()` luôn phải theo ngay sau đúng một lần `next()`.
- **Nhầm `Collection` (interface) với `Collections` (class tiện ích, có "s")** — hai cái tên rất dễ gõ nhầm, ý nghĩa hoàn toàn khác nhau.
- **Cố sửa danh sách qua tham chiếu trả về từ `Collections.unmodifiableList(...)`** — ném `UnsupportedOperationException` có chủ đích, không phải bug.
- **Quên rằng `Comparator.comparing(...)` cần method reference hoặc lambda hợp lệ** — nếu phương thức tham chiếu (`NguoiDung::getTuoi`) không trả về kiểu có thể so sánh được (không implements `Comparable`), trình biên dịch báo lỗi — sẽ hiểu rõ hơn cơ chế này sau khi học lambda ở Chương 30.

Chương 28 — Thuật toán tìm kiếm: Linear, Binary, Jump, Interpolation Search

Mục tiêu học

Sau chương này, bạn sẽ:

- Cài đặt được 4 thuật toán tìm kiếm cơ bản trên mảng.
- Hiểu khái niệm **độ phức tạp thời gian** (time complexity, ký hiệu Big-O) ở mức đủ dùng, không cần nền tảng toán học sâu.
- Biết khi nào nên dùng thuật toán nào, dựa trên đặc điểm dữ liệu (đã sắp xếp hay chưa, phân bố đều hay không).

Code mẫu đầy đủ: [code/ch28-thuat-toan-tim-kiem/](#).

Độ phức tạp thời gian (Big-O) — hiểu ở mức thực dụng

Trước khi so sánh các thuật toán, cần một cách **đo lường** tốc độ không phụ thuộc vào máy tính cụ thể đang chạy. **Big-O** mô tả tốc độ tăng của thời gian chạy **khi kích thước dữ liệu (n) tăng lên**, không phải thời gian chạy chính xác tính bằng giây:

- **$O(n)$** — thời gian chạy tăng **tỷ lệ thuận** với n . Dữ liệu tăng gấp đôi, thời gian chạy tăng gấp đôi (trường hợp xấu nhất).
- **$O(\log n)$** — thời gian chạy tăng **rất chậm** so với n . Dữ liệu tăng gấp đôi, thời gian chạy chỉ tăng thêm **một lượng cố định nhỏ** (không tăng gấp đôi). Với $n = 1,000,000$, $\log_2 n$ chỉ khoảng 20 — chênh lệch với $O(n)$ là rất lớn.
- **$O(1)$** — thời gian chạy **không đổi**, bất kể n lớn cỡ nào (ví dụ: `HashMap.get()` ở Chương 26, trung bình).

Không cần chứng minh toán học chặt chẽ ở giai đoạn này — chỉ cần hiểu: **số mũ hoặc bậc thấp hơn trong Big-O luôn tốt hơn** khi n đủ lớn, dù đôi khi thuật toán "phức tạp hơn" (nhiều bước cài đặt hơn) trong từng bước tính toán riêng lẻ.

Linear Search — $O(n)$

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Duyệt tuần tự từng phần tử, so sánh với `target`. **Đơn giản nhất, không đòi hỏi gì** về dữ liệu đầu vào (mảng không cần sắp xếp). Trường hợp xấu nhất (phần tử cần tìm ở cuối, hoặc không tồn tại): phải duyệt qua **toàn bộ** n phần tử — độ phức tạp **$O(n)$** .

Binary Search — $O(\log n)$

```
public static int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Yêu cầu bắt buộc: mảng phải đã được sắp xếp tăng dần. Ý tưởng: so sánh `target` với phần tử ở giữa — nếu bằng, tìm thấy; nếu `target` lớn hơn, chắc chắn nó (nếu có) chỉ nằm ở **nửa bên phải**, loại bỏ hẳn nửa bên trái khỏi phạm vi tìm kiếm tiếp theo (và ngược lại). Mỗi bước loại bỏ **một nửa** không gian tìm kiếm còn lại — đây là lý do độ phức tạp chỉ **$O(\log n)$** , nhanh hơn rất nhiều so với Linear Search khi `n` lớn (với 1 triệu phần tử, Binary Search cần tối đa khoảng 20 lần so sánh, Linear Search có thể cần tới 1 triệu lần).

`left + (right - left) / 2` (thay vì viết trực tiếp `(left + right) / 2`) là một chi tiết cài đặt đáng chú ý: cách viết trực tiếp có nguy cơ **tràn số** (overflow, đã học ở Chương 5) nếu `left` và `right` đều rất lớn (tổng của chúng vượt phạm vi `int`), dù hiếm gặp trong thực tế nhưng là thói quen cài đặt an toàn nên biết.

Jump Search — $O(\sqrt{n})$

```
int step = (int) Math.floor(Math.sqrt(n));
```

Yêu cầu mảng đã sắp xếp, nằm giữa Linear Search và Binary Search về ý tưởng: thay vì kiểm tra từng phần tử một (Linear Search) hay chia đôi liên tục (Binary Search), Jump Search "nhảy" theo từng khối kích thước $\sqrt{n}$, tới khi tìm thấy khối **có thể chứa** `target` (khối mà giá trị cuối khối $\geq$ `target`), rồi tìm kiếm tuyến tính **trong phạm vi khối đó**. Độ phức tạp **$O(\sqrt{n})$** — chậm hơn Binary Search nhưng nhanh hơn Linear Search. Trong thực tế ít khi dùng Jump Search thay Binary Search (Binary Search gần như luôn tốt hơn khi có thể truy cập ngẫu nhiên vào mảng), nhưng nó minh hoạ tốt ý tưởng "đánh đổi giữa hai thái cực" và hữu ích với một số cấu trúc dữ liệu chỉ hỗ trợ nhảy theo khối hiệu quả hơn truy cập ngẫu nhiên từng phần tử.

Interpolation Search — $O(\log \log n)$ trung bình

```
int pos = left + ((target - arr[left]) * (right - left)) / (arr[right] - arr[left]);
```

Yêu cầu mảng đã sắp xếp, hoạt động tốt nhất khi dữ liệu **phân bố đều** (uniformly distributed). Thay vì luôn nhắm vào giữa như Binary Search, Interpolation Search **ước lượng** vị trí có khả năng chứa `target` dựa trên tỷ lệ giữa giá trị `target` và khoảng giá trị `[arr[left], arr[right]]` — giống cách con người tìm một cái tên trong danh bạ điện thoại giấy: nếu tìm tên bắt đầu bằng "Y", bạn mở thẳng gần cuối cuốn sách, không mở đúng giữa rồi chia đôi dần như Binary Search.

Với dữ liệu phân bố đều, độ phức tạp trung bình đạt **$O(\log \log n)$** — nhanh hơn cả Binary Search. Nhưng với dữ liệu phân bố **không đều** (ví dụ phần lớn giá trị dồn về một khoảng hẹp, vài giá trị ngoại lệ rất xa), công thức ước lượng có thể sai lệch nhiều, khiến độ phức tạp xấu nhất tụt xuống **$O(n)$** — kém tin cậy hơn Binary Search (luôn đảm bảo $O(\log n)$ bất kể phân bố dữ liệu ra sao). Đây là lý do Binary Search vẫn là lựa chọn mặc định phổ biến hơn trong thực tế, dù Interpolation Search có thể nhanh hơn trong điều kiện dữ liệu lý tưởng.

Bảng so sánh

Thuật toán	Yêu cầu dữ liệu	Độ phức tạp trung bình	Độ phức tạp xấu nhất
Linear Search	Không yêu cầu	$O(n)$	$O(n)$
Binary Search	Đã sắp xếp	$O(\log n)$	$O(\log n)$
Jump Search	Đã sắp xếp	$O(\sqrt{n})$	$O(\sqrt{n})$
Interpolation Search	Đã sắp xếp, phân bố đều	$O(\log \log n)$	$O(n)$

Bài tập

- Chạy `SoSanhHieuNang.java`, thử tăng `kichThuc` lên `20_000_000` — quan sát chênh lệch giữa Linear Search và các thuật toán còn lại càng rõ rệt hơn khi `n` càng lớn.
- Sửa `target` trong `SoSanhHieuNang.java` thành một giá trị **không tồn tại** trong mảng (ví dụ một số lẻ, vì mảng mẫu chỉ chứa số chẵn) — quan sát cả 4 thuật toán đều trả về `-1`, và so sánh lại thời gian chạy trong trường hợp "không tìm thấy" (thường là trường hợp xấu nhất).

3. Tự cài đặt một biến thể của Binary Search **đệ quy** (gọi lại chính nó thay vì dùng `while`) — sách chưa dạy đệ quy chính thức, đây là bài tập mở rộng để tự khám phá; so sánh kết quả với bản `while` đã có.

Lỗi thường gặp

- **Dùng Binary/Jump/Interpolation Search trên mảng CHƯA sắp xếp** — cả ba thuật toán này giả định ngầm dữ liệu đã sắp xếp; chạy trên mảng chưa sắp xếp cho kết quả **sai** (có thể báo "không tìm thấy" dù phần tử thực sự tồn tại) mà không có bất kỳ cảnh báo lỗi nào — luôn `Arrays.sort()` (Chương 9) trước nếu dữ liệu chưa chắc chắn đã sắp xếp.
- **Viết $(left + right) / 2$ thay vì $left + (right - left) / 2$ trong Binary Search** — nguy cơ tràn số với mảng cực lớn, dù hiếm gặp trong bài tập thông thường vẫn nên tập thói quen viết đúng.
- **Chọn Interpolation Search cho dữ liệu phân bố không đều rồi ngạc nhiên vì hiệu năng tệ hơn dự kiến** — nhắc lại: độ phức tạp xấu nhất của nó là $O(n)$, không phải lúc nào cũng nhanh như quảng cáo lý thuyết. Với dữ liệu không rõ đặc tính phân bố, Binary Search là lựa chọn an toàn hơn.
- **Nhầm lẫn giá trị trả về -1 (không tìm thấy) với một chỉ số hợp lệ** — luôn kiểm tra `if (index != -1)` trước khi dùng kết quả tìm kiếm để truy cập mảng, tránh `ArrayIndexOutOfBoundsException` (Chương 9).

Chương 29 — Duyệt đồ thị/cây: BFS & DFS

Mục tiêu học

Sau chương này, bạn sẽ:

- Biểu diễn được đồ thị/cây trong Java bằng danh sách kề (adjacency list).
- Cài đặt BFS (duyet theo chiều rộng) dùng `Queue`.
- Cài đặt DFS (duyet theo chiều sâu) dùng đệ quy, và hiểu **đệ quy** là gì (khái niệm mới, lần đầu xuất hiện trong sách).
- Nhận ra BFS và DFS thực chất chỉ khác nhau ở **cấu trúc dữ liệu** dùng để ghi nhớ thứ tự xử lý (Queue vs Stack).

Code mẫu đầy đủ: [code/ch29-thuat-toan-duyet-do-thi/](#).

Biểu diễn đồ thị bằng danh sách kề

Đồ thị (graph) gồm các **đỉnh** (node/vertex) và **cạnh** (edge) nối giữa chúng. Cách biểu diễn phổ biến nhất trong code: **danh sách kề** (adjacency list) — với mỗi đỉnh, lưu danh sách các đỉnh nó nối trực tiếp tới:

```
List<List<Integer>> doThi = new ArrayList<>();
for (int i = 0; i < soDinh; i++) {
    doThi.add(new ArrayList<>());
}
doThi.get(0).add(1); // dinh 0 noi toi dinh 1
doThi.get(0).add(2); // dinh 0 noi toi dinh 2
```

`doThi.get(i)` trả về danh sách các đỉnh **kề** (nối trực tiếp) với đỉnh `i`. Cây (tree) — cấu trúc bạn có thể quen thuộc hơn (thư mục lồng nhau, cây gia phả...) — thực chất chỉ là một dạng **đặc biệt** của đồ thị (không có chu trình, mỗi đỉnh chỉ có đúng một đỉnh cha), nên mọi thuật toán duyệt đồ thị trong chương này áp dụng được trực tiếp cho cây.

BFS (Breadth-First Search) — duyệt theo chiều rộng

```
private static void bfs(List<List<Integer>> doThi, int dinhBatDau) {
    boolean[] daTham = new boolean[doThi.size()];
    Queue<Integer> hangDoi = new LinkedList<>();

    daTham[dinhBatDau] = true;
    hangDoi.offer(dinhBatDau);

    while (!hangDoi.isEmpty()) {
        int dinh = hangDoi.poll();
        System.out.print(dinh + " ");

        for (int dinhKe : doThi.get(dinh)) {
            if (!daTham[dinhKe]) {
                daTham[dinhKe] = true;
                hangDoi.offer(dinhKe);
            }
        }
    }
}
```

BFS thăm **tất cả đỉnh ở cùng khoảng cách** (số cạnh) từ đỉnh bắt đầu, trước khi sang khoảng cách xa hơn — giống cách sóng lan toả đều ra mọi hướng từ một điểm ném xuống nước. Với cây ví dụ trong code mẫu (đỉnh 0 là gốc, 1-2 là con của 0, 3-4-5-6 là cháu), BFS cho kết quả `0 1 2 3 4 5 6` — đúng **từng tầng** một.

Cơ chế: dùng `Queue` (FIFO, Chương 23) để ghi nhớ các đỉnh **đã phát hiện nhưng chưa xử lý** — mỗi lần lấy một đỉnh ra khỏi hàng đợi để xử lý, đồng thời **thêm mọi đỉnh kề chưa thăm** của nó vào cuối hàng đợi. Vì hàng đợi luôn xử lý phần tử **cũ nhất** trước, các đỉnh ở tầng gần hơn (được thêm vào trước) luôn được xử lý trước các đỉnh ở tầng xa hơn.

Mảng `daTham` (visited) **bắt buộc phải có** — nếu không, với đồ thị có chu trình (một đỉnh có đường quay lại chính nó qua các cạnh khác), thuật toán sẽ **lặp vô hạn**, liên tục thăm lại các đỉnh đã thăm.

DFS (Depth-First Search) — duyệt theo chiều sâu, và khái niệm đệ quy

```
private static void dfs(List<List<Integer>> doThi, int dinh, boolean[] daTham) {
    daTham[dinh] = true;
    System.out.print(dinh + " ");

    for (int dinhKe : doThi.get(dinh)) {
        if (!daTham[dinhKe]) {
            dfs(doThi, dinhKe, daTham); // phương thức TU GOI LAI CHÍNH NÓ
        }
    }
}
```

DFS đi **thật sâu theo một nhánh trước**, chỉ quay lại thử nhánh khác khi không còn đường đi tiếp — giống cách bạn dò một mê cung: đi theo một lối tới khi bí, mới lùi lại thử lối khác. Với cùng cây ví dụ, DFS cho kết quả `0 1 3 4 2 5 6` — đi hết nhánh trái ($0 \rightarrow 1 \rightarrow 3$, lùi lại $\rightarrow 4$) rồi mới sang nhánh phải ($0 \rightarrow 2 \rightarrow 5$, lùi lại $\rightarrow 6$).

Đệ quy (recursion) là gì?

Đây là **lần đầu tiên** sách dùng kỹ thuật này: `dfs(...)` **tự gọi lại chính nó** bên trong thân của nó — gọi là **đệ quy**. Mỗi lần `dfs` gọi tiếp `dfs` cho một đỉnh kề, một "phiên bản mới" của phương thức bắt đầu chạy (với `dinh` là tham số khác), phiên bản gọi nó tạm "treo" lại, đợi phiên bản mới chạy xong hoàn toàn rồi mới tiếp tục vòng lặp `for` của chính nó. Java tự quản lý việc "treo và tiếp tục" này bằng một cấu trúc bên trong gọi là **call stack** (ngăn xếp lời gọi) — mỗi lần gọi đệ quy chồng thêm một "lớp" mới lên ngăn xếp, lớp đó chỉ được gỡ ra khi lời gọi đó **hoàn toàn kết thúc** (chạy hết thân phương thức, hoặc gặp `return`).

Đệ quy luôn cần một **điều kiện dừng** — ở đây, điều kiện `if (!daTham[dinhKe])` đảm bảo `dfs` không bao giờ tự gọi lại cho một đỉnh **đã thăm**, nên cuối cùng sẽ hết đỉnh mới để gọi tiếp và tự dừng. Thiếu điều kiện dừng đúng đắn sẽ gây đệ quy **vô hạn**, cuối cùng ném lỗi `StackOverflowError` (ngăn xếp lời gọi đầy, không phải lỗi bạn tự bắt bằng `try/catch` thông thường — đây là dấu hiệu gần như chắc chắn của lỗi logic đệ quy).

DFS không dùng đệ quy — làm rõ mối quan hệ BFS/DFS

```
Deque<Integer> nganXep = new ArrayDeque<>(); // dung lam STACK (LIFO)
nganXep.push(dinhBatDau);
while (!nganXep.isEmpty()) {
    int dinh = nganXep.pop(); // lay dinh THEM VAO GAN DAY NHAT
    // ...
}
```

Mọi thuật toán đệ quy đều **viết lại được** dưới dạng vòng lặp thường, dùng một cấu trúc dữ liệu tương minh thay cho call stack ngầm định của Java. Với DFS, cấu trúc đó là **ngăn xếp (stack, LIFO — Vào Sau Ra Trước)**: `push` thêm vào "đỉnh" ngăn xếp, `pop` lấy ra đúng phần tử **vừa thêm gần nhất**.

So sánh với BFS: **BFS dùng Queue (FIFO), DFS dùng Stack (LIFO)** — đây là khác biệt cốt lõi duy nhất giữa hai thuật toán, phần logic còn lại (đánh dấu đã thăm, duyệt đỉnh kề) gần như giống hệt nhau. Hiểu rõ điểm này giúp bạn nhớ và cài đặt lại cả hai thuật toán dễ dàng hơn nhiều so với học thuộc lòng từng đoạn code riêng biệt.

Bài tập

1. Thêm một đỉnh thứ 8 vào đồ thị mẫu, nối với đỉnh 3, chạy lại cả BFS và DFS, dự đoán kết quả trước khi chạy để kiểm tra.
2. Viết một đồ thị **có chu trình** (ví dụ thêm cạnh nối đỉnh 6 quay lại đỉnh 0), chạy BFS/DFS, xác nhận thuật toán vẫn dừng đúng nhờ mảng `daTham` (không bị lặp vô hạn).
3. Viết một biến thể BFS trả về **khoảng cách** (số cạnh) từ đỉnh bắt đầu tới từng đỉnh khác, thay vì chỉ in ra thứ tự duyệt (gợi ý: dùng thêm một mảng `int[] khoangCach`, cập nhật `khoangCach[đỉnhKe] = khoangCach[đỉnh] + 1` mỗi khi thêm `đỉnhKe` vào hàng đợi).

Lỗi thường gặp

- **Quên đánh dấu `daTham`, hoặc đánh dấu sai thời điểm** — gây duyệt lặp lại vô hạn với đồ thị có chu trình, hoặc duyệt trùng đỉnh không cần thiết. Trong BFS, nên đánh dấu **ngay khi thêm vào hàng đợi** (không phải khi lấy ra xử lý) để tránh thêm trùng một đỉnh nhiều lần trước khi nó được xử lý.
- **`StackOverflowError` khi DFS đệ quy trên đồ thị/cây rất sâu** — call stack có giới hạn kích thước; với dữ liệu đủ lớn (hàng chục nghìn tầng sâu trở lên), bản đệ quy có thể tràn ngăn xếp. Bản không đệ quy (dùng `Deque` làm stack tường minh) không gặp giới hạn này vì ngăn xếp tự cấp phát trên heap thay vì call stack cố định.
- **Nhầm lẫn BFS luôn "tốt hơn" hoặc "đúng hơn" DFS (hoặc ngược lại)** — cả hai chỉ là hai chiến lược duyệt khác nhau, phù hợp cho bài toán khác nhau: BFS thường dùng khi cần tìm đường đi **ngắn nhất** (tính theo số cạnh) trong đồ thị không trọng số; DFS thường dùng khi cần khám phá **toàn bộ** một nhánh trước khi xét nhánh khác (ví dụ: kiểm tra chu trình, sắp xếp tô-pô).
- **Trong bản DFS không đệ quy, quên đảo ngược thứ tự khi `push` các đỉnh kề** — vì Stack là LIFO, nếu không đảo ngược thứ tự thêm vào, thứ tự duyệt cuối cùng sẽ **ngược lại** so với bản đệ quy (dù vẫn là DFS hợp lệ, chỉ khác thứ tự cụ thể) — xem chi tiết cách xử lý trong `DepthFirstSearchIterative.java`.

Chương 30 — Lambda expression & functional interface

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **functional interface** là gì, viết được lambda expression thay thế anonymous class.
- Dùng thành thạo 5 functional interface có sẵn trong `java.util.function`.
- Dùng **method reference** — cách viết gọn hơn nữa cho một số trường hợp lambda phổ biến.
- Hiểu quy tắc "effectively final" khi lambda dùng biến từ phạm vi bên ngoài.

Code mẫu đầy đủ: [code/ch30-lambda/](#).

Từ anonymous class tới lambda

Nhắc lại Chương 19, tạo object implements interface "ngay tại chỗ":

```
PhepToan cong = new PhepToan() {  
    @Override  
    public int apDung(int a, int b) {  
        return a + b;  
    }  
};
```

Khi interface đó **chỉ có đúng một phương thức abstract** (gọi là **functional interface**), phần lớn đoạn code trên (`new PhepToan() { @Override public int apDung(...) { ... } }`) là **thừa thãi về mặt hình thức** — trình biên dịch thừa sức tự suy luận ra bạn đang implement phương thức nào. **Lambda expression** loại bỏ hoàn toàn phần thừa đó:

```
PhepToan cong = (a, b) -> a + b;
```

Đọc là: "nhận vào `a, b`, trả về `a + b`". Cú pháp: (tham số) -> biểu thức_hoặc_khối_lệnh.

```
@FunctionalInterface  
interface PhepToan {  
    int apDung(int a, int b);  
}
```

`@FunctionalInterface` là annotation **tuỳ chọn nhưng nên dùng** — báo cho trình biên dịch biết "interface này được thiết kế để dùng với lambda"; nếu ai đó vô tình thêm phương thức abstract thứ hai vào interface (phá vỡ điều kiện "chỉ đúng một phương thức abstract"), trình biên dịch **báo lỗi ngay** thay vì để lỗi âm thầm lan rộng ra mọi chỗ đang dùng lambda với interface đó.

Lambda nhiều dòng

```
PhepToan congCoLog = (a, b) -> {  
    System.out.println("Đang cong " + a + " va " + b);  
    return a + b;  
};
```

Khi thân lambda có **nhiều hơn một câu lệnh**, cần dấu ngoặc nhọn `{ }` và `return` tường minh (giống thân một phương thức bình thường) — khác với lambda một dòng (`(a, b) -> a + b`), nơi giá trị của biểu thức **tự động** là giá trị trả về, không cần viết `return`.

Quy tắc "effectively final"

```
int heSo = 100;
PhepToan nhanHeSo = (a, b) -> (a + b) * heSo;
```

Lambda **dùng được** biến cục bộ từ phạm vi bao quanh nó (`heSo`), nhưng với điều kiện: biến đó phải là **effectively final** — nghĩa là, dù không bắt buộc ghi từ khoá `final`, biến đó **không được gán lại giá trị** ở bất kỳ đâu sau khi khai báo, trong toàn bộ phạm vi chứa nó. Nếu bạn gán lại `heSo = 200;` ở bất kỳ đâu sau đó, trình biên dịch báo lỗi ngay tại **lambda** đang dùng `heSo`, dù dòng gán lại nằm ở chỗ khác. Lý do kỹ thuật: lambda có thể được lưu trữ và gọi **muộn hơn nhiều** so với chỗ nó được tạo ra (thậm chí sau khi phương thức chứa nó đã kết thúc chạy) — Java đảm bảo an toàn bằng cách **chỉ cho lambda "chụp ảnh" giá trị tại thời điểm tạo**, không cho phép giá trị đó thay đổi sau này để tránh mơ hồ.

java.util.function — các functional interface có sẵn

Phần lớn trường hợp dùng lambda trong thực tế **không cần tự định nghĩa interface riêng** như `PhepToan` — thư viện chuẩn đã cung cấp sẵn các functional interface tổng quát trong package `java.util.function`:

Interface	Chữ ký	Ý nghĩa
<code>Predicate<T></code>	<code>boolean test(T t)</code>	Kiểm tra điều kiện, trả về đúng/sai
<code>Function<T, R></code>	<code>R apply(T t)</code>	Nhận vào kiểu T, trả về kiểu R (khác kiểu)
<code>Consumer<T></code>	<code>void accept(T t)</code>	Nhận vào, thực hiện hành động, không trả về gì
<code>Supplier<T></code>	<code>T get()</code>	Không nhận gì, tạo/trả về một giá trị
<code>BiFunction<T, U, R></code>	<code>R apply(T t, U u)</code>	Giống Function nhưng nhận hai tham số

```
Predicate<Integer> laSoChan = n -> n % 2 == 0;
Function<String, Integer> doDaiChuoi = s -> s.length();
Consumer<String> inHoaToanBo = s -> System.out.println(s.toUpperCase());
Supplier<String> taoChaoMung = () -> "Chao mung!";
BiFunction<Integer, Integer, Integer> congHaiSo = (a, b) -> a + b;
```

Bạn đã **thực chất dùng qua** một functional interface có sẵn từ Chương 27: `Comparator<T>` — có đúng một phương thức `abstract (compare)`, nên `Comparator.comparing(...)` nhận được lambda/method reference thay vì phải viết anonymous class dài dòng.

Method reference — gọn hơn nữa

Khi thân lambda chỉ đơn giản là **gọi lại một phương thức đã tồn tại sẵn**, có thể viết gọn hơn bằng method reference, cú pháp `Kiểu::tenPhuongThuc`:

```
// Loại 1: tham chiếu phương thức STATIC
BiFunction<Integer, Integer, Integer> cong = Integer::sum;           // thay cho (a, b) -> Integer.sum(a, b)

// Loại 2: tham chiếu phương thức INSTANCE của MỘT object cụ thể
String ten = "Xin chào";
Supplier<String> layHoa = ten::toUpperCase;                         // thay cho () -> ten.toUpperCase()

// Loại 3: tham chiếu phương thức INSTANCE, áp dụng TREN THAM SỐ được truyền vào
Function<String, Integer> layDoDai = String::length;               // thay cho s -> s.length()

// Loại 4: tham chiếu CONSTRUCTOR
Supplier<List<Integer>> taoMoi = ArrayList::new;                    // thay cho () -> new ArrayList<>()
```

Method reference **không phải cú pháp mới về bản chất** — nó chỉ là cách viết tắt cho một dạng lambda rất phổ biến (lambda mà thân của nó chỉ gọi đúng một phương thức có sẵn, không thêm logic gì khác). Dùng khi nó làm code **rõ ràng hơn**; nếu lambda cần thêm logic (dù chỉ một chút), viết lambda tường minh vẫn thường dễ đọc hơn ép dùng method reference.

Bài tập

- Viết lại `PhepToan` để có thêm phiên bản `chia` bằng lambda, xử lý (in cảnh báo, trả về `0`) khi chia cho 0 thay vì để `ArithmeticException` (Chương 21) xảy ra.
- Dùng `Predicate<String>` kiểm tra một chuỗi có phải palindrome hay không (tận dụng lại logic đã viết ở bài tập Chương 11), áp dụng nó để lọc một `List<String>` (dùng vòng lặp với `test(...)`, chưa cần Stream — sẽ học cách lọc gọn hơn nhiều ở Chương 31).
- Viết 3 method reference khác nhau cho cùng một `Function<String, String>`: một tham chiếu tới `String::toUpperCase`, một tới `String::trim`, thử áp dụng lần lượt và so sánh kết quả.

Lỗi thường gặp

- Cố dùng lambda cho interface có từ 2 phương thức abstract trở lên** — lỗi biên dịch, vì trình biên dịch không biết lambda tương ứng với phương thức nào. Chỉ functional interface (đúng một phương thức abstract) mới dùng được lambda.
- Gán lại giá trị biến ngoài sau khi đã dùng trong lambda** — vi phạm "effectively final", lỗi biên dịch tại chính lambda. Nếu cần một giá trị "thay đổi được" mà lambda vẫn thấy, cần dùng cấu trúc khác (ví dụ một mảng một phần tử, hoặc field của object) — kỹ thuật nâng cao, ít cần tới ở giai đoạn học này.
- Quên `return` trong lambda nhiều dòng** — khác với lambda một dòng (tự động trả về giá trị biểu thức), lambda dùng `{ }` **bắt buộc** `return` tường minh nếu functional interface yêu cầu trả về giá trị.
- Method reference không khớp chữ ký functional interface mong đợi** — ví dụ cố dùng `String::length` (nhận 0 tham số ngoài chính `this`, trả `int`) cho một `BiFunction` (mong đợi nhận 2 tham số) — lỗi biên dịch báo rõ không khớp kiểu, đọc kỹ thông báo lỗi để đối chiếu đúng loại method reference (1 trong 4 loại đã học) phù hợp với ngữ cảnh.

Chương 31 — Stream API

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu Stream là gì, khác gì so với Collection (Chương 23).
- Dùng thành thạo các thao tác trung gian phổ biến: `filter`, `map`, `sorted`.
- Dùng các thao tác kết thúc: `collect`, `count`, `anyMatch` / `allMatch`, `findFirst`, `reduce`.
- Dùng Collectors để gom kết quả thành List, Map, hoặc nhóm theo tiêu chí (`groupingBy`).

Code mẫu đầy đủ: [code/ch31-stream-api/](#).

Stream là gì?

```
List<String> tenDaiCu = new ArrayList<>();
for (String t : ten) {
    if (t.length() > 5) {
        tenDaiCu.add(t.toUpperCase());
    }
}
```

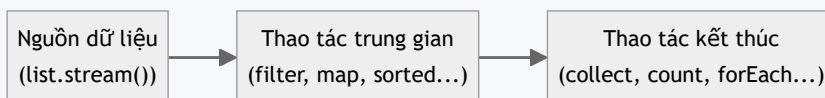
Đây là mẫu code rất quen thuộc: lọc phần tử theo điều kiện, biến đổi từng phần tử, gom kết quả lại. **Stream** (Java 8+) cung cấp một cách viết **khai báo (declarative)** cho đúng loại thao tác này — mô tả "**muốn gì**" thay vì viết từng bước "**làm thế nào**":

```
List<String> tenDaiMoi = ten.stream()
    .filter(t -> t.length() > 5)
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

Đọc gần như tiếng Anh: "lấy stream từ `ten`, lọc giữ lại phần tử dài hơn 5, biến đổi thành chữ hoa, gom lại thành một List".

Stream không phải là một cấu trúc dữ liệu (khác List / Set / Map đã học ở Phần 5) — nó là một "đường ống xử lý" tạm thời, chạy qua dữ liệu từ một nguồn (thường là một Collection có sẵn) rồi biến mất, **không lưu trữ gì lâu dài** và **không sửa đổi** nguồn dữ liệu gốc.

Ba phần của một pipeline Stream



1. **Nguồn** — thường gọi `.stream()` trên một Collection có sẵn.
2. **Thao tác trung gian (intermediate)** — trả về một Stream khác, có thể nối tiếp nhiều thao tác liên tục (gọi là "chaining"). Không thực sự chạy ngay — Stream có tính chất **lazy** (lười biếng): các thao tác trung gian chỉ thực sự thực thi khi có một thao tác kết thúc được gọi.
3. **Thao tác kết thúc (terminal)** — trả về một kết quả cụ thể (không còn là Stream nữa), và **kích hoạt** toàn bộ pipeline chạy. Sau khi gọi một thao tác kết thúc, Stream đó **không dùng lại được nữa** (gọi thêm thao tác trên cùng Stream đó sẽ ném lỗi).

Thao tác trung gian phổ biến

```
.filter(t -> t.length() > 5)    // giu lai phan tu THOA DIEU KIEN (Predicate, Chuong 30)
.map(String::toUpperCase)      // BIEN DOI tung phan tu thanh gia tri khac (Function, Chuong 30)
.sorted()                      // sap xep theo thu tu tu nhien (hoac truyen Comparator)
```

`filter` nhận một `Predicate<T>`, `map` nhận một `Function<T, R>` — đây chính xác là các functional interface đã học ở Chương 30, được dùng làm "tham số cấu hình" cho từng bước xử lý của Stream.

Thao tác kết thúc phổ biến

```
.collect(Collectors.toList())    // gom thanh List
.count()                        // dem so luong phan tu con lai sau cac buoc loc
.anyMatch(t -> t.startsWith("A")) // co IT NHAT MOT phan tu thoa dieu kien khong?
.allMatch(t -> t.length() > 2)   // TAT CA phan tu deu thoa dieu kien khong?
.findFirst()                    // lay phan tu DAU TIEN con lai, tra ve Optional (Chuong 32)
```

`reduce()` — gộp toàn bộ Stream thành một giá trị

```
double tongGiaTri = danhSach.stream()
    .map(SanPham::getGia)
    .reduce(0.0, Double::sum);
```

`reduce(giaTriKhoiDau, hamGop)` áp dụng `hamGop` liên tiếp lên **giá trị đang tích lũy** và **từng phần tử tiếp theo**, bắt đầu từ `giaTriKhoiDau`. Về bản chất, `reduce` **tương đương** một vòng lặp tích lũy bạn đã viết tay nhiều lần trong sách này:

```
double tong = 0.0;
for (SanPham sp : danhSach) {
    tong = Double.sum(tong, sp.getGia()); // tuong duong tong = tong + sp.getGia()
}
```

Collectors — các cách gom kết quả phổ biến

```
Collectors.toList()                // gom thanh List
Collectors.toMap(SanPham::getTen, SanPham::getGia) // gom thanh Map (key -> value)
Collectors.groupingBy(SanPham::getLoai)           // nhom thanh Map<Loai, List<SanPham>>
Collectors.groupingBy(SanPham::getLoai, Collectors.summingDouble(SanPham::getGia)) // nhom + tinh tong tung nhom
Collectors.joining(", ")                // noi cac String lai, cach nhau boi dau phan cach
```

`groupingBy` đặc biệt hữu ích: chỉ với một dòng, thay thế hoàn toàn khối code thủ công (tạo `Map`, kiểm tra key tồn tại, khởi tạo `List` mới cho key lần đầu, thêm phần tử...) mà bạn từng phải tự viết ở bài tập Chương 26.

Khi nào nên dùng Stream, khi nào nên dùng vòng lặp thường?

Stream **không thay thế hoàn toàn** vòng lặp `for / for-each` — cả hai đều là công cụ hợp lệ:

- **Ưu tiên Stream** khi thao tác là "lọc - biến đổi - gom kết quả" theo kiểu tuyến tính, không có logic điều khiển phức tạp (không cần `break` giữa chừng, không có nhiều nhánh rẽ phụ thuộc lẫn nhau) — Stream thường đọc rõ ràng và ngắn gọn hơn nhiều trong các trường hợp này.
- **Ưu tiên vòng lặp thường** khi logic phức tạp, cần `break / continue` có điều kiện phức tạp, cần cập nhật nhiều biến cùng lúc theo cách không tự nhiên với mô hình "biến đổi tuần tự" — cố ép logic phức tạp vào Stream nhiều khi làm code **khó đọc hơn**, phản tác dụng.

Bài tập

1. Dùng Stream tính tổng bình phương của các số chẵn trong một `List<Integer>` (`filter` → `map` → `reduce` , hoặc dùng `mapToInt(...).sum()`).
2. Dùng `Collectors.groupingBy` nhóm một `List<NguoiDung>` (tương tự Chương 27) theo độ tuổi chẵn hay lẻ (key là `"chan"` / `"le"` , tự viết một `Function` trả về chuỗi đó dựa trên `getTuoi() % 2`).
3. Viết lại bài toán "đếm số lần xuất hiện" đã làm ở Chương 26 (dùng `Map` / `merge`) bằng Stream, sử dụng `Collectors.groupingBy(tu -> tu, Collectors.counting())` .

Lỗi thường gặp

- **Dùng lại một Stream đã gọi thao tác kết thúc** — ném `IllegalStateException` (`stream has already been operated upon or closed`). Mỗi Stream chỉ dùng được **một lần**; cần gọi lại `.stream()` từ nguồn nếu muốn xử lý lại.
- **Tưởng `filter` / `map` sửa danh sách gốc** — Stream **không bao giờ** sửa nguồn dữ liệu ban đầu, luôn tạo ra kết quả **mới** qua thao tác kết thúc (`collect`). Danh sách gốc (`ten` trong ví dụ trên) luôn giữ nguyên.
- **Viết pipeline Stream quá dài, quá nhiều bước lồng nhau, khó đọc hơn cả vòng lặp thường** — nếu một pipeline Stream cần đọc đi đọc lại nhiều lần mới hiểu, đó là dấu hiệu nên tách thành các bước nhỏ hơn (gán kết quả trung gian cho biến có tên rõ ràng), hoặc quay lại dùng vòng lặp thường nếu hợp lý hơn.
- **Gọi thao tác kết thúc mà quên gán/dùng kết quả trả về** — ví dụ chỉ viết `danhSach.stream().filter(...)` một mình (thiếu thao tác kết thúc) — do tính chất "lazy", dòng này **không làm gì cả**, không có lỗi biên dịch/runtime, chỉ đơn giản là vô nghĩa. Luôn kết thúc pipeline bằng một thao tác kết thúc và dùng kết quả của nó.

Chương 32 — Optional và xử lý null an toàn

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vấn đề `null` gây ra trong code Java, vì sao nó được gọi đùa là "lỗi tỷ đô".
- Dùng `Optional<T>` để biểu diễn tường minh "giá trị có thể không tồn tại".
- Dùng đúng các phương thức của `Optional`: `orElse`, `orElseGet`, `ifPresent`, `map`.
- Biết khi nào nên và không nên dùng `Optional`.

Code mẫu đầy đủ: [code/ch32-optional/](#).

Vấn đề với null

Từ Chương 13, bạn đã biết `null` là giá trị "chưa trỏ tới object nào" của kiểu tham chiếu, và `NullPointerException` là lỗi runtime cực kỳ phổ biến. Vấn đề cốt lõi: **không có gì trong chữ ký phương thức cho biết giá trị trả về có thể là `null` hay không**:

```
String email = diaChiEmail.get("chi"); // co the la null, nhưng KHÔNG CÓ GI bao truoac dieu do
System.out.println(email.length());    // NullPointerException neu "chi" khong ton tai
```

Người gọi `diaChiEmail.get("chi")` **không có cách nào biết chắc** (chỉ dựa vào chữ ký phương thức) liệu kết quả có thể `null` hay không — phải đọc tài liệu, hoặc đọc code cài đặt bên trong, hoặc đơn giản là **quên kiểm tra** và gặp lỗi lúc chạy. `Optional<T>` (Java 8+) giải quyết đúng vấn đề này.

Optional<T> — biểu diễn tường minh "có thể không có giá trị"

```
Optional<String> optCoGiaTri = Optional.of("gia tri co san");    // BAT BUOC khac null, nem loi ngay neu truyen n
Optional<String> optRong = Optional.empty();                    // KHÔNG co gia tri
Optional<String> opt = Optional.ofNullable(diaChiEmail.get("chi")); // AN TOAN du gia tri co the null
```

- `Optional.of(giaTri)` — bọc một giá trị **chắc chắn không `null`**; nếu bạn truyền `null` vào, nó **chủ động ném `NullPointerException` ngay lập tức** (thà lỗi sớm, rõ ràng, còn hơn để lỗi lan xa mới phát hiện).
- `Optional.empty()` — biểu diễn tường minh "không có giá trị gì" (thay thế cho `null`).
- `Optional.ofNullable(giaTriCoTheNull)` — cách dùng phổ biến nhất: tự động chuyển giá trị `null` thành `Optional.empty()`, giá trị không `null` thành `Optional.of(...)`.

Kiểm tra và lấy giá trị an toàn

```
opt.isPresent(); // co gia tri khong?
opt.isEmpty();  // RONG khong? (nguoc lai voi isPresent, tu Java 11)

String ketQua = opt.orElse("gia-tri-mac-dinh"); // lay gia tri, hoac gia tri thay the neu rong
```

`orElse(giaTriMacDinh)` là cách an toàn để "mở" `Optional` ra lấy giá trị bên trong, luôn có kết quả cụ thể (không bao giờ `null`), không cần `if/else` kiểm tra `isPresent()` thủ công.

orElseGet() — chỉ tính giá trị thay thế khi thực sự cần

```
String ketQua = opt.orElseGet(() -> taoGiaTriMacDinhTonKem());
```

Khác biệt với `orElse`: `orElseGet` nhận một `Supplier<T>` (Chương 30) — hàm này **chỉ được gọi khi `Optional` thực sự rỗng**. Với `orElse(giaTriMacDinh)`, biểu thức `giaTriMacDinh` **luôn được tính toán trước**, dù `Optional` có giá trị hay không (rồi bị bỏ đi nếu không cần dùng tới) — nếu việc tính giá trị mặc định đó tốn kém (gọi API, truy vấn database...), `orElseGet` hiệu quả hơn hẳn vì tránh được chi phí không cần thiết.

Thực hiện hành động có điều kiện

```
opt.ifPresent(giaTri -> System.out.println("Co gia tri: " + giaTri));

opt.ifPresentOrElse(
    giaTri -> System.out.println("Co gia tri: " + giaTri),
    () -> System.out.println("Khong co gia tri nao")
);
```

`ifPresent` (nhận một `Consumer<T>`, Chương 30) chỉ chạy hành động **nếu có giá trị**, thay thế cho:

```
if (opt.isPresent()) {
    System.out.println("Co gia tri: " + opt.get());
}
```

`ifPresentOrElse` (Java 9+) thêm nhánh xử lý cho trường hợp rỗng, tương đương `if/else` đầy đủ.

map() — biến đổi giá trị bên trong Optional

```
Optional<String> optHoa = opt.map(String::toUpperCase);
```

Nếu `opt` **có** giá trị, `map` áp dụng hàm biến đổi và trả về `Optional` chứa kết quả mới; nếu `opt` **rỗng**, `map` trả về `Optional.empty()` luôn, **không** gọi hàm biến đổi (tránh `NullPointerException` nếu hàm biến đổi không xử lý được `null`). Đây chính là ý tưởng tương tự `map` của `Stream` (Chương 31) — thực tế `Optional` có thể xem như "một `Stream` chứa tối đa một phần tử".

Thiết kế phương thức trả về Optional<T> thay vì T (có nguy cơ null)

```
static Optional<String> timEmail(Map<String, String> danhBa, String ten) {
    return Optional.ofNullable(danhBa.get(ten));
}
```

Đây là cách dùng `Optional` **quan trọng và phổ biến nhất** trong thực tế: khi viết một phương thức mà kết quả **có thể không tồn tại**, khai báo kiểu trả về là `Optional<T>` thay vì `T`. Chữ ký phương thức tự nó đã "nói rõ": "*kết quả này có thể không có, bạn (người gọi) bắt buộc phải xử lý trường hợp đó*" — người gọi **không thể** vô tình quên kiểm tra như với `null` (họ phải chủ động gọi `orElse` / `get()` / `ifPresent()` ... để lấy giá trị ra, quá trình đó tự nhiên buộc họ đối mặt với khả năng rỗng).

Khi nào KHÔNG nên dùng Optional

- **Không dùng `Optional` làm kiểu của field trong class** — `Optional` được thiết kế chủ yếu cho **kiểu trả về của phương thức**, không phải để lưu trữ trạng thái lâu dài; dùng làm field thường gây phức tạp không cần thiết (và không implements `Serializable` — sẽ gặp khi học Chương 36).
- **Không dùng `Optional` làm kiểu tham số phương thức** — nếu một tham số có thể "không có giá trị", cách thông thường hơn là dùng overload (Chương 10) cung cấp hai phiên bản phương thức, hoặc chấp nhận `null` với tài liệu rõ ràng, thay vì bắt người gọi phải bọc `Optional.of(...)` mỗi lần gọi.
- **Không gọi `.get()` trực tiếp mà không kiểm tra trước** — `Optional.get()` (không có trong ví dụ ở trên, cố tình tránh dùng) ném `NoSuchElementException` nếu `Optional` rỗng — dùng trực tiếp `get()` mà không qua `isPresent()` / `orElse()` gần như tái tạo lại đúng vấn đề `null` mà `Optional` sinh ra để giải quyết.

Bài tập

- Viết phương thức `Optional<Integer> tìmViTri(int[] mang, int giaTri)` trả về vị trí đầu tiên tìm thấy `giaTri` trong `mang` (dùng `Optional.empty()` nếu không tìm thấy), thay cho việc trả về `-1` như Chương 28.
- Dùng `orElseThrow(...)` (tự tra cứu tài liệu Java, không có trong ví dụ trên) để ném một exception tự định nghĩa (Chương 21) khi `Optional` rỗng, thay vì dùng giá trị mặc định.
- Viết một chuỗi `map()` liên tiếp trên `Optional<String>`: đầu tiên `trim()`, sau đó `toLowerCase()`, cuối cùng `orElse("khong-co")` — quan sát nếu `Optional` gốc rỗng, mọi bước `map` ở giữa đều tự động bị bỏ qua mà không cần kiểm tra thủ công ở từng bước.

Lỗi thường gặp

- Gọi `Optional.get()` mà không kiểm tra trước** — tái tạo lại đúng vấn đề `NullPointerException` dưới một cái tên khác (`NoSuchElementException`). Luôn dùng `orElse` / `orElseGet` / `ifPresent`.
- Dùng `Optional.of(giaTri)` khi `giaTri` có thể là `null`** — ném `NullPointerException` ngay tại đó. Dùng `Optional.ofNullable(...)` nếu không chắc chắn giá trị luôn khác `null`.
- So sánh hai `Optional` bằng `==`** — giống mọi kiểu tham chiếu khác (Chương 11, 13), `==` so sánh tham chiếu, không so sánh nội dung; dùng `.equals()` nếu cần so sánh nội dung hai `Optional`.
- Lạm dụng `Optional` ở khắp mọi nơi, kể cả khi giá trị chắc chắn không bao giờ rỗng** — thêm độ phức tạp không cần thiết. Chỉ dùng `Optional` khi việc "không có giá trị" là một tình huống hợp lệ, có thật trong logic nghiệp vụ.

Chương 33 — Java hiện đại: var, record, sealed, pattern matching

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng `var` đúng chỗ, hiểu nó không phải "kiểu động".
- Dùng `record` thay cho class dữ liệu bất biến, tránh code lặp lại đã phải tự viết ở Chương 20.
- Hiểu `sealed` giới hạn tường minh danh sách lớp con hợp lệ.
- Dùng pattern matching cho `switch` (Java 21+) — cách viết hiện đại thay cho `instanceof` + ép kiểu thủ công đã học ở Chương 17.

Code mẫu đầy đủ: [code/ch33-java-hien-dai/](#).

Đây là chương tổng hợp các tính năng hiện đại của Java (11-21) mà các chương trước đã vài lần nhắc trước tên và hẹn giải thích đầy đủ ở đây — bao gồm `record` (nhắc ở Chương 20), `sealed` / pattern matching cho `switch` (nhắc ở Chương 17).

var — suy luận kiểu cục bộ (Java 10+)

```
var ten = "Nguyen Van A";           // suy luận là String
var tuoi = 25;                       // suy luận là int
var danhSach = new ArrayList<String>(); // suy luận là ArrayList<String>
```

Hiểu lầm phổ biến nhất cần tránh ngay: `var` **không** biến Java thành ngôn ngữ kiểu động (dynamic typing) như JavaScript/Python. Kiểu của biến vẫn **hoàn toàn cố định**, được trình biên dịch **suy luận và gán chết** ngay tại dòng khai báo, dựa trên giá trị khởi tạo — bạn chỉ đơn giản không phải **gõ ra** kiểu đó. Gán lại giá trị **khác kiểu** cho biến `var` sau đó vẫn là lỗi biên dịch, y hệt như khai báo kiểu tường minh.

`var` hữu ích nhất khi kiểu về phải đã **rõ ràng và dài dòng**, tránh lặp lại thừa thãi:

```
Map<String, List<Integer>> banDo = new HashMap<String, List<Integer>>(); // lặp lại kiểu 2 LẦN
var banDo = new HashMap<String, List<Integer>>();                       // gọn hơn, vẫn RÕ RÀNG kiểu gì
```

Giới hạn của `var`: chỉ dùng được cho **biến cục bộ có giá trị khởi tạo ngay tại chỗ khai báo** — không dùng được cho field của class, tham số phương thức, hay khai báo biến không gán giá trị ngay. Cũng nên **tránh lạm dụng** `var` khi kiểu về phải không rõ ràng ngay từ tên biến/giá trị (ví dụ `var ketQua = xuLy();` — nếu không đọc kỹ `xuLy()` trả về gì, code khó hiểu hơn so với ghi rõ kiểu tường minh).

record — class dữ liệu bất biến, không code lặp lại

Nhắc lại Chương 20: viết một class dữ liệu đúng cách đòi hỏi tự viết constructor, getter, `equals`, `hashCode`, `toString` — khá nhiều code lặp đi lặp lại cho mọi class dữ liệu tương tự. `record` (Java 16+) tự động sinh **toàn bộ** phần đó chỉ từ một dòng khai báo:

```
record DiemToaDo(int x, int y) {
}
```

Dòng trên **tự động có sẵn**:

```
DiemToaDo d = new DiemToaDo(3, 4);
d.x();           // getter TỰ SINH, KHÔNG có tiền tố "get" (khác quy ước getter thông thường)
d.y();
```

```
d.toString(); // "DiemToaDo[x=3, y=4]"
d.equals(...); // so sanh theo NOI DUNG (tung field), giống het khuôn mau Chuong 20
d.hashCode(); // nhat quan voi equals(), dung quy tac
```

record **luôn bất biến (immutable)** — không có setter, không có cách nào gán lại `x / y` sau khi tạo object. Đây là lựa chọn thiết kế có chủ đích: `record` dành riêng cho các trường hợp bạn cần một "gói dữ liệu" cố định, không cần (và không nên) thay đổi sau khi tạo.

Compact constructor — kiểm tra hợp lệ

```
record KhoangCach(double giaTri) {
    KhoangCach { // "compact constructor" - KHÔNG lap lai danh sach tham so trong ngoac
        if (giaTri < 0) {
            throw new IllegalArgumentException("Khoang cach khong the am");
        }
    }
}
```

Nếu cần kiểm tra tính hợp lệ (như validation ở setter Chương 14), `record` cho phép viết **compact constructor** — không cần lặp lại (`double giaTri`) và `this.giaTri = giaTri;` (Java tự thêm phần gán field ở cuối, sau đoạn kiểm tra bạn viết).

Khi nào dùng record, khi nào dùng class thường?

Dùng `record` khi class **chỉ đơn thuần chứa dữ liệu bất biến** (tương tự các ví dụ `SanPham` ở Chương 20, 31 — nếu viết lại bằng `record` sẽ ngắn hơn hẳn). Dùng class thường (Chương 13) khi cần **field có thể thay đổi** (như `TaiKhoanNganHang` ở Chương 14, số dư thay đổi theo giao dịch), hoặc cần logic kế thừa phức tạp (record không `extends` được class khác, dù vẫn `implements` được interface).

sealed — giới hạn tường minh danh sách lớp con (Java 17+)

```
sealed interface HinhDang permits HinhTron, HinhVuong, HinhTamGiac {
}

record HinhTron(double banKinh) implements HinhDang { }
record HinhVuong(double canh) implements HinhDang { }
record HinhTamGiac(double day, double chieuCao) implements HinhDang { }
```

Nhắc lại Chương 18: interface /abstract class thông thường cho phép **bất kỳ** class nào implements/extends nó, kể cả những class được viết ở nơi hoàn toàn khác, sau này. `sealed` **giới hạn tường minh**: chỉ đúng các class liệt kê trong `permits` mới được phép implements/extends — không class nào khác được phép, kể cả trong tương lai (trừ khi sửa lại chính khai báo `permits`).

Pattern matching cho switch — kết hợp với sealed

```
static double tinhDienTich(HinhDang hinh) {
    return switch (hinh) {
        case HinhTron h -> Math.PI * h.banKinh() * h.banKinh();
        case HinhVuong h -> h.canh() * h.canh();
        case HinhTamGiac h -> h.day() * h.chieuCao() / 2;
    };
}
```

So sánh với cách làm ở Chương 17 (`instanceof` + ép kiểu thủ công từng `if`), pattern matching cho `switch` (Java 21+) gộp tất cả thành **một khối gọn gàng**: mỗi `case Kiểu biến -> ...` tự động kiểm tra kiểu **và** ép kiểu vào `biến` cùng lúc.

Lợi ích lớn nhất khi kết hợp với `sealed` : trình biên dịch **biết chắc** danh sách `permits` là đầy đủ mọi khả năng, nên **không đòi hỏi nhánh `default`** — nếu bạn (hoặc đồng nghiệp) sau này thêm một `record` `HinhChuNhat` implements `HinhDang` mới vào `permits` , nhưng **quên** thêm `case` tương ứng trong `tinhDienTich` , trình biên dịch sẽ **báo lỗi ngay** (switch không còn bao phủ đủ mọi trường hợp) — thay vì để lỗi âm thầm lọt qua tới lúc chạy (như sẽ xảy ra nếu dùng `if/else` / `instanceof` thông thường, dễ quên bổ sung nhánh mới).

Bài tập

- Viết `record` `NguoiDung(String ten, int tuoi)` , thêm compact constructor kiểm tra `tuoi >= 0` .
- Thêm `HinhChuNhat` vào `permits` của `HinhDang` trong code mẫu, cập nhật `tinhDienTich` — quan sát nếu bạn **quên** cập nhật `tinhDienTich` , trình biên dịch báo lỗi ngay ("switch không bao phủ đủ mọi trường hợp").
- Viết lại bài tập Chương 20 (class `SanPham` với `equals` / `hashCode` / `toString` tự viết tay) bằng `record` , so sánh độ dài code giữa hai cách.

Lỗi thường gặp

- Tưởng `var` cho phép đổi kiểu biến sau này** — kiểu vẫn cố định ngay từ lúc khai báo, `var` chỉ là cú pháp tắt, không phải kiểu động.
- Dùng `var` cho biến không có giá trị khởi tạo ngay, hoặc cho field/tham số** — không hợp lệ, lỗi biên dịch. `var` chỉ dùng cho biến cục bộ có khởi tạo tại chỗ.
- Cố sửa field của `record` sau khi tạo** — không có setter, đây là thiết kế cố ý (bất biến), lỗi biên dịch nếu cố gán trực tiếp (`d.x = 10`; không tồn tại cú pháp này với `record`).
- Cố tạo một class implements một `sealed` interface mà không có trong `permits`** — lỗi biên dịch. Phải sửa `permits` của interface gốc để thêm class mới vào danh sách được phép.
- Quên rằng pattern matching switch không cần `default` chỉ khi switch trên kiểu `sealed` đầy đủ mọi nhánh** — với các kiểu khác (không `sealed`), vẫn cần `default` hoặc bao phủ đủ mọi trường hợp có thể, nếu không trình biên dịch báo lỗi thiếu nhánh xử lý.

Chương 34 — File I/O: [java.io](#) và [java.nio.file](#)

Mục tiêu học

Sau chương này, bạn sẽ:

- Đọc/ghi file bằng cách cổ điển (`java.io` : `BufferedReader` / `BufferedWriter`).
- Đọc/ghi file bằng cách hiện đại, ngắn gọn hơn (`java.nio.file` : `Files` , `Path`).
- Hiểu `IOException` là checked exception (nhắc lại Chương 21), và luôn dùng `try-with-resources` khi làm việc với file.

Code mẫu đầy đủ: [code/ch34-file-io/](#) .

Cách cổ điển: `java.io`

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("data.txt"))) {
    writer.write("Dong thu nhat");
    writer.newLine();
    writer.write("Dong thu hai");
} catch (IOException e) {
    System.out.println("Loi khi ghi file: " + e.getMessage());
}
```

`FileWriter` mở file để ghi; bọc thêm `BufferedWriter` bên ngoài để **ghi theo vùng đệm** (buffer) — gom nhiều thao tác ghi nhỏ lại rồi ghi xuống đĩa một lần, hiệu quả hơn nhiều so với ghi trực tiếp từng ký tự/dòng một xuống đĩa (thao tác đĩa vật lý chậm hơn thao tác bộ nhớ rất nhiều lần). `writer.newLine()` xuống dòng — khác với `println` (Chương 3), `write` **không tự động** xuống dòng, phải gọi `newLine()` tường minh.

`IOException` — nhắc lại từ Chương 21: đây là **checked exception**, vì thao tác file có thể thất bại vì nhiều lý do nằm ngoài tầm kiểm soát của code (hết dung lượng ổ đĩa, mất quyền ghi, file bị chương trình khác khoá...) — Java buộc bạn phải chủ động xử lý khả năng này, không cho phép bỏ qua.

```
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String dong;
    while ((dong = reader.readLine()) != null) {
        System.out.println(dong);
    }
} catch (IOException e) {
    System.out.println("Loi khi doc file: " + e.getMessage());
}
```

Mẫu `while ((dong = reader.readLine()) != null)` là khuôn mẫu **kinh điển** để đọc file theo từng dòng trong Java: `readLine()` trả về `null` khi đã đọc hết file — vòng lặp vừa **gán** giá trị đọc được cho `dong`, vừa **kiểm tra** điều kiện dừng trong cùng một biểu thức.

Cách hiện đại: `java.nio.file`

```
Path duongDan = Path.of("data.txt");

Files.write(duongDan, List.of("Dong 1", "Dong 2", "Dong 3")); // ghi TOAN BO 1 lan
List<String> cacDong = Files.readAllLines(duongDan);           // doc TOAN BO thanh List<String>
String noiDung = Files.readString(duongDan);                  // doc TOAN BO thanh 1 String (Java 11+)
```

`java.nio.file` (New I/O, hoàn thiện dần từ Java 7 đến Java 11) cung cấp cách viết **ngắn gọn hơn nhiều** cho các tác vụ file phổ biến — không cần tự quản lý vòng lặp đọc từng dòng hay `BufferedReader` / `BufferedWriter` thủ công cho những trường

hợp đơn giản. `Path` là kiểu hiện đại thay thế `File` (lớp cũ từ `java.io`), dùng `Path.of(...)` để tạo.

Ghi thêm vào cuối file (append)

```
Files.writeString(duongDan, "\nDong moi", StandardOpenOption.APPEND);
```

Mặc định, `Files.write` / `Files.writeString` **ghi đè toàn bộ** nội dung file cũ. Truyền thêm `StandardOpenOption.APPEND` để **thêm vào cuối** thay vì ghi đè.

Các thao tác tiện ích khác

```
Files.exists(duongDan); // file co ton tai khong?
Files.delete(duongDan); // xoa file
```

Khi nào chọn cách nào?

Tình huống	Nên chọn
File nhỏ, đọc/ghi toàn bộ nội dung một lần	<code>java.nio.file</code> (<code>Files.readAllLines/readString/write</code>) — ngắn gọn nhất
File lớn, cần xử lý từng dòng khi đọc (không muốn nạp cả file vào bộ nhớ)	<code>java.io</code> (<code>BufferedReader</code>) hoặc <code>Files.lines(...)</code> (trả về <code>Stream<String></code> , Chương 31)
Code cũ đã có sẵn, cần đọc hiểu/bảo trì	<code>java.io</code> — vẫn rất phổ biến trong code hiện có

Bài tập

- Viết chương trình đọc một file văn bản (tự tạo file mẫu vài dòng), đếm tổng số dòng và tổng số ký tự, dùng cả hai cách (`java.io` và `java.nio.file`) để so sánh độ ngắn gọn.
- Viết chương trình ghi danh sách điểm số (nhập từ `Scanner`, Chương 5) ra file, mỗi điểm một dòng, dùng `Files.write`.
- Viết chương trình đọc lại file điểm số ở bài 2, tính điểm trung bình (gợi ý: đọc bằng `Files.readAllLines`, dùng `Stream` — Chương 31 — để `map` từng dòng `String` sang `Double` rồi `reduce` / tính trung bình).

Lỗi thường gặp

- `FileNotFoundException`** (một loại `IOException`) khi đọc file không tồn tại — kiểm tra `Files.exists(...)` trước, hoặc bắt exception và xử lý phù hợp (ví dụ tạo file mới nếu hợp lý).
- Quên `try-with-resources`, tự quản lý `close()` thủ công rồi quên gọi khi có lỗi giữa chừng** — dẫn tới rò rỉ tài nguyên (file handle không được giải phóng). Luôn ưu tiên `try-with-resources` (Chương 21) với `java.io`.
- Ghi đè nhầm file quan trọng vì quên `Files.write` mặc định ghi đè toàn bộ nội dung cũ** — dùng `StandardOpenOption.APPEND` nếu ý định là thêm vào, không phải thay thế.
- Đọc file rất lớn bằng `Files.readAllLines` / `readString` (nạp toàn bộ vào bộ nhớ cùng lúc)** — với file quá lớn so với bộ nhớ máy, có thể gây `OutOfMemoryError`. Với file lớn, ưu tiên đọc theo dòng (`BufferedReader` hoặc `Files.lines(...)` trả về `Stream`, xử lý tuần tự không nạp hết vào bộ nhớ cùng lúc).
- Đường dẫn file tương đối ("data.txt") chạy đúng trong IDE nhưng lỗi khi chạy bằng dòng lệnh từ thư mục khác** — đường dẫn tương đối luôn tính từ **thư mục làm việc hiện tại** (nơi bạn gõ lệnh `java`), không phải thư mục chứa file `.java` / `.class`. Dùng đường dẫn tuyệt đối nếu cần chắc chắn vị trí file, hoặc luôn chạy từ đúng thư mục dự kiến.

Chương 35 — Xử lý JSON/CSV

Mục tiêu học

Sau chương này, bạn sẽ:

- Đọc/ghi file CSV bằng các công cụ đã học (`String.split` , `StringBuilder`), không cần thư viện ngoài.
- Hiểu cấu trúc JSON, tự viết được một bộ sinh JSON đơn giản.
- Biết cách dùng thư viện Gson để đọc/ghi JSON đúng chuẩn, và hiểu vì sao quản lý thư viện ngoài bằng tay khá phiền phức — điều Chương 42 sẽ giải quyết triệt để.

Code mẫu đầy đủ: [code/ch35-json-csv/](#) .

CSV — định dạng đơn giản, không cần thư viện

CSV (Comma-Separated Values) là định dạng văn bản đơn giản: mỗi dòng là một bản ghi, các giá trị cách nhau bởi dấu phẩy. Với dữ liệu đơn giản (không có dấu phẩy bên trong giá trị), bạn hoàn toàn tự đọc/ghi được chỉ bằng kiến thức đã học:

```
// Ghi
StringBuilder noiDung = new StringBuilder();
noiDung.append("ten,tuoi,diemTrungBinh\n");
for (SinhVien sv : danhSach) {
    noiDung.append(sv.ten()).append(",").append(sv.tuoi()).append(",").append(sv.diemTrungBinh()).append("\n");
}
Files.writeString(duongDan, noiDung.toString());

// Doc
List<String> cacDong = Files.readAllLines(duongDan);
for (int i = 1; i < cacDong.size(); i++) { // bo qua dong header
    String[] cot = cacDong.get(i).split(",");
    String ten = cot[0];
    int tuoi = Integer.parseInt(cot[1]);
    double diem = Double.parseDouble(cot[2]);
    // ...
}
```

Dòng đầu tiên (header) thường ghi tên các cột, và khi đọc lại bạn bắt đầu duyệt từ dòng **thứ hai** (`i = 1`) để bỏ qua nó. Với dữ liệu thực tế phức tạp hơn (giá trị chứa dấu phẩy, xuống dòng bên trong một trường...), cần thư viện chuyên dụng (ví dụ Apache Commons CSV) thay vì tự viết `split` đơn giản — nhưng với phần lớn bài tập và dữ liệu đơn giản, cách trên là đủ dùng.

JSON — cấu trúc và cách tự viết đơn giản

JSON (JavaScript Object Notation, dù tên có "JavaScript" nhưng là định dạng trung lập, dùng được với mọi ngôn ngữ) là định dạng phổ biến nhất để trao đổi dữ liệu có cấu trúc giữa các hệ thống — ứng dụng web, API, file cấu hình đều thường dùng JSON.

```
{
  "ten": "Nguyen Van A",
  "tuoi": 20,
  "diemTrungBinh": 8.5
}
```

Cấu trúc cơ bản, dễ liên tưởng tới các kiểu dữ liệu Java đã học:

- **Object** — `{ "key": value, ... }` — giống `Map<String, ...>` (Chương 26).
- **Array** — `[ value, value, ... ]` — giống `List` (Chương 24).

- **Kiểu giá trị:** chuỗi ("..."), số, `true / false`, `null`, hoặc lồng thêm object/array khác.

Tự viết một bộ sinh JSON đơn giản (chỉ để hiểu cấu trúc, chưa xử lý đầy đủ các trường hợp đặc biệt như ký tự cần escape trong chuỗi):

```
static String toJson(SinhVien sv) {
    return "{"
        + "\"ten\": \"" + sv.ten() + "\", "
        + "\"tuoi\": " + sv.tuoi() + ", "
        + "\"diemTrungBinh\": " + sv.diemTrongBinh()
        + "}";
}
```

Đây là cách để hiểu định dạng, không phải cách nên dùng trong dự án thật — tự viết code nối chuỗi thủ công như trên rất dễ sai (quên escape ký tự đặc biệt, dấu phẩy thừa/thiếu ở object rỗng...) khi dữ liệu phức tạp hơn một chút. Dự án thực tế luôn dùng thư viện chuyên dụng.

Dùng thư viện Gson

```
Gson gson = new GsonBuilder().setPrettyPrinting().create();

String json = gson.toJson(sv); // Java object -> chuỗi JSON
SinhVien svDaDoc = gson.fromJson(jsonDauVao, SinhVien.class); // chuỗi JSON -> Java object
```

Gson (thư viện mã nguồn mở của Google) tự động **ánh xạ** giữa object Java và JSON dựa trên **tên field** — bạn không cần tự viết code nối chuỗi hay parse thủ công như `SimpleJsonWriter` ở trên, Gson xử lý đúng mọi trường hợp phức tạp (escape ký tự, object/array lồng nhau, kiểu dữ liệu khác nhau...) mà bạn không cần tự lo.

Vấn đề: làm sao "có" được thư viện Gson?

Khác với mọi thứ bạn dùng từ đầu sách (`String`, `ArrayList`, `Scanner` ...) — những thứ này nằm **sẵn trong JDK**, không cần cài đặt gì thêm — Gson là thư viện **của bên thứ ba**, không đi kèm JDK. Muốn dùng nó, phải:

1. Tải file `.jar` chứa thư viện (từ trang chính thức, hoặc kho lưu trữ như Maven Central).
2. Thêm đường dẫn tới file `.jar` đó vào **classpath** khi biên dịch và chạy — tham số `-cp` của `javac / java` (xem chi tiết trong README code mẫu).

Với **một** thư viện, việc này còn quản lý được thủ công. Nhưng dự án thực tế thường dùng **hàng chục** thư viện, mỗi thư viện lại phụ thuộc vào các thư viện khác nữa (gọi là "dependency của dependency") — quản lý tất cả bằng tay gần như bất khả thi. Đây chính xác là vấn đề **Maven** và **Gradle** (Chương 42) sinh ra để giải quyết: khai báo thư viện cần dùng trong một file cấu hình, công cụ tự động tải về đúng phiên bản và xử lý toàn bộ classpath cho bạn.

Bài tập

1. Mở rộng `CsvDemo.java` để ghi thêm một cột `xepLoai` (tính từ `diemTrungBinh`, dùng lại logic Chương 7), đọc lại và xác nhận dữ liệu đúng.
2. Mở rộng `SimpleJsonWriter.java` để sinh JSON cho một **danh sách** `SinhVien` (dạng JSON array `[ {...}, {...} ]`), dùng `Stream` (Chương 31) để nối các object JSON lại với nhau.
3. (Nếu đã tải được Gson) Thử `gson.toJson(...)` với một `List<SinhVien>`, quan sát Gson tự sinh ra đúng định dạng JSON array mà không cần bạn tự viết logic nối chuỗi nào.

Lỗi thường gặp

- Dùng `split(",")` với dữ liệu CSV có giá trị chứa dấu phẩy bên trong (ví dụ địa chỉ "123 Đường ABC, Quận 1") — bị tách sai thành nhiều cột hơn dự kiến. Với dữ liệu CSV thực tế phức tạp, cần thư viện CSV chuyên dụng thay vì `split` đơn

giản.

- **Tự viết JSON bằng nối chuỗi cho dữ liệu có ký tự đặc biệt** (dấu " , xuống dòng, ký tự Unicode trong chuỗi) — sinh ra JSON không hợp lệ, không parse lại được đúng. Đây chính là lý do nên dùng thư viện thật (Gson) cho bất kỳ việc gì ngoài mục đích học tập.
- **ClassNotFoundException: com.google.gson.Gson** khi chạy `GsonExample.java` — quên thêm `-cp` trỏ tới file `.jar` đã tải, hoặc dùng sai dấu phân cách classpath (; trên Windows, : trên macOS/Linux).
- **NumberFormatException** khi `Integer.parseInt` / `Double.parseDouble` một cột CSV không đúng định dạng số (ví dụ ô trống, hoặc lẫn ký tự chữ) — luôn cân nhắc `try/catch` (Chương 21) khi dữ liệu đầu vào không đảm bảo sạch tuyệt đối.

Chương 36 — Serialization cơ bản

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu serialization là gì, dùng `Serializable` để ghi/đọc object xuống/từ file.
- Dùng `serialVersionUID` và `transient` đúng cách.
- Biết khi nào **nên** và **không nên** dùng serialization kiểu Java gốc trong dự án thực tế.

Code mẫu đầy đủ: [code/ch36-serialization/](#).

Serialization là gì?

Serialization (tuần tự hoá) là quá trình chuyển một **object đang tồn tại trong bộ nhớ** thành một dạng có thể **lưu trữ** (xuống file) hoặc **truyền đi** (qua mạng), rồi khôi phục lại (**deserialization**) thành object y hệt trạng thái ban đầu ở nơi khác/thời điểm khác. Bạn đã thấy một dạng serialization ở Chương 35 (chuyển object thành JSON) — Java còn có cơ chế serialization **riêng, gắn liền với ngôn ngữ**, dùng định dạng nhị phân của chính nó (không phải văn bản như JSON).

Serializable — interface đánh dấu

```
public class NguoIDung implements Serializable {
    private static final long serialVersionUID = 1L;
    private String ten;
    private int tuoi;
}
```

`Serializable` (trong `java.io`) là một **marker interface** — không có phương thức nào cần override (khác với mọi interface đã học từ Chương 18), nó chỉ đơn thuần "đánh dấu" cho JVM biết class này được phép serialize. Cố serialize một object của class **không** implements `Serializable` sẽ ném `NotSerializableException` lúc chạy.

Ghi và đọc object

```
try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("data.ser"))) {
    out.writeObject(nguoIDung);
}

NguoIDung ndDaDoc;
try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("data.ser"))) {
    ndDaDoc = (NguoIDung) in.readObject(); // BAT BUOC ep kieu - readObject() tra ve Object
}
```

`ObjectOutputStream.writeObject(...)` chuyển toàn bộ trạng thái của object (giá trị mọi field không `transient` — xem phần dưới) thành dạng nhị phân, ghi xuống file. `ObjectInputStream.readObject()` đọc ngược lại, khôi phục một object **mới**, có cùng giá trị field như object gốc lúc ghi — `readObject()` trả về kiểu `Object` (Chương 20), nên luôn cần ép kiểu về đúng loại mong đợi.

serialVersionUID — "phiên bản" của cấu trúc class

```
private static final long serialVersionUID = 1L;
```

Khi đọc lại một file đã serialize từ trước, JVM kiểm tra `serialVersionUID` của class **hiện tại** có khớp với giá trị được lưu trong file hay không — nếu không khớp, ném `InvalidClassException`. Nếu bạn **không** khai báo `serialVersionUID` tường minh, Java **tự tính** một giá trị dựa trên cấu trúc class hiện tại (tên field, kiểu, phương thức...) — hệ quả: chỉ cần **sửa nhỏ** class sau này (thêm một field mới chẳng hạn), giá trị tự tính đó **thay đổi**, khiến mọi file `.ser` cũ (serialize từ phiên bản class trước khi sửa) **không đọc lại được nữa**. Luôn khai báo `serialVersionUID` tường minh (một số bất kỳ, thường bắt đầu từ `1L`) để chủ động kiểm soát khi nào coi là "không tương thích", thay vì để Java tự quyết định.

transient — loại trừ field khỏi serialization

```
private transient String matKauTamThoi;
```

Field đánh dấu `transient` **không được lưu** khi serialize — sau khi đọc lại (deserialize), field đó luôn có giá trị mặc định (`null` với kiểu tham chiếu, `0` / `false` với kiểu nguyên thủy), bất kể giá trị lúc ghi là gì. Dùng cho: dữ liệu **nhạy cảm** không nên lưu trữ lâu dài (mật khẩu tạm, token phiên đăng nhập), hoặc dữ liệu **không có ý nghĩa** khi khôi phục ở ngữ cảnh khác (kết nối mạng đang mở, file handle, các loại "tài nguyên sống" — Chương 21 — không thể "đóng băng" rồi khôi phục nguyên trạng).

Khi nào NÊN và KHÔNG NÊN dùng Java serialization

Java serialization (`Serializable`) ngày càng ít được khuyến khích dùng trong dự án thực tế hiện đại, vì một số lý do quan trọng:

- **Định dạng nhị phân riêng của Java** — chỉ chương trình Java khác mới đọc lại được, không trao đổi được với hệ thống viết bằng ngôn ngữ khác (JSON ở Chương 35 không có hạn chế này — bất kỳ ngôn ngữ nào cũng đọc/ghi được JSON).
- **Vấn đề bảo mật đã biết** — `readObject()` có thể bị lợi dụng để thực thi mã độc nếu chương trình deserialize dữ liệu từ nguồn **không đáng tin cậy** (ví dụ nhận dữ liệu serialize từ người dùng qua mạng) — đây là một trong những lỗ hổng bảo mật Java nổi tiếng nhất trong lịch sử, từng ảnh hưởng nhiều hệ thống lớn.
- **Khó quản lý phiên bản** — như đã thấy với `serialVersionUID`, thay đổi cấu trúc class theo thời gian dễ gây lỗi tương thích ngược với dữ liệu cũ.

Vấn hợp lý để dùng trong các trường hợp hẹp: lưu trạng thái tạm thời **nội bộ** trong cùng một hệ thống Java (cache, session tạm), hoặc học tập/thử nghiệm nhanh không cần trao đổi dữ liệu ra ngoài. Với hầu hết nhu cầu lưu trữ/trao đổi dữ liệu thực tế (file cấu hình, API, cơ sở dữ liệu), **JSON (Chương 35) hoặc lưu vào cơ sở dữ liệu (Chương 46 — JDBC) là lựa chọn phù hợp hơn nhiều**.

Bài tập

1. Chạy `SerializationDemo.java`, xác nhận `matKauTamThoi` là `null` sau khi đọc lại — thử xóa từ khóa `transient` khỏi field đó, chạy lại, quan sát giờ nó **được** lưu và khôi phục đúng.
2. Thử đổi `serialVersionUID` thành một giá trị khác **sau khi** đã ghi file `.ser` (không xóa file cũ), chạy lại phần đọc — quan sát `InvalidClassException` xảy ra.
3. Viết một class `DonHang` chứa một field `List<String>` các sản phẩm — xác nhận `List` (interface chuẩn của Java, và `ArrayList` — cài đặt cụ thể của nó) cũng implements `Serializable` sẵn, nên serialize được bình thường mà không cần thêm gì đặc biệt.

Lỗi thường gặp

- **Quên implements Serializable** — `NotSerializableException` lúc chạy khi cố `writeObject`.
- **Không khai báo serialVersionUID, sửa class sau đó, không đọc lại được file cũ** — luôn khai báo tường minh ngay từ đầu, xem phần giải thích ở trên.
- **Cố serialize object có field tham chiếu tới một object của class KHÔNG implements Serializable** — ném `NotSerializableException`, chỉ ra đúng class gây lỗi trong thông báo. Mọi field tham chiếu (không phải `transient`) cũng phải trỏ tới object serialize được.
- **Deserialize dữ liệu từ nguồn không tin cậy** (nhận qua mạng từ người dùng bên ngoài) — rủi ro bảo mật nghiêm trọng đã nêu ở trên; không bao giờ làm điều này trong hệ thống thực tế mà không có các lớp phòng vệ bổ sung nghiêm ngặt

(nằm ngoài phạm vi sách nhập môn này).

Chương 37 — Thread cơ bản

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu thread (luồng) là gì, vì sao cần lập trình đa luồng.
- Tạo thread bằng hai cách: `extends Thread` và `implements Runnable`, biết cách nào nên dùng hơn.
- Hiểu vòng đời của thread, dùng `start()`, `join()`, `sleep()` đúng cách.
- Hiểu thứ tự chạy giữa các thread độc lập **không được đảm bảo**, và vì sao điều đó quan trọng.

Code mẫu đầy đủ: [code/ch37-thread-co-ban/](#).

Thread là gì?

Mọi chương trình từ đầu sách đến giờ chạy theo **một luồng duy nhất** (single-threaded) — các câu lệnh thực thi tuần tự, từng dòng một, chỉ có một "việc" diễn ra tại một thời điểm. **Thread** (luồng) là một "đường chạy" độc lập của chương trình — một chương trình có thể có **hiều thread chạy song song** (đồng thời, hoặc xen kẽ rất nhanh nếu máy chỉ có một lõi CPU), mỗi thread thực hiện một công việc riêng, không phải đợi thread khác làm xong mới bắt đầu.

Dùng đa luồng khi cần: xử lý nhiều việc **độc lập** cùng lúc (ví dụ vừa tải dữ liệu từ mạng, vừa giữ giao diện phản hồi được với người dùng), hoặc tận dụng **hiều lõi CPU** để tính toán nhanh hơn bằng cách chia nhỏ công việc.

Cách 1: `extends Thread`

```
class DemSo extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("dem: " + i);
        }
    }
}

DemSo t = new DemSo();
t.start(); // KHÔNG gọi t.run() trực tiếp!
```

Override `run()` — đây là nơi chứa code sẽ chạy **trên thread mới**. **Điểm quan trọng nhất cần nhớ:** gọi `t.start()` để **thực sự bắt đầu chạy trên một thread mới**; nếu bạn vô tình gọi `t.run()` trực tiếp (như gọi một phương thức bình thường), code vẫn chạy, nhưng chạy **trên chính thread hiện tại** (thường là `main`), **không** tạo thread mới nào cả — mất hoàn toàn ý nghĩa của việc dùng Thread.

Cách 2: `implements Runnable` — được khuyến dùng hơn

```
class TacVu implements Runnable {
    @Override
    public void run() {
        System.out.println("Dang chay...");
    }
}

Thread t = new Thread(new TacVu());
t.start();
```

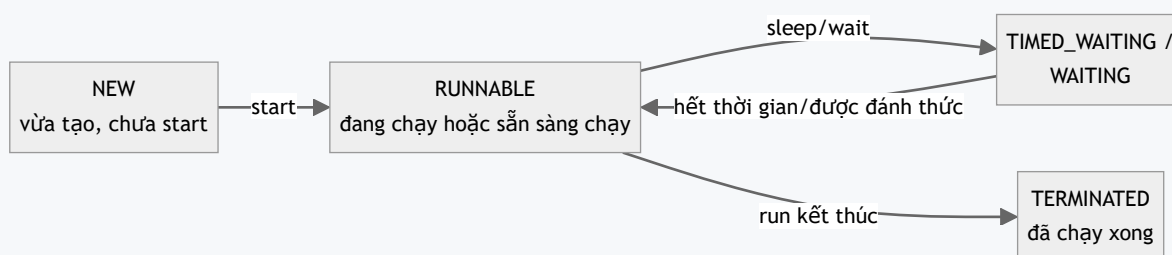
Thay vì `extends Thread` trực tiếp, tạo một class `implements Runnable` (chỉ định nghĩa **công việc cần làm**), rồi truyền nó vào constructor của `Thread`. Lý do cách này được khuyến dùng hơn: nhắc lại Chương 16, Java **không hỗ trợ đa kế thừa class** — nếu class của bạn đã `extends Thread`, nó **không còn "chỗ trống"** để `extends` thêm bất kỳ class nào khác nếu sau này cần. Dùng `implements Runnable` giữ được sự linh hoạt đó, vì một class `implements` được nhiều interface (Chương 18).

Viết gọn hơn bằng lambda (Chương 30)

```
Thread t = new Thread(() -> {
    System.out.println("Đang chạy...");
});
```

`Runnable` chỉ có **đúng một** phương thức abstract (`run()`) — đây chính xác là một functional interface (Chương 30), nên dùng lambda thay cho việc tự viết hẳn một class riêng là cách viết phổ biến nhất trong code Java hiện đại.

Vòng đời của Thread



`Thread.getState()` cho biết trạng thái hiện tại. Vài trạng thái quan trọng:

- **NEW** — object `Thread` đã tạo (`new Thread(...)`) nhưng **chưa** gọi `start()`.
- **RUNNABLE** — đã `start()`, đang chạy hoặc sẵn sàng được CPU cấp thời gian chạy.
- **TIMED_WAITING** — đang tạm dừng có thời hạn, ví dụ đang trong `Thread.sleep(...)`.
- **TERMINATED** — `run()` đã chạy xong hoàn toàn (hoặc kết thúc do lỗi không bắt được).

join() — chờ một thread khác chạy xong

```
t1.start();
t2.start();
t1.join(); // main DUNG LAI o day, cho t1 chạy xong HOÀN TOÀN
t2.join(); // rồi cho t2 chạy xong HOÀN TOÀN
System.out.println("Cả hai đã xong");
```

Nếu **không** gọi `join()`, thread gọi `start()` (thường là `main`) tiếp tục chạy ngay lập tức, **không đợi** thread mới tạo chạy xong — chương trình có thể kết thúc (hoặc `main` chạy tới dòng in kết quả) **trước khi** thread mới kịp làm xong việc. `join()` chặn thread hiện tại lại cho tới khi thread được gọi `join()` chuyển sang `TERMINATED`.

Thread.sleep(...) — tạm dừng có thời hạn

```
Thread.sleep(1000); // tạm dừng thread NAY 1000 mili-giay (1 giây)
```

`sleep` chỉ tạm dừng **thread đang gọi nó** — các thread khác (kể cả `main`, nếu gọi `sleep` từ một thread con) không bị ảnh hưởng, vẫn chạy bình thường. `sleep` ném `InterruptedException` (checked exception, Chương 21) nếu thread bị "đánh thức sớm" một cách chủ động từ bên ngoài — cần `try/catch` khi gọi.

Thứ tự chạy giữa các thread KHÔNG được đảm bảo

```
Thread a = new Thread(() -> System.out.println("A"));
Thread b = new Thread(() -> System.out.println("B"));
a.start();
b.start();
```

Đây là điều **quan trọng nhất** cần khắc sâu ở chương này: sau khi cả hai thread `start()`, **không có gì đảm bảo** `a` sẽ in "A" trước khi `b` in "B" — hệ điều hành quyết định thứ tự cấp thời gian chạy cho từng thread, có thể khác nhau **mỗi lần chạy chương trình**, thậm chí trên cùng một máy. Chạy `ThreadLifecycleDemo.java` vài lần để tự quan sát thứ tự "A"/"B" thay đổi giữa các lần chạy. Đây chính là nguồn gốc của nhiều lỗi đa luồng tinh vi — Chương 38 sẽ học cách xử lý khi nhiều thread cùng truy cập chung một dữ liệu, nơi thứ tự không đảm bảo này thực sự gây ra sai lệch dữ liệu.

Bài tập

1. Tạo 5 thread (dùng `Runnable` + lambda), mỗi thread in ra số thứ tự của nó (0 đến 4), chạy chương trình nhiều lần và quan sát thứ tự in ra không cố định.
2. Viết chương trình dùng `Thread.sleep` để mô phỏng "tải dữ liệu" mất 2 giây trên một thread riêng, trong khi `main` in ra "Đang xử lý..." mỗi 500ms cho tới khi thread tải xong (gợi ý: dùng vòng lặp kiểm tra `thread.isAlive()`).
3. So sánh thời gian chạy: tính tổng bình phương các số từ 1 đến 10.000.000 bằng (a) một vòng lặp đơn giản, (b) chia thành 2 thread mỗi thread xử lý một nửa dãy số rồi cộng kết quả lại (dùng `join()` để chờ cả hai xong). Trên máy nhiều lõi, cách (b) có thể nhanh hơn.

Lỗi thường gặp

- **Gọi `t.run()` thay vì `t.start()`** — chạy trên thread hiện tại, không tạo thread mới, mất hoàn toàn ý nghĩa đa luồng. Đây là lỗi rất phổ biến với người mới, vì cả hai đều biên dịch được bình thường, không có cảnh báo.
- **Quên `join()`, chương trình kết thúc trước khi thread con làm xong việc** — với chương trình console đơn giản, `main` kết thúc kéo theo toàn bộ tiến trình JVM kết thúc, thread con có thể bị "cắt ngang" nếu chưa kịp chạy xong (thread không phải "daemon" mặc định thì JVM sẽ đợi, nhưng để chắc chắn về thứ tự hoàn thành, luôn dùng `join()` tường minh nếu cần).
- **Gọi `start()` hai lần trên cùng một object `Thread`** — ném `IllegalThreadStateException`. Mỗi object `Thread` chỉ chạy được **đúng một lần**; cần tạo object `Thread` mới nếu muốn chạy lại.
- **Giả định thứ tự chạy giữa các thread độc lập** (viết logic phụ thuộc vào việc "thread A luôn in trước thread B") — không đảm bảo, có thể chạy đúng "tình cờ" trên máy bạn nhưng sai trên máy khác hoặc lần chạy khác. Nếu cần đảm bảo thứ tự, phải dùng cơ chế đồng bộ hoá (Chương 38) hoặc `join()` đúng chỗ.

Chương 38 — Đồng bộ hoá: synchronized, Lock

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu **race condition** là gì, tự tay tái hiện được lỗi này.
- Sửa race condition bằng `synchronized` (phương thức và khối lệnh).
- Dùng `ReentrantLock` như một lựa chọn thay thế linh hoạt hơn `synchronized`.
- Hiểu **deadlock** là gì, nguyên nhân gây ra nó, và cách phòng tránh cơ bản.

Code mẫu đầy đủ: <code/ch38-dong-bo-hoa/>.

Race condition — khi nhiều thread cùng sửa chung dữ liệu

```
static int dem = 0;

// 10 thread, mỗi thread chạy: for (...) { dem++; }
```

Trực giác thông thường: nếu 10 thread mỗi thread tăng `dem` đúng 100.000 lần, kết quả cuối cùng phải là `1.000.000`. **Thực tế không phải vậy** — chạy `RaceConditionDemo.java` để tự thấy kết quả thường **nhỏ hơn** con số mong đợi, và **khác nhau** giữa các lần chạy.

Nguyên nhân: `dem++` **trông như một thao tác**, nhưng thực chất gồm **ba bước** ở tầng thấp hơn: (1) đọc giá trị hiện tại của `dem`, (2) cộng thêm 1, (3) ghi giá trị mới trở lại. Nếu hai thread cùng thực hiện ba bước này **xen kẽ nhau** (ví dụ cả hai cùng đọc được giá trị `5` trước khi thread nào kịp ghi lại `6`), cả hai đều tính ra `6` và ghi đè lẫn nhau — kết quả là chỉ tăng được **một lần** dù cả hai thread đều đã thực thi `dem++`. Hiện tượng này gọi là **race condition** ("chạy đua điều kiện") — kết quả phụ thuộc vào **thứ tự ngẫu nhiên** các thread được CPU cấp thời gian chạy xen kẽ nhau (nhắc lại Chương 37: thứ tự này **không được đảm bảo**).

synchronized — đảm bảo chỉ một thread vào một lúc

Trên phương thức

```
static synchronized void tangDem() {
    dem++;
}
```

`synchronized` trên một phương thức đảm bảo: **tại một thời điểm, chỉ đúng một thread** được phép đang thực thi bên trong phương thức đó — mọi thread khác muốn gọi cùng phương thức phải **chờ** tới khi thread đang chạy thoát ra. Điều này loại bỏ hoàn toàn tình huống "đọc-sửa-ghi xen kẽ" đã gây ra race condition ở trên.

Trên một khối lệnh

```
Object khoa = new Object(); // object bất kỳ, dùng làm "mã khóa" chung

synchronized (khoa) {
    dem++;
}
```

Thay vì khoá **toàn bộ phương thức**, `synchronized (khoa) { ... }` chỉ khoá **đoạn code cụ thể** bên trong khối `{ }`, dùng `khoa` (một object bất kỳ) làm "chìa khoá chung" — mọi khối `synchronized (khoa)` ở bất kỳ đâu trong chương trình, dùng **cùng object** `khoa`, đều loại trừ lẫn nhau (chỉ một thread vào được tại một thời điểm). Cách này linh hoạt hơn: chỉ khoá đúng

phần code thực sự cần thiết, để phần còn lại của phương thức (nếu có) vẫn chạy song song bình thường, không bị chặn không cần thiết.

ReentrantLock — thay thế linh hoạt hơn cho synchronized

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

Lock khoa = new ReentrantLock();

khoa.lock();
try {
    dem++;
} finally {
    khoa.unlock(); // BAT BUOC trong finally - dam bao LUON mo khoa, ke ca khi co loi
}
```

`ReentrantLock` (trong `java.util.concurrent.locks`) làm việc tương tự `synchronized`, nhưng linh hoạt hơn ở vài điểm: `unlock()` phải gọi **tường minh** (không tự động như `synchronized` khi thoát khỏi lệnh) — đây là lý do **bắt buộc** đặt trong `finally` (Chương 21), để đảm bảo khoá luôn được giải phóng dù có exception xảy ra ở giữa. Đối lại, `ReentrantLock` cung cấp thêm khả năng như `tryLock()` — thử xin khoá mà **không chờ vô thời hạn**, trả về `true / false` ngay lập tức tùy có lấy được khoá hay không, hữu ích khi bạn muốn "thử rồi bỏ cuộc" thay vì chờ mãi mãi.

Lời khuyên thực dụng: `synchronized` đơn giản hơn, đủ dùng cho phần lớn tình huống thông thường; chỉ cần tới `ReentrantLock` khi thực sự cần các tính năng nâng cao của nó (`tryLock`, khoá có thể ngắt được, nhiều điều kiện chờ...).

Deadlock — hai (hoặc nhiều) thread khoá chéo nhau

```
// Thread-1: giu khoaA, roi CHO khoaB
synchronized (khoaA) {
    synchronized (khoaB) { ... }
}

// Thread-2: giu khoaB, roi CHO khoaA
synchronized (khoaB) {
    synchronized (khoaA) { ... }
}
```

Nếu Thread-1 giữ `khoaA` và đang chờ `khoaB`, **đồng thời** Thread-2 giữ `khoaB` và đang chờ `khoaA` — cả hai sẽ **chờ nhau mãi mãi**, không bên nào chịu nhường trước. Đây gọi là **deadlock** ("khoá chết") — chương trình **treo vĩnh viễn**, không có exception nào được ném ra để báo hiệu, chỉ đơn giản là không bao giờ tiến triển thêm được nữa.

Chạy `DeadlockDemo.java` để tự thấy tình huống này xảy ra thật (chương trình tự thoát cưỡng bức sau vài giây để không làm treo terminal của bạn, nhưng hai thread deadlock thực sự **không bao giờ** tự kết thúc nếu không có sự can thiệp đó).

Cách phòng tránh deadlock cơ bản

Nguyên tắc quan trọng nhất: luôn xin (lock) các khoá theo **cùng một thứ tự cố định** ở mọi nơi trong chương trình. Nếu cả Thread-1 và Thread-2 đều xin `khoaA` **trước**, `khoaB` **sau** (thay vì Thread-2 xin ngược lại như ví dụ trên), deadlock kiểu này không thể xảy ra — một trong hai thread sẽ luôn xin được `khoaA` trước, thread còn lại buộc phải chờ, nhưng không có tình huống chờ vòng tròn lẫn nhau.

Bài tập

- Sửa `DeadlockDemo.java` để cả hai thread xin khoá theo **cùng thứ tự** (`khoaA` trước, `khoaB` sau), chạy lại và xác nhận không còn deadlock, chương trình kết thúc bình thường.

- Viết một class `TaiKhoanNganHang` (tương tự Chương 14) với phương thức `rutTien` / `napTien` dùng `synchronized`, rồi cho nhiều thread cùng gọi `rutTien` / `napTien` đồng thời trên cùng một tài khoản — xác nhận số dư cuối cùng luôn đúng, không bị race condition.
- Thử bỏ `synchronized` khỏi bài tập 2, chạy lại nhiều lần, quan sát số dư cuối cùng có thể sai lệch — đối chiếu trực tiếp với `RaceConditionDemo.java`.

Lỗi thường gặp

- Chỉ đồng bộ hoá MỘT trong nhiều chỗ cùng sửa dữ liệu chung** — ví dụ `tangDem()` có `synchronized` nhưng vẫn còn chỗ khác trong code đọc/sửa trực tiếp `dem` mà không qua đồng bộ hoá — race condition vẫn xảy ra qua "cửa" không được bảo vệ đó. Mọi điểm truy cập dữ liệu dùng chung giữa nhiều thread đều cần đồng bộ hoá nhất quán.
- Dùng hai object khoá khác nhau (`khoa1` , `khoa2`) ở hai chỗ lẽ ra phải loại trừ lẫn nhau** — nếu hai khối `synchronized` dùng khoá khác nhau, chúng **không loại trừ nhau**, cả hai vẫn chạy được đồng thời — mất tác dụng đồng bộ hoá dù trông có vẻ đã "khóa" cẩn thận.
- Quên `unlock()` khi dùng `ReentrantLock` (đặc biệt khi có exception)** — khoá bị giữ vĩnh viễn, mọi thread khác chờ mãi mãi (tương tự deadlock). Luôn đặt `unlock()` trong `finally`.
- Đồng bộ hoá quá rộng** (khóa cả một phương thức lớn, trong khi chỉ một dòng thực sự cần bảo vệ) — làm giảm hiệu năng đa luồng không cần thiết, vì các thread khác phải chờ lâu hơn mức cần thiết. Cân nhắc dùng khối `synchronized` nhỏ, đúng phạm vi thay vì cả phương thức.
- Xin khoá theo thứ tự không nhất quán ở các chỗ khác nhau trong code** — nguyên nhân phổ biến nhất gây deadlock trong thực tế. Luôn thống nhất thứ tự xin khoá xuyên suốt chương trình.

Chương 39 — ExecutorService & CompletableFuture

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vì sao tự tạo `Thread` thủ công cho từng tác vụ nhỏ không hiệu quả, và `ExecutorService` giải quyết vấn đề đó thế nào.
- Dùng `Future` để lấy kết quả từ một tác vụ chạy bất đồng bộ.
- Xây dựng pipeline xử lý bất đồng bộ bằng `CompletableFuture`.

Code mẫu đầy đủ: [code/ch39-executor-completablefuture/](#).

Vấn đề với tự tạo Thread cho mọi tác vụ

Chương 37-38 tạo `Thread` trực tiếp cho từng tác vụ. Với **số lượng tác vụ nhỏ**, cách này ổn. Nhưng với hệ thống xử lý **hàng nghìn tác vụ nhỏ liên tục** (ví dụ server xử lý nhiều yêu cầu cùng lúc), tạo một `Thread` mới cho mỗi tác vụ rất **tốn kém**: mỗi thread chiếm một vùng bộ nhớ riêng (cho call stack — nhắc lại Chương 29), và bản thân việc tạo/hủy thread cũng có chi phí xử lý riêng.

ExecutorService — thread pool tái sử dụng

```
ExecutorService pool = Executors.newFixedThreadPool(4); // chỉ tạo DUNG 4 thread

for (int i = 0; i < 10; i++) {
    pool.submit(() -> { /* tac vu */ });
}

pool.shutdown();
pool.awaitTermination(5, TimeUnit.SECONDS);
```

`ExecutorService` quản lý một **"hồ bơi" (pool) thread cố định**, tái sử dụng cho nhiều tác vụ liên tiếp thay vì tạo mới mỗi lần — `submit(...)` nộp một tác vụ (`Runnable` hoặc `Callable`, xem phần dưới) vào hàng đợi, một trong các thread có sẵn trong pool sẽ nhận và chạy nó khi rảnh.

- `shutdown()` — báo hiệu "không nhận tác vụ mới nữa", nhưng vẫn để các tác vụ **đã nộp** chạy hết.
- `awaitTermination(...)` — chờ tối đa một khoảng thời gian cho mọi tác vụ đã nộp chạy xong, trả về `true` / `false` tùy đã xong kịp hay chưa.

Với ví dụ 10 tác vụ, pool 4 thread: chỉ **4 thread thực sự được tạo**, mỗi thread xử lý tuần tự vài tác vụ trong hàng đợi — tiết kiệm hơn hẳn tạo 10 thread riêng biệt.

Callable và Future — tác vụ có giá trị trả về

Nhắc lại Chương 37: `Runnable.run()` không trả về giá trị gì. `Callable<T>` là phiên bản **có giá trị trả về**:

```
Callable<Integer> tinhToan = () -> {
    Thread.sleep(1000);
    return 42;
};

Future<Integer> ketQuaTuongLai = pool.submit(tinhToan);
System.out.println("Da nop tac vu, main tiep tục chạy ngay...");
```

```
int ketQua = ketQuaTuongLai.get(); // CHAN (block) o day, cho toi khi co ket qua
```

`submit(...)` trả về **ngay lập tức** một `Future<T>` — một "giấy hẹn" đại diện cho kết quả **sẽ có trong tương lai**, không phải kết quả thật ngay lúc đó. Thread gọi `submit` **không bị chặn**, tiếp tục chạy code tiếp theo bình thường. Chỉ khi gọi `.get()`, nếu tác vụ **chưa** chạy xong, thread gọi `get()` mới thực sự **dừng lại chờ** cho tới khi có kết quả.

CompletableFuture — xây dựng pipeline bất đồng bộ

`CompletableFuture` (Java 8+) mở rộng ý tưởng của `Future`, cho phép **nối tiếp nhiều bước xử lý** theo phong cách gần giống `Stream` (Chương 31), mà không cần tự gọi `.get()` chặn giữa chừng từng bước:

```
CompletableFuture<Integer> pipeline = CompletableFuture
    .supplyAsync(() -> layDuLieuTho()) // chạy BAT DONG BO tren mot thread khac
    .thenApply(giaTri -> giaTri * 2) // BIEN DOI ket qua, giống map() của Stream
    .thenApply(giaTri -> giaTri + 5); // noi tiep them mot buoc bien doi nua

int ketQua = pipeline.get(); // chỉ CHO o buoc CUOI CUNG, khi thuc su can ket qua
```

`supplyAsync(...)` khởi động một tác vụ chạy bất đồng bộ (trên một thread pool mặc định do JVM quản lý). Mỗi `thenApply(...)` nối thêm **một bước xử lý**, chỉ chạy **sau khi** bước trước đó đã có kết quả — nhưng bản thân việc **nối các bước** không chặn thread gọi nó; toàn bộ pipeline chỉ thực sự "chờ" khi bạn gọi `.get()` ở cuối.

thenCombine() — kết hợp kết quả của hai future độc lập

```
CompletableFuture<Integer> f1 = CompletableFuture.supplyAsync(() -> tinhA());
CompletableFuture<Integer> f2 = CompletableFuture.supplyAsync(() -> tinhB());

CompletableFuture<Integer> tongHop = f1.thenCombine(f2, (a, b) -> a + b);
```

`f1` và `f2` chạy **song song** (không tuần tự) — `thenCombine` chờ **cả hai** hoàn thành rồi mới áp dụng hàm kết hợp. Đây là cách viết tự nhiên để tận dụng chạy song song nhiều tác vụ độc lập, không cần tự quản lý `Thread` / `join()` thủ công như Chương 37.

exceptionally() — xử lý lỗi trong pipeline bất đồng bộ

```
CompletableFuture<Integer> coLoi = CompletableFuture
    .supplyAsync(() -> { throw new RuntimeException("Loi!"); })
    .exceptionally(loi -> {
        System.out.println("Da bat duoc loi: " + loi.getMessage());
        return -1; // gia tri THAY THE khi co loi
    });
```

Nếu bất kỳ bước nào trong pipeline ném exception, các bước `thenApply` tiếp theo **bị bỏ qua**, pipeline "nhảy thẳng" tới khối `exceptionally` gần nhất (tương tự cách `try/catch` — Chương 21 — bỏ qua phần còn lại của `try` khi có lỗi) — `exceptionally` cung cấp giá trị thay thế để pipeline vẫn có kết quả cuối cùng thay vì crash hoàn toàn.

Bài tập

- Viết chương trình dùng `ExecutorService` với pool 3 thread, xử lý 20 "tác vụ" (mỗi tác vụ chỉ in ra số thứ tự và `Thread.sleep(100)`), đo tổng thời gian chạy bằng `System.nanoTime()` (Chương 6), so sánh với thời gian ước tính nếu chạy tuần tự hoàn toàn ($20 \times 100\text{ms}$).
- Viết pipeline `CompletableFuture` mô phỏng: "tải dữ liệu người dùng" → "tính điểm trung bình từ dữ liệu đó" → "định dạng kết quả thành chuỗi", mỗi bước `Thread.sleep` một chút để mô phỏng độ trễ thực tế.

3. Thử `thenCombine` với ba `CompletableFuture` độc lập thay vì hai (gợi ý: kết hợp từng cặp, hoặc tra cứu `CompletableFuture.allOf(...)`).

Lỗi thường gặp

- **Quên gọi `shutdown()` trên `ExecutorService`** — các thread trong pool tiếp tục sống (chờ tác vụ mới), khiến chương trình **không tự kết thúc** dù mọi việc đã xong (JVM đợi mọi non-daemon thread kết thúc trước khi thoát).
- **Gọi `.get()` ngay sau `submit()` / `supplyAsync()` mà không tận dụng thời gian chờ để làm việc khác** — về kỹ thuật không sai, nhưng làm mất hoàn toàn lợi ích của việc chạy bất đồng bộ (tương đương lại chạy tuần tự như không dùng Future).
- **Không xử lý exception trong `CompletableFuture`** — nếu thiếu `exceptionally` (hoặc `handle` / `whenComplete`), lỗi trong pipeline sẽ được "gói" lại và chỉ lộ ra khi gọi `.get()` dưới dạng `ExecutionException` bọc lỗi gốc bên trong — dễ bỏ sót nếu không chủ động kiểm tra.
- **Chọn kích thước thread pool tùy tiện** — pool quá nhỏ khiến tác vụ phải chờ lâu trong hàng đợi; pool quá lớn (nhiều hơn hẳn số lõi CPU thực có) không giúp nhanh hơn mà còn tốn overhead quản lý thread. Với tác vụ chủ yếu chờ I/O (mạng, file), pool có thể lớn hơn số lõi CPU khá nhiều; với tác vụ tính toán nặng CPU, thường nên gần bằng số lõi CPU thực tế của máy.

Chương 40 — Giới thiệu Virtual Thread

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vấn đề mà Virtual Thread (Java 21+) giải quyết: giới hạn số lượng thread thông thường của hệ thống.
- Tạo và dùng virtual thread bằng `Thread.ofVirtual()` và `Executors.newVirtualThreadPerTaskExecutor()`.
- Biết khi nào virtual thread thực sự hữu ích, khi nào không tạo ra khác biệt.

Code mẫu đầy đủ: [code/ch40-virtual-thread/](#). Cần **Java 21** trở lên để chạy.

Giới hạn của thread thông thường (platform thread)

Mọi Thread từ Chương 37-39 đến giờ đều là **platform thread** — mỗi platform thread tương ứng **trực tiếp** với một thread thật của hệ điều hành bên dưới. Hệ điều hành chỉ quản lý hiệu quả được một số lượng thread **giới hạn** (thường vài nghìn trên một máy chủ thông thường) — mỗi thread chiếm bộ nhớ riêng cho call stack (nhắc lại Chương 29), và việc hệ điều hành chuyển đổi qua lại giữa hàng nghìn thread cũng tốn chi phí đáng kể.

Vấn đề lộ rõ với các hệ thống cần xử lý **rất nhiều** tác vụ đang **chờ I/O** cùng lúc (chờ phản hồi mạng, chờ đọc file, chờ truy vấn database) — ví dụ một server cần phục vụ 50.000 kết nối đồng thời, mỗi kết nối phần lớn thời gian chỉ đang **chờ**, không thực sự tính toán gì. Tạo 50.000 platform thread cho tình huống này thường **không khả thi** về tài nguyên hệ thống.

Virtual Thread — thread "nhẹ", do JVM quản lý

```
Thread virtualThread = Thread.ofVirtual().start(() -> {
    System.out.println("Chạy tren: " + Thread.currentThread());
});
virtualThread.join();
```

Virtual thread (Java 21+, chính thức ổn định) là một loại thread **do JVM tự quản lý**, không ánh xạ trực tiếp 1-1 với thread hệ điều hành. JVM có thể tạo ra **hàng triệu** virtual thread mà không gặp vấn đề tài nguyên như platform thread, vì nhiều virtual thread **chia sẻ chung** một số lượng nhỏ platform thread thực sự bên dưới — khi một virtual thread đang **chờ** (ví dụ chờ I/O), JVM "gỡ" nó khỏi platform thread bên dưới, để platform thread đó rảnh ra phục vụ virtual thread khác đang cần chạy, rồi "gắn lại" khi virtual thread ban đầu có việc để tiếp tục.

Từ góc độ code, **cách viết gần như không khác gì** platform thread — đây là điểm thiết kế cố ý của virtual thread: bạn không cần học lại mô hình lập trình mới.

Executors.newVirtualThreadPerTaskExecutor()

```
try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        pool.submit(() -> {
            Thread.sleep(100); // mô phỏng cho I/O
        });
    }
}
```

Khác với `Executors.newFixedThreadPool(4)` ở Chương 39 (giới hạn cố định số lượng thread thực sự được tạo), `newVirtualThreadPerTaskExecutor()` tạo **một virtual thread mới cho mỗi tác vụ**, không giới hạn số lượng cố định — vì virtual thread "rẻ" hơn platform thread rất nhiều, cách tiếp cận này khả thi ở quy mô hàng chục nghìn tác vụ đồng thời mà `newFixedThreadPool` không đáp ứng nổi.

Từ Java 19, `ExecutorService` cũng implements `AutoCloseable` (Chương 21) — dùng được trong `try-with-resources`, tự động chờ mọi tác vụ đã nộp hoàn thành rồi mới đóng pool khi ra khỏi khối `try`, không cần tự gọi `shutdown()` / `awaitTermination()` tường minh như Chương 39.

`AtomicInteger` trong code mẫu (`demHoanThanh.incrementAndGet()`) là một biến đếm **an toàn với đa luồng** có sẵn trong thư viện chuẩn — nhắc lại race condition ở Chương 38: dùng `int` thường và `demHoanThanh++` trực tiếp ở đây sẽ gặp đúng vấn đề race condition đã học, `AtomicInteger` cung cấp sẵn thao tác tăng giá trị **an toàn** mà không cần tự viết `synchronized`.

Khi nào virtual thread thực sự hữu ích?

Virtual thread giúp ích rõ rệt nhất với khối lượng lớn tác vụ dành phần lớn thời gian chờ I/O (mạng, file, database) — đúng như minh hoạ ở `ScaleComparisonDemo.java`. **Virtual thread KHÔNG giúp tăng tốc** các tác vụ tính toán nặng CPU (ví dụ tính toán số học phức tạp liên tục, không chờ đợi gì) — với loại tác vụ này, số lượng platform thread hiệu quả vẫn bị giới hạn bởi số lõi CPU thực có trên máy, dùng virtual thread cho trường hợp này không mang lại lợi ích, đôi khi còn phức tạp hoá code không cần thiết.

Tóm gọn: virtual thread giải quyết vấn đề *số lượng* thread đồng thời cho tác vụ chờ I/O nhiều, không giải quyết vấn đề *tốc độ tính toán*.

Bài tập

1. Chạy `ScaleComparisonDemo.java`, thử tăng `soTacVu` lên `100_000`, quan sát chương trình vẫn chạy được và đo thời gian hoàn thành.
2. Thử thay `Executors.newVirtualThreadPerTaskExecutor()` bằng `Executors.newFixedThreadPool(200)` (Chương 39) trong cùng bài toán 10.000 tác vụ — so sánh thời gian chạy giữa hai cách (với `soTacVu` lớn, pool cố định phải "xếp hàng" xử lý dần, trong khi virtual thread xử lý gần như đồng thời hết).
3. Viết một tác vụ tính toán nặng CPU đơn giản (ví dụ tính số Fibonacci lớn bằng vòng lặp, không có `sleep` / chờ I/O nào), so sánh thời gian chạy 8 tác vụ như vậy bằng virtual thread và bằng `newFixedThreadPool` bằng đúng số lõi CPU của máy bạn — quan sát virtual thread không tạo ra khác biệt đáng kể trong trường hợp này.

Lỗi thường gặp

- **Dùng virtual thread cho tác vụ tính toán nặng CPU, mong đợi tăng tốc** — không có tác dụng như mong đợi, vì giới hạn thực sự nằm ở số lõi CPU, không phải số lượng thread có thể tạo ra.
- **Chạy code Chương này trên Java cũ hơn 21** — `Thread.ofVirtual()` và `newVirtualThreadPerTaskExecutor()` không tồn tại ở các bản Java cũ hơn, lỗi biên dịch `cannot find symbol`. Xác nhận `java -version` cho ra bản 21 trở lên trước khi chạy.
- **Đồng bộ hoá (`synchronized`, Chương 38) quá rộng trên virtual thread trong khối lượng lớn** — một số kỹ thuật đồng bộ hoá cũ có thể làm giảm bớt lợi ích của virtual thread trong tình huống cụ thể (chi tiết kỹ thuật sâu, nằm ngoài phạm vi sách nhập môn — chỉ cần biết đây là điều cần lưu ý khi làm việc ở quy mô rất lớn trong thực tế).
- **Tưởng virtual thread thay thế hoàn toàn `ExecutorService` / `CompletableFuture` (Chương 39)** — virtual thread là một **loại thread mới**, vẫn dùng chung với các API đã học ở Chương 39, không phải một mô hình lập trình hoàn toàn khác cần học lại từ đầu.

Chương 41 — Unit test với JUnit 5

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu unit test là gì, vì sao viết test quan trọng không kém viết code chính.
- Viết được test bằng JUnit 5: `@Test`, `assertEquals`, `assertThrows`.
- Dùng `@BeforeEach` để chuẩn bị trạng thái sạch cho mỗi test.
- Hiểu tư duy "thiết kế để test" — tách logic khỏi I/O.

Code mẫu đầy đủ: [code/ch41-junit/](#).

Unit test là gì, vì sao cần?

Từ đầu sách, bạn kiểm tra code "đúng" bằng cách chạy chương trình và **tự mắt** xem kết quả in ra có đúng không. Cách này có vấn đề rõ ràng khi chương trình lớn dần: mỗi lần sửa code, bạn phải **tự kiểm tra lại thủ công** mọi tính năng để chắc chắn không làm hỏng gì — tốn thời gian, dễ bỏ sót, và không lặp lại được một cách đáng tin cậy.

Unit test là code **tự động kiểm tra** một đơn vị nhỏ của chương trình (thường là một phương thức) có hoạt động đúng như mong đợi hay không — viết một lần, chạy lại **bất kỳ lúc nào** (đặc biệt sau mỗi lần sửa code) chỉ bằng một lệnh, không cần tự tay kiểm tra lại bằng mắt.

Viết test đầu tiên với JUnit 5

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorTest {
    @Test
    void testCongHaiSoDuong() {
        Calculator calculator = new Calculator();
        double ketQua = calculator.cong(2, 3);
        assertEquals(5, ketQua);
    }
}
```

- **Quy ước đặt tên:** class test tên `<TenLopCanTest>Test` (`CalculatorTest` cho `Calculator`) — không bắt buộc về mặt kỹ thuật, nhưng là quy ước gần như phổ quát trong cộng đồng Java.
- `@Test` — annotation đánh dấu một phương thức là **một test case**, JUnit tự nhận diện và chạy nó khi bạn chạy bộ test.
- `assertEquals(mongDoi, thucTe)` — khẳng định hai giá trị **phải bằng nhau**; nếu không, test **thất bại (fail)**, JUnit báo rõ giá trị mong đợi và giá trị thực tế khác nhau ra sao.

@BeforeEach — chuẩn bị trạng thái sạch cho mỗi test

```
private Calculator calculator;

@BeforeEach
void setUp() {
    calculator = new Calculator();
}
```

Phương thức đánh dấu `@BeforeEach` chạy **trước mỗi** phương thức `@Test` trong cùng class — đảm bảo mỗi test bắt đầu với một object `calculator` **hoàn toàn mới**, không bị ảnh hưởng bởi bất kỳ thay đổi trạng thái nào từ test chạy trước đó. Đây là

nguyên tắc quan trọng: **mỗi test phải độc lập hoàn toàn** với các test khác — thứ tự chạy test không nên ảnh hưởng tới kết quả.

assertThrows — kiểm tra exception được ném đúng cách

```
@Test
void testChiaChoKhong() {
    ArithmeticException loi = assertThrows(
        ArithmeticException.class,
        () -> calculator.chia(5, 0)
    );
    assertEquals("Khong the chia cho 0", loi.getMessage());
}
```

`assertThrows(KieuException.class, lambda)` (Chương 30) xác nhận đoạn code trong lambda **thực sự ném ra đúng loại exception mong đợi** — nếu không ném gì cả, hoặc ném **sai loại** exception, test thất bại. Đây là cách kiểm thử **cả những tình huống lỗi** (Chương 21) một cách có hệ thống, không chỉ kiểm tra "đường đi đúng" (happy path).

@DisplayName — mô tả test dễ đọc hơn

```
@Test
@DisplayName("Chia cho 0 phai nem ArithmeticException")
void testChiaChoKhong() { ... }
```

Tuỳ chọn, nhưng hữu ích: tên hiển thị trong báo cáo kết quả test rõ ràng hơn tên phương thức (vốn thường bị giới hạn theo quy tắc đặt tên biến/phương thức của Java, không có dấu cách hay ký tự đặc biệt).

Thiết kế code "dễ test"

So sánh `Calculator` trong code mẫu chương này (chỉ chứa phương thức tính toán thuần túy) với việc viết logic đó **lẫn trực tiếp** vào một chương trình đọc `Scanner /in System.out` — nếu logic tính toán và logic nhập/xuất trộn lẫn vào nhau, viết test cho riêng phần tính toán trở nên khó khăn hơn nhiều (phải giả lập input bàn phím, bắt output console...). **Tách biệt logic nghiệp vụ khỏi I/O** (đọc/ghi file, console, mạng...) là một nguyên tắc thiết kế quan trọng, không chỉ giúp code rõ ràng hơn mà còn giúp viết test dễ dàng hơn hẳn — bạn sẽ gặp lại tư duy này khi học kiến trúc phần mềm ở Chương 44-45.

Bài tập

- Viết `SinhVienTest` kiểm thử class `SinhVien` (Chương 13): `test tinhDiemTrungBinh()` với danh sách điểm rỗng (kết quả mong đợi `0.0`), và với vài điểm cụ thể.
- Viết test cho phương thức `isPrime` bạn đã viết ở bài tập Chương 10, kiểm tra cả trường hợp số nguyên tố, không phải nguyên tố, và trường hợp biên (`0`, `1`, số âm).
- Viết một test **cố ý sai** (ví dụ `assertEquals(6, calculator.cong(2, 3))` — sai vì kết quả đúng là `5`), chạy để tự thấy JUnit báo lỗi thế nào, đọc kỹ thông báo "expected/actual" để quen mặt định dạng báo lỗi.

Lỗi thường gặp

- Test phụ thuộc lẫn nhau** (test B chỉ chạy đúng nếu test A chạy trước và để lại trạng thái nào đó) — vi phạm nguyên tắc độc lập, gây lỗi khó hiểu khi JUnit chạy test theo thứ tự khác (JUnit **không đảm bảo** thứ tự chạy test mặc định). Luôn dùng `@BeforeEach` để đảm bảo trạng thái sạch.
- Chỉ test "đường đi đúng", quên test trường hợp lỗi/biên** — ví dụ chỉ test `chia(5, 2)` mà quên test `chia(5, 0)`. Luôn nghĩ tới các trường hợp biên: giá trị `0`, âm, rỗng, `null`, giá trị cực lớn/nhỏ.
- Viết test rồi không bao giờ chạy lại** — mất hết giá trị của việc có test tự động. Test chỉ hữu ích khi được chạy thường xuyên (lý tưởng: sau mỗi lần sửa code, hoặc tự động qua CI — nằm ngoài phạm vi sách này).

- **So sánh số thực (double) bằng assertEquals không có sai số cho phép** — số thực có thể có sai số làm tròn rất nhỏ (ví dụ $0.1 + 0.2$ không chính xác bằng 0.3 tuyệt đối ở tầng bit); JUnit có overload `assertEquals(mongDoi, thucTe, saiSoChoPhep)` dành riêng cho `double` , nên dùng khi so sánh kết quả phép tính số thực phức tạp.

Chương 42 — Build tool: Maven và Gradle

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu vấn đề Maven/Gradle giải quyết (đã gặp trực tiếp ở Chương 35, 41).
- Đọc hiểu cấu trúc thư mục chuẩn của một project Maven/Gradle.
- Khai báo dependency trong `pom.xml` (Maven) và `build.gradle` (Gradle).
- Chạy build, test, và chương trình bằng lệnh của build tool thay vì `javac` / `java` thủ công.

Code mẫu đầy đủ: [code/ch42-maven-gradle/](#).

Nhắc lại vấn đề đã gặp

Ở Chương 35 (dùng Gson) và Chương 41 (dùng JUnit), bạn đã phải tự tải file `.jar`, tự nhớ đường dẫn, tự ghép vào `-cp`. Với một thư viện, việc đó còn chịu được. Với **hiều** thư viện — mỗi thư viện có thể phụ thuộc thêm các thư viện khác nữa ("dependency của dependency") — quản lý thủ công gần như không khả thi. **Maven** và **Gradle** giải quyết đúng vấn đề này: khai báo thư viện cần dùng trong một file cấu hình, công cụ **tự động tải về** (từ kho lưu trữ trung tâm, mặc định là Maven Central) đúng phiên bản, xử lý toàn bộ classpath, và cung cấp lệnh chuẩn để biên dịch/test/đóng gói/chạy.

Cài đặt

- **Maven**: tải từ [maven.apache.org](#), giải nén, thêm thư mục `bin` vào `PATH` (giống cách cấu hình `PATH` cho JDK ở Chương 2). Kiểm tra: `mvn -version`.
- **Gradle**: tải từ [gradle.org](#), tương tự. Kiểm tra: `gradle -version`. (Trong dự án Gradle thực tế, thường dùng **Gradle Wrapper** — một script đi kèm project tự tải đúng phiên bản Gradle cần thiết, người dùng không cần tự cài Gradle riêng — nằm ngoài phạm vi demo đơn giản của chương này.)

Cả hai đều tích hợp sẵn trong IntelliJ IDEA (Chương 4) — khi tạo project mới, bạn có thể chọn ngay Maven hoặc Gradle làm build tool thay vì "Java thuần" đã chọn từ đầu sách.

Cấu trúc thư mục chuẩn

Cả Maven và Gradle đều theo (mặc định) cùng một quy ước cấu trúc thư mục:

```
project/
├─ pom.xml (Maven) hoặc build.gradle (Gradle)  # file cấu hình
├─ src/
│   ├─ main/java/...  # code CHÍNH của chương trình
│   └─ test/java/...  # code TEST (Chương 41)
```

So với cách tổ chức code tự do bạn dùng từ đầu sách (mọi file `.java` nằm chung một thư mục), cấu trúc này **tách biệt rõ ràng** code chính và code test, và là quy ước **gần như mọi** project Java thực tế đều tuân theo — giúp bất kỳ ai quen Maven/Gradle cũng lập tức biết tìm code ở đâu khi mở một project Java mới, bất kể dự án đó là gì.

Maven — pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.sach</groupId>
  <artifactId>maven-demo</artifactId>
  <version>1.0.0</version>
```

```

<dependencies>
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
</project>

```

- `groupId / artifactId / version` — định danh **project của bạn** (giống ý tưởng tên miền đảo ngược ở Chương 15 khi đặt tên package).
- Mỗi `<dependency>` khai báo một thư viện cần dùng — `groupId / artifactId / version` xác định chính xác **thư viện nào, phiên bản nào**. `<scope>test</scope>` nghĩa là thư viện đó **chỉ cần** lúc chạy test (JUnit), không đóng gói kèm khi chạy chương trình thật.

Lệnh Maven cơ bản: `mvn compile` (biên dịch), `mvn test` (chạy test), `mvn package` (đóng gói thành file `.jar`).

Gradle — build.gradle

```

plugins {
    id 'java'
    id 'application'
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'com.google.code.gson:gson:2.10.1'
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.2'
}

test {
    useJUnitPlatform()
}

```

Cùng ý tưởng như Maven, cú pháp gọn hơn (dùng Groovy hoặc Kotlin DSL thay vì XML). `implementation` tương ứng dependency dùng lúc chạy chương trình chính, `testImplementation` tương ứng `<scope>test</scope>` của Maven. Lệnh Gradle cơ bản: `gradle build` (biên dịch + test + đóng gói), `gradle test`, `gradle run` (nếu có plugin `application`, như trong code mẫu).

Maven hay Gradle?

	Maven	Gradle
Cú pháp cấu hình	XML (pom.xml), tường minh, hơi dài dòng	Groovy/Kotlin DSL, ngắn gọn hơn, linh hoạt hơn (cho phép viết logic build tùy chỉnh)
Độ phổ biến	Rất phổ biến, đặc biệt hệ sinh thái Spring	Rất phổ biến, đặc biệt hệ sinh thái Android

	Maven	Gradle
Tốc độ build	Ổn định, đơn giản	Thường nhanh hơn với project lớn (cache build thông minh hơn)

Không có lựa chọn "đúng tuyệt đối" — cả hai đều giải quyết cùng vấn đề, khác nhau chủ yếu về cú pháp và triết lý thiết kế. Nhiều công ty/dự án chọn một trong hai theo thói quen của team hoặc theo framework chính đang dùng (ví dụ dự án Spring Boot thường dùng Maven hoặc Gradle tùy đội, dự án Android gần như luôn dùng Gradle).

Từ đây trở đi

Từ chương này, các chương còn lại trong sách (Chương 43-47) có thể tham khảo cách tổ chức project theo chuẩn Maven/Gradle khi cần dùng thư viện ngoài — nhưng để giữ mọi chương **tự chạy độc lập** không đòi hỏi cài đặt phức tạp, phần lớn code mẫu vẫn ưu tiên chạy được bằng `javac / java` thuần khi hợp lý, chỉ giới thiệu cấu trúc Maven/Gradle khi thư viện ngoài thực sự cần thiết (như JDBC driver ở Chương 46).

Bài tập

1. Cài Maven (hoặc Gradle), chạy `mvn test` (hoặc `gradle test`) trong `code/ch42-maven-gradle/maven-demo/` (hoặc `gradle-demo/`), xác nhận cả hai test trong `SinhVienTest` đều pass.
2. Thêm một dependency mới vào `pom.xml / build.gradle` (ví dụ Apache Commons Lang: `org.apache.commons:commons-lang3:3.14.0`), dùng thử một phương thức tiện ích của nó trong `App.java`.
3. Tạo một project Maven mới trong IntelliJ IDEA (File → New → Project → chọn Maven), so sánh cấu trúc thư mục IDE tự sinh ra với cấu trúc đã học trong chương.

Lỗi thường gặp

- `mvn / gradle không nhận diện được (command not found)` — chưa thêm vào `PATH` đúng cách, xem lại phần "Cài đặt", tương tự cách khắc phục JDK ở Chương 2.
- **Khai báo sai `groupId / artifactId / version` của dependency** — Maven/Gradle báo lỗi không tìm thấy thư viện (`could not resolve dependency`). Kiểm tra chính xác thông tin trên trang chính thức của thư viện hoặc [Maven Central](#).
- **Đặt file `.java` sai vị trí trong cấu trúc thư mục chuẩn** (ví dụ đặt ngoài `src/main/java`) — build tool không tìm thấy để biên dịch. Luôn tuân theo đúng cấu trúc `src/main/java/... / src/test/java/...`.
- **Trộn lẫn code chính và code test trong cùng thư mục `src/main/java`** — về mặt kỹ thuật vẫn biên dịch được, nhưng phá vỡ quy ước chuẩn, khiến `mvn package / gradle build` có thể vô tình đóng gói cả code test vào sản phẩm cuối cùng.

Chương 43 — Debug & logging

Mục tiêu học

Sau chương này, bạn sẽ:

- Dùng debugger trong IDE thành thạo hơn nữa (mở rộng từ Chương 4): watch expression, conditional breakpoint, Step Into/Over/Out.
- Hiểu vì sao `System.out.println` không đủ tốt để theo dõi chương trình chạy trong thực tế.
- Dùng `java.util.logging` để ghi log có mức độ (level), có sẵn trong JDK không cần thư viện ngoài.
- Biết về SLF4J + Logback/Log4j2 — bộ công cụ logging phổ biến trong dự án thực tế.

Code mẫu đầy đủ: [code/ch43-debug-logging/](#).

Debug nâng cao trong IDE

Chương 4 đã giới thiệu breakpoint, Step Over, Step Into. Vài kỹ thuật debug nâng cao hữu ích thêm:

- **Step Out (Shift+F8)** — khi đang Step Into bên trong một phương thức, Step Out chạy nốt phần còn lại của phương thức đó rồi quay về nơi gọi nó, không cần Step Over từng dòng còn lại.
- **Watch expression** — thêm một biểu thức tùy ý (không chỉ biến đơn lẻ, ví dụ `map.length - i`) vào panel Debug để IDE tự tính và hiển thị giá trị hiện tại mỗi lần dừng lại.
- **Conditional breakpoint** — chuột phải vào breakpoint, thêm điều kiện (ví dụ `i == 3`) — chương trình chỉ thực sự dừng lại khi điều kiện đó đúng, rất hữu ích khi lỗi chỉ xảy ra ở một vòng lặp cụ thể trong hàng nghìn lần lặp, tránh phải bấm Step Over hàng nghìn lần.

Thực hành trực tiếp với `BuggyProgram.java` (xem hướng dẫn từng bước trong README code mẫu) — đây là kỹ năng chỉ thành thạo qua thực hành, không chỉ đọc lý thuyết.

Vì sao `System.out.println` không đủ cho theo dõi chương trình thực tế

Từ đầu sách, bạn dùng `System.out.println` để vừa hiển thị kết quả **cho người dùng**, vừa để **debug** khi cần xem giá trị biến. Cách này ổn cho bài tập nhỏ, nhưng có vấn đề khi chương trình lớn/chạy lâu dài:

- **Không có mức độ ưu tiên** — không phân biệt được "thông tin bình thường" với "lỗi nghiêm trọng cần chú ý ngay", tất cả đều in y hệt nhau.
- **Không tắt/bật được theo nhóm** — muốn tạm ẩn bớt các dòng debug chi tiết mà vẫn giữ lại cảnh báo quan trọng, phải tự tay xóa/thêm lại từng dòng `println`.
- **Không có ngữ cảnh tự động** (thời gian, tên class/phương thức đang chạy, tên thread nếu đa luồng — Chương 37) — phải tự viết thêm vào mỗi dòng nếu cần.
- **Khó quản lý ở quy mô lớn** — hệ thống chạy hàng ngày, hàng tuần cần ghi lại lịch sử hoạt động có tổ chức (thường vào file, không phải chỉ hiện trên console rồi mất đi).

Logging (ghi log) là giải pháp chuyên dụng cho vấn đề này.

`java.util.logging` — có sẵn trong JDK

```
import java.util.logging.Logger;

Logger logger = Logger.getLogger(LoggingDemo.class.getName());

logger.fine("Chi tiết go loi");      // mức THẤP nhất trong ví dụ này
logger.info("Thông tin bình thường");
```

```
logger.warning("Co the co van de");
logger.severe("Loi nghiêm trọng");    // mức CAO nhất
```

Các mức độ (level) từ thấp đến cao: `FINE` (chi tiết debug) → `INFO` (thông tin bình thường) → `WARNING` (cảnh báo, chương trình vẫn chạy tiếp được) → `SEVERE` (lỗi nghiêm trọng). Có thể **lọc** theo mức tối thiểu:

```
logger.setLevel(Level.WARNING); // chỉ ghi WARNING trở lên, INFO/FINE bị AN
```

Đây chính là lợi ích cốt lõi so với `println`: khi triển khai thực tế, bạn có thể **tắt** log chi tiết (`FINE` / `INFO`) để giảm nhiễu, chỉ giữ lại `WARNING` / `SEVERE` — mà **không cần sửa code**, chỉ cần đổi cấu hình.

Ghi log kèm exception

```
try {
    // ...
} catch (ArithmeticException e) {
    logger.log(Level.SEVERE, "Loi khi tinh toan", e);
}
```

`logger.log(level, thôngDiệp, exception)` ghi lại **cả** thông điệp mô tả **lần** toàn bộ stack trace của exception — hữu ích hơn nhiều so với chỉ `System.out.println(e.getMessage())` (chỉ có thông điệp ngắn, mất hết thông tin về **nơi** lỗi xảy ra trong code).

SLF4J + Logback/Log4j2 — bộ công cụ logging phổ biến trong thực tế

`java.util.logging` có sẵn trong JDK, nhưng phần lớn dự án Java thực tế dùng **SLF4J** (một "giao diện" logging chung, không tự ghi log) kết hợp với một **cài đặt cụ thể** như **Logback** hoặc **Log4j2** (thực sự ghi log ra file/console theo định dạng cấu hình được). Ý tưởng SLF4J tương tự interface/implementation đã học ở Chương 18: code của bạn chỉ gọi qua API chung của SLF4J, không phụ thuộc trực tiếp cài đặt cụ thể nào — có thể đổi Logback sang Log4j2 mà không cần sửa code gọi log.

```
// Voi SLF4J (can khai bao dependency qua Maven/Gradle - Chuong 42)
private static final Logger logger = LoggerFactory.getLogger(MyClass.class);
logger.info("He thong khoi dong xong sau {} ms", thoiGianKhoiDong);
```

Cú pháp `{}` là chỗ trống được điền giá trị tự động (tránh phải nối chuỗi `+` thủ công như `println`), và **chỉ tính toán chuỗi thật sự khi cần ghi** (nếu mức log đó đang bị tắt, SLF4J không lãng phí công sức tạo chuỗi). Việc thiết lập SLF4J/Logback cần khai báo dependency qua Maven/Gradle (Chương 42) — sách giới thiệu ở đây để bạn nhận diện được khi gặp trong code thực tế/dự án nhóm, không đi sâu cấu hình chi tiết (nằm ngoài phạm vi sách nhập môn).

Bài tập

- Mở `BuggyProgram.java` trong IDE, làm theo đúng các bước trong README để tìm và sửa lỗi bằng debugger — không được nhìn trước vào code để "biết trước" lỗi ở đâu, thực hành đúng quy trình.
- Thêm logging (`java.util.logging`) vào một chương trình bạn đã viết ở chương trước (ví dụ `TaiKhoanNganHang` — Chương 14), ghi `INFO` khi giao dịch thành công, `WARNING` khi giao dịch bị từ chối (số dư không đủ...).
- Thử đặt conditional breakpoint trong một vòng lặp `for` chạy 100 lần, chỉ dừng khi biến đếm bằng một giá trị cụ thể — xác nhận chương trình chạy nhanh qua 99 lần đầu và chỉ dừng đúng lần bạn chỉ định.

Lỗi thường gặp

- Để lại nhiều dòng `System.out.println` debug trong code sau khi đã sửa xong lỗi** — làm nhiễu output thực tế của chương trình, dễ quên xoá. Cần nhắc dùng logging (để tắt hàng loạt) thay vì `println` ngay từ đầu cho các thông tin chẩn đoán.

- **Debug bằng Run thay vì Debug** — breakpoint không có tác dụng gì nếu chạy ở chế độ Run thông thường (nhắc lại Chương 4).
- **Log ở mức SEVERE cho những tình huống không thực sự nghiêm trọng** (hoặc ngược lại, log lỗi thực sự nghiêm trọng ở mức FINE) — làm log mất tác dụng phân loại mức độ ưu tiên, khó lọc ra vấn đề thực sự quan trọng khi hệ thống có vấn đề.
- **Log thông tin nhạy cảm** (mật khẩu, số thẻ tín dụng, token...) — log thường được lưu trữ lâu dài và nhiều người có thể xem được, đây là lỗi bảo mật nghiêm trọng thường gặp trong thực tế. Luôn cân nhắc kỹ dữ liệu nào an toàn để ghi log.

Chương 44 — Design pattern cơ bản

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu design pattern là gì — không phải "luật bắt buộc", mà là **giải pháp đã được đúc kết** cho các vấn đề thiết kế lặp đi lặp lại.
- Nhận diện và cài đặt được 4 pattern cơ bản, phổ biến nhất: Singleton, Factory, Strategy, Observer.
- Biết khi nào **nên** và **không nên** dùng mỗi pattern — tránh áp dụng máy móc.

Code mẫu đầy đủ: [code/ch44-design-pattern/](https://github.com/duyquang101/duyquang101.github.io/blob/master/code/ch44-design-pattern/).

Design pattern là gì?

Qua nhiều thập kỷ, cộng đồng lập trình nhận ra một số **vấn đề thiết kế lặp đi lặp lại** trong rất nhiều dự án khác nhau, và đúc kết ra các **giải pháp chung** đã được kiểm chứng — gọi là **design pattern** (mẫu thiết kế). Đây **không phải** đoạn code sao chép dán được, mà là **ý tưởng cấu trúc** bạn tự áp dụng vào tình huống cụ thể của mình. Biết design pattern giúp bạn: (1) giải quyết vấn đề nhanh hơn (không phải "phát minh lại bánh xe"), và (2) **giao tiếp hiệu quả hơn** với lập trình viên khác — nói "dùng Observer ở đây" truyền đạt được rất nhiều ý nghĩa chỉ trong một từ.

Singleton — đảm bảo chỉ có một instance

```
class CauHinhUngDung {
    private static final CauHinhUngDung INSTANCE = new CauHinhUngDung();

    private CauHinhUngDung() { /* ... */ } // constructor PRIVATE

    public static CauHinhUngDung getInstance() {
        return INSTANCE;
    }
}
```

Dùng khi bạn cần **đảm bảo chỉ có đúng một** object của một class tồn tại trong suốt vòng đời chương trình — ví dụ cấu hình ứng dụng, kết nối tới một tài nguyên chia sẻ duy nhất. Cách cài đặt trên (tạo `INSTANCE` sẵn khi class được nạp) an toàn với đa luồng (Chương 38) mà **không cần** `synchronized`, vì JVM tự đảm bảo việc nạp class (và khởi tạo field `static final`) chỉ xảy ra một lần, thread-safe theo đặc tả ngôn ngữ.

Cẩn trọng khi dùng: Singleton dễ bị **lạm dụng** thành "biến toàn cục" (global state) — nhiều Singleton trong một hệ thống lớn có thể khiến code các phần khác nhau **ngầm phụ thuộc** vào nhau qua trạng thái chia sẻ, khó kiểm thử (Chương 41 — vì mỗi test không dễ có một `CauHinhUngDung` "sạch" riêng) và khó suy luận. Chỉ dùng khi thực sự có **đúng một** thứ như vậy trong bản chất bài toán, không dùng chỉ vì "tiện truy cập từ mọi nơi".

Factory — gom logic tạo object

```
class ThongBaoFactory {
    static ThongBao tao(String loai) {
        return switch (loai) {
            case "email" -> new ThongBaoEmail();
            case "sms" -> new ThongBaoSms();
            default -> throw new IllegalArgumentException("Loai khong ho tro: " + loai);
        };
    }
}
```

Dùng khi việc **quyết định tạo object loại cụ thể nào** khá phức tạp, hoặc có thể tạo bằng nhiều cách/nhiều loại con — thay vì rải `new ThongBaoEmail()` / `new ThongBaoSms()` khắp nơi trong code gọi (khiến mọi nơi gọi đều phải biết hết các lớp con cụ thể tồn tại), gom logic đó vào **một chỗ**. Nếu sau này thêm loại thông báo mới, chỉ cần sửa **đúng một chỗ** (bên trong Factory), mọi nơi gọi `ThongBaoFactory.tao(...)` không cần sửa gì.

Strategy — hoán đổi thuật toán lúc chạy

```
interface ChienLuocGiamGia {
    double apDung(double giaGoc);
}

class DonHang {
    private ChienLuocGiamGia chienLuoc;

    double tinhGiaCuoiCung() {
        return chienLuoc.apDung(giaGoc);
    }
}
```

Dùng khi có **nhiều cách** thực hiện cùng một việc (ở đây: tính giá sau giảm giá), và bạn muốn **chọn/đổi** cách thực hiện **lúc chạy**, không cố định lúc viết code. So với viết một khối `if/else` / `switch` dài liệt kê mọi loại giảm giá ngay trong `DonHang` (khiến `DonHang` phải "biết" về mọi loại chiến lược, và phải sửa `DonHang` mỗi khi thêm loại mới), Strategy tách mỗi cách xử lý thành một class riêng implements chung một interface — `DonHang` chỉ cần biết "có một `ChienLuocGiamGia` nào đó", không cần biết là loại nào cụ thể (đây chính là **đa hình**, Chương 17, áp dụng vào một tình huống thiết kế cụ thể).

Với chiến lược đơn giản (một dòng logic), dùng thẳng **lambda** (Chương 30) thay vì tạo hẳn một class riêng — như minh họa ở cuối `StrategyDemo.java` (`gia -> gia * 0.5`).

Observer — thông báo tự động khi trạng thái thay đổi

```
interface NguoiQuanSat {
    void nhanThongBao(double giaMoi);
}

class CoPhieu {
    private List<NguoiQuanSat> danhSachQuanSat = new ArrayList<>();

    void dangKy(NguoiQuanSat nqs) {
        danhSachQuanSat.add(nqs);
    }

    void capNhatGia(double giaMoi) {
        this.gia = giaMoi;
        for (NguoiQuanSat nqs : danhSachQuanSat) {
            nqs.nhanThongBao(giaMoi);
        }
    }
}
```

Dùng khi một object (**subject**, ở đây là `CoPhieu`) cần **thông báo** cho một số lượng **không cố định, có thể thay đổi** các object khác (**observer**) mỗi khi trạng thái của nó thay đổi — mà `CoPhieu` **không cần biết trước** (lúc viết code) sẽ có bao nhiêu, hay những loại `NguoiQuanSat` cụ thể nào sẽ đăng ký theo dõi nó. Đăng ký (`dangKy`) và hủy đăng ký (`huyDangKy`) có thể thực hiện **bất kỳ lúc nào** trong lúc chương trình chạy.

Pattern này là nền tảng của rất nhiều hệ thống lớn hơn: xử lý sự kiện giao diện người dùng (nhấn nút → thông báo cho các "listener" đã đăng ký), hệ thống pub/sub trong kiến trúc phân tán.

Nguyên tắc quan trọng nhất: không lạm dụng pattern

Đây là điều cần khắc sâu hơn cả 4 pattern cụ thể: **design pattern là công cụ giải quyết vấn đề cụ thể, không phải mục tiêu để "chứng tỏ" bạn biết pattern**. Áp dụng pattern khi bài toán **thực sự có** đặc điểm phù hợp (nhiều cách xử lý cần hoán đổi → Strategy; nhiều observer không xác định trước → Observer...) — cố nhét pattern vào bài toán đơn giản không cần tới nó chỉ làm code **phức tạp hơn** một cách không cần thiết, vi phạm trực tiếp nguyên tắc "giữ thiết kế đơn giản nhất có thể cho vấn đề đang giải quyết" mà chính các pattern này hướng tới phục vụ.

Bài tập

- Viết một `HinhFactory` (tương tự `ThongBaoFactory`) tạo các loại `HinhHoc` từ Chương 18 (`HinhTron`, `HinhChuNhat`) dựa trên tên loại truyền vào dạng chuỗi.
- Mở rộng `StrategyDemo.java` thêm một chiến lược `GiamGiaTheoBac` (giảm % khác nhau tùy khoảng giá trị đơn hàng — ví dụ dưới 100k không giảm, 100k-500k giảm 5%, trên 500k giảm 10%).
- Viết một `Observer` mới cho `ObserverDemo.java`: `LichSuGiaCoPhieu` ghi lại toàn bộ lịch sử các mức giá đã thông báo vào một `List<Double>`, có phương thức in ra toàn bộ lịch sử.

Lỗi thường gặp

- **Dùng Singleton cho mọi class "chỉ cần một object" một cách máy móc**, kể cả khi không thực sự cần chia sẻ trạng thái toàn cục — làm code khó kiểm thử hơn không cần thiết. Cần nhắc có thực sự cần **đúng một** instance duy nhất trong toàn bộ chương trình hay không.
- **Factory chỉ có một loại object duy nhất để tạo** — không cần Factory, `new` trực tiếp là đủ. Factory chỉ thực sự có giá trị khi việc chọn loại cụ thể có logic đáng gom lại.
- **Strategy với chỉ một chiến lược duy nhất, không bao giờ đổi** — thêm interface không cần thiết, chỉ nên áp dụng khi thực sự có nhiều cách xử lý cần hoán đổi.
- **Observer nhưng subject và observer phụ thuộc chặt vào nhau** (observer cần biết quá nhiều chi tiết nội bộ của subject để xử lý đúng) — làm mất đi lợi ích tách rời (decoupling) chính là mục tiêu cốt lõi của pattern này.
- **Quên `huyDangKy` khi observer không còn cần theo dõi nữa** — nếu subject sống lâu hơn observer (ví dụ observer là một phần giao diện đã bị đóng), observer "chết" nhưng vẫn còn trong danh sách đăng ký gây rò rỉ bộ nhớ (memory leak) — vấn đề thực tế phổ biến khi dùng Observer pattern.

Chương 45 — Nguyên lý SOLID

Mục tiêu học

Sau chương này, bạn sẽ:

- Hiểu và áp dụng được 5 nguyên lý SOLID — bộ nguyên tắc thiết kế OOP kinh điển, thường được hỏi trong phỏng vấn và dùng làm chuẩn tham chiếu khi review code trong công việc thực tế.
- Nhận diện code vi phạm từng nguyên lý, và biết cách tái cấu trúc (refactor) để tuân thủ.

Code mẫu đầy đủ: [code/ch45-solid/](#).

SOLID là 5 chữ cái đầu của 5 nguyên lý, do Robert C. Martin ("Uncle Bob") tổng hợp. Đây **không phải quy tắc máy móc phải áp dụng mọi lúc** — chương 44 đã nhấn mạnh "không lạm dụng pattern", tinh thần đó cũng áp dụng ở đây: SOLID là **kim chỉ nam** giúp nhận diện code có dấu hiệu thiết kế kém, không phải luật phải tuân theo tuyệt đối trong mọi dòng code.

S — Single Responsibility Principle (Nguyên lý đơn nhiệm)

Một class chỉ nên có đúng một lý do để thay đổi.

```
// VI PHAM: mot class lam CA BA viec
class NhanVienViPham {
    String taoBaoCao() { ... } // ly do thay doi 1: doi dinh dang bao cao
    void luuVaoFile(String d) { ... } // ly do thay doi 2: doi noi luu tru
}
```

Nếu một class đảm nhận nhiều trách nhiệm không liên quan (lưu dữ liệu, tính toán báo cáo, ghi file...), **mỗi lý do thay đổi khác nhau đều buộc phải sửa cùng một class** — tăng rủi ro một thay đổi nhỏ (ví dụ đổi định dạng báo cáo) vô tình ảnh hưởng phần không liên quan (dữ liệu nhân viên). Giải pháp: tách thành các class riêng, mỗi class **một trách nhiệm rõ ràng** (`NhanVien` , `BaoCaoNhanVien` , `LuuTruBaoCao`).

O — Open/Closed Principle (Nguyên lý đóng/mở)

Class nên mở cho việc mở rộng, nhưng đóng cho việc sửa đổi.

```
// VI PHAM: them loai hinh MOI phai SUA ham nay
double tinhDienTich(String loaiHinh, ...) {
    if (loaiHinh.equals("hinh_chu_nhat")) { ... }
    else if (loaiHinh.equals("hinh_tron")) { ... }
    // them hinh moi -> PHAI sua ham nay
}
```

Nhắc lại Chương 17-18: dùng **đa hình** thay cho `if/else / switch` liệt kê loại — mỗi loại hình là một lớp con override `tinhDienTich()` riêng. Thêm loại hình mới = **thêm một class mới**, không cần **sửa** bất kỳ code cũ nào đã hoạt động đúng (và đã được test — Chương 41) từ trước. "Đóng cho sửa đổi" giảm rủi ro làm hỏng chức năng đang hoạt động khi mở rộng hệ thống.

L — Liskov Substitution Principle (Nguyên lý thay thế Liskov)

Object của lớp con phải thay thế được object của lớp cha mà không làm sai lệch hành vi mong đợi.

```
class HinhVuongViPham extends HinhChuNhatViPham {
    @Override void datChieuDai(double d) {
        this.chieuDai = d;
        this.chieuRong = d; // BAT NGO thay doi ca chieuRong!
    }
}
```

```
}  
}
```

Đây là ví dụ kinh điển: về mặt toán học, hình vuông "là một" hình chữ nhật đặc biệt — có vẻ hợp lý để `HìnhVuong` extends `HìnhChuNhat`. Nhưng `HìnhChuNhat` "hứa hẹn" một hành vi ngầm định: đổi chiều dài **không ảnh hưởng** chiều rộng. `HìnhVuong` (để giữ tính chất "hình vuông") buộc phải **phá vỡ** lời hứa ngầm đó — bất kỳ code nào viết đúng cho `HìnhChuNhat` (như `kiemTraDienTich` trong code mẫu) có thể cho kết quả **sai lệch bất ngờ** khi nhận vào một `HìnhVuong`. Vi phạm LSP thường là dấu hiệu **quan hệ kế thừa được chọn sai** — giải pháp ở đây: không ép `HìnhVuong` kế thừa `HìnhChuNhat`, để cả hai độc lập cùng implements một interface `HìnhDung` chung.

Bài học tổng quát: kế thừa nên phản ánh đúng quan hệ "là một" **theo hành vi**, không chỉ theo trục giác/toán học bề ngoài. Nếu lớp con phải "ghi đè" hành vi theo cách khiến code viết đúng cho lớp cha hoạt động sai, đó là dấu hiệu vi phạm LSP.

I — Interface Segregation Principle (Nguyên lý phân tách interface)

Không nên buộc một class implements những phương thức nó không dùng tới.

```
// VI PHAM: interface "beo", may in don gian buoc phai co ca scan()/fax()  
interface MayVanPhongDaNangViPham {  
    void in(String noiDung);  
    void scan(String noiDung);  
    void fax(String noiDung);  
}
```

Một interface quá lớn, gộp nhiều khả năng không liên quan, buộc **mọi** class implements nó phải cung cấp thân hàm cho **mọi** phương thức — kể cả những phương thức nó không thực sự hỗ trợ (dẫn tới thân hàm rỗng vô nghĩa, hoặc ném `UnsupportedOperationException`, cả hai đều là dấu hiệu thiết kế kém). Giải pháp: tách thành các interface **nhỏ, tập trung** (`CoTheIn`, `CoTheScan`, `CoTheFax`), mỗi class chỉ implements đúng những interface phản ánh đúng khả năng thật của nó.

D — Dependency Inversion Principle (Nguyên lý đảo ngược phụ thuộc)

Module cấp cao không nên phụ thuộc trực tiếp vào module cấp thấp — cả hai nên phụ thuộc vào một sự trừu tượng (abstraction) chung.

```
// VI PHAM: DichVuThongBao phu thuc TRUC TIEP vao EmailSenderCuThe  
class DichVuThongBaoViPham {  
    private EmailSenderCuThe sender = new EmailSenderCuThe();  
}
```

```
// TUAN THU: ca hai phu thuc vao interface KenhGui chung  
class DichVuThongBao {  
    private KenhGui kenh;  
    DichVuThongBao(KenhGui kenh) { this.kenh = kenh; } // "tiem" phu thuc tu ben ngoai  
}
```

Đây chính là ứng dụng cụ thể của ý tưởng Strategy (Chương 44) và "lập trình theo interface" (Chương 24) vào mối quan hệ giữa các **thành phần lớn** của hệ thống: thay vì `DichVuThongBao` (logic nghiệp vụ quan trọng, "cấp cao") tự tạo (`new`) trực tiếp một `EmailSenderCuThe` cụ thể ("cấp thấp", chi tiết cài đặt), nó chỉ phụ thuộc vào **interface** `KenhGui` — và **nhận** cài đặt cụ thể từ **bên ngoài** (qua constructor — kỹ thuật này gọi là **dependency injection**, "tiêm phụ thuộc"). Kết quả: đổi kênh gửi (Email → SMS) **không cần sửa** `DichVuThongBao` — đúng tinh thần OCP đã học ở trên, áp dụng ở tầm module lớn hơn.

SOLID và các chương trước — một bức tranh thống nhất

Nếu bạn thấy các nguyên lý SOLID "quen quen" — đúng vậy: chúng chính là sự **hệ thống hoá** những ý tưởng đã rải rác xuất hiện xuyên suốt sách, từ đa hình (Chương 17), interface (Chương 18), tới design pattern (Chương 44). SOLID không giới thiệu

khái niệm hoàn toàn mới, mà đưa ra **5 góc nhìn có tên gọi rõ ràng** để đánh giá một thiết kế OOP có "khoẻ mạnh" hay không — hữu ích nhất khi dùng làm **danh sách kiểm tra (checklist)** lúc review code của chính bạn hoặc đồng nghiệp.

Bài tập

1. Tìm trong code bạn đã viết ở các bài tập trước (bất kỳ chương nào) một chỗ có thể vi phạm SRP (một class làm quá nhiều việc không liên quan) — thử tách lại theo đúng tinh thần chương này.
2. Viết một ví dụ vi phạm OCP của riêng bạn (một hàm `switch` theo loại, dễ phải sửa khi thêm loại mới), rồi refactor sang dùng đa hình.
3. Giải thích bằng lời (không cần code): tại sao ví dụ `HìnhVuong / HìnhChuNhat` vi phạm LSP lại **không** xảy ra nếu cả hai chỉ implements chung một interface `HìnhDung` có mỗi `tínhDienTich()` , thay vì kế thừa lẫn nhau?

Lỗi thường gặp

- **Áp dụng SOLID cứng nhắc cho mọi đoạn code, kể cả code rất đơn giản** — tách một class 5 dòng thành 3 class riêng "cho đúng SRP" là phản tác dụng nếu độ phức tạp thực tế không cần tới. SOLID phục vụ việc **quản lý độ phức tạp khi nó thực sự tồn tại**, không phải mục tiêu tự thân.
- **Nhầm lẫn "một class một phương thức" với Single Responsibility** — SRP nói về "một lý do để thay đổi", không phải "chỉ một phương thức". Một class có nhiều phương thức vẫn tuân thủ SRP nếu tất cả phương thức đó phục vụ **cùng một trách nhiệm** gắn kết chặt chẽ.
- **Tạo interface cho mọi class "phòng khi cần sau này"** — vi phạm ngược lại tinh thần đơn giản: chỉ tạo trừu tượng hoá (interface, DIP) khi **thực sự có từ hai cài đặt cụ thể trở lên**, hoặc có lý do rõ ràng cần thay thế được trong tương lai gần — không phải "cứ thêm interface cho chắc".
- **Cố nhớ thuộc lòng 5 chữ SOLID mà không hiểu bản chất từng nguyên lý** — hiểu **vì sao** mỗi nguyên lý tồn tại (vấn đề gì nó giải quyết) quan trọng hơn nhiều so với thuộc lòng định nghĩa — quay lại đọc phần ví dụ "TRƯỚC/SAU" trong code mẫu bất cứ khi nào cần nhớ lại.

Chương 46 — Kết nối cơ sở dữ liệu: JDBC cơ bản

Mục tiêu học

Sau chương này, bạn sẽ:

- Kết nối được tới cơ sở dữ liệu quan hệ từ Java bằng JDBC.
- Thực thi câu lệnh SQL (tạo bảng, thêm, truy vấn dữ liệu) và đọc kết quả bằng `ResultSet`.
- Hiểu lỗ hổng **SQL injection** là gì, và dùng `PreparedStatement` để phòng tránh — đây là kỹ năng **bắt buộc phải nắm vững** trước khi viết bất kỳ code tương tác database thực tế nào.

Code mẫu đầy đủ: [code/ch46-jdbc/](#).

JDBC là gì?

JDBC (Java Database Connectivity) là API chuẩn của Java để giao tiếp với cơ sở dữ liệu quan hệ (MySQL, PostgreSQL, Oracle, H2...). Mỗi loại cơ sở dữ liệu có một **driver** riêng (một thư viện `.jar` — nhắc lại khái niệm dependency ở Chương 35/42) cài đặt các interface chuẩn của JDBC, nên code Java của bạn viết theo **cùng một API** dù đang giao tiếp với loại cơ sở dữ liệu nào — chỉ cần đổi driver và connection URL, phần lớn code còn lại giữ nguyên.

Chương này dùng **H2** — một cơ sở dữ liệu **nhúng** (embedded), chạy trực tiếp trong bộ nhớ của chương trình Java, không cần cài đặt/khởi động một server database riêng biệt như MySQL/PostgreSQL — lý tưởng để học và thử nghiệm nhanh mà không cần thiết lập môi trường phức tạp.

Kết nối cơ sở dữ liệu

```
String url = "jdbc:h2:mem:testdb";
try (Connection conn = DriverManager.getConnection(url, "sa", "")) {
    // ...
}
```

`DriverManager.getConnection(url, user, password)` mở một kết nối. `Connection` là `AutoCloseable` (Chương 21) — luôn dùng `try-with-resources` để đảm bảo kết nối được đóng đúng cách, tránh rò rỉ tài nguyên (một vấn đề nghiêm trọng nếu chương trình mở nhiều kết nối mà không đóng, dần dần làm cạn kiệt tài nguyên hệ thống/database).

Thực thi SQL và đọc kết quả với Statement

```
try (Statement stmt = conn.createStatement()) {
    stmt.execute("CREATE TABLE sinh_vien (id INT PRIMARY KEY, ten VARCHAR(100), tuoi INT)");
    stmt.execute("INSERT INTO sinh_vien VALUES (1, 'Nguyen Van A', 20)");

    try (ResultSet rs = stmt.executeQuery("SELECT id, ten, tuoi FROM sinh_vien")) {
        while (rs.next()) {
            int id = rs.getInt("id");
            String ten = rs.getString("ten");
            System.out.println(id + " - " + ten);
        }
    }
}
```

- `execute(...)` — chạy câu lệnh không cần lấy kết quả dạng bảng (tạo bảng, thêm/sửa/xóa dữ liệu).
- `executeQuery(...)` — chạy câu `SELECT`, trả về `ResultSet` — đại diện cho **kết quả dạng bảng**.
- `rs.next()` — di chuyển "con trỏ" tới **dòng tiếp theo** của kết quả, trả về `false` khi hết dữ liệu (mẫu quen thuộc, tương tự `readLine()` ở Chương 34). `rs.getInt("id") / rs.getString("ten")` đọc giá trị của **cột** tương ứng trong dòng hiện

tại.

SQL injection — lỗ hổng nghiêm trọng cần tránh

```
// SAI - RUI RO SQL INJECTION
String tenNguoiDung = layTuNguoiDungNhap(); // ví dụ: "admin' OR '1'='1"
String sql = "SELECT * FROM nguoi_dung WHERE ten = '" + tenNguoiDung + "'";
```

Nếu `tenNguoiDung` là dữ liệu do **người dùng nhập** (không đáng tin cậy), và bạn **nối chuỗi trực tiếp** vào câu SQL, kẻ tấn công có thể nhập một giá trị được **thiết kế đặc biệt** để thay đổi hoàn toàn ý nghĩa câu lệnh SQL. Với ví dụ trên, nhập `admin' OR '1'='1` biến câu lệnh thành:

```
SELECT * FROM nguoi_dung WHERE ten = 'admin' OR '1'='1'
```

`'1'='1'` **luôn đúng**, khiến điều kiện `WHERE` trở nên vô nghĩa — câu lệnh trả về **toàn bộ** bảng `nguoi_dung`, thay vì đúng một dòng như dự định (trong tình huống thực tế nghiêm trọng hơn, kỹ thuật này có thể dùng để bỏ qua xác thực đăng nhập, đọc trộm dữ liệu, hoặc thậm chí xoá/sửa dữ liệu tùy ý). Đây gọi là **SQL injection** — một trong những lỗ hổng bảo mật web/ứng dụng phổ biến và nguy hiểm nhất trong lịch sử lập trình.

PreparedStatement — cách phòng tránh đúng cách

```
String sql = "SELECT * FROM nguoi_dung WHERE ten = ?";
try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, tenNguoiDung); // giá trị được truyền RIENG, KHÔNG nối vào chuỗi SQL
    try (ResultSet rs = pstmt.executeQuery()) {
        // ...
    }
}
```

Dấu `?` là **tham số giữ chỗ** (placeholder) — giá trị thật được truyền **riêng biệt** qua `setString(vị_trí, giá_trị)` (tương tự `setInt`, `setDouble` ... tùy kiểu dữ liệu), **không bao giờ** được database hiểu là một phần **cấu trúc** câu lệnh SQL, dù giá trị đó chứa ký tự đặc biệt gì đi nữa (`'` , `;` , `--` ...). Database driver tự đảm bảo tách biệt hoàn toàn giữa "cấu trúc câu lệnh" và "dữ liệu", loại bỏ hoàn toàn khả năng SQL injection theo cách trên.

Quy tắc bắt buộc, không có ngoại lệ: luôn dùng `PreparedStatement` với tham số `?` cho bất kỳ giá trị nào **có nguồn gốc từ bên ngoài** (người dùng nhập, dữ liệu từ file/API...) khi xây dựng câu lệnh SQL. Không bao giờ nối chuỗi trực tiếp giá trị không đáng tin cậy vào câu SQL, dù chỉ để "viết nhanh cho tiện" trong lúc học hay thử nghiệm — đây là thói quen cần hình thành **ngay từ đầu**, không để dành "sửa sau" trong dự án thực tế.

`PreparedStatement` còn xử lý đúng các ký tự đặc biệt trong dữ liệu (như dấu nháy đơn trong tên `"O'Brien"`, xem `PreparedStatementDemo.java`) mà không cần bạn tự "escape" thủ công — nối chuỗi thủ công với dữ liệu chứa dấu nháy đơn thường gây **lỗi cú pháp SQL**, ngoài rủi ro bảo mật.

Bài tập

- Mở rộng `JdbcBasicsDemo.java` thêm câu lệnh `UPDATE` (sửa tuổi một sinh viên) và `DELETE` (xoá một sinh viên), dùng `PreparedStatement` cho cả hai.
- Viết phương thức `List<String> timSinhVienTheoTen(Connection conn, String tuKhoa)` dùng `PreparedStatement` với `LIKE ?` (tìm kiếm gần đúng theo tên), truyền `"%" + tuKhoa + "%"` làm tham số.
- Thử tự chạy lại thí nghiệm SQL injection trong `PreparedStatementDemo.java` với vài giá trị đầu vào "độc hại" khác (tự nghĩ thêm), xác nhận `PreparedStatement` luôn trả về đúng kết quả mong đợi (0 hoặc đúng số bản ghi khớp thật sự).

Lỗi thường gặp

- **Nối chuỗi trực tiếp dữ liệu người dùng vào câu SQL** — lỗ hổng SQL injection, lỗi nghiêm trọng nhất có thể mắc phải trong chương này. Luôn dùng `PreparedStatement`.
- **Quên đóng `Connection` / `Statement` / `ResultSet`** — rò rỉ tài nguyên, dần dần làm cạn kiệt số kết nối khả dụng của database. Luôn dùng `try-with-resources` (Chương 21).
- **`SQLException: No suitable driver found`** — thiếu driver JDBC tương ứng trong classpath (quên `-cp` trỏ tới file `.jar` của H2, hoặc chưa khai báo dependency đúng trong Maven/Gradle).
- **Đánh chỉ số tham số ? sai vị trí trong `PreparedStatement`** (`setString` / `setInt` với chỉ số sai) — chỉ số bắt đầu từ **1**, không phải **0** (khác với chỉ số mảng/ `List` đã quen từ Chương 9, 24) — gán sai vị trí dẫn tới lỗi hoặc dữ liệu bị gán nhầm cột.
- **Không đóng `ResultSet` trước khi chạy truy vấn khác trên cùng `Statement`** — một số driver database yêu cầu đóng `ResultSet` hiện tại trước khi tái sử dụng cùng `Statement` cho câu lệnh khác; dùng `try-with-resources` lồng nhau đúng cách (như trong code mẫu) tránh được vấn đề này.

Chương 47 — Dự án tổng hợp

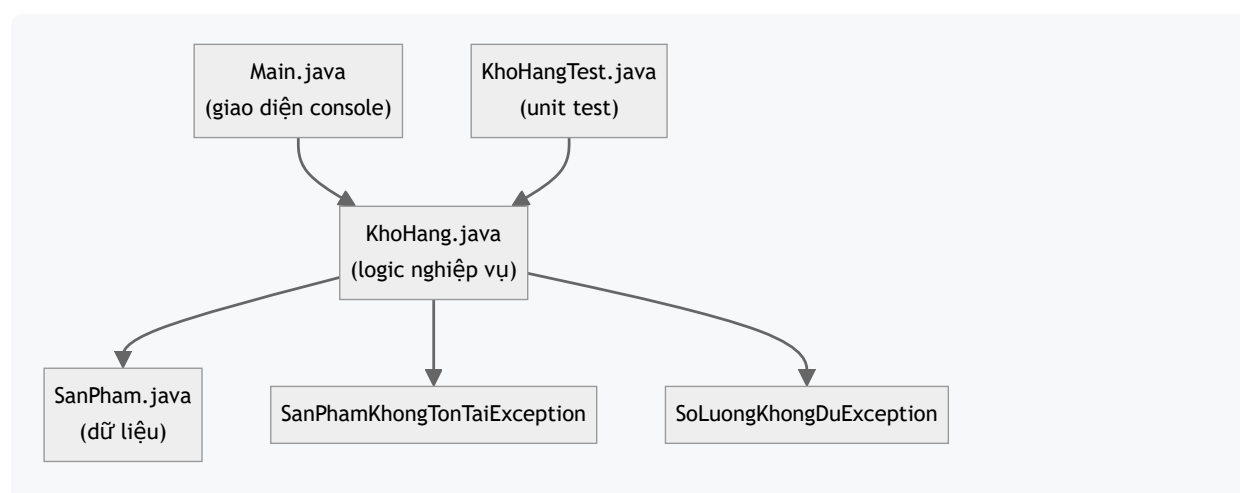
Mục tiêu

Đây là chương **thực hành tổng hợp**, không giới thiệu khái niệm mới — mục tiêu là xây dựng một ứng dụng console hoàn chỉnh, áp dụng lại kiến thức từ Phần 2 đến Phần 10 vào **một dự án thống nhất**, thay vì các ví dụ nhỏ tách rời từng chương như trước.

Dự án: **Quản lý kho hàng** — thêm/sửa/xoá sản phẩm, nhập/xuất kho, tìm sản phẩm sắp hết hàng, tính tổng giá trị kho, lưu/đọc dữ liệu giữa các lần chạy, có unit test.

Code đầy đủ: code/ch47-du-an-tong-hop/ — khuyến khích **tự mình đọc kỹ toàn bộ 5 file** trước khi đọc phần phân tích dưới đây, cố gắng tự nhận diện các khái niệm đã học ở từng chỗ.

Thiết kế tổng thể



Kiến trúc này áp dụng trực tiếp **SRP** (Chương 45): `Main` chỉ lo đọc/hiển thị console, không chứa logic nghiệp vụ; `KhoHang` chỉ lo logic nghiệp vụ, không biết gì về `Scanner / System.out`. Đây chính là lý do `KhoHangTest.java` viết test được dễ dàng cho `KhoHang` — không cần giả lập bàn phím/màn hình gì cả (nhắc lại nguyên tắc "thiết kế dễ test" ở Chương 41).

Đối chiếu từng phần với kiến thức đã học

`SanPham.java` — Chương 13, 14, 20

Class dữ liệu với field `private`, getter, và setter **có chọn lọc** (`datSoLuong / datGia` — có setter, khác `TaiKhoanNganHang` ở Chương 14 cố tình không có setter cho `soDu`, vì ở đây việc "đặt lại số lượng" là hành động hợp lệ, khác bản chất bài toán ngân hàng). Override `equals / hashCode` dựa trên `maSanPham` (mã sản phẩm là định danh duy nhất — đúng tinh thần Chương 20: chọn field nào thực sự xác định "hai object có cùng là một sản phẩm hay không"), và `toString()` định dạng đẹp để hiển thị console.

`SanPhamKhongTonTaiException.java`, `SoLuongKhongDuException.java` — Chương 21

Hai checked exception tự định nghĩa, đại diện cho hai tình huống lỗi nghiệp vụ **có thể lường trước** (thao tác trên mã sản phẩm không tồn tại, xuất kho vượt số lượng tồn) — nhắc lại lý do chọn checked exception thay vì chỉ in cảnh báo (đã bàn ở Chương 21): buộc `Main` phải **chủ động** xử lý, không thể "vô tình" bỏ qua.

`KhoHang.java` — Chương 26, 31, 34, 35, 45

- `Map<String, SanPham>` (cụ thể là `LinkedHashMap`, giữ thứ tự thêm vào — Chương 26) làm cấu trúc lưu trữ chính, tra cứu theo mã sản phẩm.

- `timSanPhamSapHet(...)` dùng Stream API (`filter` , `sorted` , `collect` — Chương 31) thay vì vòng lặp thủ công.
- `lưuRaFile` / `docTuFile` dùng `Files.write` / `Files.readAllLines` (Chương 34), định dạng CSV (Chương 35) qua `toCsvLine` / `tuCsvLine` trong `SanPham` .
- Toàn bộ class chỉ đảm nhận **một trách nhiệm**: quản lý dữ liệu và quy tắc nghiệp vụ của kho hàng (SRP, Chương 45) — không biết gì về cách dữ liệu được hiển thị hay nhập vào.

Main.java — Chương 5, 7, 21

`Scanner` đọc input (Chương 5), `switch` expression điều hướng menu (Chương 7), `try/catch` multi-catch xử lý cả hai loại exception nghiệp vụ cùng lúc, cộng thêm `NumberFormatException` khi người dùng nhập sai định dạng số (Chương 21) — chương trình **không crash** dù người dùng nhập bậy, luôn quay lại menu để thử tiếp.

KhoHangTest.java — Chương 41

Test cho từng nghiệp vụ cốt lõi: thêm/tìm, nhập kho, xuất kho (cả trường hợp thành công lẫn thất bại — `assertThrows`), tìm sản phẩm sắp hết, tính tổng giá trị. `@BeforeEach` đảm bảo mỗi test bắt đầu với một `KhoHang` chứa đúng một sản phẩm mẫu, độc lập hoàn toàn với các test khác.

Những gì dự án CHƯA áp dụng, và vì sao

Dự án không cố nhồi nhét **mọi** kiến thức đã học — đây là lựa chọn có chủ đích, phản ánh đúng thực tế: không phải dự án nào cũng cần đa luồng (Chương 37-40), design pattern phức tạp (Chương 44), hay JDBC (Chương 46). README code mẫu gợi ý một số hướng mở rộng **không bắt buộc** nếu bạn muốn luyện tập thêm — ví dụ thay lưu trữ CSV bằng JDBC + H2, hoặc thêm Strategy pattern cho quy tắc giảm giá. **Biết khi nào một kỹ thuật là cần thiết** (thay vì áp dụng máy móc mọi thứ đã học vào mọi dự án) chính là một phần quan trọng của tư duy kỹ sư phần mềm trưởng thành — nhắc lại đúng tinh thần đã nhấn mạnh ở Chương 44: không lạm dụng công cụ chỉ vì "biết nó tồn tại".

Bài tập mở rộng

1. Chạy chương trình, thử toàn bộ các chức năng menu, cố tình nhập sai (chữ thay vì số, mã sản phẩm không tồn tại, xuất kho vượt tồn) để xác nhận chương trình xử lý đúng, không crash.
2. Thêm chức năng "sửa thông tin sản phẩm" (đổi tên/giá mà không qua nhập/xuất kho) vào menu.
3. Thêm chức năng "tìm kiếm sản phẩm theo tên" (tìm gần đúng, không cần khớp chính xác — dùng `String.contains` , Chương 11) vào `KhoHang` và `Main` .
4. (Nâng cao) Thực hiện một trong các gợi ý mở rộng trong README code mẫu: chuyển lưu trữ sang JDBC
 - H2 (Chương 46), hoặc thêm Strategy pattern (Chương 44) cho quy tắc giảm giá khi xuất kho số lượng lớn.
5. Viết thêm test cho các chức năng bạn tự thêm ở bài 2-3, giữ đúng nguyên tắc mỗi test độc lập (Chương 41).

Hoàn thành dự án này là cột mốc kết thúc phần thân chính của sách — bạn đã đi từ `System.out.println("Xin chào, Java!")` ở Chương 3 tới một ứng dụng console hoàn chỉnh có kiến trúc rõ ràng, xử lý lỗi đúng cách, và có test tự động. Phụ lục A-B (chương tiếp theo) cung cấp tài liệu tra cứu nhanh cho quá trình bạn tiếp tục thực hành sau khi đọc xong sách.

Phụ lục A — Bảng tổng hợp lỗi thường gặp & cách xử lý

Tra cứu nhanh theo **tên lỗi/triệu chứng**. Mỗi mục ghi rõ chương giải thích chi tiết, để quay lại đọc kỹ nếu cần.

Lỗi biên dịch (compile-time)

Thông báo/triệu chứng	Nguyên nhân thường gặp	Chương
class X is public, should be declared in a file named X.java	Tên file không khớp tên class public	3
';' expected	Thiếu dấu chấm phẩy cuối câu lệnh	3
Error: Main method not found	Sai chữ ký public static void main(String[] args)	3
incompatible types: possible lossy conversion	Gán kiểu lớn (double) cho biến kiểu nhỏ (int) mà không ép kiểu	5
cannot assign a value to final variable	Gán lại giá trị cho biến final	5
unreachable catch block	Đặt catch lớp cha trước lớp con	21
missing return statement	Có nhánh logic không trả về giá trị dù kiểu trả về khác void	10
variable might not have been initialized	Dùng biến cục bộ trước khi gán giá trị lần đầu	5, 10
Lambda báo lỗi liên quan biến ngoài	Biến bị gán lại ở đâu đó (vi phạm "effectively final")	30
X is abstract; cannot be instantiated	Cố new trực tiếp một abstract class	18
constructor ... cannot be applied to given types	Quên gọi super(...) đúng tham số khi lớp cha không có constructor rỗng	16
package ... does not exist / cannot find symbol	Sai cấu trúc thư mục so với tên package, hoặc quên biên dịch đủ file	15
could not resolve dependency (Maven/Gradle)	Sai groupId/artifactId/version của thư viện	42

Exception lúc chạy (runtime) — theo tên

Exception	Ý nghĩa	Nguyên nhân thường gặp	Chương
NullPointerException	Gọi phương thức/truy cập field trên biến null	Quên new, hoặc Map.get()/tìm kiếm trả về null	13, 20, 26
ArrayIndexOutOfBoundsException	Truy cập chỉ số mảng ngoài phạm vi [0, length)	Sai điều kiện vòng lặp (<= thay vì <)	9
IndexOutOfBoundsException	Tương tự trên, nhưng với List	get/remove sai chỉ số	24
StringIndexOutOfBoundsException	substring/charAt sai chỉ số	Tính sai vị trí bắt đầu/kết thúc	11
ClassCastException	Ép kiểu (downcasting) sai loại thực sự của object	Không kiểm tra instanceof trước khi ép kiểu	17
ArithmeticException	Chia số nguyên cho 0	Thiếu kiểm tra mẫu số trước khi chia	6, 21
NumberFormatException	Integer.parseInt/Double.parseDouble với chuỗi không đúng định dạng số	Dữ liệu đầu vào không sạch (CSV, input người dùng)	21, 35, 47
InputMismatchException	Scanner.nextInt()/nextDouble() khi người dùng nhập sai kiểu	Không validate trước khi đọc	5
ConcurrentModificationException	Sửa collection trong lúc for-each đang duyệt nó	Dùng Iterator.remove() thay vì sửa trực tiếp	26, 27

Exception	Ý nghĩa	Nguyên nhân thường gặp	Chương
UnsupportedOperationException	Sửa một collection/danh sách bất biến	List.of(...), Collections.unmodifiableList(...), field record	24, 27, 33
IllegalStateException	Gọi Iterator.remove() không đúng sau một lần next()	Gọi remove() hai lần liên tiếp, hoặc chưa gọi next()	27
NoSuchElementException	Gọi Optional.get() khi rỗng	Không kiểm tra isPresent()/dùng orElse trước	32
StackOverflowError	Đệ quy không có (hoặc sai) điều kiện dừng	Thiếu/base case sai trong hàm đệ quy	29
IllegalThreadStateException	Gọi start() hai lần trên cùng một Thread	Thread chỉ chạy được đúng một lần	37
NotSerializableException	Serialize object của class không implements Serializable	Quên khai báo, hoặc field tham chiếu tới class không serialize được	36
InvalidClassException	serialVersionUID không khớp khi đọc lại file .ser	Sửa cấu trúc class mà không cập nhật serialVersionUID	36
ClassNotFoundException	Thiếu thư viện trong classpath	Quên -cp trỏ tới file .jar cần thiết	35, 41
SQLException: No suitable driver found	Thiếu JDBC driver trong classpath	Quên thêm driver database vào -cp/dependency	46

Lỗi logic (không có exception, kết quả sai)

Triệu chứng	Nguyên nhân thường gặp	Chương
Phép chia ra kết quả bị cắt, không có phần thập phân	Chia hai số int, phép chia xảy ra trước khi gán cho double	6
So sánh String bằng == "có lúc đúng có lúc sai"	Nên dùng .equals(), không dùng ==, cho mọi so sánh nội dung String	11
Sửa object qua biến này ảnh hưởng biến khác	Gán tham chiếu (b = a) không sao chép, cả hai trỏ chung object	9, 13
Field không được gán đúng giá trị trong constructor	Quên this. khi tham số trùng tên field	13
switch chạy "thừa" qua nhánh không mong muốn	Quên break trong switch statement (fall-through)	7
HashSet/HashMap chứa "trùng lặp" dù nội dung giống hệt	Object chưa override equals/hashCode đúng cách	20, 25, 26
Kết quả tính toán đa luồng sai, khác nhau mỗi lần chạy	Race condition — nhiều thread cùng sửa dữ liệu chung không đồng bộ hoá	38
Chương trình treo vĩnh viễn, không exception	Deadlock — hai thread khoá chéo nhau	38
Dữ liệu bị tách sai cột khi đọc CSV	Giá trị chứa dấu phẩy bên trong, split(",") đơn giản không xử lý được	35
Truy vấn SQL trả về nhiều/sai dữ liệu bất thường	SQL injection — nối chuỗi trực tiếp dữ liệu không đáng tin cậy vào câu SQL	46

Quy trình gỡ lỗi tổng quát (khi chưa biết lỗi thuộc loại nào)

1. **Đọc kỹ thông báo lỗi** — tên exception, thông điệp, và **dòng code** được chỉ ra trong stack trace thường trỏ thẳng tới (hoặc rất gần) nguyên nhân.
2. **Tra bảng ở trên** theo tên exception/triệu chứng để biết nhóm nguyên nhân phổ biến.
3. **Dùng debugger** (Chương 4, 43) — đặt breakpoint gần dòng gây lỗi, Step Over/Into để quan sát giá trị biến thực tế so với kỳ vọng.

4. Nếu là lỗi logic không có exception, viết **unit test** (Chương 41) tái hiện tình huống lỗi ở quy mô nhỏ nhất có thể, dễ cô lập nguyên nhân hơn chạy cả chương trình lớn.

Phụ lục B — Tài liệu tham khảo & nguồn học thêm

Tài liệu chính thức

- **Java SE Documentation** — docs.oracle.com/en/java — tài liệu chính thức của Oracle, gồm Java Language Specification và JavaDoc của toàn bộ thư viện chuẩn (`java.lang` , `java.util` , `java.io` ...). Đây là nguồn **đáng tin cậy nhất** khi cần tra cứu chính xác một class/phương thức làm gì — thói quen tra JavaDoc trực tiếp (thay vì chỉ hỏi công cụ AI hay tìm trên diễn đàn) là kỹ năng quan trọng nên rèn từ sớm.
- **OpenJDK** — openjdk.org — dự án mã nguồn mở là nền tảng của hầu hết các bản JDK miễn phí hiện nay (kể cả Eclipse Temurin đã dùng ở Chương 2).
- **Maven Central** — search.maven.org — kho lưu trữ thư viện trung tâm, tra cứu `groupId` / `artifactId` / `version` chính xác của bất kỳ thư viện nào trước khi khai báo dependency (Chương 42).

Công cụ đã dùng trong sách

- **Eclipse Temurin (Adoptium)** — adoptium.net — bản JDK dùng xuyên suốt sách (Chương 2).
- **IntelliJ IDEA** — jetbrains.com/idea — IDE minh họa chính (Chương 4), bản Community miễn phí đủ dùng cho toàn bộ nội dung sách.
- **Apache Maven** — maven.apache.org và **Gradle** — gradle.org — hai build tool đã học ở Chương 42.
- **JUnit 5** — junit.org/junit5 — framework kiểm thử dùng ở Chương 41, 47.
- **Gson** — github.com/google/gson — thư viện xử lý JSON dùng ở Chương 35, 42.
- **H2 Database** — h2database.com — cơ sở dữ liệu nhúng dùng để học JDBC ở Chương 46.

Hướng đi tiếp theo sau khi đọc xong sách

Sách này dừng lại ở nền tảng Java SE vững chắc — đủ để bắt đầu học các hướng chuyên sâu hơn tùy mục tiêu của bạn:

- **Phát triển backend/web** — học **Spring Framework/Spring Boot** (spring.io) — framework phổ biến nhất cho ứng dụng Java doanh nghiệp, xây dựng trực tiếp trên nền OOP, Collection, Stream, generics đã học trong sách này.
- **Phát triển Android** — Android SDK dùng Java (và Kotlin) làm ngôn ngữ chính — xem tài liệu chính thức tại developer.android.com.
- **Xử lý dữ liệu lớn/hệ thống phân tán** — các hệ thống như Apache Kafka, Apache Spark, Hadoop đều viết bằng/chạy trên JVM, dùng nhiều tới đa luồng (Phần 8) và kiến trúc (Phần 10).
- **Đào sâu JVM** — tìm hiểu về garbage collector, JIT compiler, tối ưu hiệu năng — hữu ích khi làm việc với hệ thống quy mô lớn, nơi hiểu cơ chế bên dưới JVM (đã giới thiệu sơ lược ở Chương 1) trở nên quan trọng.

Thói quen học tập nên giữ sau khi đọc xong sách

- **Đọc lại JavaDoc trước khi đoán** — khi không chắc một phương thức thư viện chuẩn hoạt động thế nào, tra cứu trực tiếp thay vì đoán dựa trên tên gọi.
- **Viết code, không chỉ đọc** — mọi chương trong sách đều có code mẫu và bài tập; kỹ năng lập trình chỉ hình thành qua thực hành, đọc lý thuyết một mình không đủ.
- **Đọc code người khác viết** — tìm các dự án mã nguồn mở Java trên GitHub phù hợp trình độ, đọc cách người khác tổ chức code thực tế — đối chiếu với nguyên tắc đã học (SOLID, design pattern — Chương 44-45) trong ngữ cảnh một dự án lớn thật sự.
- **Quay lại các chương OOP nền tảng (Phần 2-3) khi thấy mơ hồ** — đây là phần khó nhất và quan trọng nhất của sách; hiểu chưa vững ở giai đoạn đầu sẽ gây khó khăn dồn tích khi học các phần sau (đặc biệt Phần 6, 10) — không ngại đọc lại.

